BES TWS

SOFTWARE DEVELOPMENT USER MANUAL

BES Confidential and Proprietary

CONTACT US:

Company: *Bestechnic (Shanghai) Co., Ltd. (“**BES**”)*

Address: *Room 201, Tower B, Chamtime Plaza, Lane 2889 Jinke Road, Pudong New District, Shanghai 201203, China*

Phone: *(86)21 6877 1788*

For products inquires and more information, please visit www.bestechnic.com*.*

DISCLAIMER:

No part of this document may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying and recording, for any purpose, without the express written permission of BES. BES retains the right to make changes to this document at any time, without notice. BES makes no warranty of any kind, expressed or implied, with regard to any information contained in this document, including, but not limited to, the implied warranties of merchant ability or fitness for any particular purpose. Further, BES does not warrant the accuracy or completeness of the information, text, graphics, or other items contained within this document.

Revision History

The following table lists the technical content changes for all revisions.

Revision	Date	Description
1.0	July 2022	Initial release
1.1	April 2023	Made function names and order of sample code conisistant with those in code base; included more detailed guidence for Software Development Environment Setup section; grammatical and formatting editing
1.2	May 2023	Updated prompt usage guide: add prompt in flash functionality

CONFIDENTIAL

Table of Contents

List of Tables 6
List of Figures 8
List of Sample Code 12
1. General Description 14
2. Terms and Definitions 15
3. Get Started with our SDK 16
3.1 Software Development Environment Setup 16
3.1.1 Set up Compilation Environment 16
3.1.2 Build the Project 16
3.1.3 Usage of Hardware Development Kit 17
3.1.4 Image Programming 18
3.1.5 Implement Basic Functions 21
4. Software System Introduction 24
4.1 Software Architecture 24
4.2 SDK Folder Structure 25
4.3 Introduction of Main MCU-dedicated Threads 26
5. Software Development Guide 28
5.1 Target Configuration 28
5.1.1 Common.mk 28
5.1.2 Target.mk 28
5.1.3 Link Script 28
5.1.4 Target Hardware Configuration 28
5.2 Hardware Abstract Layer Drives 32
5.2.1 Use GPIO 32
5.2.2 Use UART 36
5.2.3 Use I2C 39
5.2.4 Use SPI 42
5.2.5 Use I2S 44
5.2.6 Use ADC 47
5.2.7 Use Hardware Timer 48
5.2.8 Use PWM 49
5.2.9 DMA 52
5.3 Memory Map 54
5.3.1 Logic Address Map 55
5.3.2 Flash Memory Map 55
5.3.3 Audio Buffer Usage 56
5.3.4 Memory Usage in Link Script 57
5.3.5 Define a Specific Section and Use it 60
5.3.6 Coordinate Ram Size 60
5.3.7 Overlay Usage 61
5.4 System API 62
5.4.1 System Ticks 62
5.4.2 Battery Management 62

CONFIDENTIAL

5.4.3 UART Data Path 64
5.4.4 Dynamic Frequency 68
5.4.5 Core Bridge 69
5.4.6 Sensor Hub 72
5.4.7 M55 74
5.4.8 Co-processor 75
5.4.9 Power Management 77
5.4.10 Button Action Handler 86
5.4.11 Nor-Flash API 92
5.4.12 PMU Watchdog 103
5.4.13 TWS Synchronization Operation API 104
5.4.14 Dynamic Heap Usage 105
5.4.15 Retention Information Management 106
5.5 RTOS 107
5.5.1 Multi-task 107
5.5.2 Task Scheduler 107
5.5.3 OS Abstract Layer 109
5.5.4 Create a Task 110
5.5.5 OS Event 111
5.5.6 OS Mailbox 113
5.5.7 OS Timer 115
5.5.8 OS Semaphore 116
5.5.9 OS Delay 117
5.5.10 OS Thread Statistics 118
5.6 Audio Function Usage Guide 120
5.6.1 Audio Flinger 120
5.6.2 Audio Data Path Introduction 126
5.6.3 Left and Right Playback Channel Configuration 128
5.6.4 Configure Music Jitter Buffer Length 131
5.7 Callback API 132
5.7.1 Prompt Usage Guide 132
5.7.2 Volume Level Configuration and Volume Control 142
5.8 Bluetooth Usage Guide 144
5.8.1 Bluetooth Framework 144
5.8.2 Classic BT 146
5.8.3 BLE 180
5.8.4 TWS Communication Data Path 236
5.8.5 GATT over BR/EDR 239
5.8.6 DUT Test Mode 241
5.8.7 Throughput Usage Guide 248
5.9 OTA 260
5.9.1 OTA Flash Memory Map 260
5.9.2 OTA Flow 261
5.9.3 OTA Protocol 262
5.9.4 OTA User Guide 277

CONFIDENTIAL

5.10 TOTA 298
5.10.1 TOTA Protocol 298
5.10.2 TOTA User Guide 301
5.11 Add a Custom UART Command 304
5.12 Debugging Methods 305
5.12.1 UART Log Print 305
5.12.2 Crash Analysis 305
5.12.3 Full CPU Workload Power Consumption Measurement 324
5.12.4 Thread Information Periodic Print 327
5.12.5 Audio Dump Tool Usage 330

CONFIDENTIAL

List of Tables

Table 1 Thread Introduction 26
Table 2 GPIO IOMuX Coniguration 32
Table 3 UART Configuration 36
Table 4 I2C Configuration 40
Table 5 Parameter List of Function hal_i2c_send 41
Table 6 Parameter List of Function hal_i2c_recv 42
Table 7 SPI Configuration 42
Table 8 I2S Configuration 44
Table 9 GPADC Configurable Items 47
Table 10 Hardware Timer Type 48
Table 11 PWM IOMuX Coniguration 49
Table 12 Breathing LED Configuration 51
Table 13 DMA Coniguration 52
Table 14 Overlay Configuration 61
Table 15 Dynamic Frequency Setting 68
Table 16 Lower Power Mode 77
Table 17 Wake Lock Types 79
Table 18 GPIO Button Configuration 86
Table 19 Flash Module ID 92
Table 20 Factory Section Information 99
Table 21 Data Format of Custom Parameters 102
Table 22 Example of Custom Parameters 103
Table 23 TWS Synchronization Operation Code 104
Table 24 Heap Information 105
Table 25 Threads Statistics Introduction 118
Table 26 Capture Configuration 120
Table 27 Playback Configuration 122
Table 28 APIs of Voice Prompt Module 136
Table 29 app_ibrt_if_get_a2dp_state Parameters 148
Table 30 app_ibrt_if_get_avrcp_state Parameters 148
Table 31 app_ibrt_if_get_hfp_state Parameters 149
Table 32 app_ibrt_if_get_hfp_call_status Parameters 149
Table 33 app_ibrt_if_get_mobile_connected_dev_list Parameters 150
Table 34 app_ibrt_if_nvrecord_get_mobile_addr Parameters 150
Table 35 app_ibrt_if_nvrecord_get_mobile_paired_dev_list Parameters 151
Table 36 app_ibrt_if_set_side Parameters 152
Table 37 app_ibrt_if_register_sync_user Parameters 153
Table 38 app_ibrt_if_deregister_sync_user Parameters 153
Table 39 app_ibrt_if_sync_info Parameters 153
Table 40 app_ibrt_if_request_modify_tws_bandwidth Parameters 154
Table 41 app_ibrt_if_write_bt_local_address Parameters 154
Table 42 app_ibrt_if_update_tws_pairing_info Parameters 154
Table 43 app_ibrt_if_start_tws_pairing Parameters 155

CONFIDENTIAL

Table 44 Ibrt_core_status_changed_cb_t Description 159
Table 45 app_ibrt_if_update_tws_pairing_info Parameters 160
Table 46 app_ibrt_if_start_tws_pairing Parameters 160
Table 47 app_ibrt_if_event_entry Description 161
Table 48 ibrt_core_status_changed_cb_t Description 162
Table 49 BES UI Property Configuration 164
Table 50 TWS_SYNC_USER_T Description 172
Table 51 app_ibrt_if_request_modify_tws_bandwidth Description 176
Table 52 BLE_ADV_PARAM_T Description 187
Table 53 Function app_ble_data_fill_enable Description 193
Table 54 Function app_ble_force_switch_adv Description 193
Table 55 Function app_ble_set_adv_type Description 194
Table 56 Function app_ble_param_set_adv_interval Description 197
Table 57 Function app_ble_set_adv_txpwr_by_actv_user Description 200
Table 58 Function app_ble_set_adv_txpwr_by_adv_user Description 200
Table 59 Function app_ble_set_all_adv_txpwr Description 200
Table 60 Function app_ble_set_adv_local_addr_by_user Description 202
Table 61 Function app_ble_set_adv_local_adddr_adv_user Description 202
Table 62 Function app_ble_refresh_adv_state Description 203
Table 63 Function app_ble_custom_adv_write_data Description 205
Table 64 Function app_ble_custom_adv_start Description 207
Table 65 Adv Event List 207
Table 66 Function app_ble_start_scan Description 208
Table 67 Scan Event List 210
Table 68 Function app_ble_start_connect Description 211
Table 69 Function app_ble_start_connect_with_init_type Description 212
Table 70 Function app_ble_is_connection_on_by_addr Description 213
Table 71 Function app_ble_is_connection_on Description 213
Table 72 Function app_ble_start_disconnect Description 214
Table 73 Connection Event List 215
Table 74 Function app_ble_update_conn_param_mode Description 219
Table 75 Function app_ble_conn_param Description 219
Table 76 Connection Parameter Management Event List 220
Table 77 Function aob_dts_register_spsm Description 231
Table 78 Function aob_dts_send_data Description 232
Table 79 Function aob_dts_disconnect Description 232
Table 80 DTS Callback handler Description 233
Table 81 Function aob_dtc_connect Decription 234
Table 82 Function aob_dtc_send_data Description 234
Table 83 Function aob_dtc_disconnect Description 235
Table 84 DTC Callback Handler Description 235
Table 85 Command Entry Definition: 237
Table 86 GATT Server User Callback Set 239
Table 87 BLE TX Test 247
Table 88 BLE RX Test 247

CONFIDENTIAL

Table 89 BLE service profile 248
Table 90 Data Format 250
Table 91 Command Code List 250
Table 92 Data Format of Command Response 250
Table 93 Error Code List 251
Table 94 Data Format 252
Table 95 OTA Command Type 263
Table 96 OTA Command Descriprion 263
Table 97 Transmission Resuming at Break Point (Request Packet) 264
Table 98 Transmission Resuming at Break Point (Verification Response Packet) 264
Table 99 Get a Current Version (Request Packet) 265
Table 100 Get a Current Version (Response Packet) 265
Table 101 Start an OTA Task (Request Packet) 265
Table 102 Start an OTA Task (Response Packet) 266
Table 103 OTA Configuration Packet 266
Table 104 OTA Configuration Respone Packet 267
Table 105 Data Packet Description 267
Table 106 Segment CRC Verification Packet 267
Table 107 Segement CRC Verification Response Packet 267
Table 108 Whole-image CRC Check Packet 268
Table 109 OTA Result Response Packet 268
Table 110 New-image Side Selection Packet 268
Table 111 New-image Side Selection Response Packet 268
Table 112 Image Overwriting Confirmation Packet 269
Table 113 Image Overwriting Confirmation Response Packet 269
Table 114 Set an OTA User 269
Table 115 Set an OTA Version (Response Packet) 269
Table 116 Get an OTA Version 270
Table 117Get an OTA Version (Response Packet) 270
Table 118 HFSR Bit Assigments 307
Table 119 MMFSR Bit Assignments 308
Table 120 BFSR Bit Assignments 311
Table 121 UFSR Bit Assignments 313

List of Figures

Figure 1 Building Command 16
Figure 2 Compilation Generated Files 17
Figure 3 BES Evaluation Board Set-up 18
Figure 4 Emulate Download Serial Ports 19
Figure 5 Enter User Account and Password 19
Figure 6 Image Downloading Configuration 20
Figure 7 Image Downloading 21
Figure 8 Image Downloading Completed 21
Figure 9 Image Bootup Log Printing in UART 22

CONFIDENTIAL

Figure 10 Log Printing of TWS Connection 22
Figure 11 Log Printing of Mobile Connection 23
Figure 12 Software Architecture 24
Figure 13 Main MCU Functions 25
Figure 14 SDK Folder Structure 25
Figure 15 SDK Data Flow 27
Figure 16 Logical Address Map 55
Figure 17 Flash Memory Map for OTA 56
Figure 18 UART Data Path 64
Figure 19 Core Bridge Communication Datapath between MCU and Sensor Hub Core 70
Figure 20 Log Printing of Sensor Hub Bootup 73
Figure 21 Log Printing of M55 Bootup 74
Figure 22 Sleep and Wake-up 78
Figure 23 Jump to Ram Run 80
Figure 24 Jump to Secondary Bootloader in Flash 81
Figure 25 Application Image Init 82
Figure 26 Sensor Core Flow 83
Figure 27 Async Flash Erasing/Programing 96
Figure 28 Sync Flash Erasing/Programing 97
Figure 29 Sync Flash Erasing/Programing Influence on DMA Chain 98
Figure 30 Location of Parameter Sector 102
Figure 31 Thread Scheduling and Interrupt Execution 108
Figure 32 Simple Event Communication 113
Figure 33 CMSIS-RTOS Semaphore 116
Figure 34 Thread Statistics Log Demo 119
Figure 35 Audio Path Flow of Phone Call 126
Figure 36 Audio Path Flow of Music Playback 127
Figure 37 Audio Info Sync of Music Playback 128
Figure 38 Convert Command 133
Figure 39 File Relocation 134
Figure 40 Summary Architecture 144
Figure 41 Software Architecture 145
Figure 42 BT Mac Address and Bt Name Configuration 147
Figure 43 Host Flow 170
Figure 44 Flow Path of Role Switch Preparation 171
Figure 45 Flow Path of Role Switch Post Operation 172
Figure 46 Role Switch Controller Operation 172
Figure 47 Configure BLE Mac Address and BLE Name 181
Figure 48 Connection Parameter Update Policy 218
Figure 49 BLE Connection Process 222
Figure 50 BLE Profile Functional Division 224
Figure 51 l2cap Connection 231
Figure 52 L2CAP Connection Datapath 237
Figure 53 Test Setup 241
Figure 54 Test Setup 243

CONFIDENTIAL

Figure 55 BLE Data Path 248
Figure 56 BT Data Path 249
Figure 57 Command 250
Figure 58 Throughput Test Procedure 252
Figure 59 Bestechnic for Testing Throughput 255
Figure 60 THROUGH PUT Interface 256
Figure 61 SW Image Power Off Flow 257
Figure 62 BES ThroughPut Main Interface 258
Figure 63 Choose Bluetooth 258
Figure 64 SPP Test 259
Figure 65 Main Flash Section 260
Figure 66 OTA Flow 261
Figure 67 Boot Area Selection 262
Figure 68 Upgrade Process 271
Figure 69 Burn with OTA BOOT 278
Figure 70 Main View 279
Figure 71 Pairing 279
Figure 72 Device Selection 280
Figure 73 Configure Upgrade 281
Figure 74 Device Connection 281
Figure 75 Image Selection 282
Figure 76 Start OTA Process 282
Figure 77 OTA Process Completed 283
Figure 78 Main View 283
Figure 79 Pairing and Connection 284
Figure 80 Device Selection 284
Figure 81 Configure Upgrade 285
Figure 82 Device Connection 286
Figure 83 Image Selection 286
Figure 84 Start OTA Process 287
Figure 85 Upgrade Completed 287
Figure 86 Main View 288
Figure 87 Pairing 289
Figure 88 Device Selection 289
Figure 89 Configure Upgrade 290
Figure 90 Device Connection 291
Figure 91 Image Selection 291
Figure 92 Start OTA Process 292
Figure 93 OTA Completed 292
Figure 94 Main View 293
Figure 95 Pairing and Connection 294
Figure 96 Device Selection 294
Figure 97 Configure Upgrade 295
Figure 98 Device Connection 296
Figure 99 Image Selection 296

CONFIDENTIAL

Figure 100 Start OTA Process 297
Figure 101 Upgrade Completed 297
Figure 102 TOTA Packet Format 298
Figure 103 Interaction Process of TOTA Test 299
Figure 104 Message Receiving Process 300
Figure 105 Message Sending Process 301
Figure 106 IOS App Operation Process 302
Figure 107 Android App Operation Process 303
Figure 108 App Console Log 303
Figure 109 Firmware Serial Port Log 304
Figure 110 SHCSR Bit Assignmonts 306
Figure 111 HardFault Status Register 306
Figure 112 MemManage Fault Status Register 308
Figure 113 BusFault Status Register 311
Figure 114 UsageFault Status Register 313
Figure 115 Mem Fault Dump 315
Figure 116 Instruction bus error Dump 316
Figure 117 Precise Data Bus Error Dump 317
Figure 118 Usage Fault Dump 318
Figure 119 Hard Fault Dump 318
Figure 120 Deadlock Causes Watchdog Crash 320
Figure 121 Stack Smashing 323
Figure 122 Install gcc-arm-none-eabi 324
Figure 123 Get Crash Location 324
Figure 124 power_consumption_thread Infomation 327
Figure 125 Thread Information 329
Figure 126 Audio Developer GUI 330
Figure 127 COM 331
Figure 128 Computer Management 331
Figure 129 COM3 Port 332
Figure 130 Save Data 333
Figure 131 Trace Manager 333
Figure 132 DUMP_PLC_ENABLE 334
Figure 133 Record 334
Figure 134 DUMP_PLC_ENABLE 334
Figure 135 BT_SCO_CHAIN_AUDIO_DUMP 335

CONFIDENTIAL

List of Sample Code

Sample Code 1: GPIO Wake-up 35
Sample Code 2: Use UART1 39
Sample Code 3: Use GPADC 48
Sample Code 4: Hardware Timer 49
Sample Code 5: Get Current Time 62
Sample Code 6: Get Current Battery Level 64
Sample Code 7: OS_Thread Creation 111
Sample Code 8 : OS Event 113
Sample Code 9: OS Timer 116
Sample Code 10: OS Semaphore 117
Sample Code 11: Audio Loopback Test 125
Sample Code 12: Music Jitter Buffer Length 132
Sample Code 13: Prompt Usage Test 138
Sample Code 14: Volume Level Configuration and Volume Control Test 144
Sample Code 15: Get and Write BT Address 147
Sample Code 16: app_ibrt_if_get_a2dp_state 148
Sample Code 17: app_ibrt_if_get_avrcp_state 149
Sample Code 18: app_ibrt_if_get_hfp_state 149
Sample Code 19: app_ibrt_if_get_hfp_call_status 149
Sample Code 20: app_ibrt_if_get_connected_mobile_count 149
Sample Code 21: app_ibrt_if_get_mobile_connected_dev_list 150
Sample Code 22: app_ibrt_if_is_tws_role_switch_on 150
Sample Code 23: app_ibrt_if_tws_role_switch_reques 150
Sample Code 24: app_ibrt_if_nvrecord_get_mobile_addr 150
Sample Code 25: app_ibrt_if_nvrecord_delete_all_mobile_record 150
Sample Code 26: app_ibrt_if_nvrecord_get_mobile_paired_dev_list 151
Sample Code 27: app_ibrt_if_is_tws_link_connected 151
Sample Code 28: app_ibrt_if_is_in_freeman_mode 151
Sample Code 29: app_ibrt_if_tws_disconnect_request 151
Sample Code 30: app_ibrt_if_mobile_disconnect_request 151
Sample Code 31: app_ibrt_if_is_tws_in_pairing_state 152
Sample Code 32: app_ibrt_if_is_left_side 152
Sample Code 33: app_ibrt_if_is_right_side 152
Sample Code 34: app_ibrt_if_unkown_side 152
Sample Code 35: app_ibrt_if_tws_disconnect_request 152
Sample Code 36: app_ibrt_if_tws_disconnect_request 152
Sample Code 37: app_ibrt_if_uirole2str 153
Sample Code 38: Simple Pairing Test 161
Sample Code 39: BES UI Usage 163
Sample Code 40: Sniff and Active Mode Management 175
Sample Code 41: get_ble_address 181
Sample Code 42: set_ble_address 182
Sample Code 43: get_ble_name 182

CONFIDENTIAL

Sample Code 44: set_ble_name 182
Sample Code 45: ble_adv_user_register 190
Sample Code 46: Enable and Disable advertising 194
Sample Code 47: ble_set_adv_type 195
Sample Code 48: set_adv_interval 197
Sample Code 49: ble_set_adv_txpwr_by_actv_user 200
Sample Code 50: ble_set_adv_txpwr_by_adv_user 200
Sample Code 51: ble_set_all_adv_txpwr 200
Sample Code 52: ble_set_adv_local_addr_by_actv_user 202
Sample Code 53: ble_set_adv_local_addr_by_adv_user 203
Sample Code 54: ble_user_adv_start 203
Sample Code 55: ble_custom_adv_start 207
Sample Code 56: ble_start_scan 209
Sample Code 57: ble_stop_scan 209
Sample Code 58: ble_start_connect 211
Sample Code 59: ble_start_connect_with_init_type 212
Sample Code 60: ble_is_connection_on_by_addr 213
Sample Code 61: ble_is_connection_on_by_conIdx 213
Sample Code 62: ble_cancel_connecting 214
Sample Code 63: ble_start_disconnect 214
Sample Code 64: ble_update_conn_param_mode 219
Sample Code 65: ble_get_conn_param 219
Sample Code 66: DTS Registration 231
Sample Code 67: dts_send_data 232
Sample Code 68: dts_disconnect 232
Sample Code 69: dts_connect 234
Sample Code 70: dts_send_data 235
Sample Code 71: dts_disconnect 235
Sample Code 72: Data Access Error 315
Sample Code 73: Instruction Bus Error 316
Sample Code 74: Precise Data Bus Error 317
Sample Code 75: Usage Fault 318
Sample Code 76: Hard Fault 318
Sample Code 77: Deadlock Causes Watchdog Crash 321
Sample Code 78: stack_overflow 323

CONFIDENTIAL

1. General Description

This document is intended for software developers who implement TWS product features based on BES chipsets. Basically, users can achieve the following objectives by using this manual:

- Learn how to use the BES SDK for quick and easy feature implementation.
- Make minimal effort in using the existing APIs and features.
- Focus on integrating their application layer features and raise requirements for new features that are not supported yet.

CONFIDENTIAL

2. Terms and Definitions

Term	Definition
SDK	Software Development Kit
BT	Bluetooth
BLE	Bluetooth Low Energy
OTA	Over-the-air Technology
TOTA	Test protocol over the air
SPP	Serial Port Profile

CONFIDENTIAL

3. Get Started with our SDK

3.1 Software Development Environment Setup

3.1.1 Set up Compilation Environment

The SDK uses the Arm GCC building system, and the recommended Arm GCC tool chain version is gcc-arm-none-eabi-10-2020-q4-major. The tool chain can be found and downloaded from the following website: https://developer.arm.com/tools-and-software/open-source-software/developer-tools/gnu-toolchain/gnu-rm/downloads#panel3a.

On Linux operating system, unzip the downloaded Arm GCC tool chain zip packet.
The default location to put the unzipped Arm gcc tool is */usr/local*, but other locations can be specified as needed.
The steps to add the tool chain into the linux environment are as follows:

1. Switch to the root account in order to edit */etc/profile*.

   sudo -i

2. Edit /etc/profile to add the path of tool chain as the bottom line.

   vi /etc/profile

e.g.: export PATH=$PATH:/usr/local/gcc-arm-none-eabi-10-2020-q4-major/bin

If users develop software on Windows, the recommended method is to install Ubuntu LTS. The steps to prepare the environment on Ubuntu LTS are the same as those on Linux. Building projects on Ubuntu LTS is much faster than on Windows. The steps of Ubuntu LTS installation can be found in the following link:

https://ubuntu.com/tutorials/ubuntu-on-windows#1-overview

3.1.2 Build the Project

Turn to the root path of the SDK, enter the following command:

```
make T=best1600_ibrt DEBUG=1 -j 2> log.txt
```

Figure 1 Building Command

- "T=" is to select the built target name. The target name can be one of the names in the configuration folders.
- "DEBUG=1" is to enable UART log printing.
- "-j" is to enable as many processors as the PC can to accelerate the compilation.
- "2> log.txt" is to output building logs, if many changes are done and the compilation cannot pass, collecting logs is useful for troubleshooting.
- After compilation, the following files are generated in the out/best1600_ibrt folder.

CONFIDENTIAL

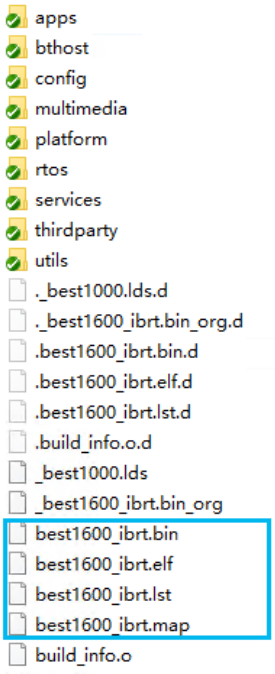


Figure 2 Compilation Generated Files

- ".lds" is the link script file generated from link script model file scripts\link\best1000.lds.S.
- ".bin" is the generated image file used for image programming.
- ".elf" is the symbol file.
- ".map" is the map file.

If users want to get the full disassemble files, use the following building command:

```
make T=best1600_ibrt DEBUG=1 -j all lst 2 > log.txt
```

3.1.3 Usage of Hardware Development Kit

The yellow Evaluation Board is compatible with various types of Bluetooth chip boards, which has differential LN/LP, RN/RP outputs and MIC inputs.

The log information can be downloaded and uploaded to the PC both through UART. The jumper cap should be changed when switching the power supply mode.

CONFIDENTIAL

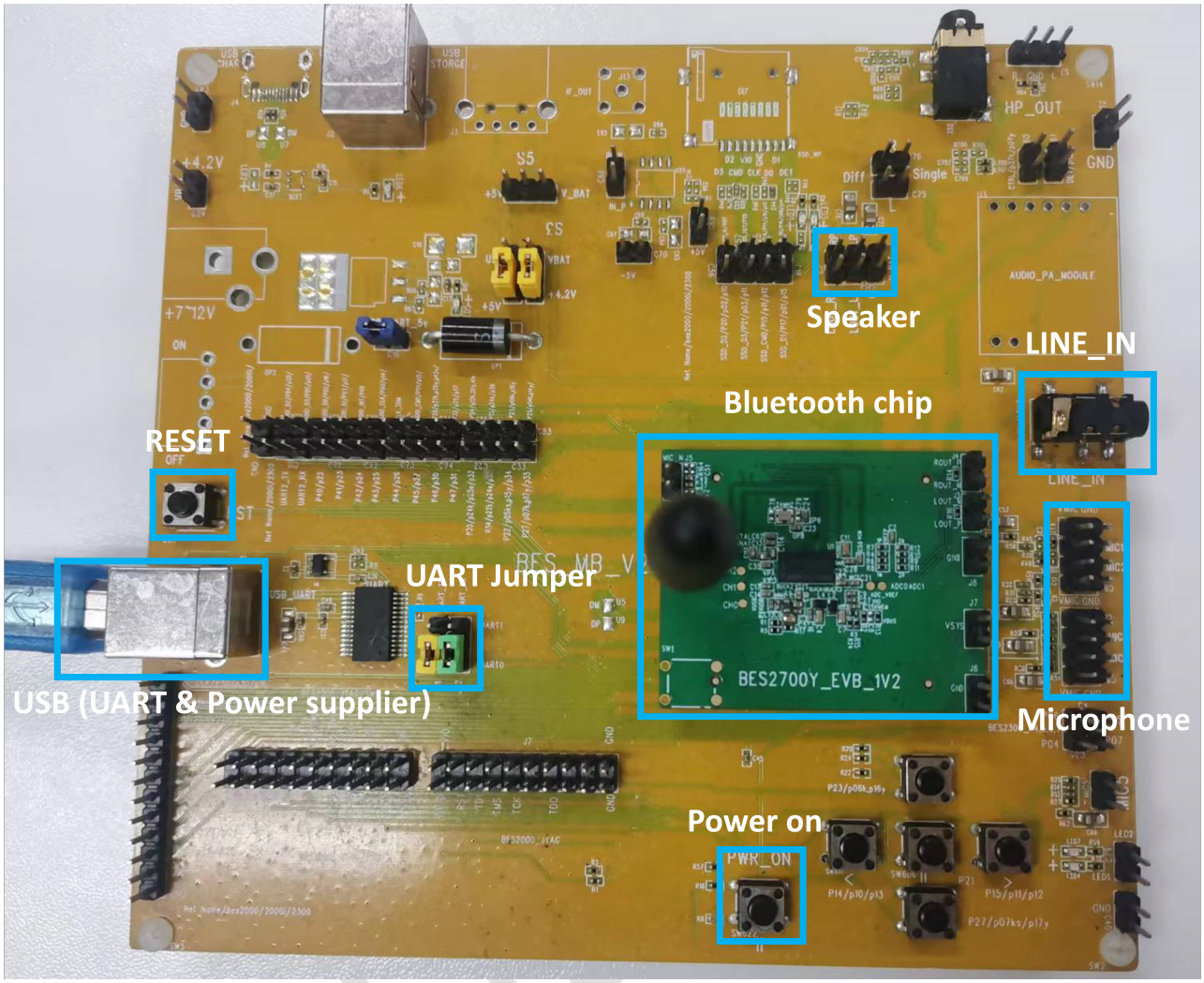


Figure 3 BES Evaluation Board Set-up

3.1.4 Image Programming

The Windows download GUI tool, **DldProductLine**, can be used for image programming.

The programming procedure is as follows:

- Set up the BES evaluation board as Figure 3 shown above in section 3.1.3.
- Connect the BES evaluation board to your computer.
- Open DldProductLine.
- Click the button pointed by arrow 1 to open the COM setting window.
- Click “list” pointed by arrow 2 to emulate all the serial ports, then click “OK” pointed by arrow 3. This step only needs to be done once.

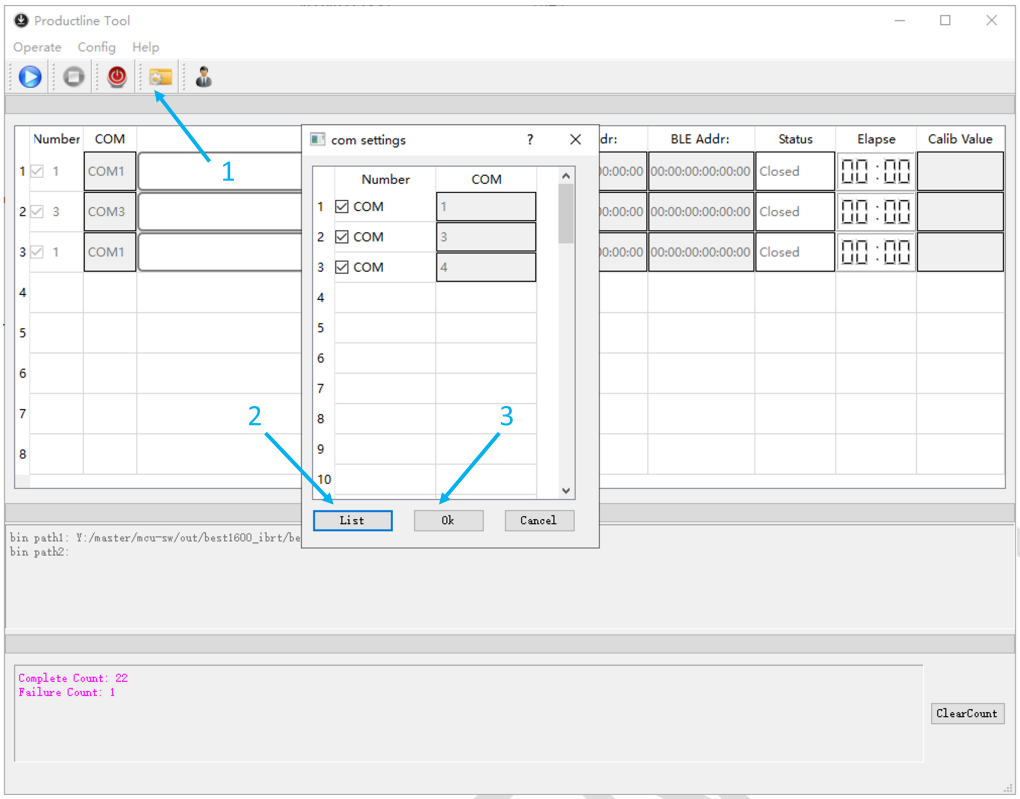


Figure 4 Emulate Download Serial Ports

- Click the button with a human figure on it, and enter “1” for both User and Pwd in the login dialog. It can also be configured <needlogin>0</needlogin> in the file “productline_cfg. XML” for a password-free login.

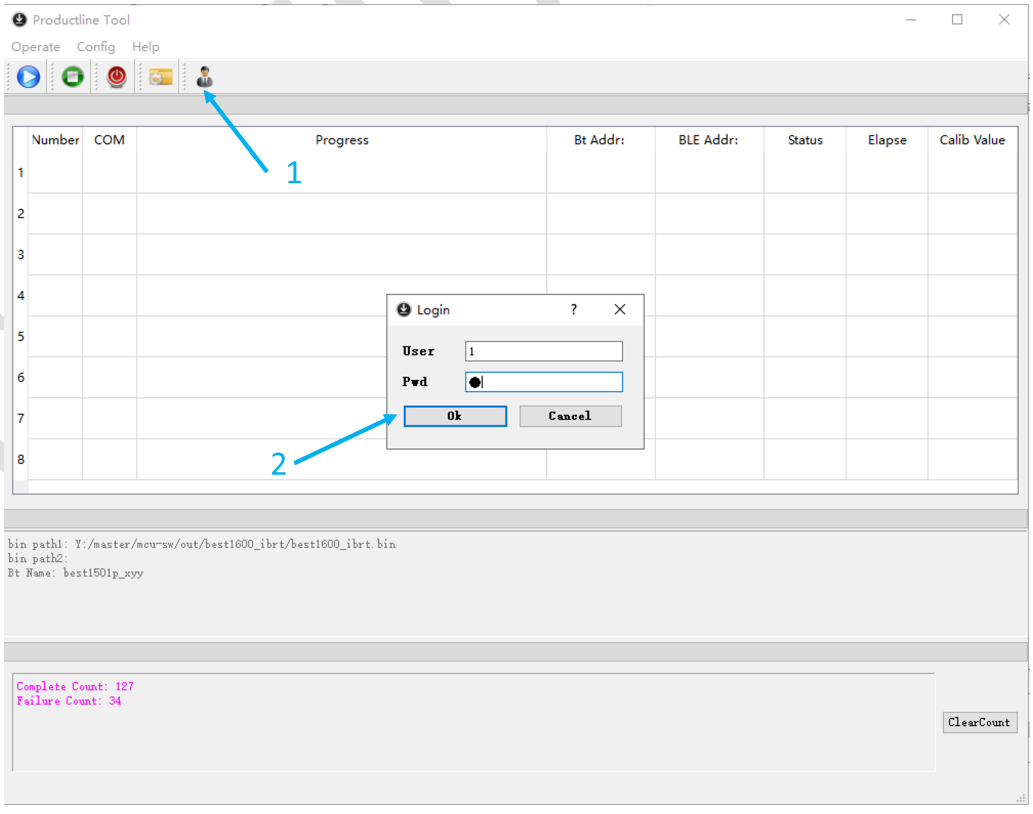


Figure 5 Enter User Account and Password

- Configure the following settings to choose the downloaded image and written non-volatile information. Note that the non-volatile information only needs to be done once. After that, you can uncheck “Erase the whole flash” and “Update Sector” to avoid unwanted programming.

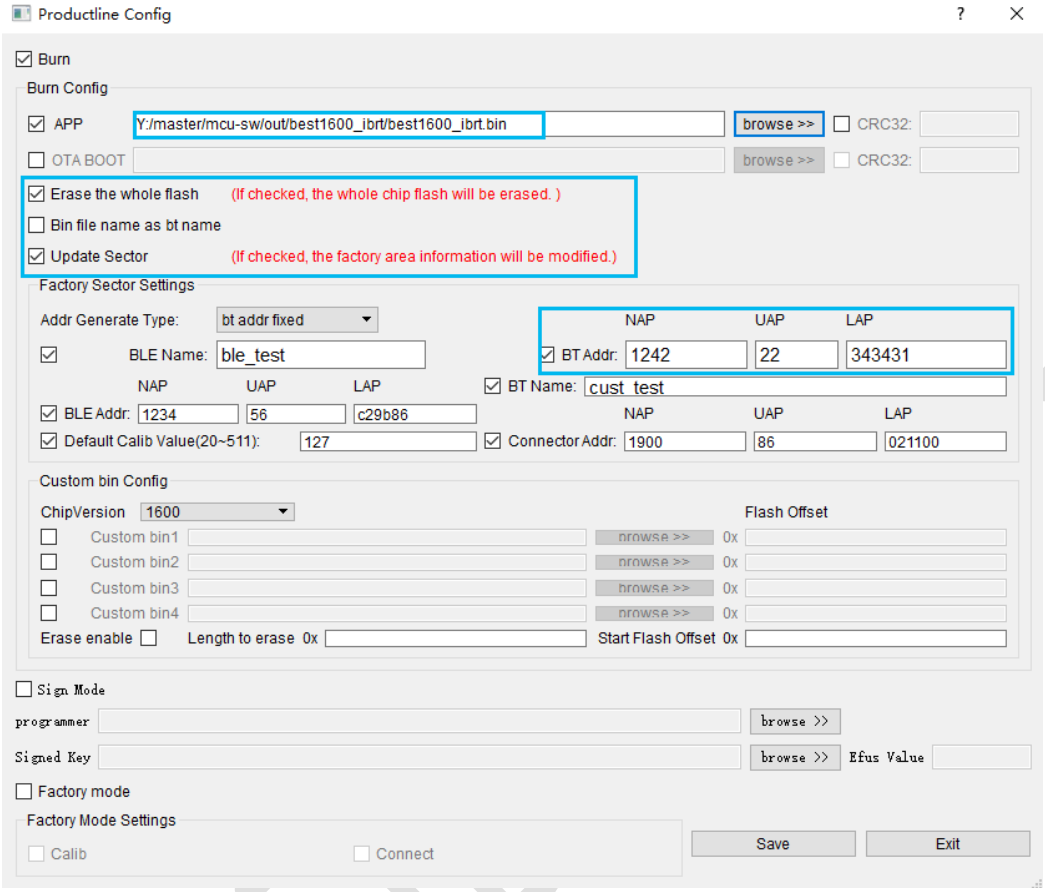


Figure 6 Image Downloading Configuration

Two test boards are needed if left and right earbuds are to be emulated. Please use different BT mac addresses for the left and right earbuds. Specifically, you should set the last bit of the BT addresses different. For example, if the last bit of the master BT address is set to 1, the last bit of the slave’s should be set to 0. Based on such BT mac addresses, default master or slave TWS roles can be assigned to two earbuds.

e.g. Master: 12 42 22 34 34 30; Slave: 12 42 22 34 34 31.

- After finishing the configuration, press “Save” button.
- Click the “start” button as shown in the figure below. Press the reset button on the EVB, and then press the Power-on button on the evaluation board (EVB) to start downloading.

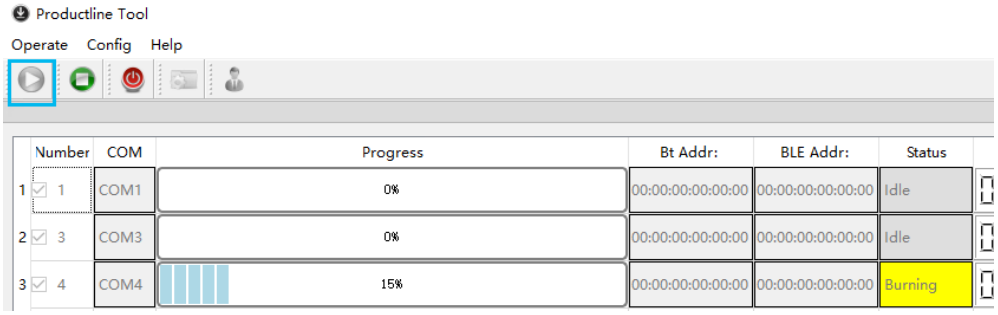


Figure 7 Image Downloading

- When the downloading is completed, click the stop button on the main view.

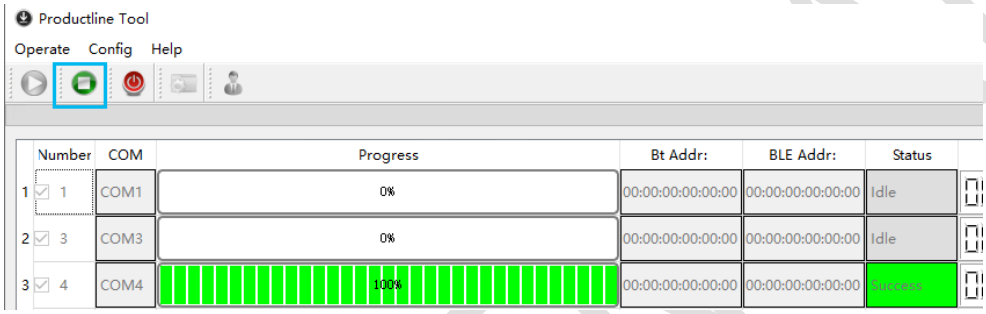


Figure 8 Image Downloading Completed

3.1.5 Implement Basic Functions

After downloading the image, TWS devices can be set to connect with each other automatically and able to be connected to mobile phones for basic operations such as playing music.

To enable the auto-pairing, these two macros should be set:

```
FREE_TWS_PAIRING_ENABLED=1 POWER_ON_ENTER_TWS_PAIRING_ENABLED=1
```

A serial monitor is needed to read the data transmitted by the evalutation board. Xshell is recommended. The default Baud is 1152000. After both of the COM ports are connected to the serial monitor, you can start pairing the two devices.

The connection process is as follows:

- Press RST button of two boards. Boot up log printing, as shown in Figure 9.

```
[17:34:58:457] CHIP=best1600
[17:34:58:457] CHIP_SUBSYS=bth
[17:34:58:457] KERNEL=RTX5
[17:34:58:457] NV_REC_DEV_VER=2
[17:34:58:457] __userdata_start=0x307FC000
[17:34:58:457] USER_SEC_SIZE=0x1000
[17:34:58:457] __aud_start=0x307FE000
[17:34:58:457] AUD_SEC_SIZE=0
[17:34:58:457] __factory_start=0x307FF000
[17:34:58:457] FACT_SEC_SIZE=0x1000
[17:34:58:457] FLASH_BASE=0x34000000
[17:34:58:457] FLASH_NC_BASE=0x30000000
[17:34:58:459] FLASH_SIZE=0x800000
[17:34:58:459] BUILD_DATE=Aug  8 2022 17:12:19
[17:34:58:459] REV_INFO=15c4aa3a2b-dirty:best1600_ibrt
[17:34:58:459]
```

Figure 9 Image Bootup Log Printing in UART

- Two boards pair with each other. A TWS connection is established, and the primary device enters into discoverable mode. The keyword for log printing in UART is “TWS-STATE”.

```
  8835/R-M/I/NONE  /  7 | TWS-STATE:DISCONNECTED EVENT=ENTRY
  8835/R-M/I/NONE  /  7 | TWS-STATE:DISCONNECTED EVENT=START
 10551/R-M/I/NONE  /  7 | TWS-STATE:DISCONNECTED EVENT=SIG_TWS_CONNECT
 10556/R-M/I/NONE  /  7 | TWS-STATE:DISCONNECTED EVENT=EXIT
 10557/R-M/I/NONE  /  7 | TWS-STATE:CONNECTING EVENT=ENTRY
 10558/R-M/I/NONE  /  7 | TWS-STATE:CONNECTING EVENT=START
 11463/R-M/I/NONE  /  7 | TWS-STATE:CONNECTING EVENT=EVT_TWS_CONNECTED
 11463/R-M/I/NONE  /  7 | TWS-STATE:CONNECTING EVENT=EXIT
 11464/R-M/I/NONE  /  7 | TWS-STATE:CONNECTED EVENT=ENTRY
 11467/R-M/I/NONE  /  7 | TWS-STATE:CONNECTED EVENT=START
 11467/R-M/I/NONE  /  7 | TWS-STATE:BESAUD-CONNECTING EVENT=ENTRY
 11471/R-M/I/NONE  /  7 | TWS-STATE:BESAUD-CONNECTING EVENT=START
 11503/R-M/I/NONE  /  7 | TWS-STATE:BESAUD-CONNECTING EVENT=EVT_TWS_BESAUD_CONNECTED
 11504/R-M/I/NONE  /  7 | TWS-STATE:BESAUD-CONNECTING EVENT=EXIT
 11504/R-M/I/NONE  /  7 | TWS-STATE:W4-SHAREINFO EVENT=ENTRY
 11504/R-M/I/NONE  /  7 | TWS-STATE:W4-SHAREINFO EVENT=START
 11628/R-M/I/NONE  /  7 | TWS-STATE:W4-SHAREINFO EVENT=EVT_SHARE_INFO_COMPLETED
 11629/R-M/I/NONE  /  7 | TWS-STATE:W4-SHAREINFO EVENT=EXIT
 11629/R-M/I/NONE  /  7 | TWS-STATE:CONNECTED EVENT=START
 11858/R-M/I/NONE  /  7 | TWS-STATE:CONNECTED EVENT=EVT_TWS_SIMPLE_PAIRING_COMPLETED
```

Figure 10 Log Printing of TWS Connection

- The primary device can be connected to the mobile phone to play music.

The name for the Bluetooth is set during the burning procedure. The name is defined in the BLE Name block of the burning setting page, as shown in Figure 6.

The keyword for log printing in UART is “MOBILE-STATE”.

```
  4852/R-M/I/NONE   /   7 | MOBILE-STATE 0 constructor addr:
  4853/R-M/I/NONE   /   7 | MOBILE-STATE 0:CONNECTING EVT=ENTRY
  4856/R-M/I/NONE   /   7 | MOBILE-STATE 0:CONNECTING EVT=START
  7952/R-M/I/NONE   /   7 | MOBILE-STATE 0:CONNECTING EVT=EVT_MOBILE_RAW_CONNECTED
  8508/R-M/I/NONE   /   7 | MOBILE-STATE 0:CONNECTING EVT=MOBILE_CONNECTED
  8508/R-M/I/NONE   /   7 | MOBILE-STATE 0:CONNECTING EVT=EXIT
  8508/R-M/I/NONE   /   7 | MOBILE-STATE 0:CONNECTED EVT=ENTRY
  8508/R-M/I/NONE   /   7 | MOBILE-STATE 0:CONNECTED EVT=START
  8508/R-M/I/NONE   /   7 | MOBILE-STATE 0:CONNECTED EVT=AUTH_COMPLETE
 12288/R-M/I/NONE   /   7 | MOBILE-STATE 0:CONNECTED EVT=IBRT_CONNECTED
 12288/R-M/I/NONE   /   7 | MOBILE-STATE 0:IBRT CONNECTED EVT=ENTRY
 12288/R-M/I/NONE   /   7 | MOBILE-STATE 0:IBRT CONNECTED EVT=START
```

Figure 11 Log Printing of Mobile Connection

CONFIDENTIAL

4. Software System Introduction

4.1 Software Architecture

The following figure is the functional description of each core in a multi-core system.

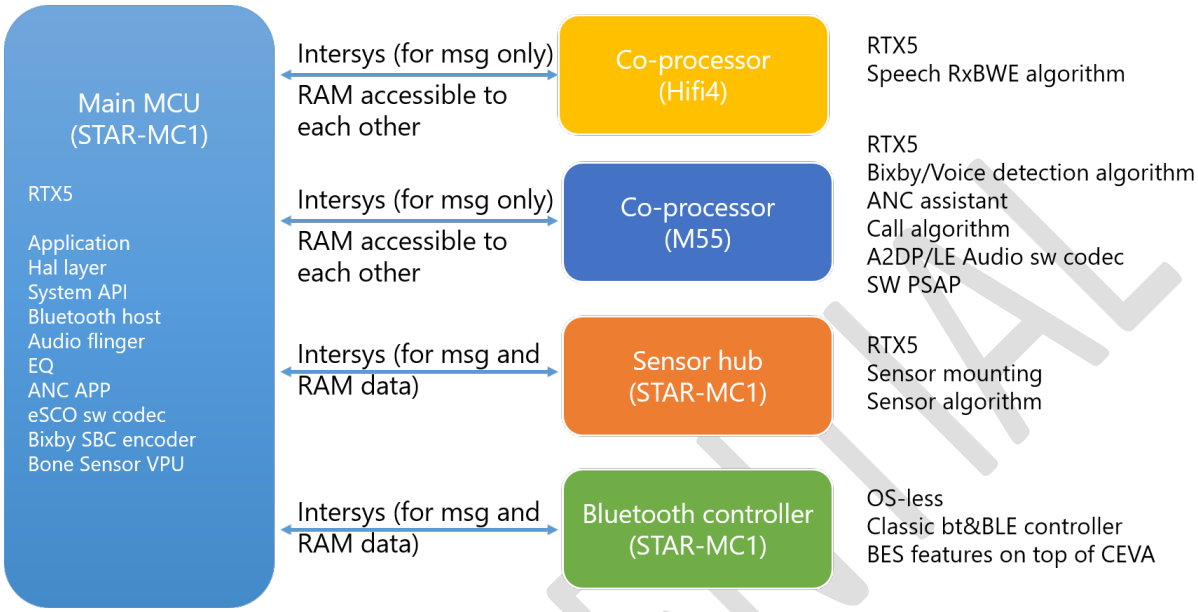


Figure 12 Software Architecture

The functions of the main MCU are introduced in Figure 13. The main MCU creates different threads based on the OS Abstract Layer, covering Audio, IBRT core, GAF, classic BT and application.

CONFIDENTIAL

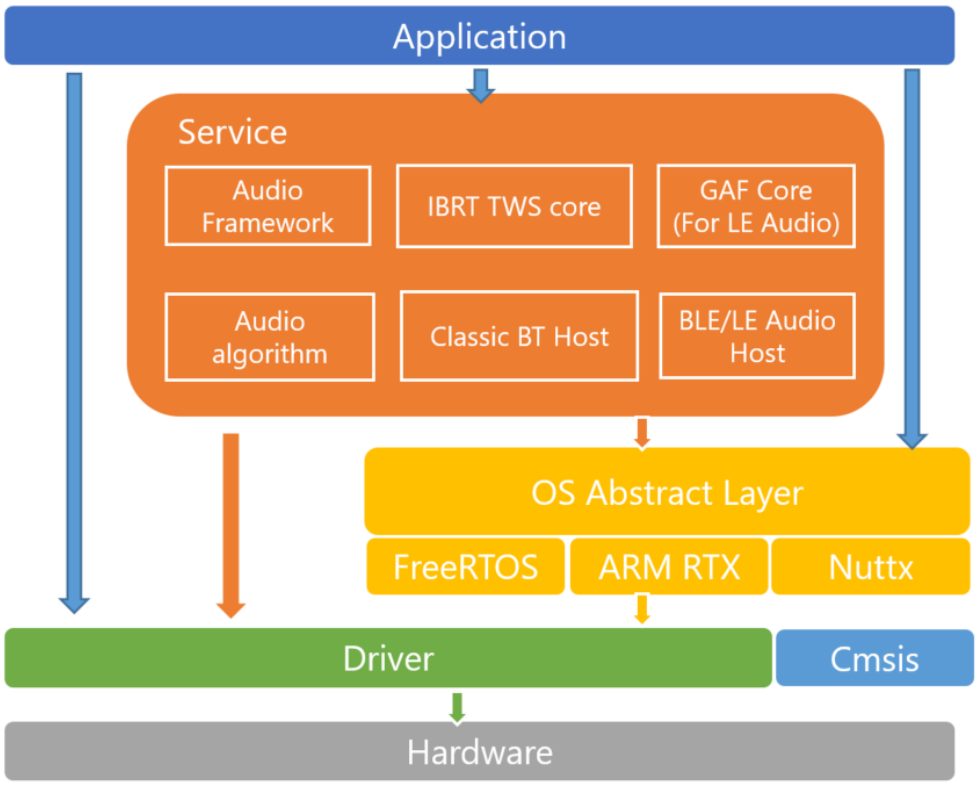
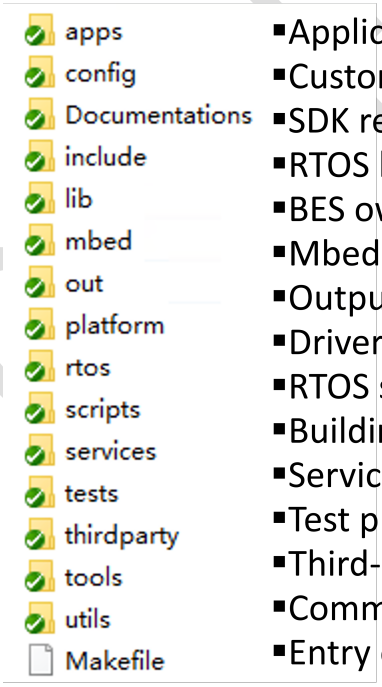


Figure 13 Main MCU Functions

4.2 SDK Folder Structure

The SDK folder contains the following directories:



- Application implementations
- Customized configurations for different projects
- SDK related basic documentations
- RTOS header files
- BES owned libraries
- Mbed related configuration files
- Output folder after buidling
- Driver codes and CMSIS codes
- RTOS source files
- Building framework scripts
- Service codes
- Test programs
- Third-party libraries
- Common utility codes
- Entry of the Makefile

Figure 14 SDK Folder Structure

- “apps” mainly stores upper-layer application-related software.
- “config” indicates the basic differentiated configuration.
- “Documentations” contains SDK related basic documents.
- “include” mainly contains RTOS header files.
- “lib” indicates BES owned libraries.
- “mbed” indicates mbed configuration files.
- “out” stores the compiled output file, select the “.bin” file when programming.
- “platform” mainly contains Driver codes and CMSIS codes.
- “rtos” indicates RTOS source files.
- “scripts” contains building framework scripts.
- “services” indicates service codes.
- “tests” contains test programs.
- “thirdparty” indicates third-party libraries.
- “utils” contains common utility codes.
- “Makefile” indicates the entry of the Makefile.

4.3 Introduction of Main MCU-dedicated Threads

Table 1 Thread Introduction

Task name	Description
app_audio_thread	Manages different types of audio streams.
app_thread	Mainly used for the processing messages from the application modules.
af_thread	Processes the audio flinger dma irq events, covering internal codec, i2s, tdm, spdif, bt pcm functionalities. This is audio thread where the audio algorithms are running.
BesbtThread	Used for running all of the classic BT and BLE host layers.
main_thread	The main thread of the SDK.
a2dp_decoder_cc_processor_thread	Communicates with M55 for A2DP music decoding.
app_custom_ux_thread	Distributes the TWS UI owned bluetooth events to the custom project for implementing the custom UI.
app_dsp_m55_bridge_rx_thread	Processes the core bridge message between main MCU and m55.
app_prompt_handler_thread	Processes the pending prompt playing requests in the queue one by one.
app_core_bridge_rx_thread	Processes the core bridge message between main MCU and sensor hub.

The following figure shows the data flow for the SDK.

CONFIDENTIAL

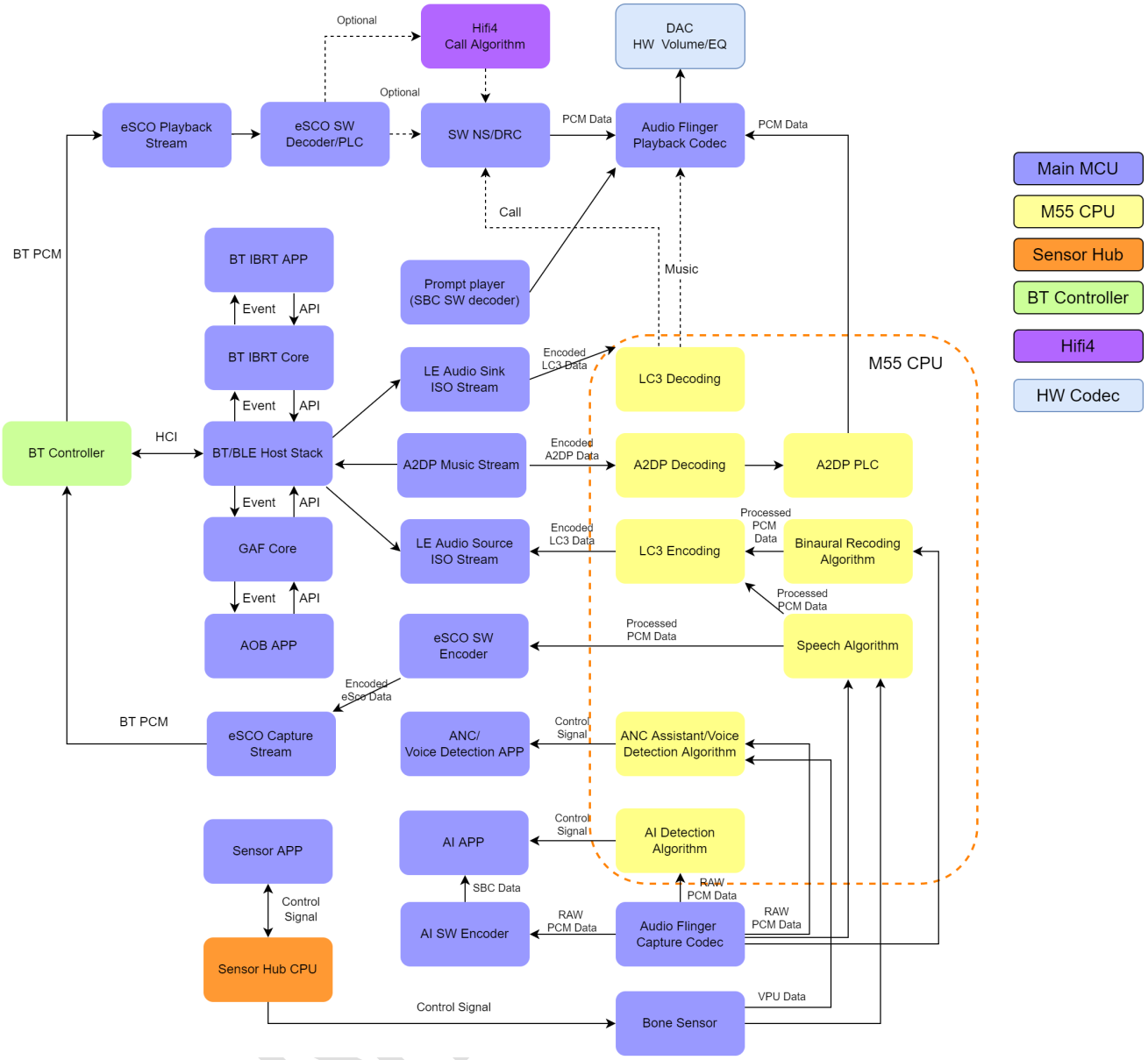


Figure 15 SDK Data Flow

5. Software Development Guide

5.1 Target Configuration

5.1.1 Common.mk

The common.mk file is where the general makefile options are included, such as compiler options, chip selection, file path, etc. The file is located in /config directory.

5.1.2 Target.mk

The target.mk files consist of build configurations for specific build targets. They are located in /config/<target name> directory.

5.1.3 Link Script

Linker scripts tell the linker which sections to include in the output file, as well as which order to put them in, what type of file is to be produced, and what is to be the address of the first instruction. Those files are located in /scripts directory.

5.1.4 Target Hardware Configuration

The configuration of the target hardware is implemented in the “tgt_hardware.c” and “tgt_hardware.h” files in the “config” directory.

The hardware configuration is as follows:

- Output channel configuration

The following playback channel configuration in “tgt_hardware.h” is used by default if the audio flinger's channel map is not configured when starting a playback stream.

```
#ifdef BLE_AUDIO_STEREO_CHANNEL_OVER_ONE_CIS
#define CFG_HW_AUD_OUTPUT_PATH_SPEAKER_DEV (AUD_CHANNEL_MAP_CH0 |
AUD_CHANNEL_MAP_CH1)
#else
#define CFG_HW_AUD_OUTPUT_PATH_SPEAKER_DEV (AUD_CHANNEL_MAP_CH0)
#endif
```

- Battery volume level

The battery voltage settings for full voltage, low voltage, and shutdown voltage are stored in “tgt_hardware.h”.

```
#define APP_BATTERY_MIN_MV (3200)
#define APP_BATTERY_PD_MV   (3100)
#define APP_BATTERY_MAX_MV (4200)
```

CONFIDENTIAL

- BT Configuration

The following BT and BLE's name and mac addresses are used by default when there is no valid information in the factory section.

```
const char *BT_LOCAL_NAME = TO_STRING(BT_DEV_NAME) "\0";
const char *BLE_DEFAULT_NAME = "BES_BLE";
uint8_t ble_global_addr[6] = {
#ifdef BLE_DEV_ADDR
    BLE_DEV_ADDR
#else
    0xBE,0x99,0x34,0x45,0x56,0x67
#endif
};
uint8_t bt_global_addr[6] = {
#ifdef BT_DEV_ADDR
    BT_DEV_ADDR
#else
    0x1e,0x57,0x34,0x45,0x56,0x67
#endif
};
```

- rfcomm uuid configuration of the TOTA service

Configure the rfcomm uuid if you want to use BES TOTA spec with different UUID values.

```
uint8_t tota_rfcomm_custom_uuid[16] =
{ 0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x88, 0x88,
0x88, 0x88, 0x88, 0x88 };
```

- Volume level configuration

Configure the playback codec volume level.

```
const struct CODEC_DAC_VOL_T codec_dac_vol[TGT_VOLUME_LEVEL_QTY] = {
    {TX_PA_GAIN,0x03,-99},
    {TX_PA_GAIN,0x03,-45},
    {TX_PA_GAIN,0x03,-42},
    {TX_PA_GAIN,0x03,-39},
    {TX_PA_GAIN,0x03,-36},
    {TX_PA_GAIN,0x03,-33},
    {TX_PA_GAIN,0x03,-30},
    {TX_PA_GAIN,0x03,-27},
```

```
    {TX_PA_GAIN,0x03,-24},
    {TX_PA_GAIN,0x03,-21},
    {TX_PA_GAIN,0x03,-18},
    {TX_PA_GAIN,0x03,-15},
    {TX_PA_GAIN,0x03,-12},
    {TX_PA_GAIN,0x03, -9},
    {TX_PA_GAIN,0x03, -6},
    {TX_PA_GAIN,0x03, -3},
    {TX_PA_GAIN,0x03,  0},  //0dBm
};
```

- Audio stream configuration

Each entry in this configuration array is used by a specific capture user scenario.

“cfg_audio_input_path_cfg.io_path” shall be filled into the audio flinger element “stream_cfg.io_path” when you set up and start an audio stream.

For example: stream_cfg.io_path = AUD_INPUT_PATH_MAINMIC.

```
const struct AUD_IO_PATH_CFG_T
cfg_audio_input_path_cfg[CFG_HW_AUD_INPUT_PATH_NUM] = {
#if defined(SPEECH_TX_AEC_CODEC_REF)
    // NOTE: If enable Ch5 and CH6, need to add channel_num when setup
audioflinger stream
    { AUD_INPUT_PATH_MAINMIC, CFG_HW_AUD_INPUT_PATH_MAINMIC_DEV |
AUD_CHANNEL_MAP_ECMIC_CH0, },
#else
    { AUD_INPUT_PATH_MAINMIC, CFG_HW_AUD_INPUT_PATH_MAINMIC_DEV, },
#endif
    { AUD_INPUT_PATH_LINEIN,  CFG_HW_AUD_INPUT_PATH_LINEIN_DEV, },
#ifdef VOICE_DETECTOR_SENS_EN
    { AUD_INPUT_PATH_VADMIC,  CFG_HW_AUD_INPUT_PATH_VADMIC_DEV, },
#else
    { AUD_INPUT_PATH_ASRMIC,  CFG_HW_AUD_INPUT_PATH_ASRMIC_DEV, },
#endif
    { AUD_INPUT_PATH_ANC_ASSIST,
CFG_HW_AUD_INPUT_PATH_ANC_ASSIST_DEV, },
#if defined(SPEECH_TX_AEC_CODEC_REF)
    { AUD_INPUT_PATH_HEARING,   CFG_HW_AUD_INPUT_PATH_HEARING_DEV |
AUD_CHANNEL_MAP_ECMIC_CH0, },
#else
```

```
    { AUD_INPUT_PATH_HEARING,    CFG_HW_AUD_INPUT_PATH_HEARING_DEV, },
#endif
};
```

- Volume level configuration

The capture channel configuration is used by default if the audio flinger's channel map is not configured when you start a capture stream.

```
struct AF_STREAM_CONFIG_T {
    uint32_t ext_mclk_freq;
    enum AUD_SAMPRATE_T sample_rate;
    enum AUD_CHANNEL_MAP_T channel_map;
    enum AUD_CHANNEL_NUM_T channel_num;
    enum AUD_BITS_T bits;
    enum AUD_STREAM_USE_DEVICE_T device;
    enum AUD_IO_PATH_T io_path;
    enum AUD_DATA_ALIGN_T align;
    enum AUD_FS_FIRST_EDGE_T fs_edge;
    uint16_t fs_cycles;
    uint8_t slot_cycles;
    bool chan_sep_buf;
    bool sync_start;
    //should define type
    uint8_t vol;

    AF_STREAM_HANDLER_T handler;

    uint8_t *data_ptr;
    uint32_t data_size;
};
```

For “CFG_HW_AUD_INPUT_PATH_MAINMIC_DEV (AUD_CHANNEL_MAP_CH0 | VMIC_MAP_CFG)” parameters, “AUD_CHANNEL_MAP_CH0” represents the ADC channel index, and “VMIC_MAP_CFG” represents the Microphone power supplier VMIC.

```
#if SPEECH_CODEC_CAPTURE_CHANNEL_NUM == 2
#define CFG_HW_AUD_INPUT_PATH_MAINMIC_DEV    (AUD_CHANNEL_MAP_CH0 |
AUD_CHANNEL_MAP_CH1 | VMIC_MAP_CFG)
```

```
#elif SPEECH_CODEC_CAPTURE_CHANNEL_NUM == 3
#define CFG_HW_AUD_INPUT_PATH_MAINMIC_DEV   (AUD_CHANNEL_MAP_CH0 |
AUD_CHANNEL_MAP_CH1 | AUD_CHANNEL_MAP_CH4 | VMIC_MAP_CFG)
#elif SPEECH_CODEC_CAPTURE_CHANNEL_NUM == 4
#define CFG_HW_AUD_INPUT_PATH_MAINMIC_DEV   (AUD_CHANNEL_MAP_CH0 |
AUD_CHANNEL_MAP_CH1 | AUD_CHANNEL_MAP_CH2| AUD_CHANNEL_MAP_CH3 |
AUD_VMIC_MAP_VMIC1)
#else
#define CFG_HW_AUD_INPUT_PATH_MAINMIC_DEV   (AUD_CHANNEL_MAP_CH0 |
VMIC_MAP_CFG)
#endif
```

5.2 Hardware Abstract Layer Drives

5.2.1 Use GPIO

GPIOs can be properly configured to achieve the dedicated functions, wake up the system and serve as a power supply.

5.2.1.1 GPIO Configuration

The following table describes the configuration items.

- IOMUX configuration

Table 2 GPIO IOMUX Coniguration

Item	Description	Notes
enum HAL_IOMUX_PIN_T	GPIO index	Depends on chipset models, normally HAL_IOMUX_PIN_Px_x or HAL_IOMUX_PIN_LEDx (Breathing LED).
enum HAL_IOMUX_FUNCTION_T	GPIO functionality	Every GPIO can be assigned to several fixed functionalities. All of the GPIOs can be configured as a general GPIO. HAL_IOMUX_FUNC_GPIO.

enum HAL_IOMUX_PIN_VOLTAGE_DOMAINS_T	Voltage domain	enum HAL_IOMUX_PIN_VOLTAGE_DOMAINS_T {
		HAL_IOMUX_PIN_VOLTAGE_VIO = 0, // 1.8v voltage level
		HAL_IOMUX_PIN_VOLTAGE_MEM, // 3.3v voltage level
		// For PMU LED pins only
		HAL_IOMUX_PIN_VOLTAGE_VBAT,
		};

enum HAL_IOMUX_PIN_PULL_SELECT_T	Default pull state	enum HAL_IOMUX_PIN_PULL_SELECT_T { HAL_IOMUX_PIN_NOPULL = 0, HAL_IOMUX_PIN_PULLUP_ENABLE, HAL_IOMUX_PIN_PULLDOWN_ENABLE,

CONFIDENTIAL

		};
		struct HAL_IOMUX_PIN_FUNCTION_MAP {
		enum HAL_IOMUX_PIN_T pin;
		enum HAL_IOMUX_FUNCTION_T function;
		enum HAL_IOMUX_PIN_VOLTAGE_DOMAINS_T volt;
		enum HAL_IOMUX_PIN_PULL_SELECT_T pull_sel;
		};

struct HAL_IOMUX_PIN_FUNCTION_MAP	IOMUX functions	

Use the following sample code to configure GPIO_3_2 as a general GPIO in the 1.8v voltage domain, no pull state.

```
#define SAMPLE_CODE_TEST_PIN (HAL_IOMUX_PIN_P3_2)
struct HAL_IOMUX_PIN_FUNCTION_MAP test_pin[] =
{
    {
    .pin = SAMPLE_CODE_TEST_PIN,
    .function = HAL_IOMUX_FUNC_AS_GPIO,
    .volt = HAL_IOMUX_PIN_VOLTAGE_VIO,
    .pull_sel = HAL_IOMUX_PIN_NOPULL,
    },
};
hal_iomux_init(&test_pin[0],
sizeof(test_pin)/sizeof(HAL_IOMUX_PIN_FUNCTION_MAP));
```

- GPIO direction configuration

A GPIO can be configured as an output or an input. You can use the following sample code to configure the GPIO as an output port with initial output high. When the direction is input, the third parameter of hal_gpio_pin_set_dir is ignored.

```
enum HAL_GPIO_DIR_T {
    HAL_GPIO_DIR_IN = 0,
    HAL_GPIO_DIR_OUT = 1,
};

hal_gpio_pin_set_dir((enum HAL_GPIO_PIN_T)SAMPLE_CODE_TEST_PIN,
HAL_GPIO_DIR_OUT, 1);
```

5.2.1.2 GPIO Interrupt

When a GPIO is configured as a general GPIO in the input direction, it can be used for GPIO interrupt to wake up the system. This feature can be used for sensor data ready to get notification, or waking up device from deep sleep mode, etc.

The following properties of the GPIO interrupt can be configured:

- A specific GPIO that can trigger the interrupt. Multiple GPIOs can be monitored at the same time.
- Whether to enable the debounce of the GPIO state change (We recommend that you enable the debounce).
- GPIO trigger methods: edge trigger or voltage level trigger.
- Event types to trigger the interrupt, high to low or low to high.
- The call-back function to call when there is a GPIO state change.

Steps to configure a GPIO:

1. Initialize the GPIO's IOMUX as a general GPIO.

```
    #define SAMPLE_CODE_TEST_PIN (HAL_IOMUX_PIN_P0_5)
    struct HAL_IOMUX_PIN_FUNCTION_MAP test_pin[] =
    {
        {
        .pin = SAMPLE_CODE_TEST_PIN,
        .function = HAL_IOMUX_FUNC_AS_GPIO,
        .volt = HAL_IOMUX_PIN_VOLTAGE_VIO,
        .pull_sel = HAL_IOMUX_PIN_NOPULL,
        },
    };

    hal_iomux_init(&test_pin[0],
sizeof(test_pin)/sizeof(HAL_IOMUX_PIN_FUNCTION_MAP));
```

2. Set up GPIO as an input

```
hal_gpio_pin_set_dir((enum HAL_GPIO_PIN_T)SAMPLE_CODE_TEST_PIN,
HAL_GPIO_DIR_IN, 0);
```

3. Configure GPIO wake-up irq. In this example, the debounce is enabled, edge trigger method is used, and low to high state change makes the GPIO interrupt happen and the callback function app_sample_code_test_gpio_irq_handler called.

```
    const struct HAL_GPIO_IRQ_CFG_T test_gpio_cfg = {
        .irq_enable = true,
        .irq_debounce = true,
```

```
        .irq_type = HAL_GPIO_IRQ_TYPE_EDGE_SENSITIVE,
        .irq_polarity = HAL_GPIO_IRQ_POLARITY_HIGH_RISING,
        .irq_handler = app_sample_code_test_gpio_irq_handler,
    };

    hal_gpio_setup_irq((enum HAL_GPIO_PIN_T)SAMPLE_CODE_TEST_PIN,
&test_gpio_cfg);
```

4. Disable GPIO wake-up irq

```
    const struct HAL_GPIO_IRQ_CFG_T disable_test_gpio_cfg = {
        .irq_enable = false,
        .irq_debounce = false,
        .irq_polarity = HAL_GPIO_IRQ_POLARITY_HIGH_RISING,
        .irq_handler = NULL,
        .irq_type = HAL_GPIO_IRQ_TYPE_EDGE_SENSITIVE,
    };

   hal_gpio_setup_irq((enum HAL_GPIO_PIN_T)SAMPLE_CODE_TEST_PIN,
&disable_test_gpio_cfg);
```

Sample Code 1: GPIO Wake-up

The demo code is implemented in /apps/tests directory. The second string of the following line is the function name for a specific demo. Tester can find actual implementation by looking into related cpp files under that directory.

{"gpio_wakeup_demo", gpio_wakeup_test},

The following line is the command to start the demo. Users need to set this macro: `APP_TRACE_RX_ENABLE=1` to enable related log printing for demo code.

[TEST, gpio_wakeup_demo]

5.2.1.3 GPIO Power Supply

Use the following sample code to configure GPIO_3_2 as a general GPIO in the 1.8v voltage domain, pull up state.

```
 #define SAMPLE_CODE_TEST_PIN (HAL_IOMUX_PIN_P3_2)
struct HAL_IOMUX_PIN_FUNCTION_MAP test_pin[] =
 {
     {
     .pin = SAMPLE_CODE_TEST_PIN,
     .function = HAL_IOMUX_FUNC_AS_GPIO,
     .volt = HAL_IOMUX_PIN_VOLTAGE_VIO,
```

```
        .pull_sel = HAL_IOMUX_PIN_PULLUP_ENABLE,
        },
    };
    hal_iomux_init(&test_pin[0],
    sizeof(test_pin)/sizeof(HAL_IOMUX_PIN_FUNCTION_MAP));
```

5.2.2 Use UART

UART can be used for data interaction between the main board and peripherals.

5.2.2.1 UART Configuration

The following table describes the configuration items.

- UART configuration

Table 3 UART Configuration

Item	Description	Notes

Struct HAL_UART_CFG_T	Configurations	struct HAL_UART_CFG_T {
		enum HAL_UART_PARITY_T parity;
		enum HAL_UART_STOP_BITS_T stop;
		enum HAL_UART_DATA_BITS_T data;
		enum HAL_UART_FLOW_CONTROL_T flow;
		enum HAL_UART_FIFO_LEVEL_T rx_level;
		enum HAL_UART_FIFO_LEVEL_T tx_level;
		uint32_t baud;
		bool dma_rx : 1;
		bool dma_tx : 1;
		bool dma_rx_stop_on_err : 1;
		};

enum HAL_UART_ID_T		enum HAL_UART_ID_T { #ifdef UART0_BASE HAL_UART_ID_0, #endif #ifdef UART1_BASE HAL_UART_ID_1, #endif #ifdef UART2_BASE HAL_UART_ID_2, #endif #ifdef UART3_BASE

CONFIDENTIAL

		HAL_UART_ID_3, #endif #ifdef BT_UART HAL_UART_ID_BT, #endif HAL_UART_ID_QTY };

union
HAL_UART_IRQ_T
	Interrupt of
	UART
		union HAL_UART_IRQ_T {
		struct {
		uint32_t RIM :1; // ri
		uint32_t CTSM :1; // cts
		uint32_t DCDM :1; // dcd
		uint32_t DSRM :1; // dsr
		uint32_t RX :1; // rx
		uint32_t TX :1; // tx
		uint32_t RT :1; // receive timeout
		uint32_t FE :1; // framing error
		uint32_t PE :1; // parity error
		uint32_t BE :1; // break error
		uint32_t OE :1; // overrun
		};
		uint32_t reg;
		};

The common configuration of UART1 is as follows.

```
struct HAL_UART_CFG_T uart_dma_cfg = {
    .parity              = HAL_UART_PARITY_NONE,
    .stop                = HAL_UART_STOP_BITS_1,
    .data                = HAL_UART_DATA_BITS_8,
    .flow                = HAL_UART_FLOW_CONTROL_NONE,
    .rx_level            = HAL_UART_FIFO_LEVEL_1_4,
    .tx_level            = HAL_UART_FIFO_LEVEL_1_2,
    .baud                = UART_BAUD_DEFAULT,
    .dma_rx              = true,
    .dma_tx              = true,
    .dma_rx_stop_on_err = false,
};
ret = hal_uart_open(HAL_UART_ID_1, &uart_dma_cfg);
```

CONFIDENTIAL

5.2.2.2 Data Transceiver

In the following example, the board sends the string 10 times via UART1, and then loses UART.

Steps to use UART:

1. Set IOMUX of UART

```
hal_iomux_set_uart1();
```

2. Open UART. The parameters are as follows

```
struct HAL_UART_CFG_T uart_dma_cfg = {
    .parity             = HAL_UART_PARITY_NONE,
    .stop               = HAL_UART_STOP_BITS_1,
    .data               = HAL_UART_DATA_BITS_8,
    .flow               = HAL_UART_FLOW_CONTROL_NONE,
    .rx_level           = HAL_UART_FIFO_LEVEL_1_4,
    .tx_level           = HAL_UART_FIFO_LEVEL_1_2,
    .baud               = UART_BAUD_DEFAULT,
    .dma_rx             = true,
    .dma_tx             = true,
    .dma_rx_stop_on_err = false,
};
ret = hal_uart_open(HAL_UART_ID_1, &uart_dma_cfg);
```

3. Configure the DMA interrupt handler. The interrupt handlers can be customized. The following code serves as an example.

```
hal_uart_irq_set_dma_handler(HAL_UART_ID_1, test_uart_dma_rx_irq_handler,
test_uart_dma_tx_irq_handler);

static void test_uart_dma_tx_irq_handler(uint32_t xfer_size, int
dma_error)
{
    if (dma_error) {
    }

    g_tx_dma_done = true;
}

static void test_uart_dma_rx_irq_handler(uint32_t xfer_size, int
dma_error, union HAL_UART_IRQ_T status)
```

```
{
    if (dma_error) {
    } else if (status.BE || status.FE || status.OE || status.PE) {
    } else {
        /* user application after received the data. */
        if (g_tx_dma_done) {
            g_tx_dma_done = 0;
            hal_uart_dma_send(HAL_UART_ID_1, g_test_rx_buf, xfer_size,
NULL, NULL);
        }
        {
            union HAL_UART_IRQ_T mask;
            mask.reg = 0;
            mask.BE = 1;
            mask.FE = 1;
            mask.OE = 1;
            mask.PE = 0;
            mask.RT = 1;
            hal_uart_dma_recv_mask(HAL_UART_ID_1, g_test_rx_buf,
RX_BUF_MAX, NULL, NULL, &mask);
        }
    }
}
```

4. Send data via DMA.

```
hal_uart_dma_send(HAL_UART_ID_1, (uint8_t *)UART_TX_STRING_1,
sizeof(UART_TX_STRING_1), NULL, NULL);
```

5. Stop the receiver after finishing using it.

```
hal_uart_stop_dma_recv(HAL_UART_ID_1);
```

Sample Code 2: Use UART1

{"uart1_transmit_demo", uart1_demo_test},

[TEST, uart1_transmit_demo]

5.2.3 Use I2C

I2C (inter-integrated circuit) serves as an interface between the microcontroller and the serial I2C bus.

5.2.3.1 I2C Configuration

The following table describes the configuration items.

- I2C configuration

Table 4 I2C Configuration

Item	Description	Notes

struct HAL_I2C_CONFIG_T	Configuration	struct HAL_I2C_CONFIG_T {
		enum HAL_I2C_API_MODE_T mode;
		uint8_t use_sync : 1;
		uint8_t use_dma : 1;
		uint8_t as_master : 1;
		uint16_t rising_time_ns;
		uint32_t speed;
		uint32_t addr_as_slave;
		};

enum HAL_I2C_ID_T	ID of I2c	enum HAL_I2C_ID_T {
		HAL_I2C_ID_0 = 0,
		#ifdef I2C1_BASE
		HAL_I2C_ID_1,
		#endif
		#ifdef I2C2_BASE
		HAL_I2C_ID_2,
		#endif
		#ifdef I2C3_BASE
		HAL_I2C_ID_3,
		#endif
		#ifdef I2C4_BASE
		HAL_I2C_ID_4,
		#endif
		#ifdef I2C5_BASE
		HAL_I2C_ID_5,
		#endif
		HAL_I2C_ID_NUM,
		};

enum HAL_I2C_ACTION_AFTER_WRITE_T	Action after write	enum HAL_I2C_ACTION_AFTER_WRITE_T {
		HAL_I2C_STOP_AFTER_WRITE = 0,
		HAL_I2C_RESTART_AFTER_WRITE,
		};

CONFIDENTIAL

Use the following sample code to configure I2C0

Steps to follow:

1. Configure I2C. In this example, the mode is task mode, DMA is disabled (When DMA is enabled, the macro:I2C_USE_DMA=1 needs to be added). Use isochronous transfer while the speed is 100000. And the MCU is set as the master.

```
struct HAL_I2C_CONFIG_T cfg;
    memset(&cfg, 0, sizeof(cfg));
    cfg.mode = HAL_I2C_API_MODE_TASK;
    cfg.use_dma = 0;
    cfg.use_sync = 1;
    cfg.speed = 100 * 1000;
cfg.as_master = 1;
```

2. Set I2C IOMUX.

```
hal_iomux_set_i2c0();
```

3. Open I2C and use the structure defined in step 1.

```
static uint8_t i2c_id = HAL_I2C_ID_0
hal_i2c_open(i2c_id, &cfg);
```

4. Close I2C after finishing using it.

```
uint32_t hal_i2c_close(i2c_id);
```

5.2.3.2 Data Transceiver

1. Send data via I2C. The meaning of each parameter is shown in Table 5. Among these parameters `HAL_I2C_ID_T id`, `device_addr`, `buf` and `value_len` are necessary.

```
ret = hal_i2c_send(i2c_id, lsm6dsox_address, lsm6dsox_buffer, 1, len, 0,
NULL);
```

Table 5 Parameter List of Function hal_i2c_send

Item	Description
`enum HAL_I2C_ID_T id`	ID of I2C
`uint32_t device_addr`	Device (such as various sensors) address
`uint8_t *buf`	Data to be sent
`uint32_t reg_len`	The starting address of data to be sent
`uint32_t value_len`	Length of data sent
`uint32_t transfer_id`	Transfer ID
`HAL_I2C_TRANSFER_HANDLER_T handler`	Transfer handler

2. Receive data from I2C. The meaning of each parameter is shown in Table 6. Among these parameters `HAL_I2C_ID_T id`, `device_addr`, `buf` and `value_len` are necessary.

```
ret = hal_i2c_recv(i2c_id, lsm6dsox_address, lsm6dsox_buffer, 1, len,
HAL_I2C_RESTART_AFTER_WRITE, 0, 0);
```

Table 6 Parameter List of Function hal_i2c_recv

Item	Description
`enum HAL_I2C_ID_T id`	ID of I2C
`uint32_t device_addr`	Device (such as various sensors) address
`uint8_t *buf`	The buffer which stored data received
`uint32_t reg_len`	The starting address of data stored
`uint32_t value_len`	Length of data
`uint8_t restart_after_write`	Action after write
`uint32_t transfer_id`	Transfer ID
`HAL_I2C_TRANSFER_HANDLER_T handler`	Transfer handler

5.2.4 Use SPI

The serial peripheral interface (SPI) allows half/full-duplex, synchronous, serial communication with external devices. The interface can be configured as the master and in this case it provides the communication clock (SCK) to the external slave device. The interface is also capable of operating in multi-master mode.

It may be used for a variety of purposes, including simplex synchronous transfers on two lines with a possible bidirectional data line or reliable communication using CRC checking.

There are two SPIs on the board, and two sets of application programming interfaces. One is with letters lcd in the functions' names, while another without them, for example, Hal_iomux_set_spilcd() and Hal_iomux_set_spi(). This paper takes SPI1 as an example, which means functions without lcd in their names are used.

5.2.4.1 SPI Configuration

The following table describes the configuration items.

- SPI configuration

Table 7 SPI Configuration

Item	Description	Notes

`struct HAL_SPI_CFG_T`	SPI configurtion	`struct HAL_SPI_CFG_T {`
		`  uint32_t rate;`
		`  bool clk_delay_half :1;`
		`  bool clk_polarity :1;`

		bool slave :1; bool dma_rx :1; bool dma_tx :1; bool rx_sep_line :1; bool ext_dss_rw :1; bool skip_en :1; uint8_t cs; uint8_t tx_bits; uint8_t rx_bits; uint8_t rx_frame_bits; uint8_t samp_delay; };

1. Configure SPI. In this example, the main board is set as the master.

```
#define     SPI_AS_MASTER
struct HAL_SPI_CFG_T spi_cfg;
spi_cfg.clk_delay_half = true;
    spi_cfg.clk_polarity = true;
#ifdef SPI_AS_MASTER
    spi_cfg.slave = false;
#else
    spi_cfg.slave = true;
#endif
#ifdef SPI_ENABLE_DMA
    spi_cfg.dma_rx = true;
    spi_cfg.dma_tx = true;
#else
    spi_cfg.dma_rx = false;
    spi_cfg.dma_tx = false;
#endif
    spi_cfg.rx_sep_line = false;
    spi_cfg.cs = 0;
#ifdef SPI_AS_MASTER
    spi_cfg.rate = 12 * 1000 * 1000;
#else
    spi_cfg.rate = 20 * 1000 * 1000;
#endif
    spi_cfg.rx_bits = 8;
    spi_cfg.tx_bits = 8;
```

```
    spi_cfg.rx_frame_bits = 0;
```

2. Set SPI MUX.

```
Hal_iomux_set_spilcd();
```

3. Open SPI using the structure defined in step 1.

```
uint32_t error = 0;
error = hal_spilcd_open(&spi_cfg);
```

4. Close SPI after finishing using it.

```
int hal_spilcd_close(uint32_t cs)
```

5.2.4.2 Data Transceiver

There are two ways to send or receive data, as shown below:

1. The CPU sends and receives data directly

Transmit:

```
ret = hal_spilcd_send(pbuf, len);
```

Receive:

```
ret = hal_spilcd_recv(cmd, outBuf, len + 1);
```

2. Send and receive data via DMA

Transmit:

```
ret = hal_spilcd_dma_send(pbuf, len, NULL);
```

Receive:

```
ret = hal_spilcd_dma_recv(cmd, outBuf, len + 1, NULL);
```

5.2.5 Use I2S

The I2S bus is dedicated to data transmission between audio devices and is widely used in various multi-media systems. It is designed to transmit clock and data signals along separate wires.

5.2.5.1 I2S Configuration

The following table describes the configuration items.

- I2S configuration

Table 8 I2S Configuration

Item	Description	Notes

struct HAL_I2S_CONFIG_T	I2S configuration	struct HAL_I2S_CONFIG_T {
		bool use_dma;
		bool chan_sep_buf;
		bool sync_start;
		uint8_t cycles;
		uint8_t bits;
		uint8_t channel_num;
		enum AUD_CHANNEL_MAP_T channel_map;
		uint32_t sample_rate;
		uint32_t ext_mclk_freq;
		};

enum HAL_I2S_MODE_T	I2S mode setting	enum HAL_I2S_MODE_T {
		HAL_I2S_MODE_NULL,
		HAL_I2S_MODE_MASTER,
		HAL_I2S_MODE_SLAVE,
		};

enum AUD_STREAM_T	Audio stream	enum AUD_STREAM_T {
		AUD_STREAM_PLAYBACK = 0,
		AUD_STREAM_CAPTURE,
		
		AUD_STREAM_NUM,
		};

I2S can be used for playing music or recording. Both functions can be tested in a board-to-board fashion. That means, one board is used for recording, the other for playing, and the two transmit audio via I2S.

5.2.5.2 Data Transceiver

I2S is a synchronous transmission protocol, and the clock must be synchronized before you use it. I2S supports full-duplex communication, but there is only one data line, and the transmission direction must be configured before data transmission.

Steps to configure I2S:

1. Get the clock.

```
hal_cmu_pll_enable(HAL_CMU_PLL_AUD, HAL_CMU_PLL_USER_AUD);
```

2. Open I2S. In this example, set I2S0 to slave mode and the audio stream is playback and capture.

```
#define SENS_I2S_ID        HAL_I2S_ID_0
int ret;
```

CONFIDENTIAL

```
ret = hal_i2s_open(SENS_I2S_ID, AUD_STREAM_PLAYBACK, HAL_I2S_MODE_SLAVE);
ret = hal_i2s_open(SENS_I2S_ID, AUD_STREAM_CAPTURE, HAL_I2S_MODE_SLAVE);
```

3. Set up the streams. No PLAYBACK exists without CAPTURE. The sync_start should be set as true when you set up the PLAYBACK stream.

```
struct HAL_I2S_CONFIG_T i2s_cfg;
memset(&i2s_cfg, 0, sizeof(i2s_cfg));
i2s_cfg.use_dma = true;
i2s_cfg.sync_start = true;
i2s_cfg.chan_sep_buf = false;
i2s_cfg.bits = 16;
i2s_cfg.channel_num = 2;
i2s_cfg.channel_map = AUD_CHANNEL_MAP_CH0 | AUD_CHANNEL_MAP_CH1;
i2s_cfg.sample_rate = AUD_SAMPRATE_192000;
ret = hal_i2s_setup_stream(SENS_I2S_ID, AUD_STREAM_PLAYBACK, &i2s_cfg);
i2s_cfg.sync_start = false;
ret = hal_i2s_setup_stream(SENS_I2S_ID, AUD_STREAM_CAPTURE, &i2s_cfg);
```

4. Initialize DMA.

```
memset(&dma_cfg, 0, sizeof(dma_cfg));
dma_cfg.dst = 0;
dma_cfg.dst_bsize = HAL_DMA_BSIZE_4;
dma_cfg.dst_periph = SENS_I2S_TX_NO;
dma_cfg.dst_width = HAL_DMA_WIDTH_HALFWORD;
dma_cfg.handler = i2s_tx_handler;
dma_cfg.src_bsize = HAL_DMA_BSIZE_4;
dma_cfg.src_tsize = sizeof(i2s_tx_buf[0]) / 2;
dma_cfg.src_width = HAL_DMA_WIDTH_HALFWORD;
dma_cfg.try_burst = 1;
dma_cfg.type = HAL_DMA_FLOW_M2P_DMA;
dma_cfg.ch = hal_audma_get_chan(dma_cfg.dst_periph, HAL_DMA_HIGH_PRIO);
for (i = 0; i < I2S_FRAME_NUM; i++) {
dma_cfg.src = (uint32_t)&i2s_tx_buf[i][0];
ret = hal_audma_init_desc(&i2s_tx_dma_desc[i], &dma_cfg,
&i2s_tx_dma_desc[(i + 1) % I2S_FRAME_NUM], (i + 1) % (I2S_FRAME_NUM / 2) ==
0);
       ASSERT(ret == 0, "Failed to init dma desc for tx");
   }
```

5. Start DMA

```
ret = hal_audma_sg_start(&i2s_rx_dma_desc[0], &dma_cfg);
```

6. Start I2S streaming.

```
ret = hal_i2s_start_stream(MCU_I2S_ID, AUD_STREAM_PLAYBACK);
ret = hal_i2s_start_stream(SENS_I2S_ID, AUD_STREAM_CAPTURE);
```

5.2.6 Use ADC

The Analog to Digital Converter converts an analog signal with continuous time and continuous amplitude into a digital signal with discrete time and discrete amplitude.

5.2.6.1 GPADC Configurable Items

The following table describes the configuration items.

Table 9 GPADC Configurable Items

Item	Description	Notes

enum
HAL_GPADC_CHAN_T
	Channels of GPADC	enum HAL_GPADC_CHAN_T {
		HAL_GPADC_CHAN_0 = 0,
		HAL_GPADC_CHAN_BATTERY = 1,
		HAL_GPADC_CHAN_2 = 2,
		HAL_GPADC_CHAN_3 = 3,
		HAL_GPADC_CHAN_4 = 4,
		HAL_GPADC_CHAN_5 = 5,
		HAL_GPADC_CHAN_6 = 6,
		HAL_GPADC_CHAN_ADCKEY = 7,
		HAL_GPADC_CHAN_QTY,
		};

enum HAL_GPADC_ATP_T	The time of signal acquisition	enum HAL_GPADC_ATP_T { HAL_GPADC_ATP_NULL = 0, HAL_GPADC_ATP_125US = 125, HAL_GPADC_ATP_500US = 500, HAL_GPADC_ATP_1MS = 1000, HAL_GPADC_ATP_2MS = 2000, HAL_GPADC_ATP_5MS = 5000, HAL_GPADC_ATP_10MS = 10000, HAL_GPADC_ATP_20MS = 20000, HAL_GPADC_ATP_30MS = 30000, HAL_GPADC_ATP_40MS = 40000,

CONFIDENTIAL

		HAL_GPADC_ATP_50MS = 50000, HAL_GPADC_ATP_60MS = 60000, HAL_GPADC_ATP_ONESHOT = -1, };

5.2.6.2 Usage

In order to use GPADCs, call the interface hal_gpadc_open to open GPADC.

```
int hal_gpadc_open(enum HAL_GPADC_CHAN_T channel, enum HAL_GPADC_ATP_T atp,
HAL_GPADC_EVENT_CB_T cb)
```

GPADCs can be configured as one shot mode or acquiring for a period of time.

```
hal_gpadc_open(HAL_GPADC_CHAN_0, HAL_GPADC_ATP_ONESHOT,
pmu_ntc_irq_handler);

hal_gpadc_open(HAL_GPADC_CHAN_3, HAL_GPADC_ATP_20MS, gpadc_key_handler);
```

Sample Code 3: Use GPADC

{"gpadc_test", gpadc_test},

[TEST,gpadc_test]

5.2.7 Use Hardware Timer

There are slow timer and fast timer in the BES2700YP, and each timer has four modes, free-running, one-shot, periodic and quantity.

Table 10 Hardware Timer Type

Item	Description	Notes

enum
HAL_TIMER_TYPE_T
	Timer type	enum HAL_TIMER_TYPE_T {
		HAL_TIMER_TYPE_FREERUNNING = 0,
		HAL_TIMER_TYPE_ONESHOT,
		HAL_TIMER_TYPE_PERIODIC,
		HAL_TIMER_TYPE_QTY
		};

Steps to use a hardware timer:

1. Initialize IOMUX.

```
static const struct HAL_IOMUX_PIN_FUNCTION_MAP gpio_output_tester[] = {
```

```
    {(enum HAL_IOMUX_PIN_T)PIN_OUT_1, HAL_IOMUX_FUNC_AS_GPIO,
AL_IOMUX_PIN_VOLTAGE_VIO, HAL_IOMUX_PIN_PULLUP_ENABLE},
    {(enum HAL_IOMUX_PIN_T)PIN_OUT_2, HAL_IOMUX_FUNC_AS_GPIO,
AL_IOMUX_PIN_VOLTAGE_VIO, HAL_IOMUX_PIN_PULLUP_ENABLE},
    };
    hal_iomux_init(gpio_output_tester, 2);
```

2. Set a direction.

```
hal_gpio_pin_set_dir((enum HAL_GPIO_PIN_T)gpio_output_tester[0].pin,
HAL_GPIO_DIR_OUT, 0);
hal_gpio_pin_set_dir((enum HAL_GPIO_PIN_T)gpio_output_tester[1].pin,
HAL_GPIO_DIR_OUT, 0);
```

3. Set up a hardware timer.

```
hal_timer_setup(HAL_TIMER_TYPE_ONESHOT, slow_timer_irq_handler);
```

4. Start the hardware timer

```
#define ONESLOT_MS_DEFAULT                  (2000)
hal_timer_start(MS_TO_TICKS(ONESLOT_MS_DEFAULT));
```

Sample Code 4: Hardware Timer

{"test_hw_timer", hw_timer_test},

[TEST, test_hw_timer]

5.2.8 Use PWM

5.2.8.1 PWM Configuration

The following table describes the configuration items.

- IOMUX configuration

Table 11 PWM IOMuX Coniguration

Item	Description	Notes

struct HAL_PWM_CFG_T	PWM configuration	struct HAL_PWM_CFG_T {
		uint32_t freq;
		uint8_t ratio;// Duty cycle
		bool inv; //Whether to reverse polarity

		bool sleep_on;//Whether enable PWM while master clock sleeping
		};

Use the following sample code to configure GPIO_3_4, GPIO_3_5 GPIO_36 and GPIO_37 as a PWM in the 1.8v voltage domain, no pull state.

```
struct HAL_IOMUX_PIN_FUNCTION_MAP pinmux_pwm[] = {
       {HAL_IOMUX_PIN_P3_4, HAL_IOMUX_FUNC_PWM0, HAL_IOMUX_PIN_VOLTAGE_VIO,
HAL_IOMUX_PIN_NOPULL},
       {HAL_IOMUX_PIN_P3_5, HAL_IOMUX_FUNC_PWM1, HAL_IOMUX_PIN_VOLTAGE_VIO,
HAL_IOMUX_PIN_NOPULL},
       {HAL_IOMUX_PIN_P3_6, HAL_IOMUX_FUNC_PWM2, HAL_IOMUX_PIN_VOLTAGE_VIO,
HAL_IOMUX_PIN_NOPULL},
       {HAL_IOMUX_PIN_P3_7, HAL_IOMUX_FUNC_PWM3, HAL_IOMUX_PIN_VOLTAGE_VIO,
HAL_IOMUX_PIN_NOPULL},
    };
hal_iomux_init(pinmux_pwm, ARRAY_SIZE(pinmux_pwm));
```

Steps to configure PWM0, PWM1, PWM2 and PWM3:

1. Initialize the GPIO's IOMUX as PWM.

```
void app_sample_code_set_pwm(void)
{
    struct HAL_IOMUX_PIN_FUNCTION_MAP pinmux_pwm[] = {
           {HAL_IOMUX_PIN_P3_4, HAL_IOMUX_FUNC_PWM0,
HAL_IOMUX_PIN_VOLTAGE_VIO, HAL_IOMUX_PIN_NOPULL},
           {HAL_IOMUX_PIN_P3_5, HAL_IOMUX_FUNC_PWM1,
HAL_IOMUX_PIN_VOLTAGE_VIO, HAL_IOMUX_PIN_NOPULL},
           {HAL_IOMUX_PIN_P3_6, HAL_IOMUX_FUNC_PWM2,
HAL_IOMUX_PIN_VOLTAGE_VBAT, HAL_IOMUX_PIN_NOPULL},
           {HAL_IOMUX_PIN_P3_7, HAL_IOMUX_FUNC_PWM3,
HAL_IOMUX_PIN_VOLTAGE_VBAT, HAL_IOMUX_PIN_NOPULL},
    };
    hal_iomux_init(pinmux_pwm, ARRAY_SIZE(pinmux_pwm));
    }
```

2. Configure PWM parameters. For example, the frequency of PWM0 is set as 8kHz, the duty cycle is 25%, not reverse the polarity, and PWM is banned when the master clock is sleeping.

```
int ret;
```

```
struct HAL_PWM_CFG_T pwm_cfg = {
        .freq = 8000,
        .ratio = 25,
        .inv = false,
        .sleep_on = false,
};
app_sample_code_set_pwm();
TRACE(0, "PWM test has started...");
TRACE(0, "Please use an oscilloscope to view the set pin waveform!");
pwm_cfg.freq = 8000;
ret = hal_pwm_enable(HAL_PWM_ID_0, &pwm_cfg);
ASSERT(ret == 0, "Failed to enable PWM0: %d", ret);
pwm_cfg.freq = 4000;
ret = hal_pwm_enable(HAL_PWM_ID_1, &pwm_cfg);
ASSERT(ret == 0, "Failed to enable PWM1: %d", ret);
pwm_cfg.freq = 2000;
ret = hal_pwm_enable(HAL_PWM_ID_2, &pwm_cfg);
ASSERT(ret == 0, "Failed to enable PWM2: %d", ret);
pwm_cfg.freq = 1000;
ret = hal_pwm_enable(HAL_PWM_ID_3, &pwm_cfg);
ASSERT(ret == 0, "Failed to enable PWM3: %d", ret);
```

After configuration, use an oscilloscope to view the set pin waveform.

5.2.8.2 Breathing LED

The following table describes the configuration items.

Table 12 Breathing LED Configuration

Item	Description	Notes

struct HAL_PWM_BR_CFG_T	Breathing LED configuration	struct HAL_PWM_BR_CFG_T {
		uint16_t off_time_ms;
		uint16_t on_time_ms;
		uint16_t fade_time_ms;
		};

Use the following sample code to enable Breathing LED and set the off time as 100ms, on time 100ms, fade time 1500ms.

```
struct HAL_PWM_BR_CFG_T pwm_br_cfg = {
    .off_time_ms = 100,
```

```
    .on_time_ms  = 100,
    .fade_time_ms = 1500,
};

ret = hal_pwm_breathing_led_enable(HAL_PWM_ID_0, &pwm_br_cfg);
TRACE(1, "open breath1:%d,%s", ret, ret ? "error" : "success");
ret = hal_pwm_breathing_led_enable(HAL_PWM_ID_1, &pwm_br_cfg);
TRACE(1, "open breath2:%d,%s", ret, ret ? "error" : "success");
ret = hal_pwm_breathing_led_enable(HAL_PWM_ID_2, &pwm_br_cfg);
TRACE(1, "open breath3:%d,%s", ret, ret ? "error" : "success");
ret = hal_pwm_breathing_led_enable(HAL_PWM_ID_3, &pwm_br_cfg);
TRACE(1, "open breath4:%d,%s", ret, ret ? "error" : "success");
```

5.2.9 DMA

Direct memory access (DMA) is used in order to provide high-speed data transfer between peripherals and memory as well as from memory to memory. Data can be quickly moved by DMA without any CPU actions. This frees CPU resources for other operations.

5.2.9.1 DMA Configuration

The following table describes the configuration items.

- DMA configuration

Table 13 DMA Coniguration

Item	Description	Notes

enum HAL_DMA_RET_T	DMA return value	enum HAL_DMA_RET_T {
		HAL_DMA_OK,
		HAL_DMA_ERR,
		};

struct HAL_DMA_CH_CFG_T	DMA channel configuration	struct HAL_DMA_CH_CFG_T { uint8_t ch; /* DMA channel number */ uint8_t try_burst; uint16_t src_tsize; /* Length/Size of transfer */ enum HAL_DMA_WDITH_T src_width; enum HAL_DMA_WDITH_T dst_width; enum HAL_DMA_BSIZE_T src_bsize; enum HAL_DMA_BSIZE_T dst_bsize;

CONFIDENTIAL

		enum HAL_DMA_FLOW_CONTROL_T type; /* Transfer Type */ enum HAL_DMA_PERIPH_T src_periph; enum HAL_DMA_PERIPH_T dst_periph; uint32_t src; /* Physical Source Address */ uint32_t dst; /* Physical Destination Address */ HAL_DMA_IRQ_HANDLER_T handler; HAL_DMA_START_CALLBACK_T start_cb; };

struct HAL_DMA_DESC_T	Transfer descriptor structure	struct HAL_DMA_DESC_T {
		uint32_t src; /* Source address */
		uint32_t dst; /* Destination address */
		uint32_t lli; /* Pointer to next descriptor structure */
		uint32_t ctrl; /* Control word that has transfer size, type etc. */
		};

enum HAL_DMA_GET_CHAN_T	DMA prioity	enum HAL_DMA_GET_CHAN_T {
		HAL_DMA_HIGH_PRIO,
		HAL_DMA_LOW_PRIO,
		HAL_DMA_LOW_PRIO_ONLY
		};

5.2.9.2 DMA Usage

DMA transfers copy data from one address space to another. When the CPU initiates the transfer, the transfer itself is performed and completed by the DMA controller. A typical example is moving a block of external memory to a faster memory area inside the chip. Operations like this will not stall processor work, but can be descheduled to handle other work. DMA transfers are important for high performance embedded system algorithms and networks.

Steps to use DMA:

1. Configure DMA return struct.

```
enum HAL_DMA_RET_T ret = HAL_DMA_OK;
```

2. Clear destination buffers and set source destination.

```
uint32_t i = 0;
```

CONFIDENTIAL

```
    for (i = 0; i < SRC_BUF_MAX; i++) {
            dst_buf[i] = 0;
            src_buf[i] = (i<<16) | i;
    }
```

3. Open DMA.

```
hal_dma_open();
```

4. Configure DMA description.

```
struct HAL_DMA_CH_CFG_T cfg = {0};
cfg.type = DMA_TYPE;
cfg.src = (uint32_t)src_buf;
cfg.src_tsize = SRC_BUF_MAX / DMA_SRC_BUSRT_SIZE;
cfg.src_bsize = DMA_SRC_BUSRT_SIZE;
cfg.src_width = DMA_SRC_XFER_WIDTH;
cfg.src_periph = DMA_SRC_PERIPH;
cfg.dst = (uint32_t)dst_buf;
cfg.dst_bsize = DMA_DST_BUSRT_SIZE;
cfg.dst_width = DMA_DST_XFER_WIDTH;
cfg.dst_periph = DMA_DST_PERIPH;
cfg.try_burst = 1;
cfg.handler = dma_handler;
cfg.start_cb = dma_start_cb;
cfg.ch = hal_dma_get_chan(DMA_PERIPH, HAL_DMA_HIGH_PRIO);
```

5. Configure transfer descriptor structure, which includes source address, destination address, pointer to next descriptor structure and control word that has transfer size, type, etc.

```
ret = hal_dma_init_desc(&g_dma_desc, &cfg, NULL, DMA_TC_IRQ_ENABLE);
```

6. Start DMA.

```
ret = hal_dma_sg_start(&g_dma_desc, &cfg);
```

Stop.

5.3 Memory Map

This section describes the logic address map and the method to use it.

5.3.1 Logic Address Map

Although the logic addresses of these cores are different, some of them point to the same physical address: BTH RAM and BTH RAMX/FRAMX. The sensor hub RAM and sensor hub RAMX point to another physical address. BTH FLASH, BTH FLASH_NC and BTH FLASHX point to the same physical address, too.

The cores with X in their names integrate ITCM (Instruction Tightly Memory) and the cores without it integrate DTCM (Data Tightly Memory). It is recommended to put instruction in cores with X in names while putting data in cores without it.

The logic address map of BES2700YP is as follows.

Shared memory

Core	Core owned (start logic address)	Core owned (size)	RAM share 0 (start logic address)	RAM share 0 (size)	RAM share 1 (start logic address)	RAM share 1 (size)	Total RAM Size
BTH RAM	0x20500000	0x60000	0x20560000	0x80000		0	(0xE0000)896KB
BTH RAMX/FRAMX	0x00500000		0x00560000				
Sensor hub RAM	0x206C0000	0x40000	0x20680000	0x40000		0	(0x80000)512KB
Sensor hub RAMX	0x006C0000						
BTC RAM (For BTC ram run)		0	0xC0460000	0xA0000		0	(0xA0000)640KB
Hifi4		0				0x100000	1MB (flexible from 128KB to 2MB)
M55		0				0x100000	1MB (flexible from 128KB to 2MB)
BTH FLASH	0x34020000	0xFE0000					0xFE0000
BTH FLASH_NC	0x30020000						(First 0x20000 is for secondary bootloader)
BTH FLASHX	0x14020000						

Figure 16 Logical Address Map

5.3.2 Flash Memory Map

There are four main flash sections:

Secondary bootloader:

Once programmed, never changed, in charge of image mapping and jumping to the dedicated image.

FLASH_OTA_BOOT_INFO_T is located at 0x1000, to tell the secondary bootloader which Image area shall be mapped and jumped to.

A area and B area:

Dual copy sections include the application image, voice prompt and other user binaries that needs to be updated all-together over the air.

A and B areas have the same size and same start offset of each sub-sections.

CONFIDENTIAL

As the BES QSPI controller support flash physical->logic address mapping, OTA procedure could program the new image to the area flip-flop according the current applied area.

Volatile sections:

Store the information that changes all over the life time. E.g. TWS&mobile pairing information, volume, etc.

Non-volatile sections:

Never changed once programmed on the production line.

The factory section is always located at the last 4KB flash page, for the BES download tool to program the non-volatile information: classic BT mac address, BT name. BLE mac address, BLE name, and XTAL trim value.

The ANC audio configuration section is to store the ANC configuration and the ANC related calibration values gathered from the production line.

The flash memory map is as follows:

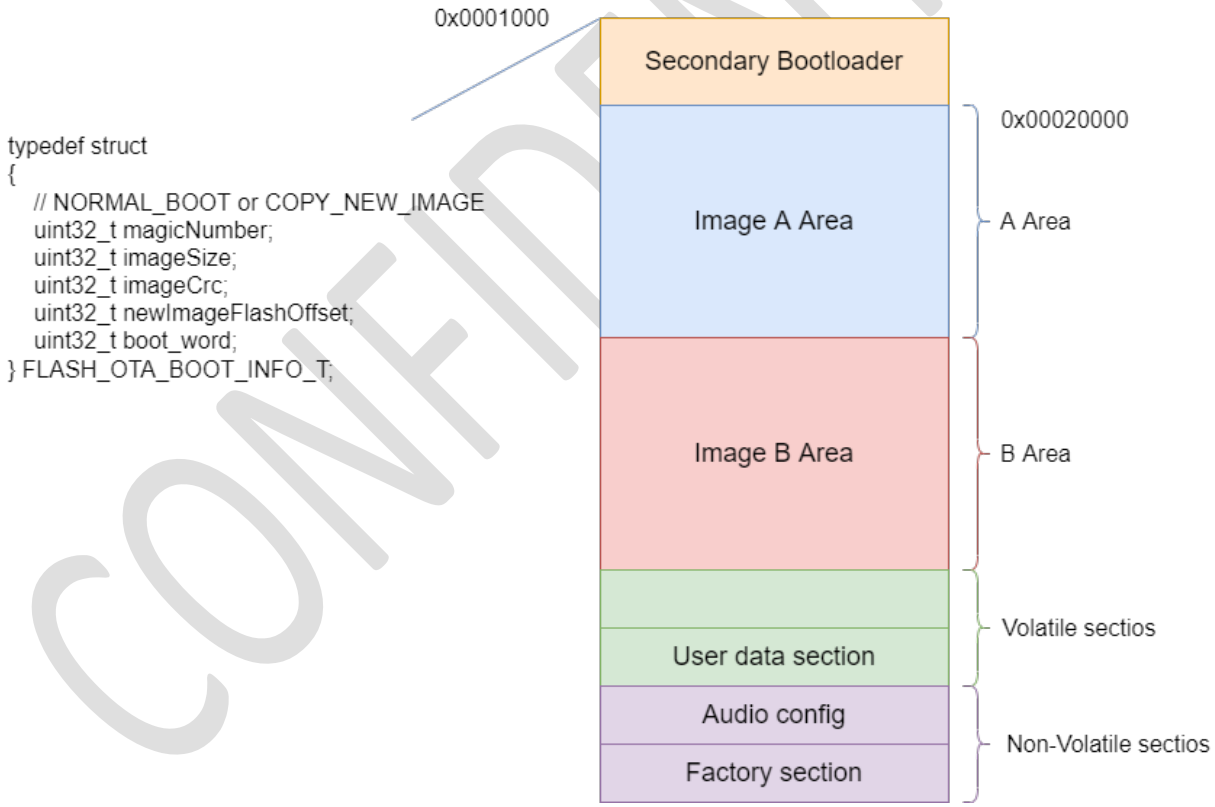


Figure 17 Flash Memory Map for OTA

5.3.3 Audio Buffer Usage

All the free RAM can be used as audio buffer. Audio buffer is divided into audio buffer and capture audio buffer. There are two macros: APP_AUDIO_BUFFER_SIZE and APP_CAPTURE_AUDIO_BUFFER_SIZE. The former one is the size of audio buffer and the last one is the size of capture audio buffer. Audio buffer ranges from the top of the free RAM

CONFIDENTIAL

space to "top + APP_AUDIO_BUFFER_SIZE" while capture audio buffer rangers from "tail - APP_CAPTURE_AUDIO_BUFFER_SIZE" to the tail of the free RAM space. Audio buffer uses syspool from the beginning, and capture buffer uses syspool from the end. Make sure that they are not overlapped with each other.

Steps to allocate memory to audio buffer (Take playing music as example):

1. Initialize the memory pool with a specific size.

```
app_audio_mempool_init_with_specific_size(APP_AUDIO_BUFFER_SIZE);
```

2. Get the address of memory that was allocated to audio buffer.

```
app_audio_mempool_get_buff(&bt_audio_buff, stream_cfg.data_size);
```

Steps to allocate memory to capture audio buffer (Take voice wake-up as example):

1. Initialize the capture audio memory pool.

```
app_capture_audio_mempool_init();
```

2. Get the address of memory that was allocated to capture audio buffer.

```
app_capture_audio_mempool_get_buff(&buf,APP_CAPTURE_AUDIO_BUFFER_SIZE)
```

In scenarios where both decoding and encoding exist, allocate memory as follows:

1. Initialize the memory pool.

```
app_audio_mempool_init();
```

2. Get the address.

```
app_audio_mempool_get_buff(&bt_audio_buff, stream_cfg.data_size);
```

5.3.4 Memory Usage in Link Script

In our project, the link script is generated from the template file. For example, in the project of BES2700YP, the template file is named best1000.lds.S which is located in scripts\link\ best1000.lds.S. The link script generated from template file is named _best1000.lds which is located in out\best1600_ibrt_best1000.lds.

Template of memory part:

```
MEMORY
{
    ROM         (rx)    : ORIGIN = ROM_BASE,              LENGTH = ROM_SIZE
    FLASH       (r)     : ORIGIN = FLASH_REGION_BASE,       LENGTH =
FLASH_REGION_SIZE
```

```
    FLASH_NC    (r)     : ORIGIN = FLASH_NC_REGION_BASE,    LENGTH =
FLASH_REGION_SIZE
#ifdef USE_MULTI_FLASH
    FLASH1      (r)     : ORIGIN = FLASH1_REGION_BASE,      LENGTH =
FLASH1_REGION_SIZE
    FLASH1_NC   (r)     : ORIGIN = FLASH1_NC_REGION_BASE,   LENGTH =
FLASH1_REGION_SIZE
#endif
#ifndef FLASHX
    FLASHX      (rx)    : ORIGIN = FLASHX_REGION_BASE,      LENGTH =
FLASH_REGION_SIZE
#endif
#ifdef MAIN_RAM_USE_TCM
    RAM         (rwx)   : ORIGIN = RAM_BASE,              LENGTH = RAM_SIZE
    RAMX        (rx)    : ORIGIN = RAMX_BASE,             LENGTH = RAMX_SIZE
#else
    RAM         (rwx)   : ORIGIN = RAM_BASE,              LENGTH = RAM_SIZE -
FAST_XRAM_SECTION_SIZE
    RAMX        (rx)    : ORIGIN = RAMX_BASE,             LENGTH = RAM_SIZE -
FAST_XRAM_SECTION_SIZE
    FRAMX       (rwx)   : ORIGIN = RAMX_BASE + RAM_SIZE -
FAST_XRAM_SECTION_SIZE,   LENGTH = FAST_XRAM_SECTION_SIZE
#endif
#ifdef RAM_NC_BASE
    RAM_NC      (rw)    : ORIGIN = RAM_NC_BASE,           LENGTH = RAM_NC_SIZE
#endif
#if defined(ARM_CMSE) && defined(INTSRAM_RUN)
    RAMX_NSC    (rx)    : ORIGIN = RAMX_BASE + RAM_S_SIZE,  LENGTH =
RAM_NSC_SIZE
#endif
#if (defined(PSRAM_ENABLE) || defined(PSRAM_HEAP_ONLY)) &&
defined(PSRAM_BASE) && (PSRAM_SIZE > 0)
    PSRAM       (rwx)   : ORIGIN = PSRAM_BASE,            LENGTH =
PSRAM_REGION_SIZE
    PSRAM_NC    (rwx)   : ORIGIN = PSRAM_NC_BASE,         LENGTH =
PSRAM_REGION_SIZE
#ifndef PSRAMX
    PSRAMX      (rwx)   : ORIGIN = PSRAMX_BASE,           LENGTH =
PSRAM_REGION_SIZE
```

```
#endif
#endif
#if defined(PSRAMUHS_ENABLE) && defined(PSRAMUHS_BASE) &&
(MCU_PSRAMUHS_SIZE > 0)
    PSRAMUHS    (rwx)   : ORIGIN = MCU_PSRAMUHS_BASE,          LENGTH =
MCU_PSRAMUHS_SIZE
    PSRAMUHS_NC (rwx)   : ORIGIN = MCU_PSRAMUHS_NC_BASE,       LENGTH =
MCU_PSRAMUHS_SIZE
#ifndef PSRAMUHSX
    PSRAMUHSX   (rx)    : ORIGIN = MCU_PSRAMUHSX_BASE,         LENGTH =
MCU_PSRAMUHS_SIZE
#endif
#endif
#if defined(CP_IN_SAME_EE) && (RAMCP_SIZE > 0)
    RAMCP       (rwx)   : ORIGIN = RAMCP_BASE,             LENGTH = RAMCP_SIZE
    RAMCPX      (rwx)   : ORIGIN = RAMCPX_BASE,            LENGTH = RAMCPX_SIZE
#endif
#ifdef MEM_POOL_BASE
    MEM_POOL    (rwx)   : ORIGIN = MEM_POOL_BASE,          LENGTH =
MEM_POOL_SIZE
#endif
}
```

Memory part in the link script:

```
MEMORY
{
 ROM (rx) : ORIGIN = 0x00020000, LENGTH = 0x00018000
 FLASH (r) : ORIGIN = (0x34000000), LENGTH = (0x800000 - ((0x34000000) -
0x34000000))
 FLASH_NC (r) : ORIGIN = (((0x34000000)) - 0x34000000 + 0x30000000), LENGTH
= (0x800000 - ((0x34000000) - 0x34000000))
 FLASHX (rx) : ORIGIN = (((0x34000000)) - 0x34000000 + 0x14000000), LENGTH
= (0x800000 - ((0x34000000) - 0x34000000))
 RAM (rwx) : ORIGIN = 0x20500000, LENGTH = 0xE0000 - 0x15500
 RAMX (rx) : ORIGIN = (0x20500000 - 0x20500000 + 0x00500000), LENGTH =
0xE0000 - 0x15500
 FRAMX (rwx) : ORIGIN = (0x20500000 - 0x20500000 + 0x00500000) + 0xE0000 -
0x15500, LENGTH = 0x15500
}
```

In order to revise the memory map, modify the value of FAST_XRAM_SECTION_SIZE. Do not modify other macros. The detailed method is described in section 5.6.

5.3.5 Define a Specific Section and Use it

In order to define a specific section and use it, you can define a section in the template of link script and use it in the code.

Here is an example:

1. Define a specific section.

```
__core_bridge_task_cmd_table_start = .;
KEEP(*(.core_bridge_task_cmd_table))
__core_bridge_task_cmd_table_end = .;
. = ALIGN(4);
```

2. Use it in the code.

```
__attribute__((used, section(".core_bridge_task_cmd_table")) =
app_esehll_bridge_tx_done_handler;
```

5.3.6 Coordinate Ram Size

Fast ram sections can be coordinated to give more space to ram/ramx sections. In order to do this, you are supposed to coordinate FAST_XRAM_SECTION_SIZE.

Steps to coordinate ram size:

1. Search __free_ in out\best1600_ibrt\best1600_ibrt.map to check if there is still free fast ram. As the following code shows, the preceding number is the free size of fast ram. The second number means the space of the whole fast ram.

```
0x000000000000e660              __free_fram = (0x21800 - (__fast_sram_end__
- __fast_sram_text_data_start__))

0x00000000000527a0              __free_ram = (__StackLimit - __HeapLimit)

0x000000000053b4d8              __free_flash = (((__tail_section_start -
0x30000000) + 0x34000000) - __flash_end)
```

2. Define macro FAST_XRAM_SECTION_SIZE, whose value is the second number found in Step 1 minus the size that you want to give to other sections. Be careful not to subtract more than the size of free ram.

```
##define FAST_XRAM_SECTION_SIZE 0x15500
```

5.3.7 Overlay Usage

Since a program does not access all parts of the program and data at all times (especially for large programs), the user space can be divided into a fixed area and several coverage areas. Put the frequently active parts in the fixed area, and the rest are segmented according to the calling relationship. First, put those segments to be accessed in the coverage area, and other segments are placed in the external memory. Before they need to be called, the system calls them into the coverage area, which replaces the original segment in the coverage area.

The role of the overlay technology is to divide the program into several program segments with relatively independent functions, and make program segments not running at the same time share the same memory area according to their own logical structure.

For example, audio buffer of music and that of call cannot be used at the same time. In this case, this section can be overlaid. The overlay section can not only be distinguished by functional types, but also by encoding such as AAC, SBC, LDPC, etc. Whichever way it is, be sure that they don't use the same section of ram at the same time. The overlay section is separated from fast ram.

The following table describes the configuration items.

- Overlay configuration

Table 14 Overlay Configuration

Item	Description	Notes
enum APP_OVERLAY_ID_T	Overlay ID	Depends on how it is coded and how it works such as APP_OVERLAY_HFP = HAL_OVERLAY_ID_0. See services\overlay\app_overlay.h for details

There are 12 overlay_text sections and 12 overlay_data sections. As the name suggests, the former is used for code and the latter is used for data.

Steps to add a custom overlay:

1. Find a section that is unused, and put files generated after compilation here. In this example, data of flac_com.o is used.

```
.overlay_data10
{
    LONG(0);
    *:flac_com.o(.data* .rodata*)
    *(.overlay_data10 .overlay_rodata10)
    . = ALIGN(4);
    }
```

CONFIDENTIAL

2. Select an overlay method. In this example, A2DP LHDC encoder is specified.

```
app_overlay_select(APP_OVERLAY_A2DP_LHDC_ENCODER);
```

3. After use, unload overlay.

```
app_overlay_unloadall();
```

5.4 System API

5.4.1 System Ticks

5.4.1.1 RTC Setting

System ticks are in the unit of 16000 Hz lower power clock ticks: 1/16000 seconds. The API to get passed system ticks since the system powers on is as follows:

```
#define GET_CURRENT_TICKS()         hal_sys_timer_get()
```

The API to get passed system mill-seconds since the system powers on is as follows:

```
#define GET_CURRENT_MS()            TICKS_TO_MS(GET_CURRENT_TICKS())
```

Sample Code 5: Get Current Time

{"get_current_ticks", get_current_ticks_test},

[TEST, get_current_ticks]

{"get_current_ms", get_current_ms_test},

[TEST, get_current_ms]

5.4.1.2 Automatically Calibrate the Clock

BES2700YP can calibrate the slave clock with the help of the slave clock. If the slave clock is found to be inaccurate, just open the macros OSTICK_USE_FAST_TIMER and FAST_TIMER_COMPENSATE.

For example:

```
export OSTICK_USE_FAST_TIMER ?= 1
export FAST_TIMER_COMPENSATE ?= 1
```

5.4.2 Battery Management

5.4.2.1 Charger Event Handler

The charger event handler is registered by the interface app_battery_register. Use the following code to register the IBRT battery callback.

CONFIDENTIAL

```
app_battery_register(app_ibrt_battery_callback);

void app_ibrt_battery_callback(APP_BATTERY_MV_T currvolt, uint8_t
currlevel,enum APP_BATTERY_STATUS_T curstatus,uint32_t status, union
APP_BATTERY_MSG_PRAMS prams)
{
    switch (curstatus)
    {
        case APP_BATTERY_STATUS_NORMAL:
        case APP_BATTERY_STATUS_CHARGING:
            app_ibrt_battery_handle_process_normal(status,prams);
            break;

        default:
            break;
    }
}
```

5.4.2.2 Battery Level Measurement

You can change the following configurations based on actual scenarios.

The battery level is divided into nine levels.

```
#define APP_BATTERY_LEVEL_MAX (9)
```

When the battery is full, the voltage is 4200 mV.

```
#define APP_BATTERY_MAX_MV (4200)
```

When the battery is exhausted and the earphone automatically powers down, the voltage is 3100 mV

```
#ifndef APP_BATTERY_PD_MV
#define APP_BATTERY_PD_MV   (3100)
#endif
```

The difference between the maximum voltage and power down voltage is divided into several equal parts, and the macro APP_BATTERY_MV_BASE in the value of each part.

```
#define APP_BATTERY_LEVEL_MIN (0)
#define APP_BATTERY_LEVEL_NUM (APP_BATTERY_LEVEL_MAX-
APP_BATTERY_LEVEL_MIN+1)
#define APP_BATTERY_MV_BASE ((APP_BATTERY_MAX_MV-
APP_BATTERY_PD_MV)/(APP_BATTERY_LEVEL_NUM)).
```

If you want to use their own battery management, set the value of macro IS_BES_BATTERY_MANAGER_ENABLED to zero.

Sample Code 6: Get Current Battery Level

```
{"current_battery_level",     get_current_battery_level_test},

[TEST, current_battery_level]
```

5.4.3 UART Data Path

5.4.3.1 UART Data Path

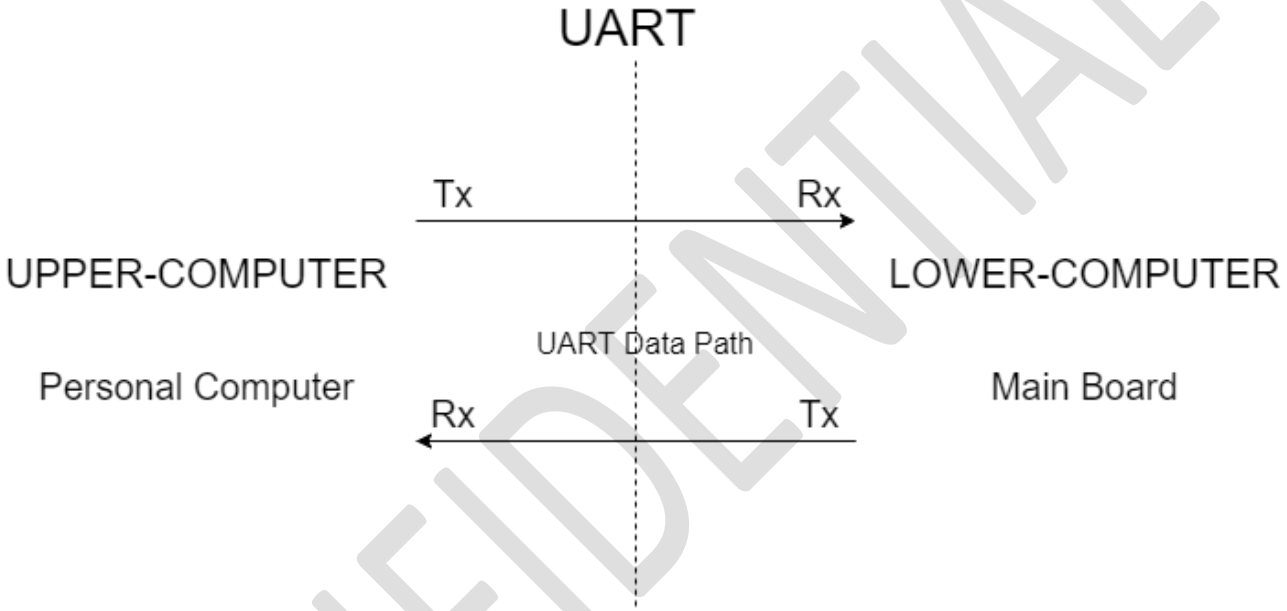


Figure 18 UART Data Path

For the UART data path, the PC connects to the main board via the data cable. The PC acts as an upper-computer while the main board is lower-computer. For downstream data transmission, the PC transfers data to the main board. For upstream data transmission, the main board transfers data to the PC.

5.4.3.2 UART Loopback Data Path

A loopback test is a test in which a signal from a communication device is returned to its original place. A loopback test is to determine whether there is a breakpoint in the line or port by shorting the transceiver of the equipment under test or the line to let the equipment under test receive the signal sent by itself. It can also hang a test instrument on the loopback line to test the transmission quality.

There is a macro TEST_UART_LOOP_BACK which controls the loopback test. To enable a loopback test, change its value to 1.

```
#define TEST_UART_LOOP_BACK 1
```

CONFIDENTIAL

In the test, APP_UART_ID is the ID of UART to be tested, and CONFIG_UART_BAUDRATE is baud rate. Use the following sample code to set UART0 for the looptest, and the baud rate is 1152000.

```
#define APP_UART_ID HAL_UART_ID_0
#define CONFIG_UART_BAUDRATE    1152000
```

The following function registers a callback. Once the main board receives the data sent from upper-computer, app_process_uart prints it via the debug port.

```
app_uart_register_data_receiver(app_process_uart);
static void app_process_uart(uint8_t *ptr, uint32_t len)
{
    TRACE(0, "app_process_uart: %s", ptr);
}
```

After registration, the board starts receiving data.

```
app_uart_start_receiving_data();
```

After receiving data from upper-computer, the board transfers it back.

```
uart_dma_send_data(ptr, bytesInBuffer);
```

5.4.3.3 Add UART Test Command

Steps to add UART test command and use it:

1. Register UART command callback function.

```
app_trace_rx_register("AOB", ble_audio_uart_cmd_callback);

unsigned int ble_audio_uart_cmd_callback(unsigned char *cmd, unsigned int
cmd_length)
{
    int param_len = 0;
    char* cmd_param = NULL;
    char* cmd_end = (char *)cmd + cmd_length;
    cmd_param = strstr((char*)cmd, (char*)"|");
    if (cmd_param)
    {
        *cmd_param = '\0';
        cmd_length = cmd_param - (char *)cmd;
        cmd_param += 1;
        param_len = cmd_end - cmd_param;
```

```
        ble_audio_uart_cmd_with_param_handler((char *)cmd, cmd_length,
cmd_param, param_len);
    }
    else
    {
        ble_audio_uart_cmd_handler((unsigned char*)cmd,strlen((char*)cmd));
    }
    return 0;
}
```

2. Define UART command handler that used in step1.

```
int ble_audio_uart_cmd_with_param_handler(
    char* cmd, uint32_t cmdLen, char* cmdParam, uint32_t cmdParamLen)
{
    int ret = 0;
    ble_audio_uart_test_function_handle_with_param handl_function =
        ble_audio_test_find_uart_handle_with_param(cmd);
    if(handl_function)
    {
        handl_function(cmdParam, cmdParamLen);
    }
    else
    {
        ret = -1;
        TRACE(0,"can not find handle function");
    }
    return ret;
}

int ble_audio_uart_cmd_handler(unsigned char *buf, unsigned int length)
{
    int ret = 0;
    ble_audio_uart_test_function_handle handl_function =
ble_audio_test_find_uart_handle(buf);
    if(handl_function)
    {
        app_bt_start_custom_function_in_bt_thread(0, 0,
            (uint32_t)handl_function);
    }
```

```
    else
    {
        ret = -1;
        TRACE(0,"can not find handle function");
    }
    return ret;
}
```

3. Find handle function with UART command.

```
ble_audio_uart_test_function_handle handl_function =
ble_audio_test_find_uart_handle(buf);

ble_audio_uart_test_function_handle
ble_audio_test_find_uart_handle(unsigned char* buf)
{
    ble_audio_uart_test_function_handle p = NULL;
    for(uint32_t i = 0; i < ARRAY_SIZE(aob_uart_test_handle); i++)
    {
        if (strncmp((char*)buf, aob_uart_test_handle[i].string,
strlen(aob_uart_test_handle[i].string))==0 ||
            strstr(aob_uart_test_handle[i].string, (char*)buf))
        {
            TRACE(1, "AOB cmd:%s", aob_uart_test_handle[i].string);
            p = aob_uart_test_handle[i].function;
            break;
        }
    }
    return p;
}
```

4. Define UART handle struct, and add command in the UART command table.

```
typedef struct
{
    const char* string;
    ble_audio_uart_test_function_handle function;
} ble_audio_uart_handle_t;

static const ble_audio_uart_handle_t aob_uart_test_handle[] = {
    {"start_adv",ble_audio_start_advertising},
}
```

5. Define handler for the command.

```
static void ble_audio_stop_adv()
{
    if(!aob_conn_stop_adv())
        TRACE(2, "%s:stop adv fail", __func__);
}
```

While sending UART commands, it should be noted that the command should be wrapped in brackets [], the first parameter of the UART command is defined when registering UART command callback function, and the second parameter is defined in the UART command table.

There are two examples of how to use UART command:

1. Use UART command without parameter.

```
[AOB,start_adv]
```

2. Use UART command with parameter. Please note that the first parameter for the handler and the handler should be separated by | , and the remaining parameters should be separated by space.

```
[AOB,update_timing|1 2 4 1 2 4 1 8]
```

5.4.4 Dynamic Frequency

The system frequency uses the vote mechanism for the choice of frequency when multiple users may request different frequencies.

Basically, the highest requested frequency will be the configured value if several users ask for different frequencies. If a user doesn't need a frequency anymore, it will release the frequency and the existing frequency request list will be went through again to choose the highest one.

- Dynamic frequency setting

Table 15 Dynamic Frequency Setting

Item	Description	Notes
enum APP_SYSFREQ_USER_T	Users requesting frequencies	The users can be found in emulator list:
enum APP_SYSFREQ_FREQ_T	Frequency	Configurable clock frequency can be found in apps\common\app_utils.h

You can use the following function to request a specific frequency.

```
int app_sysfreq_req(enum APP_SYSFREQ_USER_T user, enum APP_SYSFREQ_FREQ_T
freq);
```

By default, each user is assigned with APP_SYSFREQ_32K and the system frequency is 32KHz, which means that the system can enter into deep sleep mode.

If any user requests a frequency higher that 32KHz, the system cannot enter into deep sleep mode.

For example:

The user app_1 needs 24MHz. The system frequency is switched to 24MHz.

```
app_sysfreq_req(APP_SYSFREQ_USER_APP_1, APP_SYSFREQ_26M);
```

The user app_2 needs 48MHz. The system frequency is switched to 48MHz.

```
app_sysfreq_req(APP_SYSFREQ_USER_APP_2, APP_SYSFREQ_48M);
```

The user app_1 finishes the operation and releases the request. While app_2 still needs 48MHz, the system frequency is kept at 48MHz, and the system cannot enter into deep sleep mode.

```
app_sysfreq_req(APP_SYSFREQ_USER_APP_1, APP_SYSFREQ_32K);
```

The user app_2 finishes the operation and releases the request. No user needs active mode frequency, so the system can enter into deep sleep mode that uses low power 32KHz only.

```
app_sysfreq_req(APP_SYSFREQ_USER_APP_2, APP_SYSFREQ_32K);
```

5.4.5 Core Bridge

Core Bridge is implemented as the communication data path between the MCU and sensor hub core or M55.
The core bridge handling flow is as follows.

CONFIDENTIAL

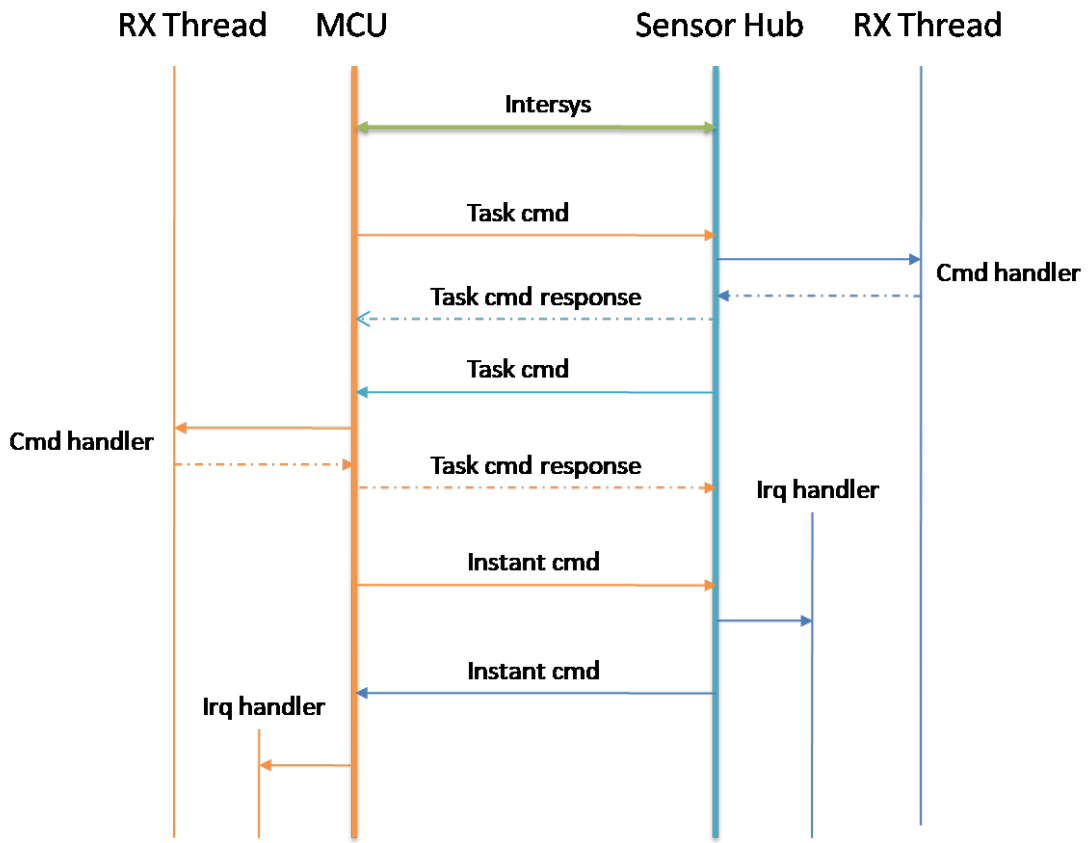


Figure 19 Core Bridge Communication Datapath between MCU and Sensor Hub Core

The MCU and sensor hub cores communicate via the hardware intersys bus. To manage the communication and support multiple communications triggered at the same time, a top level core bridge command layer is designed.

There are two kinds of core bridge commands: task command and instant command.

They differ in receiving the data from the peer core. If the data belongs to the task command, it is pushed into the queue and a signal is sent to the RX thread for command handling.

If the data belongs to the instant command, the data is immediately processed in the data receiving interrupt handler.

The task command definition is as follows:

```
typedef struct
{
    uint16_t                            cmdcode;
    const char                          *log_cmd_code_str;
    app_core_bridge_cmd_transmit_handler_t
core_bridge_cmd_transmit_handler;
    app_core_bridge_cmd_receivd_handler_t   cmdhandler;
    uint32_t                            wait_rsp_timeout_ms;
```

```
    app_core_bridge_wait_rsp_timeout_handle_t
app_core_bridge_wait_rsp_timeout_handle;
    app_core_bridge_rsp_handle_t                app_core_bridge_rsp_handle;
    app_core_bridge_cmd_transmission_done_handler_t
app_core_bridge_transmisson_done_handler;
} __attribute__((aligned(4))) app_core_bridge_task_cmd_instance_t;
```

The instant command definition is as follows:

```
typedef struct
{
    uint16_t                            cmdcode;
    app_core_bridge_cmd_transmit_handler_t
core_bridge_cmd_transmit_handler;
    app_core_bridge_cmd_receivd_handler_t   cmdhandler;
} __attribute__((aligned(4))) app_core_bridge_instant_cmd_instance_t;
```

There are two sample task command entries in “mcu_sensor_hub_app.cpp”.

- No-wait-response task command:

```
CORE_BRIDGE_TASK_COMMAND_TO_ADD(MCU_SENSOR_HUB_TASK_CMD_DEMO_REQ_NO_RSP,
                            "demo no rsp req to sensor hub core",

app_mcu_sensor_hub_transmit_demo_no_rsp_cmd_handler,

app_mcu_sensor_hub_demo_no_rsp_cmd_received_handler,
                            0,
                            NULL,
                            NULL,

app_mcu_sensor_hub_demo_no_rsp_cmd_tx_done_handler);
```

- Wait-response TWS custom command:

```
CORE_BRIDGE_TASK_COMMAND_TO_ADD(MCU_SENSOR_HUB_TASK_CMD_DEMO_REQ_WITH_RSP,
                            "demo with rsp req to sensor hub core",

app_mcu_sensor_hub_transmit_demo_wait_rsp_cmd_handler,

app_mcu_sensor_hub_demo_wait_rsp_cmd_received_handler,
                            APP_CORE_BRIDGE_DEFAULT_WAIT_RSP_TIMEOUT_MS,
```

```
app_mcu_sensor_hub_demo_wait_rsp_cmd_wait_rsp_timeout,

app_mcu_sensor_hub_demo_wait_rsp_cmd_rsp_received_handler,

app_mcu_sensor_hub_demo_with_rsp_cmd_tx_done_handler);
```

The implementation of the core bridge between the MCU and M55 is similar to that of the sensor hub.

The following example shows how to use the core bridge between the MCU and M55. The specific implementation is to define the transmitting and receiving command entry on the M55 and MCU, respectively.

The MCU defines the data transmitting command in a2dp_decoder_cc_bth.cpp.

```
RPC_CROSS_CORE_TASK_CMD_TO_ADD(CROSS_CORE_TASK_CMD_A2DP_INIT_WAITING_RSP,
                        "CC_A2DP_INIT_WAITING_RSP",
                        a2dp_decoder_bth_init_req_transmit_handler,
                        NULL,
                        3000,
                        a2dp_decoder_bth_init_wait_rsp_timeout_handler,
                        a2dp_decoder_bth_init_rsp_received_handler,
                        NULL);
```

The M55 defines the data receiving command in a2dp_decoder_cc_off_bth.cpp.

```
RPC_CROSS_CORE_TASK_CMD_TO_ADD(CROSS_CORE_TASK_CMD_A2DP_INIT_WAITING_RSP,
                        "CC_A2DP_INIT_WAITING_RSP",
                        NULL,
                        a2dp_decoder_cc_init_req_received_handler,
                        0,
                        NULL,
                        NULL,
                        NULL);
```

5.4.6 Sensor Hub

Sensor hub is a stand-alone M33 CPU which has peripheral resources like I2C/SPI/UART/etc for the external components and sensors' mounting. The sensor hub gathers sensor data and process them when the main MCU is in sleep or deep sleep mode. Considering that the sensor hub has far less hardware modules than the main MCU, by the time the system wakes up to process sensor data, the current of the sensor hub is far smaller than main MCU. Such a work mode can decrease the whole system power consumption compared with gathering and processing sensor data on main MCU.

- How to use the sensor hub

Build a sensor hub project: make T=sensor_hub -j8

Main project to add the macro to enable the sensor hub features: SENSOR_HUB=1

Trace relay for the sensor hub: If the main board has only one UART output, the sensor hub's logs are forwarded to the MCU through the core bridge to avoid the conflict between sensor hub log printing and MCU log printing. The MCU merges the sensor hub's logs into the trace buffer and prints them out together.

Macro to enable the trace relay between two projects: SENS_TRC_TO_MCU=1

The sensor hub boot-up log is shown in the following picture.

```
  6948/SENS/I/NONE  /  1 | MCU2SENS-OPEN:0: peer=0x206fffe0 (n/a) local=0x206fffe4 (0x2068f1e8)
  6948/SENS/I/NONE  /  1 | MCU2SENS-OPEN:0: rxPtr=(n/a) txPtr=0x206949a4 flowCtrl=0
  6949/SENS/I/NONE  /  1 | [multi_heap_register_impl] start=0x20690c84 size=2048 user=0x680c6f
  6950/SENS/I/NONE  /  4 | core bridge:-------TRS: cmdcode=ping mcu seq = 0 ------->
  6951/SENS/I/NONE  /  1 | app_sensor_hub_core_timer_init:
  6951/SENS/I/NONE  /  1 | SYSFREQ USER FREQ:
  6952/SENS/I/NONE  /24E | MCU2SENS-RX:0: Got peer=0x206fffe0 (0x3417969c)
  6952/SENS/I/NONE  /24E | MCU2SENS-RX:0: Got rxPtr=0x2050ea38
  6952/SENS/I/NONE  /  5 | core bridge:<-----------RCV:cmdcode=Get sensorhub crash log seq = 0 len=8-------
  6952/SENS/I/NONE  /  5 | SENS Get crash rsp command from mcu: be57eeee
  6953/SENS/I/NONE  /  5 | [app_sensor_hub_core_crash_received_cmd_handler] crash log addr 0x2069b798
  6953/SENS/I/NONE  /  4 | core bridge:-------TRS: cmdcode=Get sensorhub crash log seq = 1 ------->
 17056/SENS/I/NONE  /  2 | CPU USAGE: busy=4 light=0 sys_deep=0 chip_deep=96
 17056/SENS/I/NONE  /  4 | core bridge:-------TRS: cmdcode=ping mcu seq = 2 ------->
```

Figure 20 Log Printing of Sensor Hub Bootup

- Usage of the sensor hub

The sensor hub's project folder list: tests\sensor_hub. The main entry is in “sensor_hub_main.c”.

1. The sensor hub is running on RTOS. You can create processing threads just like on the main MCU.

2. Come to the cross core communication. A pair of cross core command handlers can be defined, one on the main MCU, and the other one on the sensor hub. The demo can be directly taken as the reference in the section 5.2.1.

The API to dynamically power on and power off the sensor hub: The sensor hub is powered on by default for sensor data processing.

```
// power off sensor hub
void app_sensor_hub_deinit(void)
// power on sensor hub
void app_sensor_hub_init(void)
// reset sensor hub
void app_mcu_reset_sesor_hub(void)
```

Note that the sensor hub can call “app_sysfreq_req” to change its CPU frequency.

```
int app_sysfreq_req(enum APP_SYSFREQ_USER_T user, enum APP_SYSFREQ_FREQ_T
freq)
{
    int ret;
```

```
    // if user is qos user
    if ((1 << user) & QOS_USERS) {
        ret = app_qosfreq_req(user, freq);
    } else { // if user is NOT qos user
        ret = hal_sysfreq_req((enum HAL_SYSFREQ_USER_T)user, (enum
HAL_CMU_FREQ_T)freq);
    }

    return ret;
}
```

5.4.7 M55

M55 is a stand-alone M55 CPU which is used for co-processing of high calculation requirement algorithms. Because of its high performance and efficient instruction sets, M55 has lower MIPS costs and lower power consumption compared with M33 when processing the algorithms with the same complexity. Generally, the music decoding and phone call algorithms are running on M55.

- How to use M55

Build a M55 project: make T=dsp_m55 -j8
Main project to add the macro to enable M55 features: DSP_M55=1

Trace relay for M55: If the main board has only one UART output, M55's logs are forwarded to the MCU through the core bridge to avoid the conflict between M55 log printing and MCU log printing. The MCU merges the M55's logs into the trace buffer and prints them out together.
Macro to enable the trace relay between two projects: DSP_M55_TRC_TO_MCU = 1, RMT_TRC_IN_MSG_CHAN = 1
The M55 boot-up log is shown in the following picture.

```
70174/DSPM/I/NONE  /  1 | M55 log main here: 247
70175/DSPM/I/NONE  /  1 | M55-BTH-OPEN:0: peer=0x2015ffe4 (n/a) local=0x2015ffe0 (0xc2002036)
70175/DSPM/I/NONE  /  1 | M55-BTH-OPEN:0: rxPtr=(n/a) txPtr=0x200e466c flowCtrl=0
70175/DSPM/I/NONE  /  1 | [multi_heap_register_impl] start=0x200e0b08 size=7680 user=0xaa813
70176/DSPM/I/NONE  /  1 | app_dsp_m55_timer_init:
70176/DSPM/I/NONE  /  1 | SYSFREQ USER FREQ:
70177/DSPM/I/CC_DEC/  9 | a2dp_decoder_cc_processor_thread
70196/DSPM/I/NONE  /40E | M55-BTH-RX:0: Got peer=0x2015ffe4 (0x3414d1a0)
70196/DSPM/I/NONE  /40E | M55-BTH-RX:0: Got rxPtr=0x2051190c
70202/DSPM/I/NONE  /  9 | off bth syspool_init: 0x20106348,0x58998
70202/DSPM/I/CC_DEC/  9 | a2dp_decoder_cc_init_handler
70202/DSPM/I/CC_DEC/  9 | codec type:0x2 ch:2, num_chnl:2, sample_rate:44100, bits:16, max_pcm:4096
70202/DSPM/I/NONE  /  9 | [off_bth_syspool_get_buff] ptr=0x20106348 size=91488 free=362904 user=0xb12ad
70203/DSPM/I/NONE  /  9 | [multi_heap_register_impl] start=0x20106348 size=91488 user=0xb12bd
70203/DSPM/I/CC_DEC/  9 | new LIST 0x200f6b60 -> 0x2010636c
70203/DSPM/I/CC_DEC/  9 | new LIST 0x200f6b6c -> 0x2010638c
70204/DSPM/I/NONE  /  9 | [off_bth_syspool_get_buff] ptr=0x2011c8a8 size=36864 free=271416 user=0xa162f
70204/DSPM/I/NONE  /  9 | [off_bth_syspool_get_buff] ptr=0x201258a8 size=14020 free=234552 user=0xa163b
70204/DSPM/I/NONE  /  9 | [off_bth_syspool_get_buff] ptr=0x20128f6c size=14020 free=220532 user=0xa1645
70204/DSPM/I/NONE  /  9 | [off_bth_syspool_get_buff] ptr=0x2012c630 size=16384 free=206512 user=0xa164f
70240/DSPM/I/NONE  /  9 | [multi_heap_register_impl] start=0x2011c8a8 size=36864 user=0xa167b
```

Figure 21 Log Printing of M55 Bootup

- Usage of M55

The M55's project folder list: tests\dsp_m55. The main entry is in dsp_m55_main.c

1. M55 is running on RTOS. You can create processing threads just like on the main MCU.

2. To use cross core communication, a pair of cross core command handlers can be defined. One is on the main MCU, and the other one is on M55. The demo can be directly taken as the reference in the section 5.2.1.

M55 defines different users in enum APP_DSP_M55_USER_E.

```
typedef enum  {
    APP_DSP_M55_USER_INIT             = 0,
    APP_DSP_M55_USER_A2DP             = 1,
    APP_DSP_M55_USER_CALL             = 2,
    APP_DSP_M55_USER_SMF_RPC           = 3,
    APP_DSP_M55_USER_DMA_AUDIO         = 4,
    APP_DSP_M55_USER_VOICE_ASSIST       = 5,
    APP_DSP_M55_USER_LEA_AUDIO         = 6,
    APP_DSP_M55_USER_AUDIO_TEST        = 7,
    APP_DSP_M55_USER_QTY
} APP_DSP_M55_USER_E;
```

The API for M55 control: When multiple users are using M55, a voting mechanism is used to ensure that M55 is powered off without any users.

```
// power on M55
void app_dsp_m55_init(APP_DSP_M55_USER_E user);
// power off M55
void app_dsp_m55_deinit(APP_DSP_M55_USER_E user);
// forced power off M55
void app_dsp_m55_force_deinit(void);
// Check whether the M55 is running
bool app_dsp_m55_is_running(void);
```

Note that M55 can call “app_sysfreq_req” in “app_utils.c” to change its CPU frequency.

5.4.8 Co-processor

There is an example to show how to add a custom calculation module into the co-processor.

In the function gaf_audio_lc3_start_cp_process, cp_accel_open is used to execute the function _cp_aob_codec_main.

```
static void gaf_audio_lc3_start_cp_process(void)
{
    if (0 == gaf_codec_cp_user_cnt)
```

```
    {

norflash_api_flush_disable(NORFLASH_API_USER_CP,(uint32_t)cp_accel_init_don
e);
        cp_accel_open(CP_TASK_AOB_CODEC, &TASK_DESC_AOB_CODEC);
        while (cp_accel_init_done() == false)
        {
            LOG_D("[%s] Delay...", __func__);
            osDelay(1);
        }
        norflash_api_flush_enable(NORFLASH_API_USER_CP);

        LOG_D("[%s]", __func__);
    }
    gaf_codec_cp_user_cnt++;
}
```

To link the function to the logical address where CP belongs, the macro CP_TEXT_SRAM_LOC is added before the function _cp_aob_codec_main.

```
CP_TEXT_SRAM_LOC
static unsigned int _cp_aob_codec_main(uint8_t event)
```

When different operations such as encode and decode need to be performed, the function cp_accel_send_event_mcu2cp is called to send an event to CP. Then, gaf_audio_lc3_start_cp_process executes the corresponding callback.

```
static int32_t gaf_codec_audio_decode_over_cp(
        GAF_CODEC_CP_PROCESSING_TYPE_E processor_type,
        uint32_t processor_func,
        void*   lc3_dec_instance,
        void*   scratch,
        void*   input_bytes,
        int32_t num_bytes,
        void*   output_samples,
        int32_t bfi_ext)
{
    gaf_codec_wait_for_cp_idle();
    gaf_codec_cp_processing_info.processing_type = processor_type;
    GAF_CODEC_CP_DECODING_INFO_T* pDecodingInfo =
        &(gaf_codec_cp_processing_info.cp_decoding_info);
```

```
    pDecodingInfo->processor_func = processor_func;
    pDecodingInfo->lc3_dec_instance = lc3_dec_instance;
    pDecodingInfo->scratch = scratch;
    pDecodingInfo->input_bytes = input_bytes;
    pDecodingInfo->num_bytes = num_bytes;
    pDecodingInfo->output_samples = output_samples;
    pDecodingInfo->bfi_ext = bfi_ext;
    gaf_codec_cp_state = GAF_CODEC_CP_STATE_IN_DECODING;
    cp_accel_send_event_mcu2cp(CP_BUILD_ID(CP_TASK_AOB_CODEC,
CP_EVENT_AOB_CODEC_PROCESSING));
    gaf_codec_wait_for_cp_idle();

    return gaf_codec_cp_processing_result;
}
```

5.4.9 Power Management

5.4.9.1 Low Power Mode

The following table describes the low power mode.

- Low power introduction:

Table 16 Lower Power Mode

Mode	Entry condition	Operation
OS is idle but still stay in active mode	- No active thread - CPU wake lock users are still active - Low power unit is still busy	In active mode
Light Sleep	- System wake lock users are still active - Active system frequency allocated and occupied by users - DMA transaction is on-going - UART trace logging is busy Register deep sleep hooker blocks the system from entering deep sleep mode	WFI
System deep sleep	- Chip wake lock users are still active	Power down everything except: - PLLs occupied by PLL users - Set sleep deep bit of Cortex-M system control register
Chip deep sleep	- No blocking item	- Power down everything

CONFIDENTIAL

The process of entering into low power mode is shown in the following figure.

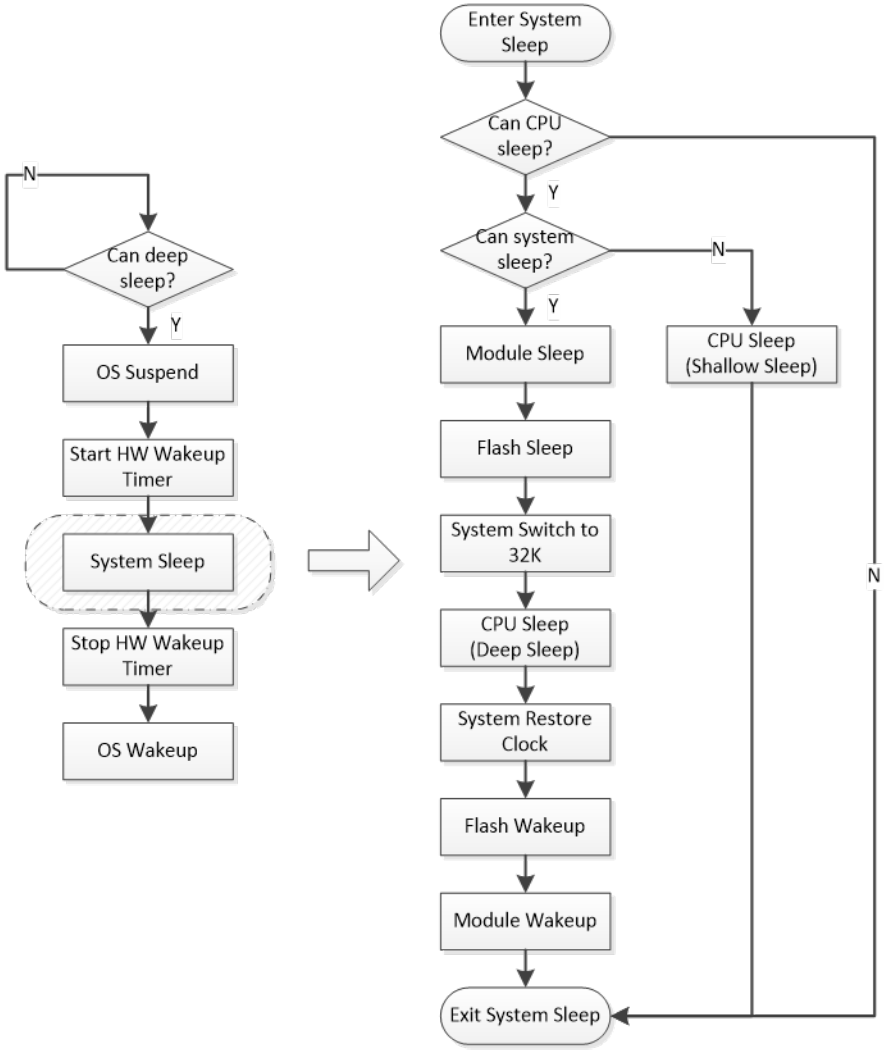


Figure 22 Sleep and Wake-up

"Can CPU sleep" checks whether there is an on-going sys clk switch.

"Can System sleep" checks the following items:

- Any CPU freq >= 24MHz occupied?
- Any software hook locks sleep mode?

Sample code to register a sleep hook user:

```
enum HAL_SLEEP_HOOK_USER_T {
    … …
    HAL_SLEEP_HOOK_TEST,  // add sleep hook user
    HAL_SLEEP_HOOK_USER_QTY
};
```

```
int test_sleep_hooker_func(void)
{
    // handling pre-enter sleep mode

    return -1; // return -1 to disallow entering sleep mode
    or
    return 0; // return 0 to allow entering sleep mode
}

void enable_sleep_hooker(void)
{
    hal_sleep_set_sleep_hook(HAL_SLEEP_HOOK_TEST,
  test_sleep_hooker_func);
}

void disable_sleep_hooker(void)
{
    hal_sleep_set_sleep_hook(HAL_SLEEP_HOOK_TEST, NULL);
}
```

- Any DMA running?

WFI wake-up sources:

- Timer: OS timer or HW timer
- VBUS event: Charger plug-in and plug-out
- GPIO Wake-up event: GPIO irq
- Intersys IRQ: Bt controller or sensor-hub traffic

- Wake lock type introduction:

Table 17 Wake Lock Types

Type	Purpose
CPU wake lock	Prevent CPU from entering WFI e.g. Across audio dma data handling
Light Sleep	WFI
System wake lock	Prevent system from entering deep sleep mode e.g. Intersys bus between bt controller and main MCU

Chip wake lock	Prevent chipset from enter deep sleep mode e.g. ANC, VAD

5.4.9.2 Power on

System power on procedure:

1. Main MCU flow 1: jump to ram run

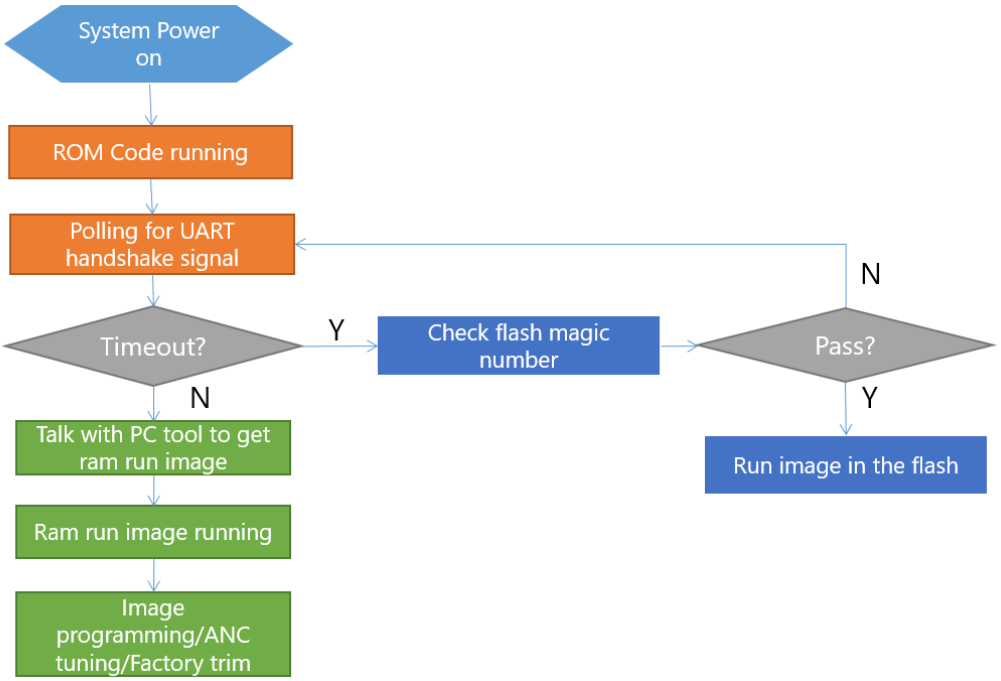


Figure 23 Jump to Ram Run

2. Main MCU flow 2: jump to the secondary bootloader in flash

CONFI

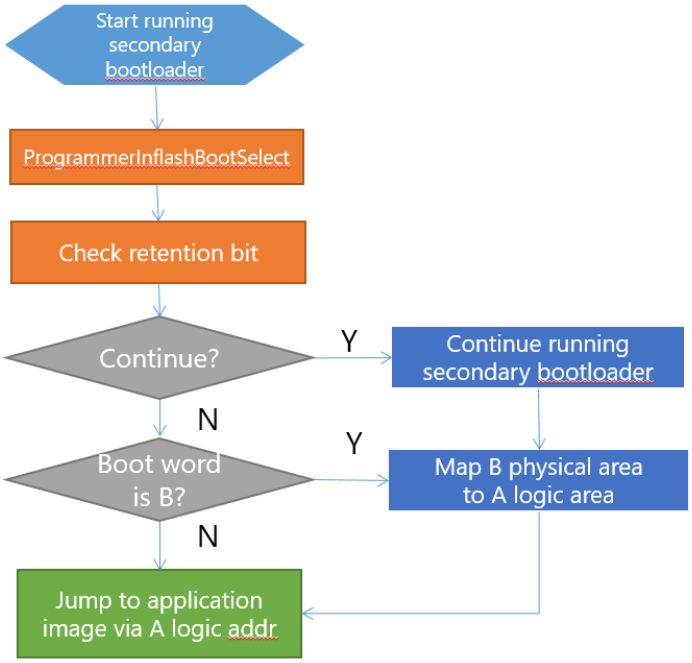


Figure 24 Jump to Secondary Bootloader in Flash

3. Main MCU flow 3: initialize the application image

CONFIDENTIAL

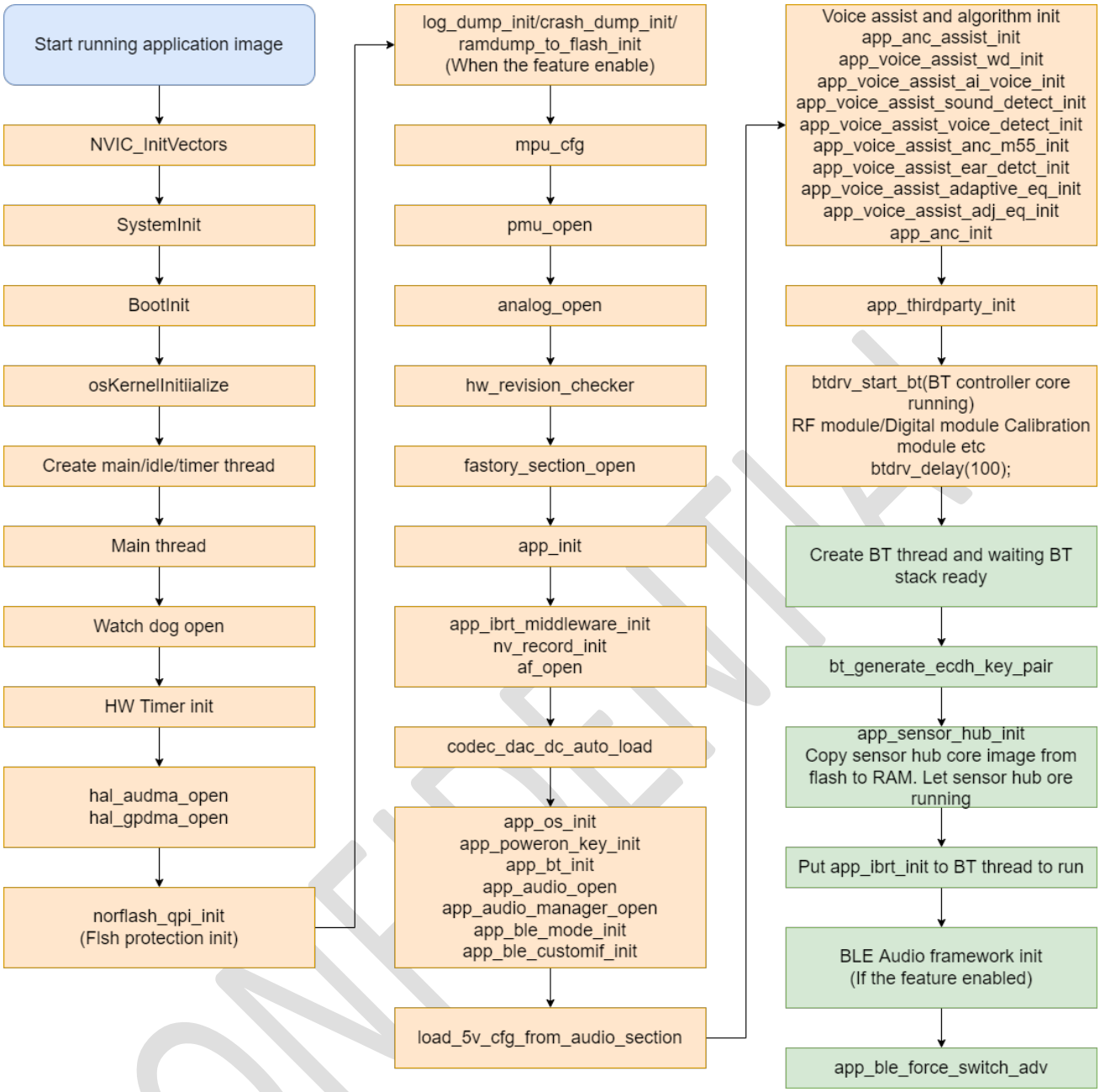


Figure 25 Application Image Init

CONFIDENTIAL

4. Sensor core flow

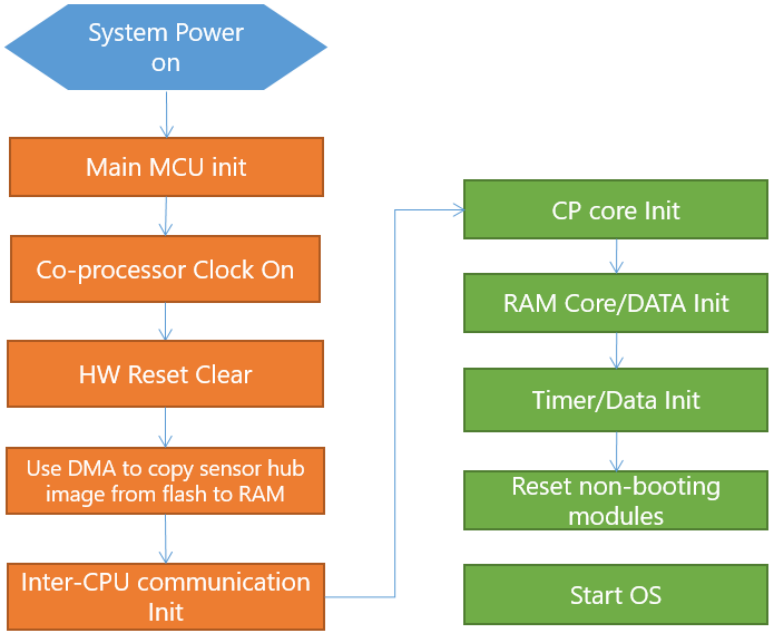


Figure 26 Sensor Core Flow

M55 and HIFI is a dynamic switch, and it does not occupy the system boot time.

There are two callbacks which can be customized. The app_reset_gpio_status is used to initialize the GPIO states to avoid the possible current leakage from the mounted external components over the GPIO. The system_power_on_callback can be customized to do anything immediately after booting.

Use the following sample code to configure GPIO_2_0 as a general GPIO in the 1.8v voltage domain, enable pull up.

```
WEAK void app_reset_gpio_status(void)
{
    struct HAL_IOMUX_PIN_FUNCTION_MAP test_pin[] =
    {
        {
            .pin = HAL_IOMUX_PIN_P2_0,
            .function = HAL_IOMUX_FUNC_AS_GPIO,
            .volt = HAL_IOMUX_PIN_VOLTAGE_MEM,
            .pull_sel = HAL_IOMUX_PIN_PULLUP_ENABLE,
        },
    };
    hal_iomux_init(&test_pin[0],
                sizeof(test_pin)/sizeof(HAL_IOMUX_PIN_FUNCTION_MAP));
```

```
}
```

Use the following code to print Hello after power-on.

```
WEAK void system_power_on_callback(void)
{
    TRACE(0,"==========Hello==========");
}
```

Functions decorated with WEAK can be redefined elsewhere. system_power_on_callback and app_reset_gpio_status can also be redefined.

5.4.9.3 Power off

In order to turn off the board, call the interface app_shudown. The specific shutdown procedure is as follows.

1. Call the interface app_shutdown. The signal flags of the main thread are set.

```
int app_shutdown(void)

TR_INFO(TR_MOD(MAIN), "system_shutdown!!");
osThreadSetPriority(main_thread_tid, osPriorityRealtime);
osSignalSet(main_thread_tid, 0x4);
```

2. After the boot, the main thread waits until any single signal flag set. Once the main thread receives the signal flag 0x4, the value of sys_case is set as 1.

```
if(evt.value.signals & 0x04)
{
    sys_case = 1;
    break;
}
```

3. The main thread calls app_deinit.

```
app_deinit(ret);
```

4. The callback system_power_off_callback can be customized to do anything before shutdown.

```
WEAK void system_power_off_callback(uint8_t sys_case)
{
    if (1 == sys_case){
        TRACE(0,"==========Poweroff==========");
    }
    else if (2 == sys_case){
```

```
        TRACE(0,"==========Reboot==========");
    }
    else {
        TRACE(0,"==========ERRROR!!!==========");
    }
}
```

5. In the end, the power manage unit is shut down.

```
pmu_shutdown();
```

5.4.9.4 Software Reset

Software reset indicates using a series of instructions to imitate the hardware reset function. In order to reset software, call the interface app_reset. The specific shutdown procedure is as follows.

1. Call the interface app_shutdown. Then, the signal flags of the main thread are set.

```
int app_reset (void)

osThreadSetPriority(main_thread_tid, osPriorityRealtime);
osSignalSet(main_thread_tid, 0x8);
```

2. After the boot, the main thread waits until any single signal flag set. Once the main thread receives the signal flag 0x8, the value of sys_case is set as 2.

```
if(evt.value.signals & 0x08)
{
    sys_case = 2;
    break;
}
```

3. The main thread calls app_deinit.

```
app_deinit(ret);
```

4. The callback system_power_off_callback can be customized to do anything before software reset

```
WEAK void system_power_off_callback(uint8_t sys_case)
{
    if (1 == sys_case){
        TRACE(0,"==========Poweroff==========");
    }
    else if (2 == sys_case){
```

```
        TRACE(0,"==========Reboot==========");
    }
    else {
        TRACE(0,"==========ERRROR!!!==========");
    }
}
```

5. In the end, the power manage unit is shut down.

```
pmu_reboot ();
```

5.4.10 Button Action Handler

There are two kinds of buttons, the normal button called GPIO button in the code and the power on button.

5.4.10.1 GPIO Button

The GPIO button can be customized by customers for key positions, pressing actions, triggering methods and functions. The following table describes the configuration items:

- GPIO button

Table 18 GPIO Button Configuration

Item	Description	Notes

struct HAL_KEY_GPIOKEY_CFG_T	GPIO button configuration	struct HAL_KEY_GPIOKEY_CFG_T {
		enum HAL_KEY_CODE_T key_code;
		struct HAL_IOMUX_PIN_FUNCTION_MAP key_config;
		enum HAL_KEY_GPIOKEY_VAL_T key_down;
		};

enum HAL_KEY_CODE_T	Key code	enum HAL_KEY_CODE_T { HAL_KEY_CODE_NONE = 0, HAL_KEY_CODE_PWR = (1 << 0), HAL_KEY_CODE_FN1 = (1 << 1), HAL_KEY_CODE_FN2 = (1 << 2), HAL_KEY_CODE_FN3 = (1 << 3), HAL_KEY_CODE_FN4 = (1 << 4), HAL_KEY_CODE_FN5 = (1 << 5), HAL_KEY_CODE_FN6 = (1 << 6), HAL_KEY_CODE_FN7 = (1 << 7), HAL_KEY_CODE_FN8 = (1 << 8), HAL_KEY_CODE_FN9 = (1 << 9),

CONFIDENTIAL

		HAL_KEY_CODE_FN10 = (1 << 10), HAL_KEY_CODE_FN11 = (1 << 11), HAL_KEY_CODE_FN12 = (1 << 12), HAL_KEY_CODE_FN13 = (1 << 13), HAL_KEY_CODE_FN14 = (1 << 14), HAL_KEY_CODE_FN15 = (1 << 15), };

enum
HAL_KEY_GPIOKEY_VAL_T
	Trigger level	enum HAL_KEY_GPIOKEY_VAL_T {
		HAL_KEY_GPIOKEY_VAL_LOW = 0,
		HAL_KEY_GPIOKEY_VAL_HIGH,
		};

struct APP_KEY_STATUS	Key status	typedef struct APP_KEY_STATUS { uint32_t code; uint8_t event; } APP_KEY_STATUS;
APP_KEY_HANDLE	Key handler	typedef struct { APP_KEY_STATUS key_status; const char* string; APP_KEY_HANDLE_CB_T function; void *param; } APP_KEY_HANDLE;

Use the following sample code to configure GPIO_1_3 as KEY1, GPIO_1_2 as KEY2 and GPIO_0_7 as KEY3, low level trigger. The macro CFG_HW_GPIOKEY_NUM is the number of GPIO buttons.

```
const struct HAL_KEY_GPIOKEY_CFG_T cfg_hw_gpio_key_cfg[CFG_HW_GPIOKEY_NUM]
= {
#if (CFG_HW_GPIOKEY_NUM > 0)
    {HAL_KEY_CODE_FN1,{HAL_IOMUX_PIN_P1_3, HAL_IOMUX_FUNC_AS_GPIO,
HAL_IOMUX_PIN_VOLTAGE_VIO, HAL_IOMUX_PIN_PULLUP_ENABLE}},
    {HAL_KEY_CODE_FN2,{HAL_IOMUX_PIN_P1_2, HAL_IOMUX_FUNC_AS_GPIO,
HAL_IOMUX_PIN_VOLTAGE_VIO, HAL_IOMUX_PIN_PULLUP_ENABLE}},
    {HAL_KEY_CODE_FN3,{HAL_IOMUX_PIN_P0_7, HAL_IOMUX_FUNC_AS_GPIO,
HAL_IOMUX_PIN_VOLTAGE_VIO, HAL_IOMUX_PIN_PULLUP_ENABLE}},
#endif
}
```

Steps to use the GPIO button:

1. Configure GPIO as button in the button mapping table.

```
const struct HAL_KEY_GPIOKEY_CFG_T
cfg_hw_gpio_key_cfg[CFG_HW_GPIOKEY_NUM] = {
#if (CFG_HW_GPIOKEY_NUM > 0)
    {HAL_KEY_CODE_FN1,{HAL_IOMUX_PIN_P1_3, HAL_IOMUX_FUNC_AS_GPIO,
HAL_IOMUX_PIN_VOLTAGE_VIO, HAL_IOMUX_PIN_PULLUP_ENABLE}},
    {HAL_KEY_CODE_FN2,{HAL_IOMUX_PIN_P1_2, HAL_IOMUX_FUNC_AS_GPIO,
HAL_IOMUX_PIN_VOLTAGE_VIO, HAL_IOMUX_PIN_PULLUP_ENABLE}},
    {HAL_KEY_CODE_FN3,{HAL_IOMUX_PIN_P0_7, HAL_IOMUX_FUNC_AS_GPIO,
HAL_IOMUX_PIN_VOLTAGE_VIO, HAL_IOMUX_PIN_PULLUP_ENABLE}},
#endif
}
```

2. Register the button handler table. For the TWS project, the table is registered in app_key_init by default.

```
void app_key_init(void)
{
#if defined(IBRT) && defined(IBRT_UI_V1)
    app_ibrt_ui_test_key_init();
#elif defined(BLE_WALKIE_TALKIE)
    app_walkie_talkie_key_init();
#elif defined(IBRT) && defined(IBRT_CORE_V2_ENABLE)
    app_tws_ibrt_raw_ui_test_key_init();
#else
    uint8_t i = 0;
    TRACE(1,"%s",__func__);

    app_key_handle_clear();
    for (i = 0; i < ARRAY_SIZE(app_key_handle_cfg); i++) {
        app_key_handle_registration(&app_key_handle_cfg[i]);
    }
#endif
}

const APP_KEY_HANDLE  app_ibrt_ui_v2_test_key_cfg[] =
{

{{APP_KEY_CODE_FN1,APP_KEY_EVENT_CLICK},"app_ibrt_service_test_key",
app_tws_ibrt_test_key_io_event, NULL},
```

```
{{APP_KEY_CODE_FN1,APP_KEY_EVENT_DOUBLECLICK},"app_ibrt_service_test_ke
y", app_tws_ibrt_test_key_io_event, NULL},

{{APP_KEY_CODE_FN2,APP_KEY_EVENT_CLICK},"app_ibrt_service_test_key",
app_tws_ibrt_test_key_io_event, NULL},

{{APP_KEY_CODE_FN2,APP_KEY_EVENT_DOUBLECLICK},"app_ibrt_service_test_ke
y", app_tws_ibrt_test_key_io_event, NULL},

{{APP_KEY_CODE_FN3,APP_KEY_EVENT_CLICK},"app_ibrt_service_test_key",
app_tws_ibrt_test_key_io_event, NULL},

{{APP_KEY_CODE_FN3,APP_KEY_EVENT_DOUBLECLICK},"app_ibrt_service_test_ke
y", app_tws_ibrt_test_key_io_event, NULL},
};

void app_tws_ibrt_raw_ui_test_key_init(void)
{
#ifdef APP_KEY_ENABLE
    TRACE(0,"app_tws_ibrt_raw_ui_test_key_init");
    app_key_handle_clear();
    for (uint8_t i=0; i<ARRAY_SIZE(app_ibrt_ui_v2_test_key_cfg); i++)
    {
        app_key_handle_registration(&app_ibrt_ui_v2_test_key_cfg[i]);
    }
#endif

#ifdef IS_ENABLE_RSSI_LOG_PRINT
    app_ibrt_rssi_print_timer_id =
osTimerCreate(osTimer(IBRT_UI_RSSI_PRINT_TIMER), \
                                osTimerPeriodic, NULL);

    osTimerStart(app_ibrt_rssi_print_timer_id, 1000);
#endif
}
```

When the button is triggered, app_tws_ibrt_test_key_io_event handles it.

```
void app_tws_ibrt_test_key_io_event(APP_KEY_STATUS *status, void *param)
```

```
{
    TRACE(0,"app_tws_ibrt_test_key_io_event");
    TRACE(3, "%s 000%d,%d",__func__, status->code, status->event);
#if defined(IBRT_UI_V2)
    switch(status->event)
    {
        case APP_KEY_EVENT_CLICK:
            if (status->code== APP_KEY_CODE_FN1)
            {
                app_ibrt_if_event_entry(IBRT_MGR_EV_CASE_OPEN);
            }
            else if (status->code== APP_KEY_CODE_FN2)
            {
                app_ibrt_if_event_entry(IBRT_MGR_EV_UNDOCK);
            }
            else
            {
                app_ibrt_if_event_entry(IBRT_MGR_EV_WEAR_UP);
            }
            break;

        case APP_KEY_EVENT_DOUBLECLICK:
            if (status->code== APP_KEY_CODE_FN1)
            {
                app_ibrt_if_event_entry(IBRT_MGR_EV_CASE_CLOSE);
            }
            else if (status->code== APP_KEY_CODE_FN2)
            {
                app_ibrt_if_event_entry(IBRT_MGR_EV_DOCK);
            }
            else
            {
                app_ibrt_if_event_entry(IBRT_MGR_EV_WEAR_DOWN);
            }
            break;

        case APP_KEY_EVENT_LONGPRESS:
            if (status->code== APP_KEY_CODE_FN1)
            {
```

```
            }
            else if (status->code== APP_KEY_CODE_FN2)
            {

            }
            else
            {
            }
            break;

        case APP_KEY_EVENT_TRIPLECLICK:
            break;

        case HAL_KEY_EVENT_LONGLONGPRESS:
            break;

        case APP_KEY_EVENT_ULTRACLICK:
            break;

        case APP_KEY_EVENT_RAMPAGECLICK:
            break;
    }
#endif
}
```

You can also customize the button event handler. The handler function can be registered by using the following code.

```
app_key_handle_clear();
for (i = 0; i < ARRAY_SIZE(app_key_handle_cfg); i++) {
    app_key_handle_registration(&app_key_handle_cfg[i]);
}
```

5.4.10.2 Power on Button

The usage is the same as GPIO button except that it doesn't need to be added into the button mapping table in tgt_hardware.c. The configuration methods can be found in 10.1

The power on button handling table registration is as follows.

```
{{APP_KEY_CODE_PWR,APP_KEY_EVENT_CLICK},"app_ibrt_ui_test_key",
app_ibrt_raw_ui_test_key, NULL},
```

5.4.11 Nor-Flash API

5.4.11.1 Flash Program/Erase

The Nor-Flash can store data and instructions. Before you use it, register flash module with the interface app_flash_register_module.

Table 19 Flash Module ID

Item	Description	Notes

enum NORFLASH_API_MODULE_ID_T	Flash module ID	enum NORFLASH_API_MODULE_ID_T
		{
		NORFLASH_API_MODULE_ID_LOG_DUMP,
		NORFLASH_API_MODULE_ID_USERDATA,
		NORFLASH_API_MODULE_ID_BOOTUP_INFO,
		NORFLASH_API_MODULE_ID_OTA,
		NORFLASH_API_MODULE_ID_UPGRADE_LOG,
		#if defined(_AUTO_TEST_)
		NORFLASH_API_MODULE_ID_CUSTOM_PARAMETER,
		NORFLASH_API_MODULE_ID_FREE,
		NORFLASH_API_MODULE_ID_RESERVED,
		NORFLASH_API_MODULE_ID_AUD,
		#endif
		NORFLASH_API_MODULE_ID_CRASH_DUMP,
		NORFLASH_API_MODULE_ID_COREDUMP,
		NORFLASH_API_MODULE_ID_RAMDUMP,
		NORFLASH_API_MODULE_ID_FACTORY,
		NORFLASH_API_MODULE_ID_HOTWORD_MODEL,
		NORFLASH_API_MODULE_ID_INTERACTION_OTA,
		NORFLASH_API_MODULE_ID_GMA_OTA,
		NORFLASH_API_MODULE_ID_AUDIO,
		NORFLASH_API_MODULE_ID_TEST1,
		NORFLASH_API_MODULE_ID_TEST2,
		NORFLASH_API_MODULE_ID_COUNT,
		};

enum NORFLASH_API_OPRATION_TYPE	Nor flash operation type.	enum NORFLASH_API_OPRATION_TYPE { NORFLASH_API_WRITTING = 0x01,

CONFIDENTIAL

		NORFLASH_API_ERASING = 0x02, NORFLASH_API_ALL = 0x03 };

The parameter module is flash module ID, dev_id can be gotten with the interface app_flash_get_dev_id_by_addr usually, baseAddr is the base address of the module, len is the length of the module, and imageHandler is a callback function which is called when the flash is flushed.

```
void app_flash_register_module(uint8_t module,
                           enum HAL_FLASH_ID_T dev_id,
                           uint32_t baseAddr,
                           uint32_t len,
                           uint32_t imageHandler);

app_flash_register_module((uint8_t)NORFLASH_API_MODULE_ID_OTA,
                      app_flash_get_dev_id_by_addr(OTA_FLASH_LOGIC_ADDR +
NEW_IMAGE_FLASH_OFFSET),
                      OTA_FLASH_LOGIC_ADDR + NEW_IMAGE_FLASH_OFFSET,
                      hal_norflash_get_flash_total_size(HAL_FLASH_ID_0) -
NEW_IMAGE_FLASH_OFFSET - 2*4096,
                      0);
```

This interface is flush pending flash operation. The parameter module is flash module ID, type is lash operation refer to enum NORFLASH_API_OPRATION_TYPE for more information.

Flash operation is usually pended in order to minimize the physical flash operation, so sometimes, the flash is not changed after you call a flash operation. This function will force the flash controller to execute pended operation.

```
void app_flash_flush_pending_op(enum NORFLASH_API_MODULE_ID_T module,
                            enum NORFLASH_API_OPRATION_TYPE type);

app_flash_flush_pending_op(mod, NORFLASH_API_ALL);
```

This interface checks it whether the flash is clean or not. The parameter is flash start address, flashOffset is length of checking. If the flash is clean, it will return true, otherwise return false.

```
bool app_flash_is_clean(uint32_t addr, uint32_t len);
```

This interface erases the whole sector (4K bytes) according to the incoming flash offset. The parameter module is flash module, flashOffset is offset according to the flash module base address.

```
void app_flash_sector_erase(enum NORFLASH_API_MODULE_ID_T module, uint32_t
flashOffset);
```

```
app_flash_sector_erase(mod, flashOffset);
```

This interface erases all blocks (32k bytes or 64k bytes) and sectors in the specified range according to the incoming flash offset and length. The parameter module is flash module, flashOffset is offset according to the flash module base address, len is erase length.

```
void app_flash_erase(enum NORFLASH_API_MODULE_ID_T module, uint32_t
flashOffset, uint32_t len);
```

This interface writes data to flash according to incoming flash module and offset. The parameter module is flash module, flashOffset is offset according to the flash module base address, ptr is pointer of data to write, len is length of data to be written, syncWrite is sync or async flag of write operation. If syncWrite is true, flash driver will flush the incoming data to flash; if syncWrite is false, the data you write probably will not present in flash immediately, if you read the flash now, you may get unexpected result. So if you want read flash right after write opeation, you should either write flash in sync way (syncWrite is ture) or flush the pending flash operation (call app_flash_flush_pending_op) after write.

```
void app_flash_program(enum NORFLASH_API_MODULE_ID_T module,
                  uint32_t flashOffset,
                  uint8_t *ptr,
                  uint32_t len,
                  bool synWrite);

app_flash_program(mod,
               offsetInFlashToProgram,
               ptrSource,
               preBytes,
               false);
```

This interface reads flash according to incoming flash module and offset. The parameter module flash module, flashOffset is offset according to the flash module base address, ptr is pointer of data to read, len is length of data to read.

```
void app_flash_read(enum NORFLASH_API_MODULE_ID_T module,
                 uint32_t flashOffset,
                 uint8_t *ptr,
                 uint32_t len);

app_flash_read(mod,
            flashOffset,
            ota_control_env.dataBufferForBurning,
```

```
            FLASH_SECTOR_SIZE_IN_BYTES);
This interface reads flash according to incoming flash module and offset.
The parameter module flash module, flashOffset is offset according to the
flash module base address, ptr is pointer of data to read, len is length of
data to read.
void app_flash_sync_read(enum NORFLASH_API_MODULE_ID_T module,
                uint32_t flashOffset,
                uint8_t *ptr,
                uint32_t len);
```

This interface gets flash id by flash module addr, the parameter addr is flash module address. Usually it is used when register module.

```
enum HAL_FLASH_ID_T app_flash_get_dev_id_by_addr(uint32_t addr);
```

All interrupts need to be suspended during flash programing/erasing. Our products support synchronous programing/erasing by default.

5.4.12.1.1 Async Flash Erasing/Programing

The async flash erase/program operation makes use of Nor-flash suspend/resume commands to allow the timing restricted threads to be scheduled if its corresponding IRQ happens during erasing/programing.

After call application flash erase/program API, if the operation queue is not full, flash erase/program into operation queue, when OS enters idle state, it will fetch operation from the queue, if the operation was suspended, OS will resume it, then excute the operation. At this time if spicific IRQ happens, the operation will be suspended until the end of IRQ handler. If the operation queue is full and it is not cleared, OS will excute operation and then push erase/program into operation queue. At the same time, f spicific IRQ happens, the operation will be suspended and delay 10ms to continue.

Following is an example of async flash erase/program.

```
app_flash_sector_erase(mod, sectorIndexInFlash *
FLASH_SECTOR_SIZE_IN_BYTES);

app_flash_program(mod,
                sectorIndexInFlash * FLASH_SECTOR_SIZE_IN_BYTES,
                ptrSource + sector * FLASH_SECTOR_SIZE_IN_BYTES,
                FLASH_SECTOR_SIZE_IN_BYTES,
                false);

app_flash_flush_pending_op(mod, NORFLASH_API_ALL);
```

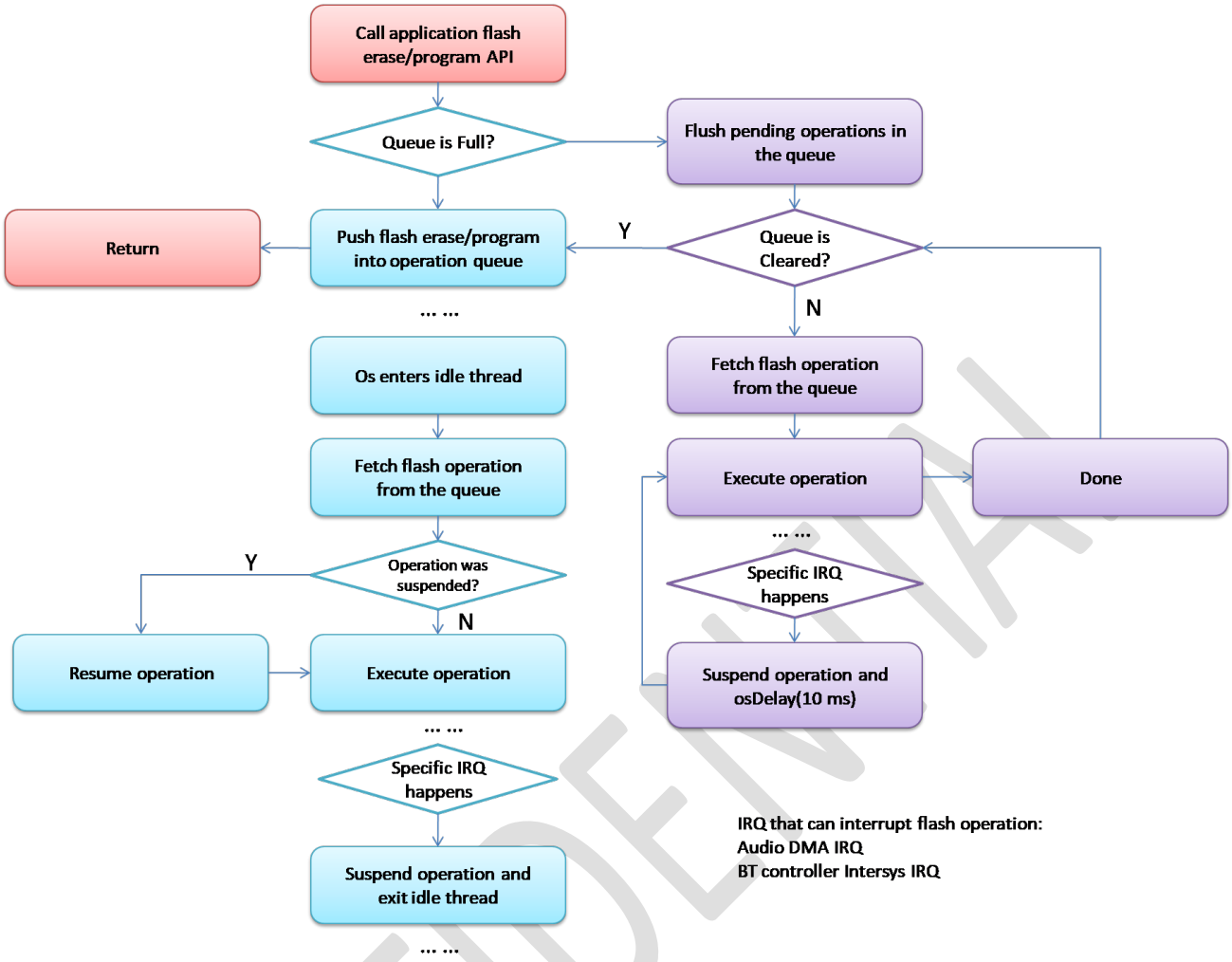


Figure 27 Async Flash Erasing/Programing

5.4.12.1.2 Sync Flash Erasing/Programing

The sync flash erase/program operation disables global IRQ and keeps waiting until the operation is completed. So if the operation costs longer time than a codec DMA interval, the codec DMA chain will miss one chunk's feeding and audio glitches may occur.

Following is an example of sync flash erase/program.

```
app_flash_sector_erase(mod, flashOffset);

app_flash_program(mod,
                flashoffset,
                ota_control_env.dataBufferForBurning,
                FLASH_SECTOR_SIZE_IN_BYTES,
                true);
```

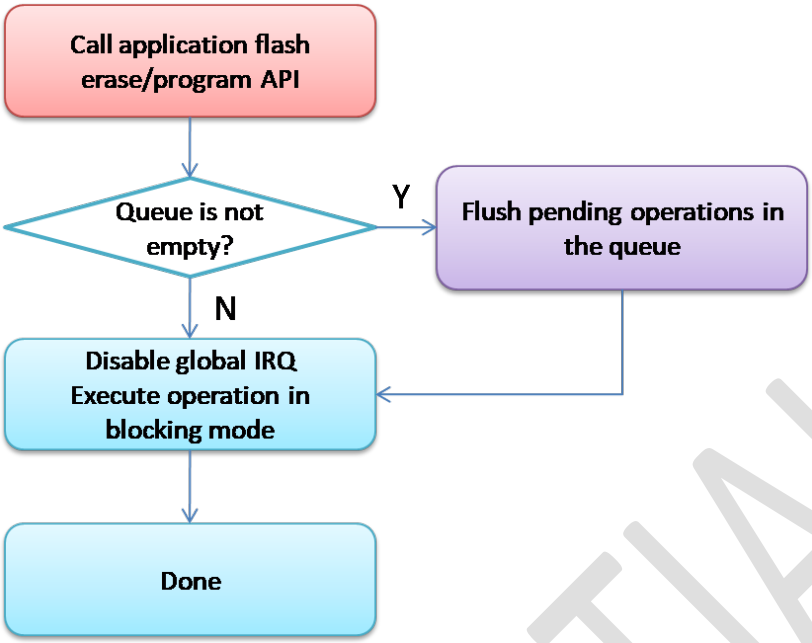


Figure 28 Sync Flash Erasing/Programing

5.4.12.1.3 Sync Flash Erasing/Programing Influence on DMA chain

Long time global IRQ disabling from sync flash erasing may cause that the audio thread misses filling PCM data into DMA buffer.

As the figure shows, if sync erase interface is called when audio thread fill PCM data into DMA Ping Buffer, global IRQ will be disabled, Ping Buffr Done DMA IRQ can not be triggered, thus audio thread cannot play PING buffer.

CONFIDENTIAL

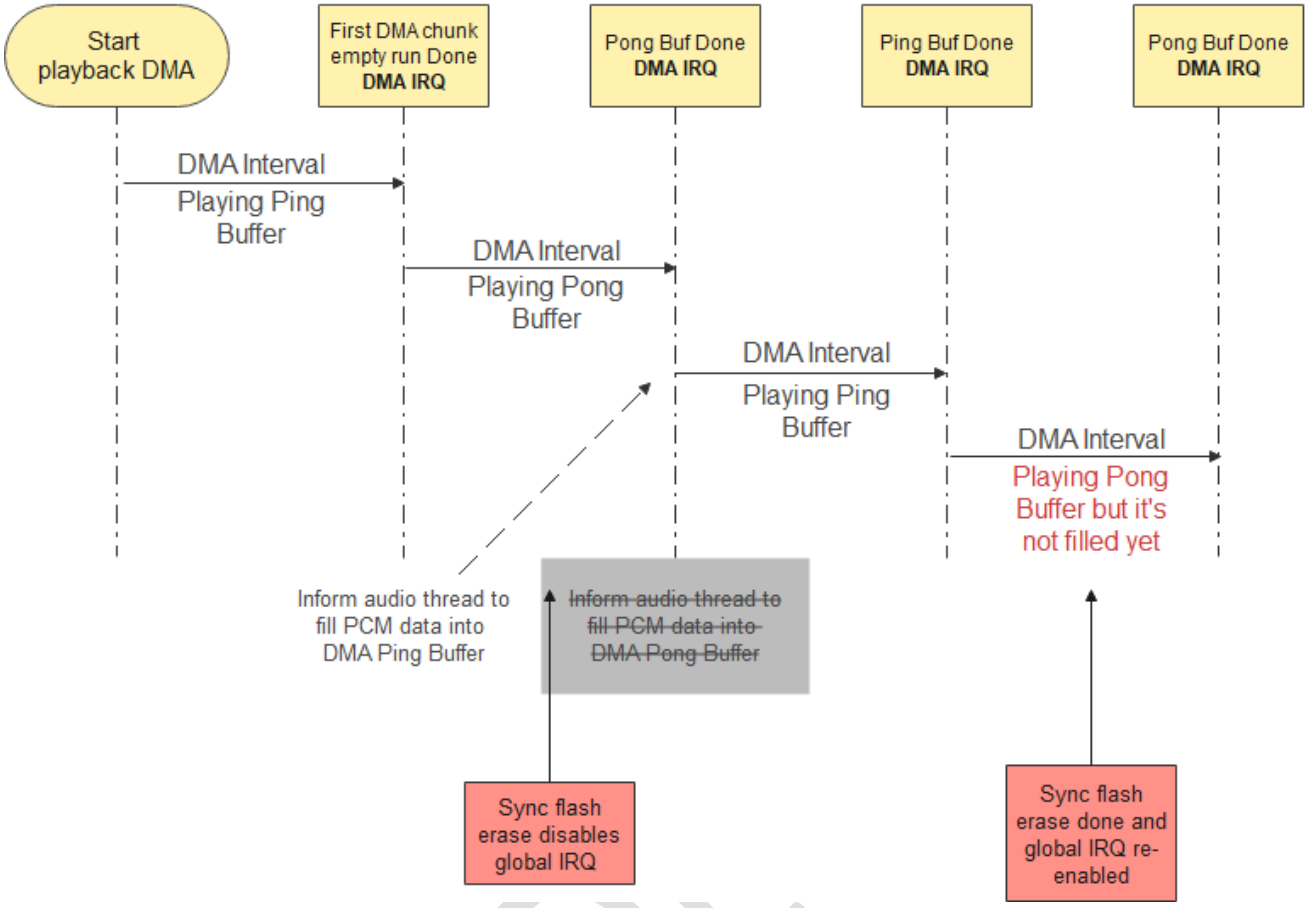


Figure 29 Sync Flash Erasing/Programing Influence on DMA Chain

5.4.11.2 Factory Section

The factory section is used to store data that never changes during the lifetime.

5.4.12.2.1 Location

For the location of the factory section, refer to 5.2 Flash memory map. The details can be found in the map. Search __factory_start in the map for the start address of the factory section and __factory_end for the end. For example, here is the location of best1600_ibrt.

```
.factory        0x00000000307ff000     0x1000
                0x00000000307ff000              __factory_start = .
                0x0000000000001000              . = 0x1000
 *fill*         0x00000000307ff000     0x1000
                0x0000000030800000              __factory_end = .
```

5.4.12.2.2 Usage

The information about data storage in the factory section is shown in the follow table.

Table 20 Factory Section Information

Item	Description	Notes
factory_section_t	Factory section	typedef struct { section_head_t head; factory_section_data_t data; }factory_section_t;

factory_section_data_t	Data stored in the factory section	typedef struct {
		unsigned char device_name[248+1] ALIGN4;
		unsigned char bt_address[8] ALIGN4;
		unsigned char ble_address[8] ALIGN4;
		// connector addr of the dldtool is the default peer bt address
		unsigned char default_peer_bt_address[8] ALIGN4;
		unsigned int xtal_fcap ALIGN4;
		unsigned int rev1_data_len;
		unsigned int rev2_data_len;
		unsigned int rev2_crc;
		unsigned int rev2_reserved0;
		unsigned int rev2_reserved1;
		unsigned int rev2_bt_name[63];
		unsigned int rev2_bt_addr[2];
		unsigned int rev2_ble_addr[2];
		// connector addr of the dldtool is the default peer bt address
		unsigned char rev2_default_peer_bt_address[6];
		unsigned char rev2_u8_reserved0;
		unsigned char rev2_default_bt_nv_role;
		unsigned int rev2_xtal_fcap;
		unsigned int rev2_ble_name[8];
		}factory_section_data_t;

The clock required for flash storage is provided by a 24MHz external crystal oscillator which can be accurately trimmed.

Here are several application programing interfaces:

CONFIDENTIAL

Get the original address of Bluetooth stored in the factory section.

```
void factory_section_original_btaddr_get(uint8_t *btAddr);
```

Set the name of Bluetooth.

```
int factory_section_set_bt_name(const char *name,int len);
```

Get the frequency of the external crystal oscillator.

```
int factory_section_xtal_fcap_get(unsigned int *xtal_fcap);
```

Set the frequency of the external crystal oscillator.

```
int factory_section_xtal_fcap_set(unsigned int xtal_fcap);
```

Get the address of Bluetooth stored in the factory section.

```
uint8_t* factory_section_get_bt_address(void);
```

Get the address of Bluetooth Low Energy stored in the factory section.

```
uint8_t* factory_section_get_ble_address(void);
```

Set the address of Bluetooth.

```
int factory_section_set_bt_address(uint8_t* btAddr);
```

Set the address of Bluetooth Low Energy.

```
int factory_section_set_ble_address(uint8_t * bleAddr);
```

Get the name of Bluetooth.

```
uint8_t* factory_section_get_bt_name(void);
```

Get the name of Bluetooth Low Energy.

```
uint8_t* factory_section_get_ble_name(void);
```

Get current version.

```
uint32_t factory_section_get_version(void);
```

Get the address of the default peer.

```
uint8_t* factory_section_get_default_peer_bt_address(void);
```

Get the default Bluetooth non-volatile memory role.

CONFIDENTIAL

```
uint8_t factory_section_get_default_bt_nv_role(void);
```

5.4.11.3 User Section

The user section is used to store data that does not change during use.

5.4.11.3.1 Location

For the location of the user section, refer to 5.2 Flash memory map. The details can be found in the map. Search __userdata_start in the map for the start address of the user section and __userdata_end for the end. For example, here is the location of best1600_ibrt.

```
.userdata      0x00000000307fc000    0x2000
               0x00000000307fc000             __userdata_start = .
               0x0000000000002000             . = 0x2000
 *fill*        0x00000000307fc000    0x2000
               0x00000000307fe000             __userdata_end = .
```

5.4.11.3.2 Usage

Use the following interface to get extension entries.

```
NV_EXTENSION_RECORD_T *nv_record_get_extension_entry_ptr(void);
```

In order to update the user data section, call the interface nv_record_btdevicevolume_set_hfp_vol.

```
void nv_record_btdevicevolume_set_hfp_vol(btdevice_volume* device_vol,
int8_t vol)
{
    uint32_t lock = nv_record_pre_write_operation();
    if (vol != device_vol->hfp_vol)
    {
        nv_record_update_runtime_userdata();
        device_vol->hfp_vol = vol;
    }
    nv_record_post_write_operation(lock);
}
```

The times of flash programing/erasing are limited, data in the flash memory is flushed only when dirty data is available. The dirty user data section may be flushed into the flash in the following cases:

1. Normal shut-down
2. Normal reset
3. Crash and reboot
4. Initiative sync flushing

```
void nv_record_flash_flush(void);
```

5. Initiative async flushing

```
void nv_record_execute_async_flush(void);
```

6. Periodical self-check and flushing when the OS is in the idle state

```
#define NV_RECORD_FLUSH_EXECUTION_INTERVAL_MS  (20*60*1000)
```

5.4.11.4 Custom Parameter Section

This section provides an interface to save and acquire the parameters unique by device and non-volatile across the product's lifetime.

Location of custom parameters

The location is fixed in the flash, ahead of the user data sector.

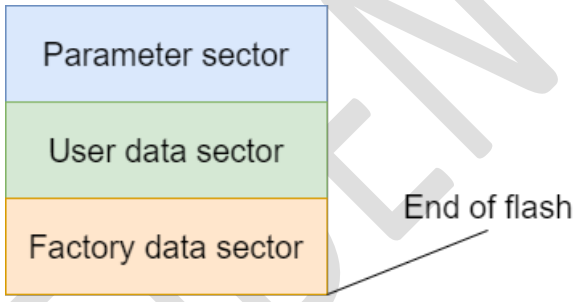


Figure 30 Location of Parameter Sector

Data format of custom parameters

Table 21 Data Format of Custom Parameters

Magic number	Version number	Total length	Parameter entries count	Parameter entry 0 - (count-1)		
				Parameter index	Parameter length	Parameter content
4 bytes	2 bytes	2 bytes	2 bytes	2 bytes	2 bytes	Depends on parameter length

Define parameters in the file with the .csv suffix. Here is an example:

Magic number,"0x42,0x45,0x53,0x54",

Version,1,

SN,0,"0x00, 0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,0x09,x00a,0x0b,0x0c,0x0d,0x0e,0x0f"

CONFIDENTIAL

Table 22 Example of Custom Parameters

Magic number	Value, small endian. E.g. “0x42,0x45,0x53,0x54” means that value of the parameter is: parameter[0]=0x42, parameter[1]=0x45,parameter[2]=0x53,parameter[3]=0x54	
Version	Value	
Name of the parameter entry	Index of the parameter entry	Value of the parameter, small endian. E.g. “0x12, 0x34” means that value of the parameter is: parameter[0]=0x12, parameter[1]=0x34

Steps to download the product serial number:

1. Fill the custom parameter into custom_parameters.csv
2. Generate the custom parameter binary file via ~$ python .\custom_parameters_bin_convertor.py custom_parameters.csv
3. Program the generated binary file into the fixed flash offset (FLASH_SIZE-0x3000-AUD_SECTION_SIZE). Note that the dldtool since rev 1.2.1 can support this feature. The "ChipVersion" needs to be chosen as "1000" for 2000 series, while "2000" for 2000i series.
4. Define the custom parameters in customparam_section.c/h

In the firmware, call nv_custom_parameter_section_get_entry to get the parameter content.

5.4.12 PMU Watchdog

The watchdog is a timer circuit. The watchdog counter starts to count automatically when the system is running. If the watchdog is not cleared after the specified time, the watchdog counter will overflow, which causes the watchdog to interrupt and the system to reset.

The watchdog can make the MCU system work non-stop even in an unguarded state, preventing the program from running away or falling into an infinite loop. The PMU watchdog can also protect the power and data.

In order to open a PMU watchdog, call the interface app_wdt_open. Note that the time set by app_wdt_open is 2.2 times the incoming parameter.

```
int app_wdt_open(int seconds)
```

After opening the watchdog, it needs to be cleared before the watchdog counter reaches the maximum value. The interface watchdog_ping is used to feed the watchdog.

```
void watchdog_ping(void)
```

Since the watchdog handler is handled by the SDK, the application code does not need to call watchdog_ping unless otherwise required.

CONFIDENTIAL

5.4.13 TWS Synchronization Operation API

The synchronization operation is required between the left and right of TWS devices. The code for TWS synchronization is as follows.

Table 23 TWS Synchronization Operation Code

Item	Description	Notes
typedef enum APP_BT_SYNC_OP_CODE_E	Bluetooth synchronous operation code	typedef enum { APP_BT_SYNC_OP_RETRIGGER = 0x00000000, APP_BT_SYNC_OP_VOLUME = 0x00000001, APP_BT_SYNC_OP_VOLUME_OFFSET = 0x00000002, } APP_BT_SYNC_OP_CODE_E;

This macro addes commands to a table. opCode is the TWS synchronization operation code, cmdhandler is the command handler, and statusNotify is the synchronization status notification

```
#define APP_BT_SYNC_COMMAND_TO_ADD(opCode, cmdhandler, statusNotify)
\
    static const app_bt_sync_instance_t syncOp##opCode##_entry
\
    __attribute__((used, section(".app_bt_sync_command_handler_table"))) =
\
    {(opCode), (cmdhandler), (statusNotify)};
```

Use the following function to enable the triggering feature of both buds. opCode is the pointer to the expected callback handler, length is the length of information shared between TWS devices, and p_buff is the pointer to the shared information.

```
bool app_bt_sync_enable(uint32_t opCode, uint8_t length, uint8_t *p_buff)
```

Here is an example of registering a synchronization volume state handler and enabling it.

```
APP_BT_SYNC_COMMAND_TO_ADD(APP_BT_SYNC_OP_VOLUME,
aob_tws_sync_volume_status_handler, NULL);

if (!app_bt_sync_enable(APP_BT_SYNC_OP_VOLUME,
sizeof(app_gaf_arc_vcs_volume_ind_t), (uint8_t*)&p_volume_ind)){
    aob_tws_sync_volume_status_handler();
```

```
}
```

5.4.14 Dynamic Heap Usage

When the size of the heap is found to be too small, it can be dynamically adjusted.

The heap information is as follows:

Table 24 Heap Information

Item	Description	Notes

typedef struct multi_heap_info	Metadata header for the heap, stored at the beginning of the heap space	typedef struct multi_heap_info {
		void *lock;
		size_t total_bytes;
		size_t free_bytes;
		size_t minimum_free_bytes;
		heap_block_t *last_block;
		heap_block_t first_block; /* initial 'free block', never allocated */
		#if defined(MULTI_HEAP_DEFAULT_INT_LOCK)
		size_t int_lock;
		#endif
		};

The following interfaces can be called to dynamically resize the heap:

1. This function initializes a heap at the specified address, and returns a handle for future heap operations. The parameter start is the start address of the memory to use for a new heap, and the parameter size is the size (in bytes) of the new heap. The return value is the handle of a new heap ready for use, or NULL if the heap region is too small to be initialized.

```
typedef struct multi_heap_info *multi_heap_handle_t;
multi_heap_handle_t heap_register(void *start, size_t size);
```

2. This function allocates a buffer in a given heap. Its semantics are the same as standard malloc(), and the returned buffer is allocated in the specified heap. The parameter heap is a handle to a registered heap, and the parameter size is the size of the desired buffer.

```
bool heap_malloc (multi_heap_handle_t heap, size_t size);
```

3. This function frees a buffer in a given heap. Semantics are the same as standard free(), but the argument 'p' must be NULL or have been allocated in the specified heap. The parameter heap is the handle to a registered heap.

```
void heap_free(multi_heap_handle_t heap, void *p);
```

CONFIDENTIAL

4. This function checks if it is available to allocate a buffer in a given head. The parameter heap is a handle to a registered heap, and the parameter size is the size of the desired buffer.

```
bool heap_check_size_malloc_available(multi_heap_handle_t heap, size_t
size);
```

5. This function returns the size that a particular pointer is allocated with. The parameter heap is a handle to a registered heap. The parameter p is a pointer, and it must have been previously returned from heap_malloc or hrap_relloc for the same heap. The return value is the size of the memory allocated at this block.

```
size_t heap_get_allocated_size(multi_heap_handle_t heap, void *p);
```

5.4.15 Retention Information Management

There are two ways to retain certain information during the reboot, retention bits and retention ram.

5.4.15.1 Retention Bits

There are 32 bits that can be set before reboot. The mode can be found in platform\hal\hal_bootmode.h.

In order to set a bit before reboot, call the interface hal_sw_bootmode_set. Use the following code to set boot mode as HAL_SW_BOOTMODE_REBOOT and HAL_SW_BOOTMODE_REBOOT_FROM_CRASH after the crash.

```
#ifdef CRASH_REBOOT
hal_sw_bootmode_set(HAL_SW_BOOTMODE_REBOOT|HAL_SW_BOOTMODE_REBOOT_FROM_CRAS
H);
```

After rebooting, call the interface hal_sw_bootmode_get get the bit that has been set.

```
if (hal_sw_bootmode_get() & HAL_SW_BOOTMODE_REBOOT_FROM_CRASH){
    hal_sw_bootmode_clear(HAL_SW_BOOTMODE_REBOOT_FROM_CRASH);
    TRACE(0,"Crash happened!!!");
#ifdef VOICE_DATAPATH
    gsound_dump_set_flag(true);
#endif
}
```

5.4.15.2 Retention Ram Usage

In order to remain certain information in ram, a section can be defined in the link script.

```
    .reboot_param (NOLOAD) :
    {
        KEEP(*(.reboot_param))
```

```
        . = REBOOT_PARAM_SECTION_SIZE;
        . = ALIGN(4);
#if defined(ASIC_SIMU)
        . = ALIGN(1024*4);
#endif
    } > RAM
```

Then use attribute to modify the information you want to keep during rebooting in the code.

```
typedef struct
{
    uint32_t magicCode;
    uint8_t  bleBdAddr[BLE_BD_ADDR_LEN];
} BLE_BD_ADDR_FOR_OTA_T;

BLE_BD_ADDR_FOR_OTA_T __attribute((section(".reboot_param")))
bleBdAddrForOta;
```

If the size of this section is not enough, modify the value of REBOOT_PARAM_SECTION_SIZE or redefine a paragraph. The method of defining a section can be found in 5.5.

5.5 RTOS

5.5.1 Multi-task

In CMSIS-RTOS2, the basic unit of execution is a "Thread". Once started, an RTOS thread must contain a loop so that it never terminates, i.e. it runs forever. With the Arm Compiler, it is possible to optimize a thread by using a __NO_RETURN macro. This attribute reduces the cost of calling a function that never returns.

An RTOS program is made up of a number of threads, which are controlled by the RTOS scheduler. This scheduler uses the SysTick timer to generate a periodic interrupt as a time base. The scheduler does not allot a certain amount of execution time to each thread. So thread1 runs for one system tick period and then be re-scheduled to allow thread2 to run for a similar period; thread2 gives way to thread3 and finally controls passes back to thread1. By allocating these slices of runtime to each thread in a round-robin fashion, you can get the appearance of all three threads running in parallel to each other.

Conceptually, you can think of each thread as performing a specific functional unit of our program with all threads running simultaneously. When a thread is created, it is also allocated with its own thread ID. This is a variable which acts as a handle for each thread and is used when you want to manage the activity of the thread.

5.5.2 Task Scheduler

RTX5 implements a low-latency preemptive scheduler. Major parts of RTX5 are executed in handler mode: SysTick_Handler used for time-based scheduling.

CONFIDEN

SVC_Handler used for lock-based scheduling.

PendSV_Handler used for interrupt-based scheduling.

In order to be low-latency with respect to ISR execution those system exceptions are configured to use the lowest priority groups available. The priorities are configured so that no preemption happens between them. Thus no interrupt critical sections (i.e. interrupt locks) are needed to protect the scheduler.

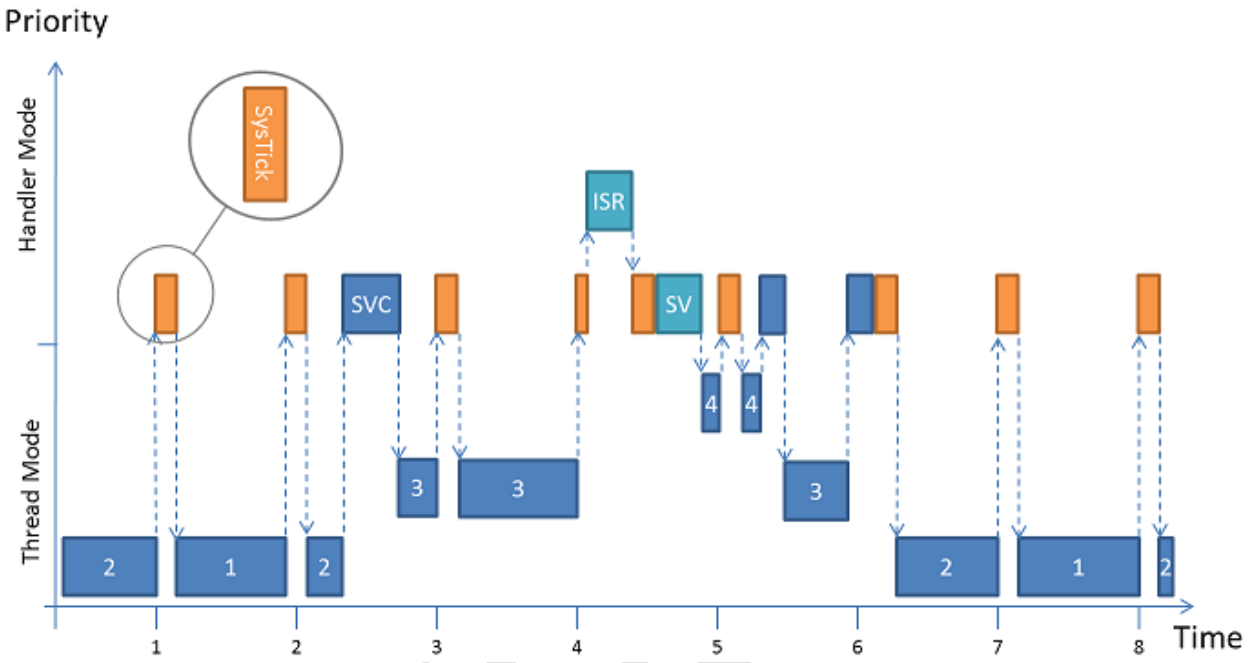


Figure 31 Thread Scheduling and Interrupt Execution

The scheduler combines priority and round-robin based context switches. The example depicted in the preceding figure contains four threads (1, 2, 3, and 4). Threads 1 and 2 share the same priority, thread 3 has a higher one and thread 4 the highest (osThreadAttr_t::priority). As long as threads 3 and 4 are blocked, the scheduler switches between thread 1 and 2 on a time-slice basis (round-robin).

Thread 2 unblocks thread 3 by an arbitrary RTOS-call (executed in SVC handler mode) at time index 2. The scheduler switches to thread 3 immediately because thread 3 has the highest priority. Thread 4 is still blocked.

At time index 4, an interrupt (ISR) occurs and preempts the SysTick_Handler. RTX does not add any latency to the interrupt service execution. The ISR routine uses an RTOS-call that unblocks thread 4. Instead of switching to thread 4 immediately, the PendSV flag is set to defer the context switching. The PendSV_Handler is executed right after the SysTick_Handler returns and the deferred context switch to thread 4 is carried out. As soon as highest priority thread 4 blocks again by using a blocking RTOS-call, the execution is switched back to thread 3 immediately during time index 5.

At time index 5, thread 3 uses a blocking RTOS-call as well. Thus the scheduler switches back to thread 2 at time index 6. At time index 7, the scheduler uses the round-robin mechanism to switch to thread 1.

CONFIDENTIAL

5.5.3 OS Abstract Layer

OS Abstract Layer is a generic API that is agnostic of the underlying RTOS kernel. Application programmers call CMSIS-RTOS2 API functions in the user code to ensure maximum portability from one RTOS to another. Middleware using OS Abstract Layer takes advantages of this approach by avoiding unnecessary porting efforts.

For the application layer, you can call “osThreadCreate” to create a thread's external interface, but the implementation of this function is different for different OS (FreeRTOS, Nuttx, RTX5, etc.). In this way, the application layer code does not need to be changed when porting between different OS.

In “cmsis_os1.c”

```
osThreadId osThreadCreate (const osThreadDef_t *thread_def, void *argument)
{

  if (thread_def == NULL) {
    return (osThreadId)NULL;
  }
  return osThreadNew((osThreadFunc_t)thread_def->pthread, argument,
&thread_def->attr);
}
```

In “nuttx_os.c”

```
osThreadId osThreadCreate (osThreadDef_t *thread_def, void *argument)
{
    swarn("thread name:%s\n",thread_def->name);
    pthread_attr_t attr = {0};
    thread_def->mutex = (pthread_mutex_t *)malloc(sizeof(pthread_mutex_t));
    thread_def->cond = (pthread_cond_t *)malloc(sizeof(pthread_cond_t));
    pthread_mutex_init(thread_def->mutex, NULL);
    pthread_cond_init(thread_def->cond,NULL);
    pthread_attr_init(&attr);
    thread_def->argument = argument;
    attr.priority = thread_def->priority;
    attr.stacksize = thread_def->stacksize;
#ifndef  CHIP_A7_DSP
#if USE_PTHREAD_PORTING
    int ret =
pthread_create(&thread_def->thread,&attr,pthread_proxy,thread_def);
    swarn("thread name:%s, thread=%d,ret=%d,
stackaddr:%x\n",thread_def->name,thread_def->thread,ret,attr.stackaddr);
```

```
    if (ret != OK)
        serr("ret=%d,errno=%d  ERROR PLEASE CHECK!!!!!\n",ret,errno);
#else
    int ret;
    void *argv[3];
    char arg1[16];

    snprintf(arg1, 16, "0x%" PRIxPTR, (uintptr_t)thread_def);

    argv[0] = "";
    argv[1] = arg1;
    argv[2] = NULL;

    /* Create the task */

    ret = kthread_create_with_stack(thread_def->name, thread_def->priority,
attr.stackaddr, attr.stacksize, main_proxy, (char *const*)argv);
    if (ret < OK)
      {
        serr("ret=%d,errno=%d  ERROR PLEASE CHECK!!!!!\n",ret,errno);
        return NULL;
      }

    thread_def->thread = ret;
#endif
    return (osThreadId)thread_def;
}
```

5.5.4 Create a Task

As we can see in the first example, the “app_main()” thread is used as a launcher thread to create the application threads. This is done in two stages. First, a thread structure is defined. The thread structure allows us to define the thread operating parameters.

```
typedef struct {
  const char                *name;   ///< name of the thread
  uint32_t               attr_bits;   ///< attribute bits
  void                   *cb_mem;    ///< memory for control block
  uint32_t                 cb_size;   ///< size of provided memory for
control block
```

```
  void                *stack_mem;    ///< memory for stack
  uint32_t             stack_size;   ///< size of stack
  osPriority_t            priority;   ///< initial thread priority (default:
osPriorityNormal)
  TZ_ModuleId_t          tz_module;   ///< TrustZone module identifier
  uint32_t               reserved;   ///< reserved (must be 0)
} osThreadAttr_t;
```

The thread structure requires us to define the name of the thread function, its thread priority, any special attribute bits, its TrustZone_ID and its memory allocation. Once the thread structure is defined, the thread can be created by calling the osThreadNew() API. Then the thread is created from within the application code.

```
#if !defined(os1_Disable_Thread)
osThreadId osThreadCreate (const osThreadDef_t *thread_def, void *argument)
{

  if (thread_def == NULL) {
    return NULL;
  }
  return osThreadNew((osThreadFunc_t)thread_def->pthread, argument,
&thread_def->attr);
}
#endif
```

Sample Code 7: OS_Thread Creation

{"os_thread_create_demo", os_thread_create_demo_test},

[TEST,os_thread_create_demo]

5.5.5 OS Event

Event flags operate in a similar fashion to thread flags but must be created and then act as a global RTOS object that can be used by all the running threads.

First, create a set of event flags. This is a similar process to creating a thread. You can define an event flag attribute structure. The attribute structure specifies an ASCII name string, attribute bits, and memory detention if we are using the static memory model.

```
typedef struct {
  const char               *name;   ///< name of the event flags
  uint32_t              attr_bits;   ///< attribute bits
  void                  *cb_mem;    ///< memory for control block
```

```
  uint32_t                cb_size;   ///< size of provided memory for
control block
} osEventFlagsAttr_t;
```

Next, a handle to control access of the event flags shall be created. Then we can create the event flag object:

```
osEventFlagsId_t osEventFlagsNew (const osEventFlagsAttr_t *attr) {
  EventGroupHandle_t hEventGroup;
  int32_t mem;

  hEventGroup = NULL;

  if (!IS_IRQ()) {
    mem = -1;

    if (attr != NULL) {
      if ((attr->cb_mem != NULL) && (attr->cb_size >=
sizeof(StaticEventGroup_t))) {
        mem = 1;
      }
      else {
        if ((attr->cb_mem == NULL) && (attr->cb_size == 0U)) {
          mem = 0;
        }
      }
    }
    else {
      mem = 0;
    }

    if (mem == 1) {
      hEventGroup = xEventGroupCreateStatic (attr->cb_mem);
    }
    else {
      if (mem == 0) {
        hEventGroup = xEventGroupCreate();
      }
    }
  }
```

```
  return ((osEventFlagsId_t)hEventGroup);
}
```

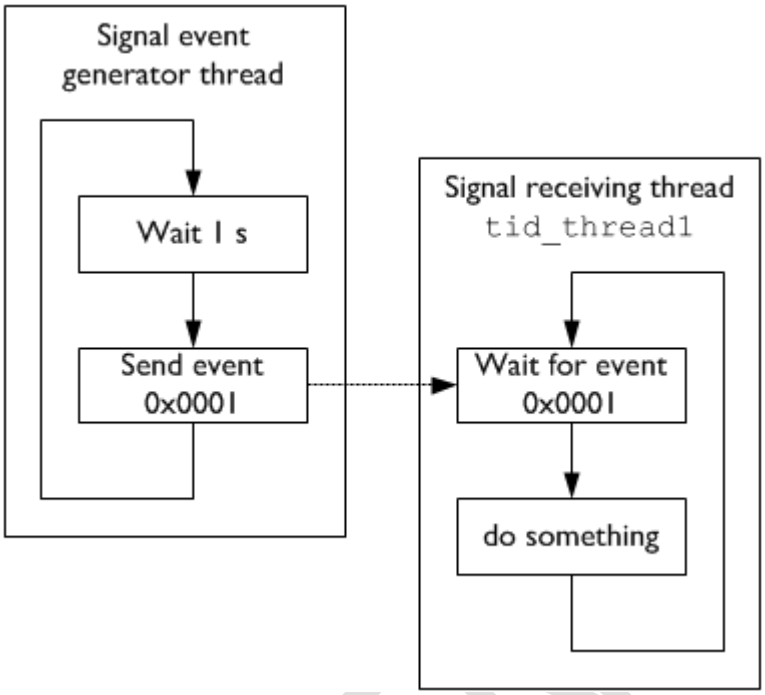


Figure 32 Simple Event Communication

Here is a simple example to show how two threads can communicate with each other using event flags.

Sample Code 8 : OS Event

{"os_event_demo", os_event_demo_test},

[TEST, os_event_demo]

5.5.6 OS Mailbox

To create a zero copy mail box system, you can combine a memory pool to store the data with a message queue which is used to transfer a pointer of the allocated memory pool slot. This way, the message data stays static and a pointer is passed between threads.

```
typedef struct os_mailQ_def {
  uint32_t                queue_sz;   ///< number of elements in the queue
  uint32_t                 item_sz;   ///< size of an item
  void                      *mail;   ///< pointer to mail
  osMemoryPoolAttr_t        mp_attr;   ///< memory pool attributes
  osMessageQueueAttr_t      mq_attr;   ///< message queue attributes
} osMailQDef_t;
```

The Mailbox can be created by calling the osMailCreate() API.

```
osMailQId osMailCreate (const osMailQDef_t *queue_def, osThreadId
thread_id) {
  os_mail_queue_t *ptr;
  (void)thread_id;

  if (queue_def == NULL) {
    return (osMailQId)NULL;
  }

  ptr = queue_def->mail;
  if (ptr == NULL) {
    return (osMailQId)NULL;
  }

  ptr->mp_id = osMemoryPoolNew  (queue_def->queue_sz, queue_def->item_sz,
&queue_def->mp_attr);
  ptr->mq_id = osMessageQueueNew(queue_def->queue_sz, sizeof(void *),
&queue_def->mq_attr);
  if ((ptr->mp_id == (osMemoryPoolId_t)NULL) || (ptr->mq_id ==
(osMessageQueueId_t)NULL)) {
    if (ptr->mp_id != (osMemoryPoolId_t)NULL) {
      osMemoryPoolDelete(ptr->mp_id);
    }
    if (ptr->mq_id != (osMessageQueueId_t)NULL) {
      osMessageQueueDelete(ptr->mq_id);
    }
    return (osMailQId)NULL;
  }

  return (osMailQId)ptr;
}
```

Here is an example to show the simple use of mailbox: Different users use app_mailbox_pull to fill in information to the structure. “app_thread” takes the information through app_mailbox_get from the structure and stores it into mod_handler.

```
static void app_thread(void const *argument)
{
    while(1){
        APP_MESSAGE_BLOCK *msg_p = NULL;
```

```
        if (!app_mailbox_get(&msg_p)) {
            if (msg_p->mod_id < APP_MODUAL_NUM) {
                if (mod_handler[msg_p->mod_id]) {
                    int ret = mod_handler[msg_p->mod_id](&(msg_p->msg_body));
                    if (ret)
                        TRACE(2,"mod_handler[%d] ret=%d", msg_p->mod_id, ret);
                }
            }
            app_mailbox_free(msg_p);
        }
    }
}
```

5.5.7 OS Timer

The CMSIS-RTOS API can be used to define any number of virtual timers which act as count down timers. When they expire, they run a user call-back function to perform a specific action. Each timer can be configured as a one shot or repeat timer. A virtual timer is created by first defining a timer structure. The timer must then be instantiated by an RTOS thread.

Note that do not use osDelay and time-consuming operations in the timer callback function, which may block other timer responses. It is recommended to send a message directly to the processing thread through the timer to trigger task processing.

This creates the timer and defines it as a periodic timer (osTimerPeriodic) or a single shot timer (osTimerOnce). The next parameter passes an argument to the callback function when the timer expires:

```
osTimerId osTimerCreate (const osTimerDef_t *timer_def, os_timer_type type,
void *argument) {

  if (timer_def == NULL) {
    return (osTimerId)NULL;
  }
  return osTimerNew((osTimerFunc_t)timer_def->ptimer, type, argument,
&timer_def->attr);
}
```

The timer can then be started at any point in a thread the timer start function invokes the timer by its handle and defines a count period in kernel ticks. Each timer can be configured as a one-shot or a periodic timer. A periodic timer repeats its operation until it is deleted or stopped. All timers can be started, restarted, or stopped.

```
// start a timer
osStatus_t osTimerStart (osTimerId_t timer_id, uint32_t ticks);
// stop a timer
osStatus_t osTimerStop (osTimerId_t timer_id);
```

Sample Code 9: OS Timer

{"os_timer_demo", os_timer_demo_test},

[TEST, os_timer_demo]

5.5.8 OS Semaphore

Semaphores are used to manage and protect access to shared resources. Semaphores are very similar to Mutexes. Whereas a Mutex permits just one thread to access a shared resource at a time, a semaphore can be used to permit a fixed number of threads/ISRs to access a pool of shared resources. Using semaphores, access to a group of identical peripherals can be managed (such as multiple DMA channels).

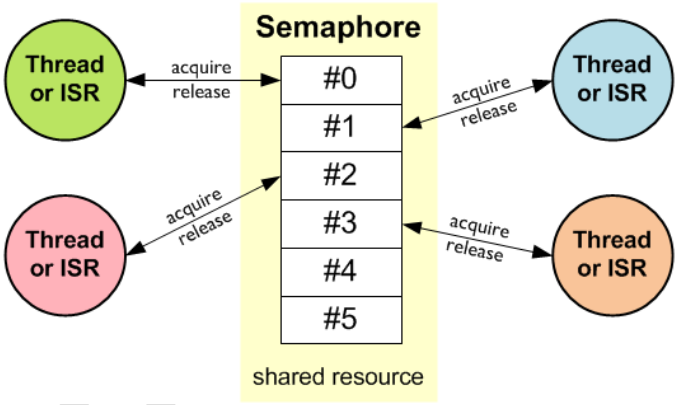


Figure 33 CMSIS-RTOS Semaphore

To use a semaphore in the CMSIS-RTOS, you must first declare the semaphore attributes.

```
typedef struct {
  const char                *name;   ///< name of the semaphore
  uint32_t                 attr_bits;   ///< attribute bits
  void                     *cb_mem;    ///< memory for control block
  uint32_t                   cb_size;   ///< size of provided memory for
control block
} osSemaphoreAttr_t;
```

Next, declare the semaphore handle. Then, within a thread the semaphore container can be initialized with a number of tokens:

```
osSemaphoreId osSemaphoreCreate (const osSemaphoreDef_t *semaphore_def,
int32_t count) {

  if (semaphore_def == NULL) {
    return (osSemaphoreId)NULL;
  }
  return osSemaphoreNew(osRtxSemaphoreTokenLimit, (uint32_t)count,
semaphore_def);
}
```

A semaphore object should be initialized to the maximum number of available tokens. This number of available resources is specified as the parameters of the osSemaphoreNew function. Each time a semaphore token is obtained with osSemaphoreAcquire (in available state), the semaphore count is decremented. When the semaphore count is 0 (i.e. depleted state), no more semaphore tokens can be obtained. The thread/ISR that tries to obtain the semaphore token needs to wait until the next token is free. Semaphores are released with osSemaphoreRelease incrementing the semaphore count.

Here is an example to show the simple use of Semaphore.

Sample Code 10: OS Semaphore

{"os_semaphore_demo", os_semaphore_demo_test},

[TEST, os_semaphore_demo]

5.5.9 OS Delay

Timer delay is the most basic function of these timing services. This is an easy way of providing timing delays within your application. Although the RTOS kernel size is quoted as 5 KB, features such as delay loops and simple scheduling loops are often part of a non-RTOS application and consume code bytes anyway, so the overhead of the RTOS can be less than it immediately appears.

This call puts the calling thread into the WAIT_DELAY state for the specified number of milliseconds. The scheduler passes the execution request to the next thread in the READY state.

When the timer expires, the thread leaves the WAIT_DELAY state and enters the READY state. The thread will resume running when the scheduler moves it to the RUNNING state. If the thread then continues executing without any further blocking OS calls, it is descheduled at the end of its time slice and be placed in the ready state, assuming another thread of the same priority is ready to run.

In addition to the osDelay() function which gives a relative time delay starting from the instant it is called, there is also a delay function which halts a thread until a specific point in time:

```
osStatus_t osDelayUntil (uint32_t ticks) {
```

CONFIDENTIAL

```
  osStatus_t status;

  EvrRtxDelayUntil(ticks);
  if (IsIrqMode() || IsIrqMasked()) {
    EvrRtxDelayError((int32_t)osErrorISR);
    status = osErrorISR;
  } else {
    status = __svcDelayUntil(ticks);
  }
  return status;
}
```

The osDelayUntil() function will halt a thread until a specific value of kernel timer ticks is reached.

5.5.10 OS Thread Statistics

By calculating the time of thread swap-in and swap-out, the statistics of thread usage can be obtained.

```
#if __RTX_CPU_STATISTICS__
  uint32_t             swap_in_time;  ///< the task schedule in time
  uint32_t            swap_out_time;  ///< the task schedule out time
  uint32_t                   rtime;  ///< the task runing time after startup
  uint32_t               step_rtime;  ///< the task runing time tile last
show
```

There is a simple example for thread statistics in the function “rtx_show_all_threads” of “rtx_thread_dump.c”. The function prints information about all threads in different states like current, next, ready, delay, wait, terminate, and the state of timer and memory. The log information printed by the function “rtx_show_all_threads” is defined in Table 25.

Table 25 Threads Statistics Introduction

Item	Information	Description
Thread	id	Object thread Identifier
	name	Object thread Name
	thread	The first pointer to memory corresponding to the struct osRtxThread_s
	priority	Thread Priority
	thread_addr	Thread entry address
	thread_next	Link pointer to next Thread in Object list
	thread_prev	Link pointer to previous Thread in Object list
	delay_next	Link pointer to next Thread in Delay list
	delay_prev	Link pointer to previous Thread in Delay list
	thread_join	Thread waiting to Join

	flags_options	Thread/Event Flags Options
	wait_flags	Waiting Thread/Event Flags
	thread_flags	Thread Flags
	stack_mem	Stack Memory
	stack_size	Stack Size
	sp	Current Stack Pointer
	min_stack_free	Minimum remaining stack space
	swap_in_time	the task schedule in time
	swap_out_time	the task schedule out time
	task runtime	the task running time
	frame	Register address
Timer	Timer	Object timer Name
	Timer state	Object timer State
	Timer type	Timer Type (Periodic/One-shot)
	Current tick	Timer current tick
	Load	Timer load value
Memory	Memory size	Memory Pool size
	Used	Used memory

The following figure shows the thread log demo:

```
Thread List:
Current List
--- Thread  4 name=app_trace_rx thread=0x20561f60, prio=24 state=RUNNING   thread_addr=0x14052DD9
    thread_next=0x00000000 thread_prev=0x00000000 delay_next=0x20562540 delay_prev=0x00000000
    thread_join=0x00000000 flags_options=0 wait_flags=0 thread_flags=0
    stack_mem=0x20522D18 stack_size=2048 sp:0x20523490 min_stack_free=1296
    swap_in_time=598473(ticks)/37404(ms) swap_out_time=10868(ticks)/679(ms)
    after last switch thread still runing, now 598514(ticks)/37407(ms)

Ready List
--- Thread  2 name=idle thread=0x20569aa0, prio=1 state=READY     thread_addr=0x00502F39
    thread_next=0x00000000 thread_prev=0x20509A84 delay_next=0x00000000 delay_prev=0x00000000
    thread_join=0x00000000 flags_options=0 wait_flags=0 thread_flags=0
    stack_mem=0x2056AAF8 stack_size=1024 sp:0x2056aea8 min_stack_free=168
    swap_in_time=554810(ticks)/34675(ms) swap_out_time=598473(ticks)/37404(ms)
    after last switch task runtime 43663(ticks)/2728 ms
    frame:0x2056AEC8
    R0 =0x00000001 R1=0x40130000 R2=0x0001A521 R3  =0xFFFFFFFF
    R12=0x00000001 LR=0x005085F5 PC=0x00508958 XPSR=0x2100F000

Possible Backtrace:
  5085F0
  502F5E

Delay List
Wait List
--- Thread  5 name=app_bt_cmd_thread thread=0x20562540, prio=24 state=WAIT_MGET thread_addr=0x14103E05
    thread_next=0x00000000 thread_prev=0x205623B8 delay_next=0x20564CB0 delay_prev=0x00000000
    thread_join=0x00000000 flags_options=0 wait_flags=0 thread_flags=0
    stack_mem=0x2054A240 stack_size=2048 sp:0x2054a960 min_stack_free=1824
    swap_in_time=11076(ticks)/692(ms) swap_out_time=11077(ticks)/692(ms)
    after last switch task runtime 1(ticks)/0 ms
    frame:0x2054A980
    R0 =0xFFFFFFFE R1=0x20562540 R2=0x2054A9B8 R3  =0x00000000
    R12=0x005047E1 LR=0x00504A03 PC=0x00504A60 XPSR=0x61000000
```

Figure 34 Thread Statistics Log Demo

OS_THREAD_TIMING_STATISTICS_ENABLE

5.6 Audio Function Usage Guide

5.6.1 Audio Flinger

5.6.1.1 Common Introduction

Audio flinger is an audio policy executor, responsible for the management of I/O STREAM devices and the processing and transmission of PCM data.

The following configuration items are available:

- Playback configuration
- Capture configuration

5.6.1.2 Capture Configuration

Table 26 Capture Configuration

Item	Description	Notes
enum AUD_BITS_T	bit width of PCM data	The bit width of PCM data is 0, 8, 12, 16, 20, 24, 32. The default channel is AUD_BITS_16.
enum AUD_CHANNEL_NUM_T	channel number	The number of channels ranges from 0 to 8. The default channel is AUD_CHANNEL_NUM_1.
enum AUD_SAMPRATE_T	sample rate of audio sample	The audio sampling rate ranges from 7350 to 812500 and DIV must be a multiple of 32. The default channel is AUD_SAMPRATE_8000.

CONFIDENTIAL

enum AUD_STREAM_USE_DEVICE_T	codec device	enum AUD_STREAM_USE_DEVICE_T{ AUD_STREAM_USE_DEVICE_NULL = 0, AUD_STREAM_USE_EXT_CODEC, AUD_STREAM_USE_INT_CODEC, AUD_STREAM_USE_INT_CODEC2, AUD_STREAM_USE_INT_CODEC3, AUD_STREAM_USE_I2S0_MASTER, AUD_STREAM_USE_I2S0_SLAVE, AUD_STREAM_USE_I2S1_MASTER, AUD_STREAM_USE_I2S1_SLAVE, AUD_STREAM_USE_TDM0_MASTER, AUD_STREAM_USE_TDM0_SLAVE, AUD_STREAM_USE_TDM1_MASTER, AUD_STREAM_USE_TDM1_SLAVE, AUD_STREAM_USE_SPDIF0, AUD_STREAM_USE_SPDIF1, AUD_STREAM_USE_BT_PCM, AUD_STREAM_USE_DPD_RX, AUD_STREAM_USE_MC, }; here we use AUD_STREAM_USE_EXT_CODEC.
enum TGT_VOLUME_LEVEL_T	volume	The default channel is TGT_VOLUME_LEVEL_15.
enum AUD_IO_PATH_T	io_path	The default channel is AUD_INPUT_PATH_MAINMIC
App_factorymode_data_come	handler	The callback function when mic data comes(capture stream)
BT_AUDIO_CACHE_2_UNCACHE	data_ptr	The pointer of buffer to cache the capture data
BT_AUDIO_FACTORMODE_BUFF_SIZE	data_size	The size of the data cache buffer. The default setting is 1024*2.

Use the following sample code to configure the Capture stream.

```
struct AF_STREAM_CONFIG_T stream_cfg;
memset(&stream_cfg, 0, sizeof(stream_cfg));
stream_cfg.bits = AUD_BITS_16;

#ifdef SPEECH_TX_AEC_CODEC_REF
       stream_cfg.channel_num = AUD_CHANNEL_NUM_2;
#else
```

CONFIDENTIAL

```
        stream_cfg.channel_num = AUD_CHANNEL_NUM_1;
#endif
#if defined(__AUDIO_RESAMPLE__) && defined(SW_CAPTURE_RESAMPLE)
        stream_cfg.sample_rate = AUD_SAMPRATE_8463;
#else
        stream_cfg.sample_rate = AUD_SAMPRATE_8000;
#endif
#if FPGA==0
        stream_cfg.device = AUD_STREAM_USE_INT_CODEC;
#else
        stream_cfg.device = AUD_STREAM_USE_EXT_CODEC;
#endif
        stream_cfg.vol = TGT_VOLUME_LEVEL_15;
        stream_cfg.io_path = AUD_INPUT_PATH_MAINMIC;
        stream_cfg.handler = app_factorymode_data_come;

        stream_cfg.data_ptr = BT_AUDIO_CACHE_2_UNCACHE(buff_capture);
        stream_cfg.data_size = BT_AUDIO_FACTORMODE_BUFF_SIZE;
        af_stream_open(AUD_STREAM_ID_0, AUD_STREAM_CAPTURE, &stream_cfg);
```

5.6.1.3 Playback Configuration

The configurations of Playback and Capture are mostly common, except for 'io_path', 'handler', 'data_ptr'and 'data_size'.

Table 27 Playback Configuration

Item	Description	Notes
enum AUD_IO_PATH_T	io_path	The default channel is AUD_OUTPUT_PATH_SPEAKER
app_factorymode_more_data	handler	callback function when playback ask for more data(playback stream)
BT_AUDIO_CACHE_2_UNCACHE	data_ptr	pointer of buffer to cache the playback data
BT_AUDIO_FACTORMODE_BUFF_SIZE	data_size	size of the data cache buffer. The default setting is 1024*4.

Use the following sample code to configure the Playback stream.

```
stream_cfg.channel_num = AUD_CHANNEL_NUM_2;
stream_cfg.io_path = AUD_OUTPUT_PATH_SPEAKER;
stream_cfg.handler = app_factorymode_more_data;
```

```
stream_cfg.data_ptr = BT_AUDIO_CACHE_2_UNCACHE(buff_play);
stream_cfg.data_size = BT_AUDIO_FACTORMODE_BUFF_SIZE*2;
af_stream_open(AUD_STREAM_ID_0, AUD_STREAM_PLAYBACK, &stream_cfg);
```

5.6.1.4 Audio Loopback Test

1. Configure the system frequency to match the audio stream process: `app_sysfreq_req`.

```
        if (freq < APP_SYSFREQ_52M) {
            freq = APP_SYSFREQ_52M;
        }
        app_sysfreq_req(APP_SYSFREQ_USER_APP_0, freq);
```

2. Initialize the audio-dedicated memory pool `app_audio_mempool_init`.

```
        app_audio_mempool_init();
```

3. Allocate buffer from the audio memory pool.

   Parameter info:

   - buff: the pointer of audio buffer to allocate.

   - size: the size of audio buffer to allocate.

```
app_audio_mempool_get_buff(&buff_capture, BT_AUDIO_FACTORMODE_BUFF_SIZE);
        app_audio_mempool_get_buff(&buff_play,
BT_AUDIO_FACTORMODE_BUFF_SIZE*2);
```

4. Configure the stream.

   Use the structure `struct AF_STREAM_CONFIG_T` to configure the stream. Description of the parameters:

   - .bit: the bit width of PCM data. The type is `enum AUD_BITS_T`.

   - .channel_num: the channel number.

   - .sample_rate: the sample rate of audio sample.

   - .device: the codec device. Here, `AUD_STREAM_USE_INT_CODEC` is used for the internal codec.

   - .vol: the volume, which is used to configure the gain of the audio data.

   - .io_path: the input/output path, `AUD_INPUT_PATH_MAINMIC` for capture and `AUD_OUTPUT_PATH_SPEAKER` for playback.

   - .handler: the callback function when mic data comes (capture stream) or playback requests more data (playback stream).

   - .data_ptr: the pointer of the buffer to cache the capture/playback data.

   - .data_size: the size of the data cache buffer.

```
struct AF_STREAM_CONFIG_T stream_cfg;
memset(&stream_cfg, 0, sizeof(stream_cfg));
stream_cfg.bits = AUD_BITS_16;

#ifdef SPEECH_TX_AEC_CODEC_REF
       stream_cfg.channel_num = AUD_CHANNEL_NUM_2;
#else
       stream_cfg.channel_num = AUD_CHANNEL_NUM_1;
#endif
#if defined(__AUDIO_RESAMPLE__) && defined(SW_CAPTURE_RESAMPLE)
       stream_cfg.sample_rate = AUD_SAMPRATE_8463;
#else
       stream_cfg.sample_rate = AUD_SAMPRATE_8000;
#endif
#if FPGA==0
       stream_cfg.device = AUD_STREAM_USE_INT_CODEC;
#else
       stream_cfg.device = AUD_STREAM_USE_EXT_CODEC;
#endif
       stream_cfg.vol = TGT_VOLUME_LEVEL_15;
       stream_cfg.io_path = AUD_INPUT_PATH_MAINMIC;
       stream_cfg.handler = app_factorymode_data_come;

       stream_cfg.data_ptr = BT_AUDIO_CACHE_2_UNCACHE(buff_capture);
       stream_cfg.data_size = BT_AUDIO_FACTORMODE_BUFF_SIZE;
```

5. Open stream with `af_stream_open`.

   Parameter info:

   - id: the stream ID.

   - stream: the stream type(capture or playback).

   - cfg: the stream configurations. Pass in the configurations in step 4.

```
       af_stream_open(AUD_STREAM_ID_0, AUD_STREAM_CAPTURE, &stream_cfg);
```

6. Start the stream with `af_stream_start`.

   Start the configured stream with the incoming stream ID and stream type, and the audio flinger starts running.

```
       af_stream_start(AUD_STREAM_ID_0, AUD_STREAM_CAPTURE);
```

7. Audio data input/output is triggered by DMA interrupt.

   The mic data is handled (or playback data is loaded) in the `.handler` configured in step 4.

```
static uint32_t app_factorymode_data_come(uint8_t *buf, uint32_t len)
{
    app_audio_pcmbuff_put(buf, len);
    if (a2dp_cache_status == APP_AUDIO_CACHE_QTY){
        a2dp_cache_status = APP_AUDIO_CACHE_OK;
    }
    return len;
}

static uint32_t app_factorymode_more_data(uint8_t *buf, uint32_t len)
{
    if (a2dp_cache_status != APP_AUDIO_CACHE_QTY){
        app_audio_pcmbuff_get((uint8_t *)app_audioloop_play_cache, len/2);
        app_bt_stream_copy_track_one_to_two_16bits((int16_t *)buf,
app_audioloop_play_cache, len/2/2);
    }
    return len;
}
```

8. Stop the audio stream.

   Call `af_stream_stop` to stop a specific audio stream according to the stream ID and stream type.

```
        af_stream_stop(AUD_STREAM_ID_0, AUD_STREAM_CAPTURE);
        af_stream_stop(AUD_STREAM_ID_0, AUD_STREAM_PLAYBACK);
```

9. Close the audio stream.

   Call `af_stream_close` to close a specific audio stream according to the stream ID and stream type

```
        af_stream_close(AUD_STREAM_ID_0, AUD_STREAM_CAPTURE);
        af_stream_close(AUD_STREAM_ID_0, AUD_STREAM_PLAYBACK);
```

Sample Code 11: Audio Loopback Test

{"factorymode_audioloop", factorymode_audioloop_test},

[TEST, factorymode_audioloop]

5.6.2 Audio Data Path Introduction

5.6.2.1 Phone Call

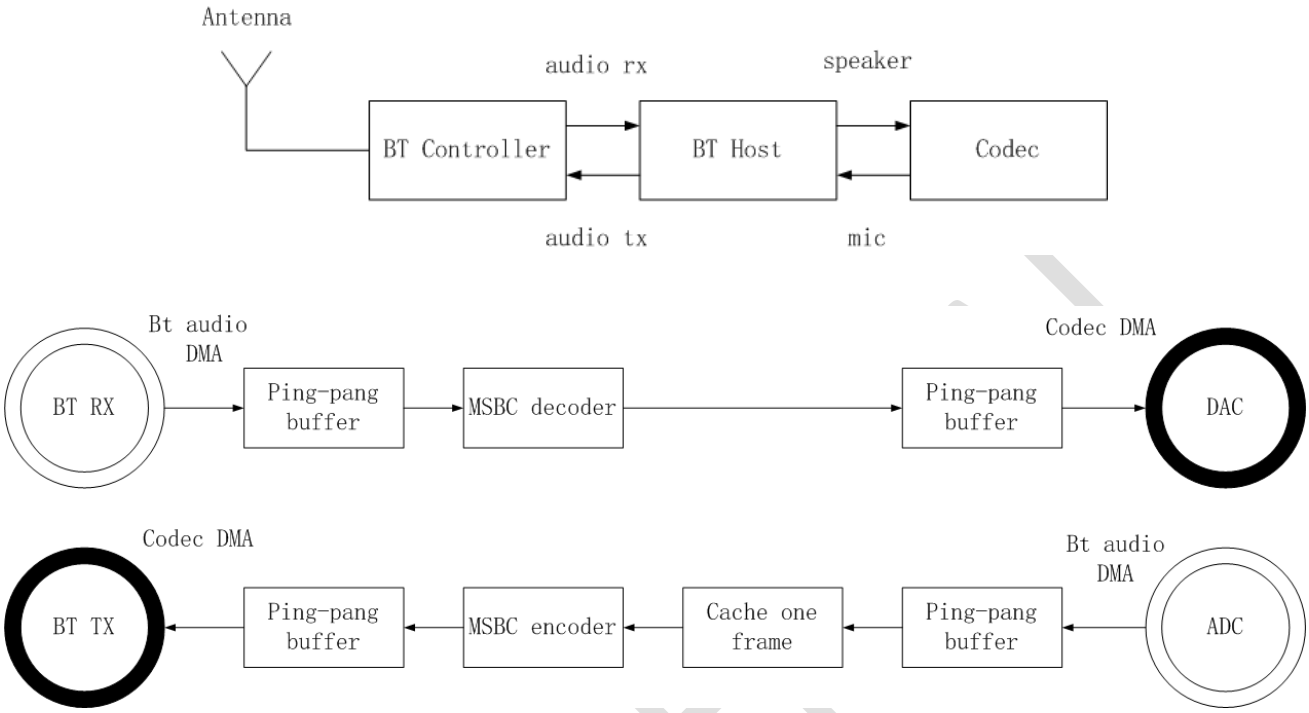


Figure 35 Audio Path Flow of Phone Call

The same source clock is used for audio RX/TX and speaker/mic, and also the sco link uses the reserved time slot for BT data packet TX and RX, so the BT controller audio streaming and codec's ticks are synchronized via hardware.

All the handling is in the audio thread.

By default, one frame is 7.5ms.

The sco delay measurement result is: 40ms for playback, less than 40ms for capturing.

Key functions:

- **Call the capture DMA IRQ handler**

  bt_sco_player()->bt_sco_codec_capture_data

```
stream_cfg.handler = bt_sco_codec_capture_data;
```

- **Call the playback DMA IRQ handler**

  bt_sco_player()->bt_sco_codec_playback_data

```
stream_cfg.handler = bt_sco_codec_playback_data;
```

5.6.2.2 Music Playback

Audio path:

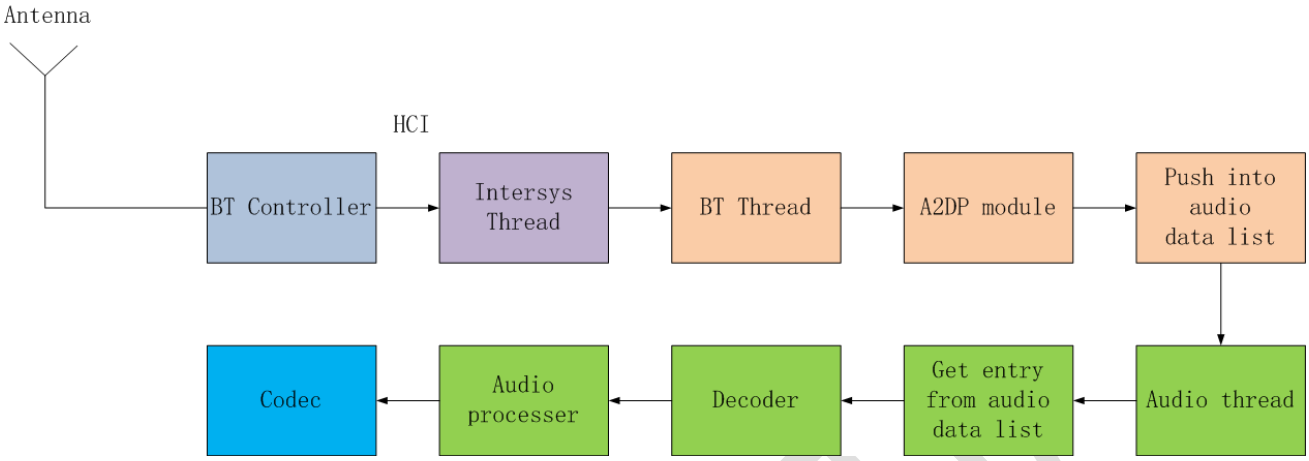


Figure 36 Audio Path Flow of Music Playback

Sample count for each audio data entry: AAC/1024, SBC/128, Scalable/864.

There is no additional PCM buffer for buffering the PCM data from decoder. The decoder's output PCM data is directly fed into the Codec.

Audio Info Sync:

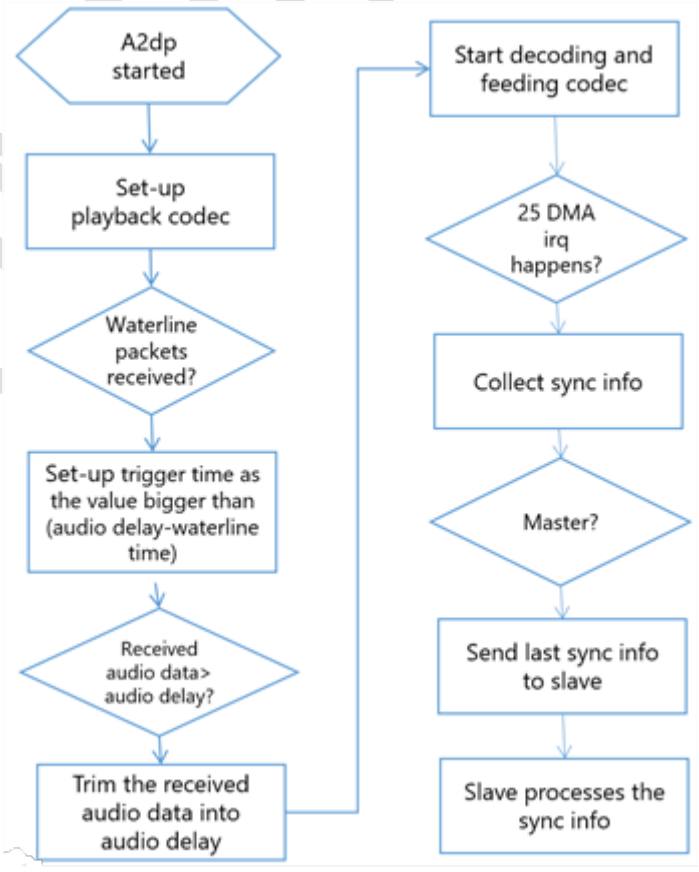


Figure 37 Audio Info Sync of Music Playback

- The codec triggering accuracy on BES2700YP is 1 us. The codec feeding unit size (DMA irq chunk size) is configured as exactly the one AAC frame (1024 samples), five SBC smallest sequence (5*128 samples), or one scalable frame (864 samples)
- Every 25 DMA irqs interval, both sides collects the sync Info of the same frame. The master sends the info of last interval to the slave for synchronizing.
- The L/R sync detection is done during sync info processing.
- Audio delay is in the unit of codec frame. E.g., 23ms*5 = 115ms for AAC codec. Re-sampling factor tuning is applied to assure that the delay is within the range.

Key functions:

- **call playback dma irq handler**

bt_sbc_player()-> bt_sbc_player_more_data

```
stream_cfg.handler = bt_sbc_player_more_data;
```

5.6.3 Left and Right Playback Channel Configuration

The left and right playback channel configuration is implemented in the app_ibrt_ui_v2_test_config_load and app_tws_ibrt_reconfig_Role function. The parameter is ibrt_config_t.

app_ibrt_init -> app_ibrt_ui_v2_test_config_load

```
typedef struct
{
    ibrt_role_e nv_role;
    bt_bdaddr_t local_addr;
    bt_bdaddr_t peer_addr;
    bt_bdaddr_t mobile_addr;
    uint32_t    audio_chnl_sel;
} ibrt_config_t;
```

5.6.3.1 Configuration Steps

1. Get the configuration parameters from NV.

```
    ibrt_config_t *ibrt_config = (ibrt_config_t *)config;
    struct nvrecord_env_t *nvrecord_env;

    nv_record_env_get(&nvrecord_env);
    factory_section_original_btaddr_get(ibrt_config->local_addr.address);
```

2. The master and slave are judged according to the mantissa of the local address. If it's 1, it's the master. If it's 0, it's slave. Finally, the program writes the result to NV.

```
app_ibrt_initialize_nv_role_callback(ibrt_config, nvrecord_env);
```

3. In the app_tws_ibrt_reconfig_Role (), configure the left and right playback channel.

```
void app_tws_ibrt_reconfig_role(uint32_t role, const uint8_t *pMasterAddr,
                    const uint8_t *pSlaveAddr, bool
isRightMasterSidePolicy)
{
... ...
    if (isRightMasterSidePolicy)
   {
      if (IBRT_MASTER == role)
      {
         p_ibrt_ctrl->audio_chnl_sel = AUDIO_CHANNEL_SELECT_RCHNL;
         app_ibrt_middleware_set_side(EAR_SIDE_RIGHT);
      }
      else if (IBRT_SLAVE == role)
      {
         p_ibrt_ctrl->audio_chnl_sel = AUDIO_CHANNEL_SELECT_LCHNL;
         app_ibrt_middleware_set_side(EAR_SIDE_LEFT);
      }
   }
   else
   {
      if (IBRT_SLAVE == role)
      {
         p_ibrt_ctrl->audio_chnl_sel = AUDIO_CHANNEL_SELECT_RCHNL;
         app_ibrt_middleware_set_side(EAR_SIDE_RIGHT);
      }
      else if (IBRT_MASTER == role)
      {
         p_ibrt_ctrl->audio_chnl_sel = AUDIO_CHANNEL_SELECT_LCHNL;
         app_ibrt_middleware_set_side(EAR_SIDE_LEFT);
      }
   }
```

```
    APP_TWS_SIDE_T custom_side = app_ibrt_ui_get_earbud_side();
    if (EAR_SIDE_NUM != custom_side)
    {
        TRACE(0, "app_tws_side_info!");
        ibrt_ui_v2_audio_chnl_custom_config(custom_side, p_ibrt_ctrl);
    }
... ...
```

4. You can reconfigure channels by using WEAK APP_TWS_SIDE_T APP_iBRT_UI_geT_earbud_side (void).

```
WEAK APP_TWS_SIDE_T app_ibrt_ui_get_earbud_side(void)
{
    return EAR_SIDE_NUM;
}

void ibrt_ui_v2_audio_chnl_custom_config( APP_TWS_SIDE_T earbudSize,
ibrt_config_t *ibrt_config, struct nvrecord_env_t *nvrecord_env)
{
    if (EAR_SIDE_LEFT == earbudSize)
    {
      TRACE(0, "EAR_SIDE_LEFT!");
      p_ibrt_ctrl->audio_chnl_sel = AUDIO_CHANNEL_SELECT_LCHNL;
      app_ibrt_if_set_side(EAR_SIDE_LEFT);
    }
    else if (EAR_SIDE_RIGHT == earbudSize)
    {
      TRACE(0, "EAR_SIDE_RIGHT!");
      p_ibrt_ctrl->audio_chnl_sel = AUDIO_CHANNEL_SELECT_RCHNL;
      app_ibrt_if_set_side(EAR_SIDE_RIGHT);
    }
}
```

5.6.3.2 Configuration of Stereo

In order to enable stereo, you need to turn A2DP_AUDIO_STEREO_MIX_CTRL into 1 in target.mk.

```
export A2DP_AUDIO_STEREO_MIX_CTRL ?= 1
```

5.6.4 Configure Music Jitter Buffer Length

5.6.4.1 Initial Jitter Buffer Length

The initial Jitter buffer Length is determined by the macro. The Initial Jitter buffer Length varies with the decoding mode. Take SCALABLE as an example.

```
#define A2DP_PLAYER_PLAYBACK_DELAY_SCALABLE_HIRES_MTU (13)
#define A2DP_PLAYER_PLAYBACK_DELAY_SCALABLE_HIRES_BASE (9000)
#define A2DP_PLAYER_PLAYBACK_DELAY_SCALABLE_HIRES_US
(A2DP_PLAYER_PLAYBACK_DELAY_SCALABLE_HIRES_BASE*A2DP_PLAYER_PLAYBACK_DELAY_
SCALABLE_HIRES_MTU)

#define A2DP_PLAYER_PLAYBACK_DELAY_SCALABLE_BASERES_MTU  ((8-
A2DP_PLAYER_PLAYBACK_WATER_LINE_BASE_LINE))
#define A2DP_PLAYER_PLAYBACK_DELAY_SCALABLE_BASERES_BASE (19500)
#define A2DP_PLAYER_PLAYBACK_DELAY_SCALABLE_BASERES_US
(A2DP_PLAYER_PLAYBACK_DELAY_SCALABLE_BASERES_BASE*A2DP_PLAYER_PLAYBACK_DELA
Y_SCALABLE_BASERES_MTU)

 if(stream_cfg->sample_rate > AUD_SAMPRATE_48000){
          tg_trigger_time = A2DP_PLAYER_PLAYBACK_DELAY_SCALABLE_HIRES_US *
a2dp_audio_latency_factor_get();

          tg_trigger_time +=
offset*A2DP_PLAYER_PLAYBACK_DELAY_SCALABLE_HIRES_BASE;
       }else{
          tg_trigger_time = A2DP_PLAYER_PLAYBACK_DELAY_SCALABLE_BASERES_US
* a2dp_audio_latency_factor_get();
          tg_trigger_time +=
offset*A2DP_PLAYER_PLAYBACK_DELAY_SCALABLE_BASERES_BASE;
       }
```

5.6.4.2 Run-time Update of Jitter Buffer Length

You can use the following interface to update the jitter buffer length during running.

```
void sbm_update_target_jitter_buf_length(uint16_t targetMs)
```

The system calculates targetBufCount based on targetMs, so the actual target time is based on the calculated targetBufCount . The minimum value of targetBufCount defaults to 5.

```
targetBufCount = (targetMs*currentSampleRate/1000)/sampleCountPerBuf;
```

Sample Code 12: Music Jitter Buffer Length

{"a2dp_change_jitter_buf", a2dp_change_jitter_buf_test},

[TEST, a2dp_change_jitter_buf|<parameter>]

5.7 Callback API

5.7.1 Prompt Usage Guide

5.7.1.1 General Introduction

This section describes the audio prompts of the project. Audio prompts are generally used to indicate event that is happening or the current status of device to the users. For example, when the device is turned on, a piece of specific audio indicating “power on” will be played. The audio can be customized to either a piece of music or recorded human voice, according to different needs.

The current default audio prompt provided is voice recordings in English. A total of two packages of prompts are supported right now. User can switch between the two packages while using the device. To switch to more packages or to update the current language package, using OTA to upgrade the device is required.

The following specification is designed to show the design of two types of voice prompt supported, i.e.: embedded prompt and prompt in flash, customization of voice prompts, and using OTA to upgrade the prompt in flash.

5.7.1.2 Design Discussion

Initally, the audio prompt files are desgined to be integrated into the general image file. By transforming prerecorded wav files to txt files, and then include them into header files, the preset audio prompts are encoded and built into the main image alongside with other promgrams and data. Those prompts are called **embedded promtp**s. Each audio is encoded into binary data and initalized to an array, and then the array is mapped to a sound mapping struct and thus they can be called and played when necessary. They can be customized before they are built and programmed into the flash of the device.

With the introduce of the OTA functionaility and the need for supporting multiple language packages to provide better user experience, the second way of integrating audio prompt: **prompt in flash**, is desgined and implemented. This type implementation of integrating audio prompt will consider each package of audio prompt as a whole and promgram them into a fix area of the flash. Currently, two slots in flash are reserved for two packages of audio prompts. By default, the English voice prompt is promgramed initially in the first slot and the other slot in flash is left empty. Comparing to the original embedded prompts, the new prompt in flash has more flexibility when it comes to upgrading audio prompt package or increasing supported languages, since it won’t need to rebuild and reprogram the whole image, it just need to create the prompt package binary file and then use OTA to program the binary file to certain location of the flash. It also provides the functionaility of switching between different audio prompt packages while using the device, without rebooting it.

CONFIDENTIAL

To summarize, for simplier built and smaller total image size, embedded prompts is recommended, and for more flexibility and maintainability, prompt in flash should be applied.

5.7.1.3 Audio File Format Conversion and Inclusion

1. RAW AUDIO FILE CONVERSION

Audio prompts package should be first transformed from *wav* audio format to sbc array in *txt* format, so as to be included by the program.

A format conversion tool called "*wav2sbcArray.exe*", is provided. It should run in the Windows command prompt, e.g.: PowerShell, by using the following command:

```
wav2sbcArray.exe Target.wav ParaA
```

Target.wav is the audio file to be conversed and ParaA indicates the ratio of the compression. The default SBC bitpool is 39 when ParaA is absent. The tools only support audio files with the sample frequency of 16000, 32000, 44100 or 48000.

Files are created with the name as Target_8k.sbc, Target_16k.sbc or Target_44.1k.sbc according to the sample frequency of the original files along with another named Target_XXk.txt.

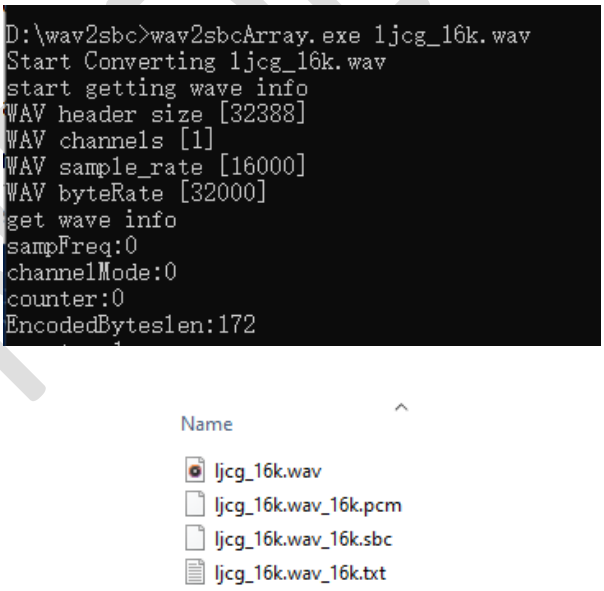


Figure 38 Convert Command

2. RESOURCE FILE RELOCATION

CONFIDENTIAL

Project folder as shown in the following figure has many subfolders which indicate the different operation environments of the voice prompts. Move the generated Target_XXk.txt into one as needed. For example, “en” contains the English prompts and “cn” contains the Chinese ones.

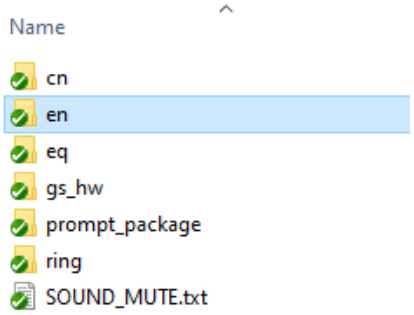


Figure 39 File Relocation

In this case, the ljcg_16k.wave_16k.txt file is copied into the “en” folder.

Note: The directory of the resource file is located in the following address by default.

```
./config/_default_cfg_src_/res/
```

3. AUDIO RESOURCE FILE INCLUSION

This part is specified for customizing embedded prompts. Since the resource files are directly included by header file in when using embedded prompt, the promgram files should be modified to replace original resource files to customized new ones.

Firstly, the new converted *txt* file should be added to the header file called res_audio_data_XX.h. The directory is the following:

```
./services/audio_bt/res_audio_data.h
```

Secondly, add the new item to the header file *res_audio_data.h* according to its corresponding content. In this example, the EN_POWER_ON item is modified. To replace the old prompt file, change the txt file name.

```
const uint8_t EN_POWER_ON [] = {
#include "res/en/SOUND_POWER_ON.txt"
};
```

To create new prompts additional to the originals, two extra steps should be performed.

Step 1: In the header file *res_audio_data.h,* add customized array name.

Step 2: Look into the file *“app_media_player.cpp”* for the function:

"void media_runtime_audio_prompt_update(uint16_t id, uint8_t** ptr, uint32_t* len)".

Change the corresponding "*case*" item of "*switch(ptr->aud_id)*" to a customized ID if necessary.

```
    switch (id) {
    case AUD_ID_POWER_ON:
        get_sound_id_info(POWER_ON, &sound_data, &length);
        break;
```

Please be noted that the default audio sample rate of the voice prompts is 16000. To replace the original ones with new sample rate, the code should be adjusted at the same time.

For stand-alone voice prompt, it's located in the function "int app_play_audio_onoff(bool onoff, APP_AUDIO_STATUS* status)" of file "App_media_player.cpp".

```
stream_cfg.sample_rate = AUD_SAMPRATE_16000;
```

4. AUDIO RESOURCE FILE CONVERSION TO BINARY

To use the prompt in flash functionaility, the audio resource file should be further convered from txt file to binary files, so as to be programmed or upgraded to flash separately.

The tool for convertion is called *"prompt_bin_generate.py"*, it is located under the following directory:

```
tools/prompt_bin_generate
```

The command to use to generate the binary file is the following:

```
python <directory-of-the-conversion-file>\prompt_bin_generate.py -i <prompt ID> -v <version> -c <directory-of-configuration-file>/prompt_gen.csv -d <directory-of-resource-file>\en
```

Here is an example of the command:

```
python E:\workbench\mcu-sw\tools\prompt_bin_generate\prompt_bin_generate.py -i 0x00000001 -v 0x00000001 -c ./prompt_gen.csv -d E:\workbench\mcu-sw\config\_default_cfg_src_\res\en
```

The default output file is *"combined_bin.bin"*. The output directory is shown in the output info by the *"prompt_bin_generate.py"*. For detailed instruction, please check the readMe file under the same directory with the python converstion file.

CONFIDENTIAL

Note: When generating the binary file, pay attention to the “-i” argument. This argument is short for “--id”, which indicates the prompt ID. In the default setting, the English prompt ID is set to 1. While other language prompt package ID are listed in the following code snippet:

```
typedef enum
{
    LANGUAGE_ID_INVALID = 0,
    LANGUAGE_ID_EN      = 1,
    LANGUAGE_ID_CN      = 2,
    LANGUAGE_ID_GN      = 3,
    LANGUAGE_ID_FN      = 4,
    LANGUAGE_ID_KN      = 5,
    LANGUAGE_ID_JN      = 6,
    LANGUAGE_ID_RN      = 7,
    LANGUAGE_ID_NUM = 0xFF,
} LANGUAGE_ID_E;
```

The above code snipped is in file “app_media_player.h”. If new language prompt package to be added, please be guided that there is a default mapping between some certain language and its ID.

5.7.1.4 VOICE PROMPT API

The Voice Prompt API amid at providing services for playing certain voice prompt audio when necessary. For example, when there is an income phone call, the promgram can call a certain function to play a voice audio indicating this event to the user. Functions of different purposes are listed in the following table. Generally, when calling those functions, prompt ID and target device ID need to be specified as input parameters. Those functions are decleared in header file “app_media_player”.

Table 28 APIs of Voice Prompt Module

Item	Description	parameters
media_PlayAudio(AUD_ID_ENUM id, uint8_t device_id);	- Prompt will be played on the both sides if they're connected. - If there are music/phone call on-going, the prompt will be mixed.	The APP_STATUS_INDICATION_T enumeration type parameter represents the prompt tone number and is used to index the prompt tone ID. typedef enum { AUD_ID_INVALID, AUD_ID_POWER_ON, AUD_ID_POWER_OFF, AUD_ID_LANGUAGE_SWITCH,
media_PlayAudio_locally(AUD_ID_ENUM id, uint8_t device_id);	- Prompt will be played locally no matter whether earbuds are connected or not	

	- If there are music/phone call on-going, the prompt will be mixed.	AUD_ID_NUM_0, AUD_ID_NUM_1, AUD_ID_NUM_2, AUD_ID_NUM_3, AUD_ID_NUM_4, AUD_ID_NUM_5, AUD_ID_NUM_6, AUD_ID_NUM_7, AUD_ID_NUM_8, AUD_ID_NUM_9, AUD_ID_BT_PAIR_ENABLE, AUD_ID_BT_PAIRING, AUD_ID_BT_PAIRING_SUC, AUD_ID_BT_PAIRING_FAIL, AUD_ID_BT_CALL_REFUSE, AUD_ID_BT_CALL_OVER, AUD_ID_BT_CALL_ANSWER, AUD_ID_BT_CALL_HUNG_UP, AUD_ID_BT_CALL_INCOMING_CALL, AUD_ID_BT_CALL_INCOMING_NUMBER, AUD_ID_BT_CHARGE_PLEASE, AUD_ID_BT_CHARGE_FINISH, AUD_ID_BT_CLEAR_SUCCESS, AUD_ID_BT_CLEAR_FAIL, AUD_ID_BT_CONNECTED, AUD_ID_BT_DIS_CONNECT, AUD_ID_BT_WARNING, AUDIO_ID_BT_ALEXA_START, AUDIO_ID_FIND_MY_BUDS, AUDIO_ID_FIND_TILE, AUDIO_ID_BT_ALEXA_STOP, AUDIO_ID_BT_GSOUND_MIC_OPEN, AUDIO_ID_BT_GSOUND_MIC_CLOSE, AUDIO_ID_BT_GSOUND_NC, AUDIO_ID_BT_MUTE, AUD_ID_RING_WARNING, #ifdef __INTERACTION__
media_PlayAudio_remotely (AUD_ID_ENUM id, uint8_t device_id);	- Prompt will be played on peer device only if they're connected. - If there are music/phone call on-going, the music or phone call output will be stopped until the prompt playing is completed.	
media_PlayAudio_standalone(AUD_ID_ENUM id, uint8_t device_id);	- Prompt will be played on the both sides if they're connected. - If there are music/phone call on-going, the music or phone call output will be stopped until the prompt playing is completed.	
media_PlayAudio_standalone_locally(AUD_ID_ENUM id, uint8_t device_id);	- Prompt will be played locally no matter whether earbuds are connected or not - If there are music/phone call on-going, the music or phone call output will be stopped until the prompt playing is completed.	
media_PlayAudio_standalone_remotely(AUD_ID_ENUM id, uint8_t device_id);	- Prompt will be played on the peer device only if they're connected. - If there are music/phone call on-going, the music or phone call output will be stopped until the prompt playing is completed.	
media_PlayAudio_continuous_start(AUD_ID_ENUM id, uint8_t device_id);	- Prompt will be played on the both sides if they're connected.	

CONFIDENTIAL

	- If there are music/phone call on-going, the prompt will be mixed. - Prompt will keep playing time and time again until media_PlayAudio_continuous_end is called	AUD_ID_BT_FINDME, #endif AUD_ID_ANC_PROMPT, AUD_ID_ANC_MUTE, AUD_ID_ANC_MODE0, AUD_ID_ANC_MODE1, #if BLE_AUDIO_ENABLED AUD_ID_LE_AUD_INCOMING_CALL, #endif MAX_RECORD_NUM, AUD_ID_ENUM_BOTTOM, } AUD_ID_ENUM; device_id indicates the device ID. The value is generally 0.
media_PlayAudio_continuous_end(AUD_ID_ENUM id, uint8_t device_id);	Stop the on-going continuous prompt playing	
app_prompt_stop_all();	Stop the current prompt playing, and clear subsequent requests to prompt	
app_prompt_start_callback_register(app_prompt_start_cb cb);	A callback function that is automatically invoked at the beginning of each prompt function	The cb is the ID of the current prompt (audio ID).
app_prompt_finish_callback_register(app_prompt_finsih_cb cb);	A callback function that is automatically invoked at the end of each prompt function	

Sample Code 13: Prompt Usage Test

{"prompt_stop_all", prompt_stop_all_test},

{"media_PlayAudio_test", media_PlayAudio_test},

{"media_PlayAudio_locally_test", media_PlayAudio_locally_test},

{"media_PlayAudio_remotely_test", media_PlayAudio_remotely_test},

{"media_PlayAudio_standalone_test", media_PlayAudio_standalone_test},

{"media_PlayAudio_standalone_locally_test", media_PlayAudio_standalone_locally_test},

{"media_PlayAudio_standalone_remotely_test", media_PlayAudio_standalone_remotely_test},

{"media_PlayAudio_continuous", media_PlayAudio_continuous_test},

CONFIDENTIAL

{"prompt_finish_callback_register", app_prompt_finish_callback_register_test},

5.7.1.5 PROMPT IN FLASH API

The Prompt In Flash API amid at providing services for lower level flash memory management. The services include providing certain prompt audio data in flash according to specified language ID and audio ID, finding the correct address space when it comes to upgrading existing prompt package or adding new prompt package to the flash, updating the language prompt package that is least recently used, etc. Functions are declared in header file *"nvrecord_prompt.h"*. The following table shows the detailed information about each function in this API.

Item	Description	parameters
nv_record_prompt_get_package_addr(uint8_t language);	Obtain start address of required language packet	param[in] `language`: the id of language to get address of Return the start address of required language package in flash
nv_record_prompt_get_default_start_addr(void)	Return the start address of default start address for prompt in flash	None
nv_record_prompt_get_prompt_info(uint8_t language, uint16_t id, uint8_t **data, uint32_t *length);	Obtain the address and length of a specific prompt of a given language	param[out] `data`: pointer to the address of the prompt param[out] `length`: pointer to the length of the prompt param[in] `language`: the id of language of the prompt info param[in] `id`: the id of the prompt to get info of Return the start address of required language package in flash; Return the start address to program, if a new langauge package is to be added by specifying addNew to be true
nv_record_get_current_language(void);	Return the id of the current language for prompt	None
nv_record_switch_current_language(void)	switch to the other language if there are two language packages in flash	None

CONFIDENTIAL

5.7.1.6 PROMGRAMING OPTION FOR PROMPT IN FLASH

The process for programming the prompt binary file along with the main image file takes extra steps comparing to the normal image programing process.

After generating the binary file using file conversion tool, the location of the prompt binary file should be acquired. This information can be obtained by checking the “map” file. The location of the file is:

```
out/<build-version> /<build-version>.map
```

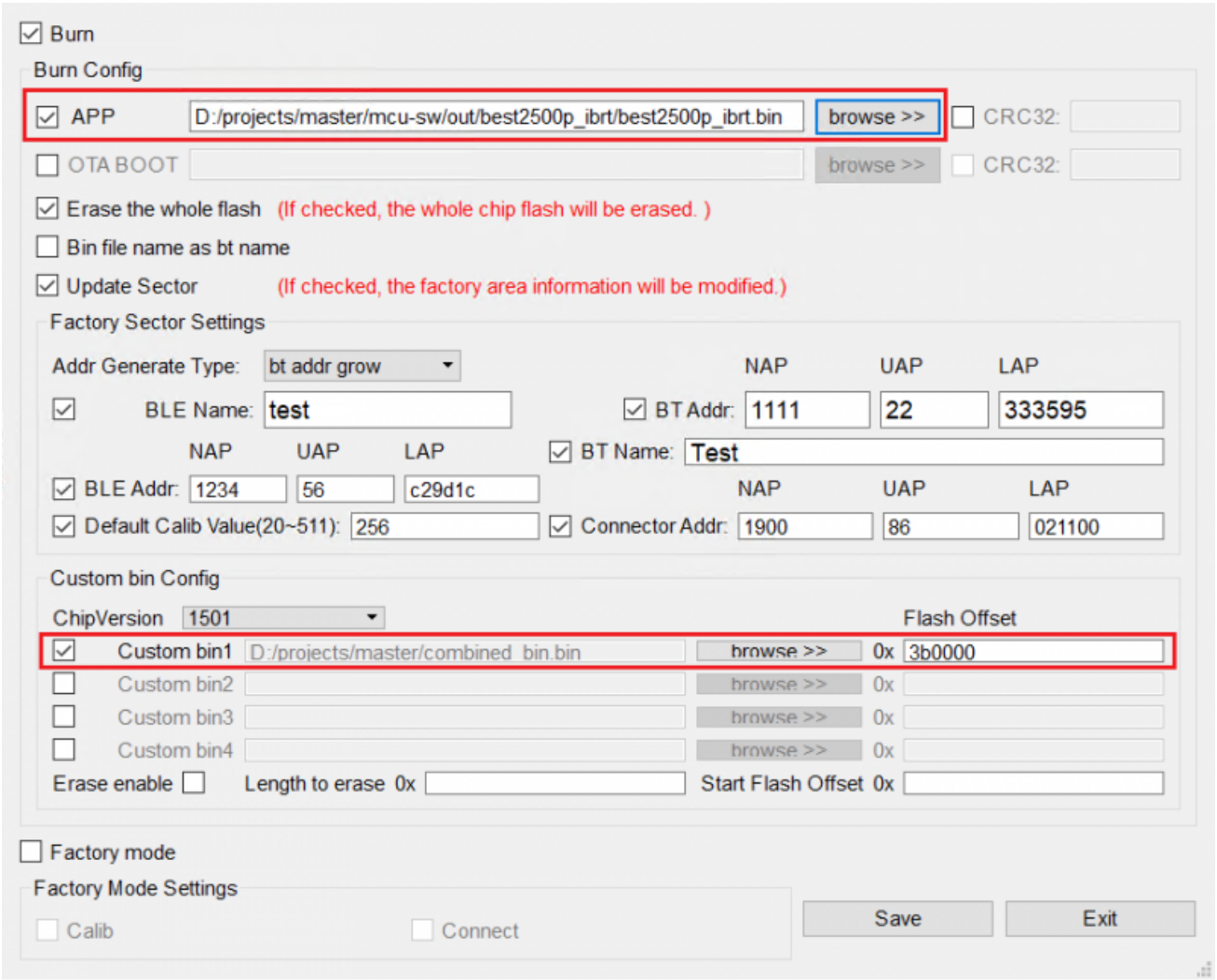
e.g.: `out/best2500p_ibrt/best2500p_ibrt.map`

In the map file, search for the line “.prompt”, this specifies the start address in flash to program. 28 is offset, which is unnecessary. The address following the 28 is the corresponding address. The following picture shows an example of the target promgram address.

```
.prompt         0x00000000283b0000    0x4b000
                0x00000000283b0000                __prompt_start = .
                0x000000000004b000                . = 0x4b000
```

Here is an example of how to program the prompt bin along with the main image. Please be noted that the location of the binary file and the flash offset to program is applied in the field of “Custom bin Config” of the Dld promgrom software.

CONFIDENTIAL



5.7.1.7 OTA FOR PROMPT IN FLASH

The OTA for prompt in flash can be achieved using mobile OTA application developed by Bestechnic. The application is supported on both IOS and Android systems. To install the application, please search for “Bestechnic” in app store or Google play. For detailed OTA process, please refer to the chapter 5.9.4 OTA User Guide.

Due to the limit of size in flash, two packages of prompt in flash is supported currently. The size of each prompt binary file should be no larger than 152 KB. For OTA purpose, a buffer with the same size as a prompt package is reserved. The following is the memory map for the voice prompt part of the flash.

When OTA is initialized, the new package will first be programmed to the buffer space. After the transmission is complete, then the language ID of the new arrival voice prompt package will be checked.

- If the new arrival package has the same language ID as one of the existing voice prompts in flash, the original content of that language prompt in flash will be overwritten by the new arrival one. And the current prompt language will be switched to the new one.

- If the new arrival package has a new language ID comparing to the existing ones, if there is empty slot will be checked.

    a. If there is an empty slot, the new arrival package will be programed to the empty slot.

    b. If there is no empty slot (there are two existing voice prompt packages), the least recently written slot will be overwritten by the new one. For instance, if there is a default language prompt package, English prompt, in slot 1, and then a Chinese voice prompt is programmed to slot 2 using OTA. After that, a Germany prompt package should be OTA to the device, then the English prompt package in slot 1 will be overwritten by the new arrival Germany prompt package.

5.7.2 Volume Level Configuration and Volume Control

5.7.2.1 Music and Call

The system has 17 levels of volume gain. The configurations are as follows.

```
const struct CODEC_DAC_VOL_T codec_dac_vol[TGT_VOLUME_LEVEL_QTY] = {
    {TX_PA_GAIN,0x03,-99},
    {TX_PA_GAIN,0x03,-45},
    {TX_PA_GAIN,0x03,-42},
    {TX_PA_GAIN,0x03,-39},
    {TX_PA_GAIN,0x03,-36},
    {TX_PA_GAIN,0x03,-33},
    {TX_PA_GAIN,0x03,-30},
    {TX_PA_GAIN,0x03,-27},
    {TX_PA_GAIN,0x03,-24},
    {TX_PA_GAIN,0x03,-21},
    {TX_PA_GAIN,0x03,-18},
    {TX_PA_GAIN,0x03,-15},
    {TX_PA_GAIN,0x03,-12},
    {TX_PA_GAIN,0x03, -9},
    {TX_PA_GAIN,0x03, -6},
    {TX_PA_GAIN,0x03, -3},
    {TX_PA_GAIN,0x03,  0},  //0dBm
};
```

The system can record the information (such as the volume) of the last connection of several devices and update information to the current connection status.

```
typedef struct {
    uint32_t pairedDevNum;
    nvrec_btdevicerecord pairedBtDevInfo[MAX_BT_PAIRED_DEVICE_COUNT];
```

```
} NV_RECORD_PAIRED_BT_DEV_INFO_T;

typedef struct {
    btif_device_record_t record;
    btdevice_volume device_vol;
    btdevice_profile device_plf;
#ifdef BTIF_DIP_DEVICE
    bt_dip_pnp_info_t pnp_info;
#endif
} nvrec_btdevicerecord;

typedef struct btdevice_profile {
    bool hfp_act;
    uint8_t a2dp_abs_vol;
    bool a2dp_act;
    uint8_t a2dp_codectype;
} btdevice_profile;

typedef struct btdevice_volume
{
    int8_t      a2dp_vol;
    int8_t      hfp_vol;
} btdevice_volume;
```

The interface for changing the volume is as follows:

- Adjust the volume through the phone.

```
void app_ibrt_if_a2dp_send_volume_up(uint8_t device_id)
void app_ibrt_if_a2dp_send_volume_down(uint8_t device_id)
void app_ibrt_if_a2dp_send_set_abs_volume(uint8_t device_id, uint8_t
volume)
```

- Adjust the volume through the earphone.

```
void app_ibrt_if_set_local_volume_up(void)
void app_ibrt_if_set_local_volume_down(void)
```

During music playback, the local volume is converted to Bluetooth volume by normalization, ranging from 0 to 127.

```
#define MAX_A2DP_VOL   (127)
uint8_t a2dp_convert_local_vol_to_bt_vol(uint8_t localVol)
```

```
{
    return unsigned_range_value_map(localVol, TGT_VOLUME_LEVEL_MUTE,
TGT_VOLUME_LEVEL_MAX, 0, MAX_A2DP_VOL);
}
```

During call playback, the local volume is converted to Bluetooth volume by normalization, ranging from 0 to 15.

```
#define MAX_HFP_VOL    (15)
#define MIN_HFP_VOL    (0)
#define HFP_DEFAULT_VOLUME (7)
uint8_t hfp_convert_bt_vol_to_local_vol(uint8_t btVol)
```

Sample Code 14: Volume Level Configuration and Volume Control Test

{"ib_send_volumeup",app_ibrt_send_volumeup_test},

{"ib_send_volumedn",app_ibrt_send_volumedown_test},

{"ib_send_setabsvol",app_ibrt_send_set_absvolume_test},

{"ib_adj_local_volup",app_ibrt_adjust_local_volumeup_test},

{"ib_adj_local_voldn",app_ibrt_adjust_local_volumedown_test},

5.8 Bluetooth Usage Guide

5.8.1 Bluetooth Framework

5.8.1.1 Summary Architecture

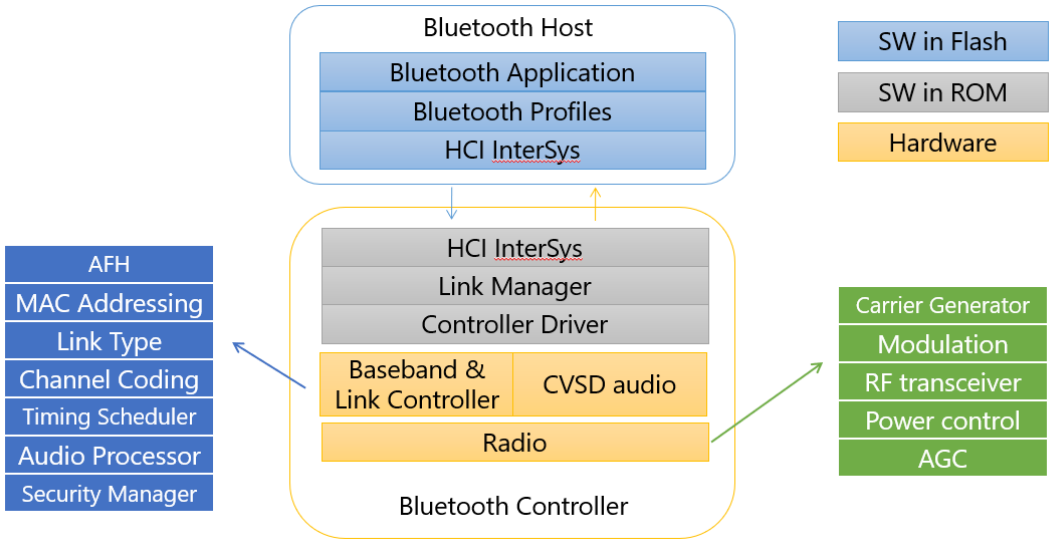


Figure 40 Summary Architecture

5.8.1.2 Software Architecture

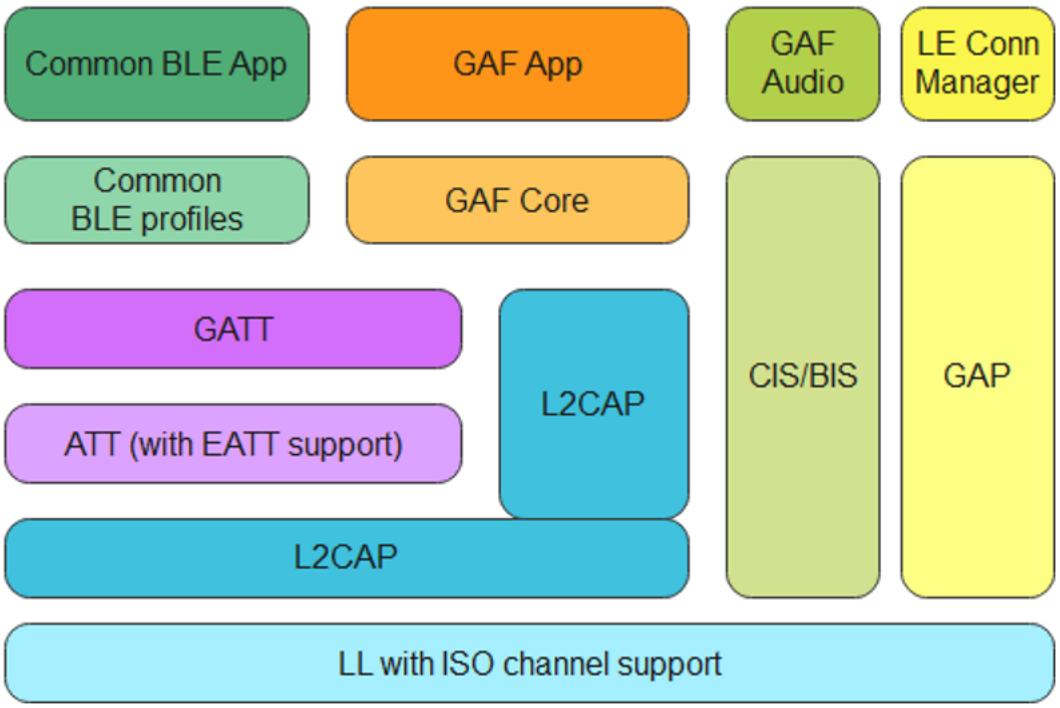


Figure 41 Software Architecture

Bond the corresponding folders of the Bluetooth framework as follows.

-> bthost

--> stack

---> bt_if

Bt interface

---> bt_profiles

The stack section is included.

----> protocols

-----> l2cap

-----> btgatt

---->sdp

… …

---> ble_if

Ble interface

---> ble_ stack

The stack contains several protocol stacks.

---> ble_profiles

---> le_bridge

--> service

---> bt_app **---> ble_app**

Apps with non-BLE functions, such as broadcast functions, etc

-----> a2dp_codecs **---> ble_audio**

Apps related to BLE

-----> audio_policy **----> aob_app**

-----> besmain.cpp **----> gaf_core**

---> bt_source **---> ble_audio**

--> adapter

-> services

--> ibrt_core_v2

IBRT Control: IO Event, SM, etc.

--> ibrt_ui_v2

peripheral manager: Open box, Wear up, etc.

5.8.2 Classic BT

5.8.2.1 BT Mac Address and BT Name Configuration

BT name and BT mac address can be configured in DldProductLine before programming. Both information is written into the factory section after programming.

CONFIDENTIAL

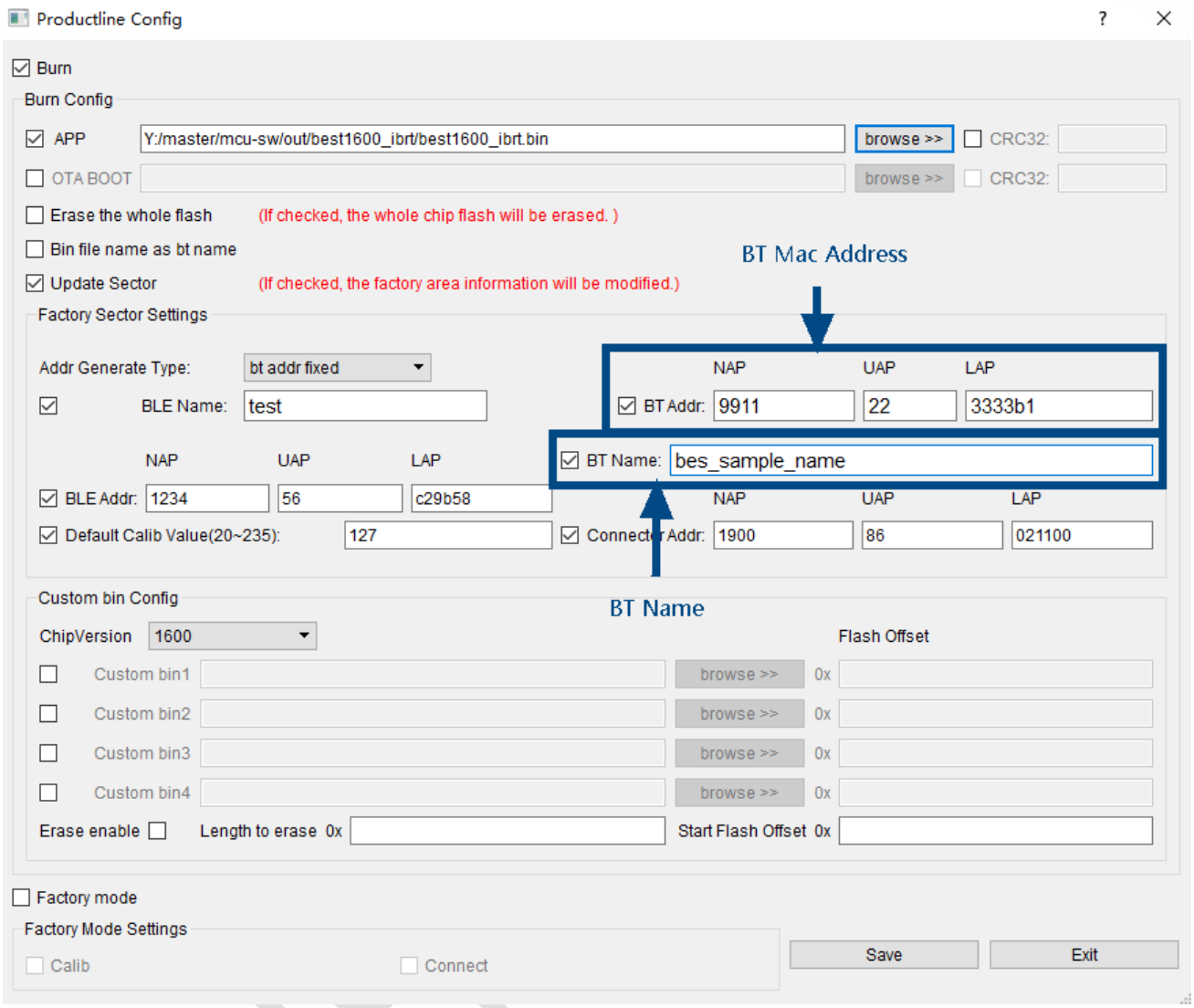


Figure 42 BT Mac Address and Bt Name Configuration

1. Get the BT address.

```
uint8_t *app_ibrt_if_get_bt_local_address(void)
```

2. Update the BT address.

```
void app_ibrt_if_write_bt_local_address(uint8_t* btAddr)
```

Sample Code 15: Get and Write BT Address

{"get_bt_local_address", app_ibrt_if_get_bt_local_address_test},

{"get_bt_peer_address", app_ibrt_if_get_bt_peer_address_test},

{"write_bt_local_address", app_ibrt_if_write_bt_local_address_test},

5.8.2.2 Pairing Test

- Single-ear model

Make sure that the following macros are properly configured in target.mk.

```
POWER_ON_ENTER_FREEMAN_PAIRING_ENABLED ?= 1
POWER_ON_ENTER_TWS_PAIRING_ENABLED ?= 0
```

- TWS model

Make sure that the following macros are properly configured in target.mk.

The master and slave are judged according to the mantissa of the local address. If it's 1, it's the master. If it's 0, it's slave.

```
POWER_ON_ENTER_FREEMAN_PAIRING_ENABLED ?= 0
POWER_ON_ENTER_TWS_PAIRING_ENABLED ?= 1
```

5.8.2.3 Common API List

5.8.2.3.1 Interface Description

1. Get the A2DP stream status of a specific mobile device address.

```
AppIbrtStatus app_ibrt_if_get_a2dp_state(bt_bdaddr_t *addr,
AppIbrtA2dpState *a2dp_state)
```

Table 29 app_ibrt_if_get_a2dp_state Parameters

Parameter	Description
addr	Specify the mobile device address to get AVRCP palyback state
a2dp_state	Used to return A2DP stream state

Sample Code 16: app_ibrt_if_get_a2dp_state

{"get_a2dp_state_test", app_ibrt_if_get_a2dp_state_test},

2. Get the AVRCP playback status of a specific mobile device address.

```
AppIbrtStatus app_ibrt_if_get_avrcp_state(bt_bdaddr_t *addr,
AppIbrtAvrcpState *avrcp_state)
```

Table 30 app_ibrt_if_get_avrcp_state Parameters

Parameter	Description
addr	Specify the mobile device address to get AVRCP palyback state

CONFIDENTIAL

a2dp_state	Used to return AVRCP palyback state

Sample Code 17: app_ibrt_if_get_avrcp_state

{"get_avrcp_state_test", app_ibrt_if_get_avrcp_state_test},

3. Get the HFP status of a specific mobile device address.

```
AppIbrtStatus app_ibrt_if_get_hfp_state(bt_bdaddr_t *addr, AppIbrtHfpState
*hfp_state)
```

Table 31 app_ibrt_if_get_hfp_state Parameters

Parameter	Description
addr	Specify the mobile device address to get HFP state
hfp_state	Used to return HFP state

Sample Code 18: app_ibrt_if_get_hfp_state

{"get_hfp_state_test", app_ibrt_if_get_hfp_state_test},

4. Get the HFP call status of specific mobile device address.

```
AppIbrtStatus app_ibrt_if_get_hfp_call_status(bt_bdaddr_t *addr,
AppIbrtCallStatus *call_status)
```

Table 32 app_ibrt_if_get_hfp_call_status Parameters

Parameter	Description
addr	Specify the mobile device address to get HFP state
hfp_state	Used to return HFP state

Sample Code 19: app_ibrt_if_get_hfp_call_status

{"get_hfp_call_status", app_ibrt_if_get_hfp_call_status_test},

5. Get the current connected mobile count.

```
uint8_t app_ibrt_if_get_connected_mobile_count(void)
```

Sample Code 20: app_ibrt_if_get_connected_mobile_count

{"connected_mobile_count", app_ibrt_if_get_connected_mobile_count_test},

6. Get current connected mobile device count and address list.

```
uint8_t app_ibrt_if_get_mobile_connected_dev_list(bt_bdaddr_t *addr_list)
```

CONFIDENTIAL

Table 33 app_ibrt_if_get_mobile_connected_dev_list Parameters

Parameter	Description
addr_list	Used to return current connected mobile device address list

Sample Code 21: app_ibrt_if_get_mobile_connected_dev_list

```
{"mobile_connected_dev_list",                    app_ibrt_if_get_mobile_connected_dev_list_test},
```

7. Check if the TWS role switch is ongoing.

```
bool app_ibrt_if_is_tws_role_switch_on(void)
```

Sample Code 22: app_ibrt_if_is_tws_role_switch_on

```
{"is_tws_role_switch_on",                    app_ibrt_if_is_tws_role_switch_on_test},
```

8. Perform the role switch operation.

```
void app_ibrt_if_tws_role_switch_request(void)
```

Sample Code 23: app_ibrt_if_tws_role_switch_reques

```
{"role_switch",                    app_ibrt_if_tws_role_switch_test},
```

9. Get the address list of the history paired mobile phones.

```
bool
app_ibrt_if_nvrecord_get_mobile_addr(bt_bdaddr_tmobile_addr_list[],uint8_t
*count)
```

Table 34 app_ibrt_if_nvrecord_get_mobile_addr Parameters

Parameter	Description
mobile_addr_list[]	Address list of the history paired mobiles
hfp_state	Address count

Sample Code 24: app_ibrt_if_nvrecord_get_mobile_addr

```
{"nvrecord_get_mobile_addr",                    app_ibrt_if_nvrecord_get_mobile_addr_test},
```

10. Clear all paired mobile records in the flash.

```
void app_ibrt_if_nvrecord_delete_all_mobile_record(void)
```

Sample Code 25: app_ibrt_if_nvrecord_delete_all_mobile_record

```
{"delete_all_mobile_record",                    app_ibrt_if_nvrecord_delete_all_mobile_record_test},
```

CONFIDENTIAL

11. Get the paired mobile records.

```
bool app_ibrt_if_nvrecord_get_mobile_paired_dev_list(nvrec_btdevicerecord
*nv_record,uint8_t *count)
```

Table 35 app_ibrt_if_nvrecord_get_mobile_paired_dev_list Parameters

Parameter	Description
nv_record	Record list of the history paired mobiles
count	Record count

Sample Code 26: app_ibrt_if_nvrecord_get_mobile_paired_dev_list

{"nvrecord_get_mobile_paired_dev_list", app_ibrt_if_nvrecord_get_mobile_paired_dev_list_test},

12. Check if the TWS link is connected.

```
bool app_ibrt_if_is_tws_link_connected(void)
```

Sample Code 27: app_ibrt_if_is_tws_link_connected

{"is_tws_link_connected", app_ibrt_if_is_tws_link_connected_test},

13. Check if freeman mode is enabled.

```
uint8_t app_ibrt_if_is_in_freeman_mode(void)
```

Sample Code 28: app_ibrt_if_is_in_freeman_mode

{"is_in_freeman_mode", app_ibrt_if_is_in_freeman_mode_test},

14. Disconnect a TWS link.

```
ibrt_status_t app_ibrt_if_tws_disconnect_request(void)
```

Sample Code 29: app_ibrt_if_tws_disconnect_request

{"tws_disconnect_request", app_ibrt_if_tws_disconnect_request_test},

15. Disconnect a mobile link.

```
ibrt_status_t app_ibrt_if_mobile_disconnect_request(const bt_bdaddr_t
*addr,ibrt_post_func post_func)
```

Sample Code 30: app_ibrt_if_mobile_disconnect_request

{"mobile_disconnect_request", app_ibrt_if_mobile_disconnect_request_test},

16. Check if it is in the TWS pairing state.

CONFIDENTIAL

```
bool app_ibrt_if_is_tws_in_pairing_state(void)
```

Sample Code 31: app_ibrt_if_is_tws_in_pairing_state

{"is_tws_in_pairing_state", app_ibrt_conn_is_tws_in_pairing_state_test},

17. Check if the earbud side is left.

```
bool app_ibrt_if_is_left_side(void)
```

Sample Code 32: app_ibrt_if_is_left_side

{"is_left_side", app_ibrt_if_is_left_side_test},

18. Check if the earbud side is right.

```
bool app_ibrt_if_is_right_side(void)
```

Sample Code 33: app_ibrt_if_is_right_side

{"is_right_side", app_ibrt_if_is_right_side_test},

19. Check if the earbud side is unknown.

```
bool app_ibrt_if_unkown_side(void)
```

Sample Code 34: app_ibrt_if_unknown_side

{"is_unknown_side", app_ibrt_if_is_unknown_side_test},

20. Set a earbud side.

```
void app_ibrt_if_set_side(APP_TWS_SIDE_T side)
```

Table 36 app_ibrt_if_set_side Parameters

Parameter	Description
Side	Only can be set to left or right side

Sample Code 35: app_ibrt_if_tws_disconnect_request

{"ibrt_if_set_side", app_ibrt_if_set_side_test},

21. Get the role of an earbud.

```
TWS_UI_ROLE_E app_ibrt_if_get_ui_role(void)
```

Sample Code 36: app_ibrt_if_tws_disconnect_request

{"get_ui_role", app_ibrt_if_get_ui_role_test},

CONFIDENTIAL

22. Convert a role value to a string.

```
const char* app_ibrt_if_uirole2str(TWS_UI_ROLE_E uiRole)
```

Sample Code 37: app_ibrt_if_uirole2str

{"ibrt_if_uirole2str", app_ibrt_if_uirole2str_test},

23. Register a user and its handling functions for specific TWS information sync.

```
void app_ibrt_if_register_sync_user(TWS_SYNC_USER_E id, TWS_SYNC_USER_T
*user)
```

Table 37 app_ibrt_if_register_sync_user Parameters

Parameter	Description
id	User id
user	Handling functions belonging to the user

For the usage of related interfaces, see 5.2.2.7.

24. Unregister a user.

```
void app_ibrt_if_deregister_sync_user(TWS_SYNC_USER_E id)
```

Table 38 app_ibrt_if_deregister_sync_user Parameters

Parameter	Description
id	User id

For the usage of related interfaces, see 5.2.2.7.

25. Initialize and make preparation for TWS synchronization

```
void app_ibrt_if_prepare_sync_info(void)
```

For the usage of related interfaces, see 5.2.2.7.

26. Fill TWS information into the pending sync list.

```
void app_ibrt_if_sync_info(TWS_SYNC_USER_E id)
```

Table 39 app_ibrt_if_sync_info Parameters

Parameter	Description
id	User id for tws information filling

For the usage of related interfaces, see 5.2.2.7.

CONFIDENTIAL

27. Flush pending TWS sync information to a peer device.

```
void app_ibrt_if_flush_sync_info(void)
```

For the usage of related interfaces, see 5.2.2.7.

28. Request a specific TWS bandwidth for a user.

```
void app_ibrt_if_request_modify_tws_bandwidth (TWS_TIMING_CONTROL_USER_E
user, bool enable)
```

Table 40 app_ibrt_if_request_modify_tws_bandwidth Parameters

Parameter	Description
user	User id
enable	true(request)/false(release)

For the usage of related interfaces, see 5.2.2.9.

29. Write a local BT address into factory section and activate it.

```
void app_ibrt_if_write_bt_local_address(uint8_t* btAddr)
```

Table 41 app_ibrt_if_write_bt_local_address Parameters

Parameter	Description
btAddr	User id for tws information filling

For the usage of related interfaces, see 5.2.2.1.

30. Get a local BT address.

```
uint8_t* app_ibrt_if_get_bt_local_address(void)
```

For the usage of related interfaces, see 5.2.2.1.

31. Get a peer BT address.

```
uint8_t* app_ibrt_if_get_bt_peer_address (void)
```

For the usage of related interfaces, see 5.2.2.1.

32. Write a local TWS role and peer device BT address. The TWS pairing won't be started.

```
void app_ibrt_if_update_tws_pairing_info(ibrt_role_e role, uint8_t*
peerAddr)
```

Table 42 app_ibrt_if_update_tws_pairing_info Parameters

CONFIDENTIAL

Parameter	Description
Role	The tws role to update
peerAddr	The peer device bt address to update

For the usage of related interfaces, see 5.2.2.4.

33. Write a local TWS role and peer device BT address. The TWS pairing is immediately started

```
void app_ibrt_if_start_tws_pairing(ibrt_role_e role, uint8_t* peerAddr)
```

Table 43 app_ibrt_if_start_tws_pairing Parameters

Parameter	Description
Role	The tws role to update
peerAddr	The peer device bt address to update

For the usage of related interfaces, see 5.2.2.4.

5.8.2.3.2 Parameter Structure Description

- bt_bdaddr_t

```
/* Bluetooth Device Address */
typedef struct {
    uint8_t address[6];
} __attribute__ ((packed)) bt_bdaddr_t;
```

- AppIbrtA2dpState

```
/* A2DP State */
typedef enum {
    APP_IBRT_IF_A2DP_IDLE               = 0,
    APP_IBRT_IF_A2DP_CODEC_CONFIGURED   = 1,
    APP_IBRT_IF_A2DP_OPEN               = 2,
    APP_IBRT_IF_A2DP_STREAMING          = 3,
    APP_IBRT_IF_A2DP_CLOSED             = 4,
}AppIbrtA2dpState;
```

- AppIbrtAvrcpState

```
/* AVRCP State */
typedef enum {
    APP_IBRT_IF_AVRCP_DISCONNECTED      = 0,
    APP_IBRT_IF_AVRCP_CONNECTED         = 1,
```

```
    APP_IBRT_IF_AVRCP_PLAYING       = 2,
    APP_IBRT_IF_AVRCP_PAUSED        = 3,
    APP_IBRT_IF_AVRCP_VOLUME_UPDATED    = 4,
}AppIbrtAvrcpState;
```

- AppIbrtHfpState

```
/* HFP State */
Typedef enum {
    APP_IBRT_IF_HFP_SLC_DISCONNECTED    = 0,
    APP_IBRT_IF_HFP_CLOSED          = 1,
    APP_IBRT_IF_HFP_SCO_CLOSED          = 2,
    APP_IBRT_IF_HFP_PENDING         = 3,
    APP_IBRT_IF_HFP_SLC_OPEN        = 4,
    APP_IBRT_IF_HFP_NEGOTIATE       = 5,
    APP_IBRT_IF_HFP_CODEC_CONFIGURED    = 6,
    APP_IBRT_IF_HFP_SCO_OPEN        = 7,
    APP_IBRT_IF_HFP_INCOMING_CALL       = 8,
    APP_IBRT_IF_HFP_OUTGOING_CALL       = 9,
    APP_IBRT_IF_HFP_RING_INDICATION     = 10,
} AppIbrtHfpState;
```

- AppIbrtCallStatus

```
/* CALL Status */
typedef enum {
    APP_IBRT_IF_NO_CALL             = 0,
    APP_IBRT_IF_CALL_ACTIVE         = 1,
    APP_IBRT_IF_HOLD                = 2,
    APP_IBRT_IF_SETUP_INCOMMING     = 3,
    APP_IBRT_IF_SETUP_OUTGOING      = 4,
    APP_IBRT_IF_SETUP_ALERT         = 5,
} AppIbrtCallStatus;
```

- nvrec_btdevicerecord

```
/* Bluetooth Device info record */
typedef struct {
btif_device_record_t record;
btdevice_volume device_vol;
btdevice_profile device_plf;
#ifdef BTIF_DIP_DEVICE
```

```
    uint16_t vend_id;
    uint16_t vend_id_source;
    uint16_t reserve;
#endif
} nvrec_btdevicerecord;
```

- ibrt_post_func

```
/* Callback Function */
typedef void (*ibrt_post_func)(void);
```

- APP_TWS_SIDE_T

```
/* Earbuds Side */
typedef enum {
    EAR_SIDE_UNKNOWN   = 0,
    EAR_SIDE_LEFT         = 1,
    EAR_SIDE_RIGHT        = 2,
    EAR_SIDE_NUM,
} APP_TWS_SIDE_T;
```

- TWS_SYNC_USER_T

```
/* Functions used to handle tws information sync*/
typedef struct {
    TWS_SYNC_INFO_PREPARE_FUNC_T sync_info_prepare_handler;
    TWS_INFO_SYNC_FUNC_T sync_info_received_handler;
    TWS_SYNC_INFO_PREPARE_FUNC_T sync_info_prepare_rsp_handler;
    TWS_INFO_SYNC_FUNC_T sync_info_rsp_received_handler;
    TWS_INFO_SYNC_FUNC_T sync_info_rsp_timeout_handler;
} TWS_SYNC_USER_T;
```

- TWS_SYNC_USER_E

```
/* Users need sync tws information */
typedef enum {
    TWS_SYNC_USER_BLE_INFO       = 0,
    TWS_SYNC_USER_OTA               = 1,
    TWS_SYNC_USER_AI_CONNECTION  = 2,
    TWS_SYNC_USER_GFPS_INFO      = 3,
    TWS_SYNC_USER_AI_INFO        = 4,
    TWS_SYNC_USER_AI_MANAGER     = 5,
    TWS_SYNC_USER_DIP               = 6,
```

```
    TWS_SYNC_USER_NUM,
} TWS_SYNC_USER_E;
```

- TWS_TIMING_CONTROL_USER_E

```
/* Users of the tws timing control */
typedef enum
{
    TWS_TIMING_CONTROL_USER_SNIFF   = 0,
    TWS_TIMING_CONTROL_USER_OTA,
    TWS_TIMING_CONTROL_USER_AI_VOICE,
    TWS_TIMING_CONTROL_USER_DEFAULT,
    TWS_TIMING_CONTROL_USER_START_IBRT,
    TWS_TIMING_CONTROL_USER_RESERVE1,
    TWS_TIMING_CONTROL_USER_NUM,
} TWS_TIMING_CONTROL_USER_E;
```

- ibrt_role_e

```
/* TWS role */
typedef uint8_t ibrt_role_e;
#define   IBRT_MASTER          0
#define   IBRT_SLAVE           1
#define   IBRT_UNKNOW          0xff
```

5.8.2.3.3 Return Value Description

- ApplbrtStatus

```
typedef enum {
    APP_IBRT_IF_STATUS_SUCCESS                = 0,
    APP_IBRT_IF_STATUS_PENDING                = 1,
    APP_IBRT_IF_STATUS_ERROR_INVALID_PARAMETERS  = 2,
    APP_IBRT_IF_STATUS_ERROR_NO_CONNECTION       = 3,
    APP_IBRT_IF_STATUS_ERROR_CONNECTION_EXISTS   = 4,
    APP_IBRT_IF_STATUS_IN_PROGRESS            = 5,
    APP_IBRT_IF_STATUS_ERROR_DUPLICATE_REQUEST   = 6,
    APP_IBRT_IF_STATUS_ERROR_INVALID_STATE       = 7,
    APP_IBRT_IF_STATUS_ERROR_TIMEOUT             = 8,
    APP_IBRT_IF_STATUS_ERROR_ROLE_SWITCH_FAILED  = 9,
    APP_IBRT_IF_STATUS_ERROR_UNEXPECTED_VALUE    = 10,
    APP_IBRT_IF_STATUS_ERROR_OP_NOT_ALLOWED      = 11,
```

```
} AppIbrtStatus;
```

- ibrt_status_t

```
typedef enum {
    IBRT_STATUS_SUCCESS                = 0,
    IBRT_STATUS_PENDING                = 1,
    IBRT_STATUS_ERROR_INVALID_PARAMETERS   = 2,
    IBRT_STATUS_ERROR_NO_CONNECTION        = 3,
    IBRT_STATUS_ERROR_CONNECTION_EXISTS  = 4,
    IBRT_STATUS_IN_PROGRESS            = 5,
    IBRT_STATUS_ERROR_DUPLICATE_REQUEST  = 6,
    IBRT_STATUS_ERROR_INVALID_STATE        = 7,
    IBRT_STATUS_ERROR_TIMEOUT          = 8,
    IBRT_STATUS_ERROR_ROLE_SWITCH_FAILED   = 9,
    IBRT_STATUS_ERROR_UNEXPECTED_VALUE     = 10,
    IBRT_STATUS_ERROR_OP_NOT_ALLOWED       = 11,
    IBRT_STATUS_VENDOR_START           = 0x80,
} ibrt_status_t;
```

- TWS_UI_ROLE_E

```
TWS_UI_MASTER      0
TWS_UI_SLAVE       1
TWS_UI_UNKNOWN     0xff
```

- TWS_SYNC_FILL_RET_E

```
typedef enum
{
    TWS_SYNC_CONTINUE_FILLING    = 0,
    TWS_SYNC_BUF_FULL            = 1,
    TWS_SYNC_NO_DATA_FILLED      = 3
} TWS_SYNC_FILL_RET_E;
```

5.8.2.4 Custom UI Usage

1. Call IBRT APIs that are defined in the common API List.
2. Register a callback handler list to receive the Bluetooth events distributed from the custom UX layer of the SDK:

```
void app_custom_ux_register_callback(ibrt_core_status_changed_cb_t *cb)
```

Table 44 Ibrt_core_status_changed_cb_t Description

Item	Description
ibrt_core_status_changed_cb_t	void (*global_state_changed)(ibrt_global_state_change_event *state);
	void (*a2dp_state_changed)(const bt_bdaddr_t *addr, ibrt_conn_a2dp_state_change *state);
	void (*hfp_state_changed)(const bt_bdaddr_t *addr, ibrt_conn_hfp_state_change *state);
	void (*avrcp_state_changed)(const bt_bdaddr_t *addr, ibrt_conn_avrcp_state_change *state);
	void (*tws_acl_state_changed)(ibrt_conn_tws_conn_state_event *state, uint8_t reason_code);
	void (*mobile_acl_state_changed)(const bt_bdaddr_t *addr, ibrt_mobile_conn_state_event *state, uint8_t reason_code);
	void (*tws_role_switch_status_ind)(const bt_bdaddr_t *addr, ibrt_conn_role_change_state state, ibrt_role_e role);
	void (*access_mode_changed)(btif_accessible_mode_t newAccessMode);
	ibrt_conn_req_rsp_t (*incoming_connect_request_response_cb)(void);
	void (*sco_disconnect_cb)(const bt_bdaddr_t *addr, uint8_t reason_code);
	void (*dip_info_queried_cb)(uint8_t *remote_addr, bt_dip_pnp_info_t *pnp_info);
	void (*bsir_event_cb)(uint8_t is_in_band_ring);

3. How to configure TWS pairing for the custom UI.

- Write a local TWS role and peer device BT address. The TWS pairing won't be started.

```
void app_ibrt_if_update_tws_pairing_info(ibrt_role_e role, uint8_t*
peerAddr);
```

Table 45 app_ibrt_if_update_tws_pairing_info Parameters

Parameter	Description
Role	The tws role to update
peerAddr	The peer device bt address to update

- Write a local TWS role and peer device BT address. The TWS pairing is immediately started.

```
void app_ibrt_if_start_tws_pairing(ibrt_role_e role, uint8_t* peerAddr);
```

Table 46 app_ibrt_if_start_tws_pairing Parameters

Parameter	Description

Role	The tws role to update
peerAddr	The peer device bt address to update

Sample Code 38: Simple Pairing Test

{"slave_update_tws_pair", app_slave_update_tws_pair_info_test},

{"master_update_tws_pair", app_master_update_tws_pair_info_test},

{"master_start_tws_pair", app_master_start_tws_pairing_test},

{"slave_start_tws_pair", app_slave_start_tws_pairing_test}

5.8.2.5 BES UI Usage

5.8.2.5.1 TWS Role's Judgment Strategy

nv-role: Not changed.

bt-role: During a BT connection, the device that initiates the connection is the master, and the device that is connected is the slave. In the connected state, no change occurs.

ibrt-role: In IBRT, the device that actually connects to the mobile phone is the master, and the device that listens is the slave. The master and slave are changed in the following events.

- When the master earphone is inserted into the box, the external slave earphone becomes the master.
- The master earphone signal is bad, the slave earphone becomes the master.
- The power level of the left and right earphone is unbalanced, and the master-slave transmission is switched.
- Events defined by the customers.

ui-role: It is consistent with IBRT-role, except the case that both earphones are masters when they are not connected.

5.8.2.5.2 API List

- Interface to enter the event:

```
void app_ibrt_if_event_entry(ibrt_mgr_evt_t event)
```

Table 47 app_ibrt_if_event_entry Description

Item	Description	Notes
ibrt_mgr_evt_t	IBRT_MGR_EV_CASE_OPEN	Open box
	IBRT_MGR_EV_UNDOCK	Fetch out box
	IBRT_MGR_EV_DOCK	Put in box
	IBRT_MGR_EV_CASE_CLOSE	Close box

CONFIDENTIAL

	IBRT_MGR_EV_WEAR_UP	Wear up
	IBRT_MGR_EV_TWS_PAIRING	Tws pairing
	IBRT_MGR_EV_FREE_MAN_MODE	Free man mode

- Interface to enter the event freeman mode:

```
void app_ibrt_if_test_enter_freeman(void)
```

- Interface to dump info:

```
void app_ibrt_if_dump_ibrt_mgr_info()
```

- Custom TWS switch interface:

```
void app_ibrt_customif_ui_tws_switch(void)
```

- Custom TWS switching check interface. Checks whether the TWS switch is performing now, and returns true or false.

```
bool app_ibrt_customif_ui_is_tws_switching(void)
```

- Callback for the BES UI:

```
void
app_ibrt_if_register_client_callback(APP_IBRT_IF_LINK_STATUS_CHANGED_CALLBACK* cbs)
{
    client_cb = cbs;
}
typedef ibrt_link_status_changed_cb_t
APP_IBRT_IF_LINK_STATUS_CHANGED_CALLBACK;
```

Table 48 ibrt_core_status_changed_cb_t Description

Item	Description
ibrt_link_status_changed_cb_t	void (*ibrt_global_state_changed)(ibrt_global_state_change_event *state);
	void (*ibrt_a2dp_state_changed)(const bt_bdaddr_t* addr, ibrt_conn_a2dp_state_change *state);
	void (*ibrt_hfp_state_changed)(const bt_bdaddr_t* addr, ibrt_conn_hfp_state_change *state);
	void (*ibrt_avrcp_state_changed)(const bt_bdaddr_t *addr, ibrt_conn_avrcp_state_change *state);

	void (*ibrt_tws_pairing_changed)(ibrt_conn_pairing_state state,uint8_t reason_code);
	void (*ibrt_tws_acl_state_changed)(ibrt_conn_tws_conn_state_event *state,uint8_t reason_code);
	void (*ibrt_mobile_acl_state_changed)(const bt_bdaddr_t *addr, ibrt_mobile_conn_state_event *state, uint8_t reason_code);
	void (*ibrt_sco_state_changed)(const bt_bdaddr_t *addr, ibrt_sco_conn_state_event *state,uint8_t reason_code);
	void (*ibrt_tws_role_switch_status_ind)(const bt_bdaddr_t *addr,ibrt_conn_role_change_state state,ibrt_role_e role);
	void (*ibrt_ibrt_state_changed)(const bt_bdaddr_t *addr, ibrt_connection_state_event* state, ibrt_role_e role,uint8_t reason_code);
	void (*ibrt_access_mode_changed)(btif_accessible_mode_t newAccessMode);

Sample Code 39: BES UI Usage

```
{"open_box_event_test",app_ibrt_if_event_entry_open_box_test},
{"fetch_out_box_event_test",app_ibrt_if_event_entry_fetch_out_box_test},
{"put_in_box_event_test",app_ibrt_if_event_entry_put_in_box_test},
{"close_box_event_test",app_ibrt_if_event_entry_close_box_test},
{"wear_up_event_test",app_ibrt_if_event_entry_wear_up_test},
{"wear_down_event_test",app_ibrt_if_event_entry_wear_down_test},
{"pairing_mode_test",app_ibrt_if_pairing_mode_test},
{"free_man_test",app_ibrt_if_event_entry_freeman_test},
{"dump_mgr_info_test",app_ibrt_if_event_entry_dump_info_test},
{"ui_tws_switch",app_ibrt_customif_ui_tws_switch_test},
{"check_ui_tws_switch",app_ibrt_customif_ui_is_tws_switching_test},
{"register_client_callback",app_ibrt_if_register_client_callback_test},
```

5.8.2.5.3 BES UI Property Configuration

Key functions for calls:

app_ibrt_customif_ui_start -> ibrt_mgr_config_t

CONFIDENTIAL

Table 49 BES UI Property Configuration

Item	Description
bool freeman_enable	freeman mode config, default should be false.
bool tws_use_same_addr	tws earphone set the same addr, UI will be flexible, default should be true
bool wear_updown_tws_switch_enable	ibrt slave will reconnect to mobile if tws connect failed, default should be true
bool slave_reconnect_enable	do tws switch when wearup or weardown, must be true because MIC will be with IBRT master
bool enter_pairing_mode	pairing mode default value, default should be set false
bool freeman_accept_mobile_new_pairing	following cases the reconnect will be fail for freeman, please set to true if you want to reconnect successful: 1. freeman has link key but mobile deleted the link key 2. freeman changed its bt address after reboot and use the new address to reconnect mobile
bool enter_pairing_on_empty_mobile_addr	for some proj no box key, default should be false;
bool enter_pairing_on_reconnect_mobile_failed	for some proj no box key, default should be false
bool enter_pairing_on_reconnect_mobile_failed_once	when mobile has connected, enter_pairing_on_reconnect_mobile_failed will be cleared, default false
bool enter_pairing_on_mobile_disconnect	for some proj no box key, default should be false
bool check_plugin_excute_closedbox_event;	default should be true
bool disc_tws_before_reconnect_mobile;	for 08 error reconnect event, default must be true
bool snoop_via_ble_enable;	exchange snoop info by BLE_box, special custom config, default should be false
bool tws_switch_according_to_rssi_value;	do tws switch when RSII value change, default should be true
bool tws_switch_according_to_noise_value;	do tws switch when noise value change, default should be true
bool disable_tws_switch;	disable tws switch, NOT recommended to open
bool disable_stop_ibrt;	disable tws switch, NOT recommended to open
bool lowlayer_monitor_enable;	controller baseband monitor
bool delay_exit_sniff;	default should be false
bool share_tws_info_done;	default should be false
uint32_t delay_ms_exit_sniff;	default should be 3000
bool mobile_incoming_filter_unpaired;	only allow paired mobile device incoming when not in paring mode, default should be false

CONFIDENTIAL

uint8_t rssi_threshold;	do tws switch when rssi value change over threshold. default should be 10dm. #define IBRT_UI_ROLE_SWITCH_THRESHOLD_WITH_RSSI (10) //dbm
int32_t noise_count_threshold;	do tws switch when noise count value change over threshold. #define IBRT_UI_ROLE_SWITCH_THRESHOLD_WITH_NOISE_COUNT (2) //times
uint8_t role_switch_timer_threshold;	do tws switch when RSII value change, timer threshold #define IBRT_UI_ROLE_SWITCH_TIME_THRESHOLD (2) //
uint8_t noise_role_switch_timer_threshold;	
uint16_t close_box_event_wait_response_timeout;	close box debounce time config #define IBRT_UI_CLOSE_BOX_EVENT_WAIT_RESPONSE_TIMEOUT (600)//ms
uint16_t reconnect_wait_ready_timeout;	reconnect event internal config wait timer when tws disconnect #define IBRT_UI_MOBILE_RECONNECT_WAIT_READY_TIMEOUT (1000)//ms
uint16_t reconnect_mobile_wait_ready_timeout;	reconnect event internal config wait timer when tws disconnect #define IBRT_UI_MOBILE_RECONNECT_WAIT_READY_TIMEOUT (1000)//ms
uint16_t reconnect_tws_wait_ready_timeout;	reconnect event internal config wait timer when tws disconnect #define IBRT_UI_TWS_RECONNECT_WAIT_READY_TIMEOUT (1500)//ms
uint16_t reconnect_mobile_wait_response_timeout;	wait time before launch reconnect event #define IBRT_UI_RECONNECT_MOBILE_WAIT_RESPONSE_TIMEOUT (5000)//5s
uint16_t reconnect_ibrt_wait_response_timeout;	#define IBRT_UI_RECONNECT_IBRT_WAIT_RESPONSE_TIMEO

CONFIDENTIAL

	UT (300)//ms
uint16_t nv_master_reconnect_tws_wait_response_timeout;	#define IBRT_UI_NV_MASTER_RECONNECT_TWS_WAIT_RESPONSE_TIMEOUT (300)//ms
uint16_t nv_slave_reconnect_tws_wait_response_timeout;	#define IBRT_UI_NV_SLAVE_RECONNECT_TWS_WAIT_RESPONSE_TIMEOUT (1000)//ms
uint32_t disable_bt_scan_timeout;	pairing mode timeout config #define IBRT_UI_DISABLE_BT_SCAN_TIMEOUT (150000)//5min
uint16_t open_reconnect_mobile_max_times;	open box reconnect mobile times config #define IBRT_UI_OPEN_RECONNECT_MOBILE_MAX_TIMES (0)
uint16_t open_reconnect_tws_max_times;	open box reconnect tws times config #define IBRT_UI_OPEN_RECONNECT_TWS_MAX_TIMES (1)
uint16_t reconnect_mobile_max_times;	open box reconnect mobile times config #define IBRT_UI_OPEN_RECONNECT_MOBILE_MAX_TIMES (0)
uint16_t reconnect_tws_max_times;	open box reconnect tws times config #define IBRT_UI_OPEN_RECONNECT_TWS_MAX_TIMES (1)
uint16_t reconnect_ibrt_max_times;	connection timeout reconnect ibrt times config #define IBRT_UI_RECONNECT_IBRT_MAX_TIMES (1)
uint8_t tws_reconnect_cycle;	reconnect tws one cycle #define IBRT_TWS_RECONNECT_ONE_CYCLE (3)
uint8_t mobile_reconnect_cycle;	reconnect mobile one cycle #define IBRT_MOBILE_RECONNECT_ONE_CYCLE (5)
uint16_t default_bt_tpoll;	Default polling time #define IBRT_TWS_BT_TPOLL_DEFAULT (80)
uint16_t tws_page_timeout_on_last_success;	for fast connect when only one headset in the nearby

CONFIDENTIAL

	#define IBRT_TWS_PAGE_TIMEOUT_ON_CONNECT_SUCCESS_LAST (0x2000)
uint16_t tws_page_timeout_on_last_failed;	#define IBRT_TWS_PAGE_TIMEOUT_ON_CONNECT_FAILED_LAST (0x2000)
uint16_t tws_page_timeout_on_reconnect_mobile_success;	#define IBRT_TWS_PAGE_TIMEOUT_ON_RECONNECT_MOBILE_SUCCESS (0x2000)
uint16_t tws_page_timeout_on_reconnect_mobile_failed;	#define IBRT_TWS_PAGE_TIMEOUT_ON_RECONNECT_MOBILE_FAILED (0x2000)
uint16_t mobile_page_timeout;	#define IBRT_MOBILE_PAGE_TIMEOUT (0x2000)
uint16_t tws_connection_timeout;	tws connection timeout #define IBRT_UI_TWS_CONNECTION_TIMEOUT (3200)//1600 slot=1s/3200 slot = 2s/8000slot=5s
uint8_t profile_concurrency_supported;	false
uint8_t connect_new_mobile_enable;	
uint16_t wait_time_before_disc_tws;	3000
bt_bdaddr_t new_mobile_address;	
uint32_t rx_seq_error_timeout;	#define IBRT_UI_RX_SEQ_ERROR_TIMEOUT (10000)
uint32_t rx_seq_error_threshold;	define IBRT_UI_RX_SEQ_ERROR_THRESHOLD (50)
uint32_t rx_seq_recover_wait_timeout;	#define IBRT_UI_RX_SEQ_ERROR_RECOVER_TIMEOUT (5000)

uint32_t rssi_monitor_timeout;	#ifdef TEST_OVER_THE_AIR_ENANBLED
	#define IBRT_UI_RSSI_MONITOR_TIMEOUT (1000)//ms
	#else
	#define IBRT_UI_RSSI_MONITOR_TIMEOUT (5000)//ms

uint32_t noise_monitor_timeout;	#define IBRT_UI_NOISE_MONITOR_TIMEOUT (1000)//ms
bool wear_updown_detect_supported;	false
uint32_t stop_ibrt_timeout;	#define IBRT_UI_STOP_IBRT_TIMEOUT (3000000)//40min

CONFIDENTIAL

uint16_t radical_scan_interval_nv_slave;	#define BTIF_BT_DEFAULT_PAGE_SCAN_INTERVAL 0x800 #define IBRT_UI_RADICAL_SAN_INTERVAL_NV_SLAVE ((BTIF_BT_DEFAULT_PAGE_SCAN_INTERVAL/4)-0x48)
uint16_t radical_scan_interval_nv_master;	#define BTIF_BT_DEFAULT_PAGE_SCAN_INTERVAL 0x800 #define IBRT_UI_RADICAL_SAN_INTERVAL_NV_MASTER (BTIF_BT_DEFAULT_PAGE_SCAN_INTERVAL/4)
uint32_t event_hung_timeout;	#define IBRT_EVENT_HUNG_TIMEOUT (200)//ms
uint16_t rssi_tws_switch_threshold;	#define IBRT_TWS_SWITCH_RSSI_THRESHOLD (30)
uint32_t stop_ibrt_wait_time_after_tws_switch;	#define IBRT_STOP_IBRT_WAIT_TIME (500)
uint32_t tws_conn_failed_wait_time;	#define TWS_CONN_FAILED_WAIT_TIME (5)
uint32_t sm_running_timeout;	
uint32_t peer_sm_running_timeout;	#define SM_RUNNING_TIMEOUT (20000)
uint32_t connect_no_03_timeout;	#define CONNECT_NO_03_TIMEOUT (15000)
uint32_t disconnect_no_05_timeout;	#define DISCONNECT_NO_05_TIMEOUT (8000)
uint32_t reconnect_peer_sm_running_timeout;	#define IBRT_UI_RECONNECT_MOBILE_MAX_TIMES (20) #define IBRT_UI_RECONNECT_MOBILE_WAIT_RESPONSE_TIMEOUT (5000)//5s #define RECONNECT_PEER_SM_RUNNING_TIMEOUT (2*(IBRT_UI_RECONNECT_MOBILE_MAX_TIMES * (IBRT_UI_RECONNECT_MOBILE_WAIT_RESPONSE_TIMEOUT+5000))) #define CONNECT_NO_03_TIMEOUT (15000)
bool tws_switch_tx_data_protect;	true
uint32_t tws_cmd_send_timeout;	#define IBRT_UI_TWS_CMD_SEND_TIMEOUT

CONFIDENTIAL

	(10000)
uint32_t tws_cmd_send_counter_threshold;	#define IBRT_UI_TWS_CMD_SEND_TIMEOUT (10000)
uint32_t tws_switch_stable_timeout;	#define IBRT_UI_TWS_SWITCH_STABLE_TIMEOUT (300)
bool invoke_event_when_box_closed;	true
uint8_t audio_sync_mismatch_resume_version;	2
uint8_t llmonitor_report_format;	
uint32_t llmonitor_report_count;	

5.8.2.5.4 Pairing Information Management

- Interface to get all paired BT link keys

```
bool app_ibrt_if_get_all_paired_bt_link_key(ibrt_if_paired_bt_link_key_info
*linkKey)

typedef struct {
    int pairedDevNum;
    ibrt_if_link_key_info linkKey[MAX_BT_PAIRED_DEVICE_COUNT];
} ibrt_if_paired_bt_link_key_info;

typedef struct {
    bt_bdaddr_t btAddr;
    uint8_t linkKey[16];
} ibrt_if_link_key_info;

typedef struct {
    bt_bdaddr_t btAddr;
    uint8_t linkKey[16];
} ibrt_if_link_key_info;
```

- Interface to delete all pairing records

```
void app_ibrt_nvrecord_delete_all_bt_pairing_record(void)
```

- Interface to delete all mobile records

```
void app_ibrt_nvrecord_delete_all_mobile_record(void);
```

- Interface to record the paired mobile records to the NV

```
bt_status_t
app_ibrt_nvrecord_get_mobile_paired_dev_list(nvrec_btdevicerecord
*nv_record,uint8_t *count);

typedef struct {
    btif_device_record_t record;
    btdevice_volume device_vol;
    btdevice_profile device_plf;
#ifdef BTIF_DIP_DEVICE
    bt_dip_pnp_info_t pnp_info;
#endif
} nvrec_btdevicerecord;
```

For the test cases of related interfaces, see 5.2.2.3.1(9-11)

5.8.2.6 IBRT Role Switch

5.8.2.6.1 Host Flow

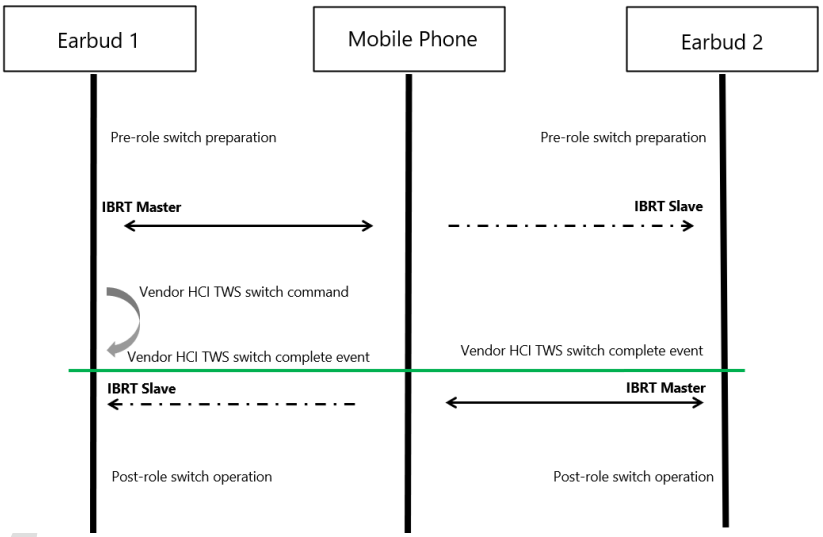


Figure 43 Host Flow

5.8.2.6.2 Role Switch Preparation

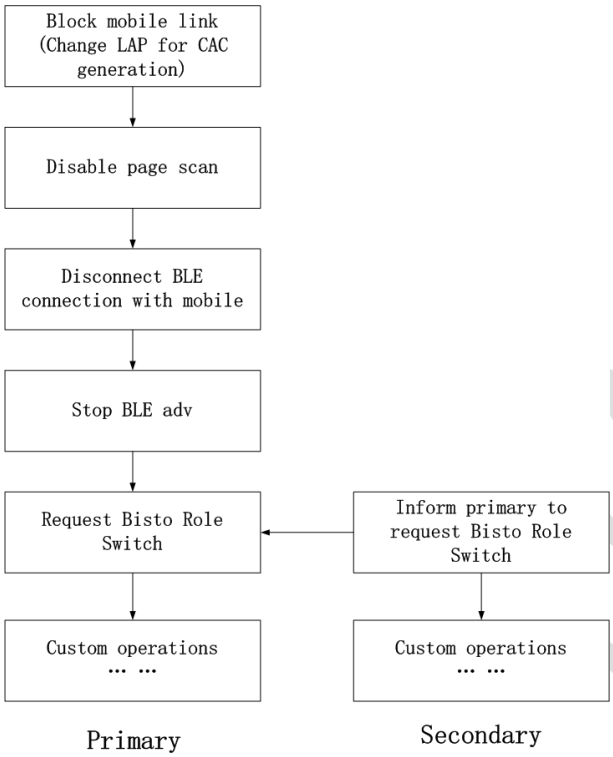
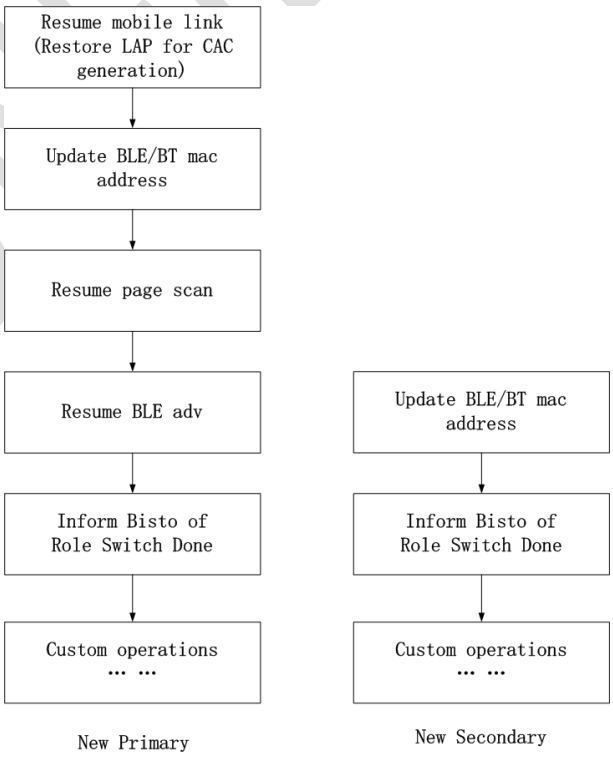


Figure 44 Flow Path of Role Switch Preparation

5.8.2.6.3 Role Switch Post Operation



CONFIDENTIAL

Figure 45 Flow Path of Role Switch Post Operation

5.8.2.6.4 Role Switch Controller Operation

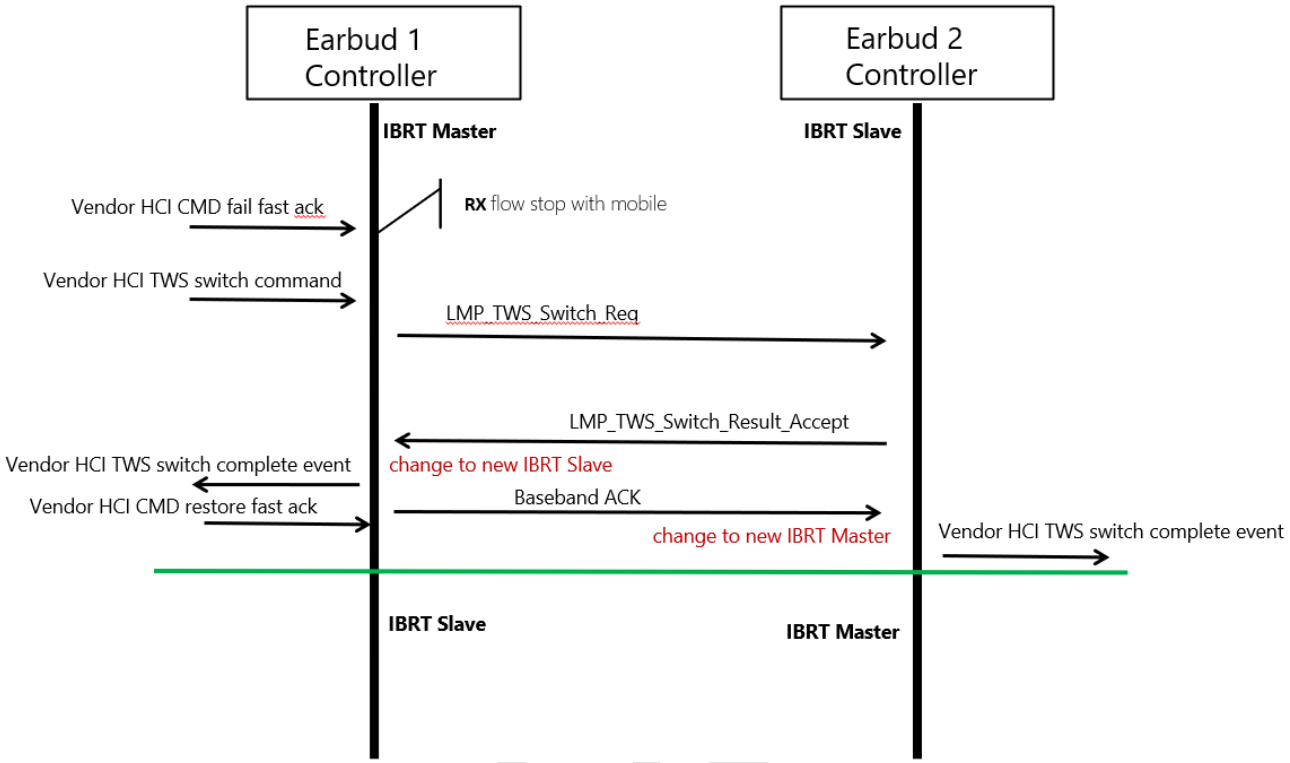


Figure 46 Role Switch Controller Operation

For the test cases of related interfaces, see 7.2.3.1 (7-8).

5.8.2.7 TWS Synchronization Interface Usage

- TWS information sync

Earbuds need to synchronize information with each other, and the different TWS information is defined into different users, such as AI, BLE, and OTA. If you want to synchronize a new kind of information, you may need to add a user.

Take user “BLE”as an example, you can add a user by performing the following steps:

1. Add “TWS_SYNC_USER_BLE_INFO”into TWS_SYNC_USER_E;
2. Implement the handling functions and fill them into TWS_SYNC_USER_T, for example, the name is “userBle”;

Table 50 TWS_SYNC_USER_T Description

Item	Description	Notes
TWS_SYNC_USER_T	sync_info_prepare_handler	Collect BLE information to be sent to a peer device
	sync_info_received_handler	Process the received BLE sync information from the peer device

	sync_info_prepare_rsp_handler	Prepare BLE information to reponse
	sync_info_rsp_received_handler	Process the received response from the peer device
	sync_info_rsp_timeout_handler	The response timeout handler

3. Register a user and its handle function to the TWS information sync list by using "app_ibrt_if_register_sync_user(TWS_SYNC_USER_BLE_INFO, &userBle)".

```
void app_ble_mode_tws_sync_init(void)
{
    TWS_SYNC_USER_T userBle = {
        ble_sync_info_prepare_handler,
        ble_sync_info_received_handler,
        ble_sync_info_prepare_handler,
        ble_sync_info_rsp_received_handler,
        NULL,
    };

    app_ibrt_if_register_sync_user(TWS_SYNC_USER_BLE_INFO, &userBle);
}
```

4. Synchronize the TWS information of the BLE user.
   Firstly, call "app_ibrt_if_prepare_sync_info"to Initialize and make preparation for TWS synchronization;
   Then, call "app_ibrt_if_sync_info"to fill TWS information into the pending sync list; The TWS sync information can be filled at this stage.
   Finally, call"app_ibrt_if_flush_sync_info"to flush the pending TWS sync information to the peer device.

```
void app_ble_sync_ble_info(void)
{
    app_ibrt_if_prepare_sync_info();
    app_ibrt_if_sync_info(TWS_SYNC_USER_BLE_INFO);
    app_ibrt_if_flush_sync_info();
}
#endif
```

5.8.2.8 Sniff and Active Mode Management

The sniff mode saves power by reducing the number of time slots for the master device to send data and the corresponding number of time slots for the slave device to listen. The sniff mode is mainly controlled by the BTC. You can control the sniff mode by using the following interfaces.

5.8.2.8.1 Prevent Entering Sniff Mode

```
void app_ibrt_if_prevent_sniff_set(uint8_t *p_mobile_addr, uint16_t
prv_sniff_bit)

/// Definition of the bits preventing the BTC from enter sniff
enum app_ibrt_if_prevent_sniff
{
    /// Flag indicating that the OTA process is ongoing
    OTA_ONGOING                     = 0x01,
    /// Flag indicating that vocie record is ongoing
    AI_VOICE_RECORD                 = 0x02,
    /// Flag indicating that AVRCP status change is ongoing
    AVRCP_STATUS_CHANING        = 0x04,
};
```

5.8.2.8.2 Disabling 'APP_IBRT_IF_PREVENT_SNIFF_SET'

```
void app_ibrt_if_prevent_sniff_clear(uint8_t *p_mobile_addr, uint16_t
prv_sniff_bit)
```

Next, take entering OTA mode as an example. When the system enters the OTA mode, exit the sniff mode and then prevent the sniff mode.

```
static POSSIBLY_UNUSED void Bes_enter_ota_state(void)
{
    LOG_DBG(0,"%s state %d", __func__, isInBesOtaState);

    if (isInBesOtaState)
    {
        return;
    }

    // 1. switch to the highest freq
    app_sysfreq_req(APP_SYSFREQ_USER_OTA, APP_SYSFREQ_104M);

    // 2. exit bt sniff mode
#ifdef IBRT
    app_ibrt_if_request_modify_tws_bandwidth(TWS_TIMING_CONTROL_USER_OTA,
true);
    app_tws_ibrt_exit_sniff_with_tws();
#if defined(IBRT_UI_V1)
```

```
    app_ibrt_if_exit_sniff_with_mobile();
#else
    uint8_t* mobileAddr = ota_basic_get_mac_addr();
    app_ibrt_if_exit_sniff_with_mobile(mobileAddr);

    if (OTA_BASIC_SPP_DATAPATH_ENABLED == ota_basic_get_datapath())
    {
        app_ibrt_if_prevent_sniff_set(mobileAddr, OTA_ONGOING);
    }
#endif
```

When the system exits the OTA mode, cancel the mode of preventing the sniff.

```
void Bes_exit_ota_state(void)
{
    LOG_DBG(0,"%s state %d", __func__, isInBesOtaState);
    if (!isInBesOtaState)
    {
        return;
    }

    app_sysfreq_req(APP_SYSFREQ_USER_OTA, APP_SYSFREQ_32K);

#ifdef IBRT
    app_ibrt_if_prevent_sniff_clear(ota_basic_get_mac_addr(), OTA_ONGOING);

    app_ibrt_if_request_modify_tws_bandwidth(TWS_TIMING_CONTROL_USER_OTA,
false);
#else
    bes_bt_me->set_stay_active_mode(false, BT_ACTIVE_MODE_KEEP_USER_OTA,
UPDATE_ACTIVE_MODE_FOR_ALL_LINKS);
#endif

#ifdef __IAG_BLE_INCLUDE__
    bes_ble_me->ble_update_conn_param_mode(BLE_CONN_PARAM_MODE_OTA, false);
    bes_ble_me->ble_update_conn_param_mode(BLE_CONN_PARAM_MODE_OTA_SLOWER,
false);
#endif
```

Sample Code 40: Sniff and Active Mode Management

{"prevent_sniff_set",app_ibrt_if_prevent_sniff_set_test},

{"prevent_sniff_clear",app_ibrt_if_prevent_sniff_clear_test}

5.8.2.9 TWS Communication Bandwidth Update

Different TWS bandwidths may be required based on scenarios. So different scenarios are defined as different users. “app_ibrt_if_request_modify_tws_bandwidth”is implemented for request a specific TWS bandwidth for a user. When a user requests or releases its TWS bandwidth, the TWS link’s bandwidth is adjusted.

```
#define app_ibrt_if_request_modify_tws_bandwidth
app_tws_ibrt_request_modify_tws_bandwidth

void app_tws_ibrt_request_modify_tws_bandwidth(TWS_TIMING_CONTROL_USER_E
user, bool enable)
```

Table 51 app_ibrt_if_request_modify_tws_bandwidth Description

Item	parameter	Notes
app_ibrt_if_request_modify_tws_bandwidth	TWS_TIMING_CONTROL_USER_E user	typedef enum { TWS_TIMING_CONTROL_USER_SNIFF = 0, TWS_TIMING_CONTROL_USER_DEFAULT, TWS_TIMING_CONTROL_USER_A2DP, TWS_TIMING_CONTROL_USER_OTA, TWS_TIMING_CONTROL_USER_AI_VOICE, TWS_TIMING_CONTROL_USER_START_IBRT , TWS_TIMING_CONTROL_USER_IBRT_SWIT CH, TWS_TIMING_CONTROL_USER_FAST_COM MUNICATION, TWS_TIMING_CONTROL_USER_NUM, } TWS_TIMING_CONTROL_USER_E;
	bool enable	

```
static TWS_TIMING_CONTROL_INFO_T
tws_timing_info[TWS_TIMING_CONTROL_USER_NUM] =

{
```

```
    {false,    TWS_LINK_SNIFF_DURATION,       TWS_LINK_SNIFF_POLL_INTERVAL,
TWS_LINK_SNIFF_POLL_INTERVAL_IN_SCO},  //SNIFF

    {false,    IBRT_TWS_LINK_DEFAULT_DURATION,
IBRT_UI_DEFAULT_POLL_INTERVAL,     IBRT_UI_DEFAULT_POLL_INTERVAL_IN_SCO},
//DEFAULT

    {false,    TWS_LINK_A2DP_DURATION,        IBRT_UI_DEFAULT_POLL_INTERVAL,
IBRT_UI_DEFAULT_POLL_INTERVAL_IN_SCO}, //A2DP,

    {false,    TWS_LINK_OTA_DURATION,         TWS_LINK_OTA_POLL_INTERVAL,
TWS_LINK_OTA_POLL_INTERVAL_IN_SCO},    //OTA

    {false,    TWS_LINK_AI_DURATION,          TWS_LINK_AI_POLL_INTERVAL,
TWS_LINK_AI_POLL_INTERVAL_IN_SCO},     //AI_VOICE

    {false,    TWS_LINK_START_IBRT_DURATION,  TWS_LINK_START_IBRT_INTERVAL,
TWS_LINK_START_IBRT_INTERVAL_IN_SCO}, //START_IBRT

    {false,    TWS_LINK_ROLE_SWITCH_DURATION,
IBRT_UI_DEFAULT_POLL_INTERVAL,     IBRT_UI_DEFAULT_POLL_INTERVAL_IN_SCO},
//IBRT_SWITCH,

};



typedef struct

{

    bool    timing_enable;

    uint8_t  acl_slot_num;

    uint16_t acl_interval;

    uint16_t acl_interval_in_sco;

} TWS_TIMING_CONTROL_INFO_T;
```

This function is called when the initialization state or Bluetooth state changes.

- App_ibrt_init

```
void app_ibrt_init(void)
```

```
{
    if (bes_bt_me->bes_cfg_get(BT_BES_CFG_BT_SINK_ENABLED))
    {
        bes_bt_me->bt_global_handle_init();
#if defined(IBRT)
        ibrt_config_t config;
        app_tws_ibrt_init();
        app_tws_ibrt_init_default_tws_bandwidth_config(
            IBRT_TWS_LINK_DEFAULT_DURATION,
            IBRT_UI_DEFAULT_POLL_INTERVAL,
            IBRT_UI_DEFAULT_POLL_INTERVAL_IN_SCO);

app_tws_ibrt_request_modify_tws_bandwidth(TWS_TIMING_CONTROL_USER_DEFAULT,
true);
#if defined(IBRT_UI_V1)
        app_ibrt_ui_init();
        app_ibrt_ui_test_init();
        app_ibrt_if_config_load(&config);
        app_ibrt_customif_ui_start();
#ifdef IBRT_SEARCH_UI
        app_tws_ibrt_start(&config, true);
        app_ibrt_search_ui_init(false,IBRT_NONE_EVENT);
```

- app_tws_ibrt_disconnect_callback

```
            error_code = p_ibrt_ctrl->ibrt_disc_reason;
            p_ibrt_ctrl->ibrt_disc_reason = INVALID_ERROR;
        }
        app_tws_ibrt_bandwidth_table_clean();

app_tws_ibrt_request_modify_tws_bandwidth(TWS_TIMING_CONTROL_USER_DEFAULT,t
rue);
        }

        p_ibrt_ctrl->snoop_connected = 0;

        //Cancel profile wait timer if wait for data exchange
        if (p_ibrt_ctrl->wait_profile == true)
        {
            TRACE(0,"ui_log:cancel wait profile timer");
```

```
            p_ibrt_ctrl->wait_profile = false;
            osTimerStop(p_ibrt_ctrl->wait_profile_ready_timer_id);
            osTimerStop(p_ibrt_ctrl->delay_profile_send_timer_id);
        }

        app_tws_ibrt_reset_tss_state();

    }
```

5.8.2.10 Add a Custom RFComm Profile

The RFcomm related functions are encapsulated in the “g_bes_bt_spp” function structure of “bt_api_export.c”.

The API for RFcomm control:

g_bes_bt_spp->spp_rfcomm_ ()

```
    // turn on rfcomm profile
    .spp_rfcomm_open = app_rfcomm_open,
    // turn off rfcomm profile
    .spp_rfcomm_close = app_rfcomm_close,
    // send data
    .spp_rfcomm_write = app_rfcomm_write,
```

The number of channels provided by RFcomm is 6. The common buffer pool is shared for all the opened RFcomm channels and each RxBuffer size is 2048 bytes.

```
#define RFCOMM_SERVER_MAX_CHANNEL_CNT    6

static RfcommService_t
RfcommServiceInstance[RFCOMM_SERVER_MAX_CHANNEL_CNT];
static uint8_t rfcomm_service_used_instance_cnt = 0;
#define RFCOMM_RECV_BUFFER_SIZE 2048
#define RFCOMM_RECV_BUFFER_POOL_SIZE (RFCOMM_SERVER_MAX_CHANNEL_CNT *
RFCOMM_RECV_BUFFER_SIZE)

static uint8_t rfcomm_rx_buf[RFCOMM_RECV_BUFFER_POOL_SIZE];  // common
buffer pool for all the opened rfcomm channels
```

The information in the RFCOMM_CONFIG_T structure should be defined during initialization, including RFcomm chanel which can be added into enum RFCOMM_CHANNEL_NUM, maximum number of RFComm channels, UUID, event callback, data callback, and mutex.

```
typedef struct
```

```
{
    uint8_t     rfcomm_ch;
    uint8_t     tx_pkt_cnt;
    const uint8_t*          rfcomm_128bit_uuid;
    rfcomm_callback_func    callback;
    spp_handle_data_event_func_t spp_handle_data_event_func;
    osMutexId   mutexId;
    osMutexId   creditMutexId;
} RFCOMM_CONFIG_T;
```

The callback function performs corresponding operations according to the status of the event.

Callback function:

```
static bool fp_rfcomm_callback(uint8_t device_id, RFCOMM_EVENT_E event,
    uint8_t instanceIndex, uint16_t connHandle,
    uint8_t* pBtAddr, uint8_t* pSentDataBuf, uint16_t sentDataLen)
```

enum RFCOMM_EVENT_E contains different types of events:

```
typedef enum
{
    RFCOMM_INCOMING_CONN_REQ = 0,
    RFCOMM_CONNECTED,
    RFCOMM_DISCONNECTED,
    RFCOMM_TX_DONE,
    RFCOMM_UNKNOWN_EVENT
} RFCOMM_EVENT_E;
```

5.8.3 BLE

5.8.3.1 Connection Management

5.8.3.1.1 BLE Mac Address and BLE Name Configuration

BLE name and BLE mac address can be configured in DldProductLine before programming. Both information is written into the factory section after programming.

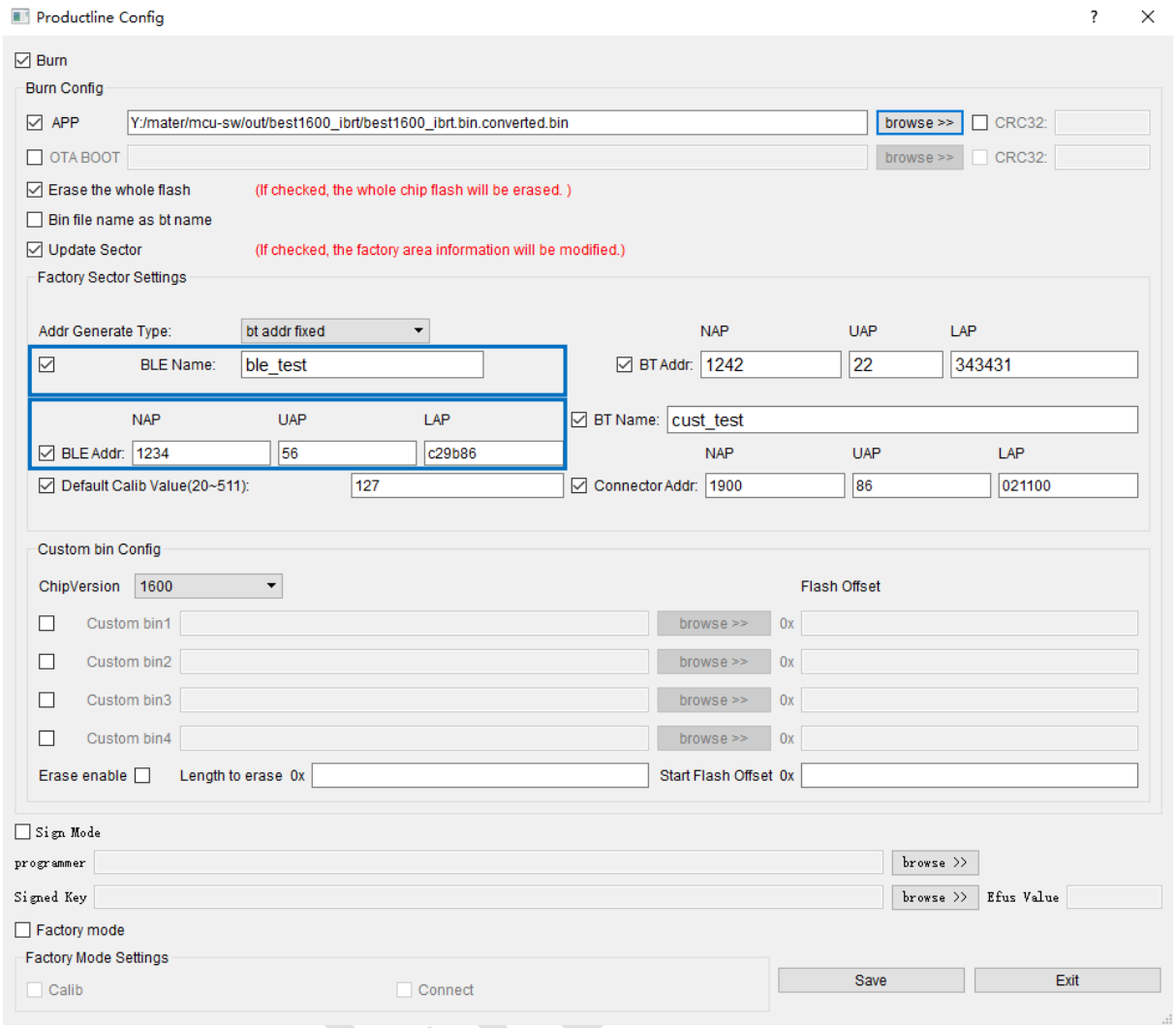


Figure 47 Configure BLE Mac Address and BLE Name

If necessary, The BLE address and BLE name can be updated and viewed through the following interfaces:

1. Get the BLE mac address.

```
uint8_t* factory_section_get_ble_address(void)
```

Sample Code 41: get_ble_address

{"get_ble_addr", get_ble_addr_test},

2. Update the BLE mac address.

```
/**
****************************************************************************************
*************
* @return
* 0:Set ble address failed
* 1:Set ble address successed
```

```
*
*************************************************************************
************
 */
int factory_section_set_ble_address(uint8_t* bleAddr)
```

Sample Code 42: set_ble_address

{"set_ble_address", set_ble_address_test},

3. Get the BLE name.

```
uint8_t* factory_section_get_ble_name(void)
```

Sample Code 43: get_ble_name

{"get_ble_name", get_ble_name_test},

4. Update the BLE name.

```
/**
*************************************************************************
************
* @return
* 0:Set ble name failed
* 1:Set ble name successed
*
*************************************************************************
************
*/
int factory_section_set_ble_name(char *name,int len)
```

Sample Code 44: set_ble_name

{"set_ble_name", set_ble_name_test},

5.8.3.1.2 BLE Adv

There are two kinds of advertising users: adv user and custom user. The advertising initiated by the adv user is called user adv, and the advertising initiated by the custom user is called custom adv. These two kinds of advertising will be interduced respectively as following.

5.8.3.1.2.1 User Adv

In order to choose user adv, the macro switch CUSTOMER_DEFINE_ADV_DATA needs to be disabled in target.mk:

CUSTOMER_DEFINE_ADV_DATA ?= 0

5.8.3.1.2.1.1 Simple Advertising

Before starting simple advertising, the macro switch BLE_AUDIO_ENABLED needs to be disabled in target file:

BLE_AUDIO_ENABLED = 0

In order to facilitate a simple adv, the default adv user -- USER_STUB is set to work when no other adv user is registered. That is, if other adv users are registered, USER_STUB will be invalid. The flag of this adv is 0x06, and there is only one BLE name and no response data exists in this adv. The default adv user can be started and stopped by using the following two interfaces when the UI role is not a slave:

Enable USER_STUB

```
void ble_core_enable_stub_adv(void)
```

Disable USER_STUB

```
void ble_core_disable_stub_adv(void)
```

5.8.3.1.2.1.2 Add an Adv User

To specify UUID or response data in the adv, you need to register an adv user. In this example, USER_BLE_ DEMO is used to introduce how to add an adv user.

Steps to add an adv user:

1. Add the enum variables.

   Add USER_BLE_DEMO1 in BLE_ADV_USER_E and add it to ble_adv_user2str

```
typedef enum
{
    USER_INUSE = 0,
    USER_ALL = USER_INUSE,
    USER_STUB,
    USER_GFPS,
    USER_SWIFT,
    USER_GSOUND,
    USER_AI,
    USER_INTERCONNECTION,
    USER_TILE,
    USER_OTA,
    USER_BLE_AUDIO,
    USER_SPOT,
```

```
    USER_BLE_CUSTOMER_0,
    USER_BLE_CUSTOMER_1,
    USER_BLE_CUSTOMER_2,
    USER_BLE_CUSTOMER_3,
    USER_BLE_DEMO0,
    USER_BLE_DEMO1,
    BLE_ADV_USER_NUM,
} BLE_ADV_USER_E;
```

```
char *ble_adv_user2str(BLE_ADV_USER_E user)
{
    switch (user)
    {
        CASE_S(USER_STUB)
        CASE_S(USER_GFPS)
        CASE_S(USER_SWIFT)
        CASE_S(USER_GSOUND)
        CASE_S(USER_AI)
        CASE_S(USER_INTERCONNECTION)
        CASE_S(USER_TILE)
        CASE_S(USER_OTA)
        CASE_S(USER_BLE_AUDIO)
        CASE_S(USER_SPOT)
        CASE_S(USER_BLE_CUSTOMER_0)
        CASE_S(USER_BLE_CUSTOMER_1)
        CASE_S(USER_BLE_CUSTOMER_2)
        CASE_S(USER_BLE_CUSTOMER_3)
        CASE_S(USER_BLE_DEMO0)
        CASE_S(USER_BLE_DEMO1)
        CASE_S(USER_BLE_DEMO)
        CASE_D()
    }
}
```

2. Bind an adv activity.

   Bind the added adv user to one of the adv activities in the table ble_adv_fill_param.

```
BLE_ADV_FILL_PARAM_T ble_adv_fill_param[BLE_ADV_ACTIVITY_USER_NUM] =
{
    {
```

```
        //idle adv interval, music adv interval, sco adv interval
        {BLE_ADV_INVALID_INTERVAL},
        {USER_BLE_CUSTOMER_0, USER_STUB, USER_GFPS, USER_GSOUND, USER_AI,
USER_INTERCONNECTION, USER_TILE,
         USER_OTA,USER_BLE_AUDIO, USER_BLE_DEMO},
         BLE_ADV_TX_POWER_LEVEL_0
    },
    {
        //idle adv interval, music adv interval, sco adv interval
        {BLE_ADV_INVALID_INTERVAL},
        {USER_SWIFT, USER_SPOT, USER_BLE_CUSTOMER_1, USER_BLE_DEMO0,
USER_INUSE},
        BLE_ADV_TX_POWER_LEVEL_1
    },
    {
        //idle adv interval, music adv interval, sco adv interval
        {BLE_ADV_INVALID_INTERVAL},
        {USER_BLE_CUSTOMER_2, USER_BLE_DEMO1, USER_INUSE},
        BLE_ADV_TX_POWER_LEVEL_2
    },
#ifdef IS_BLE_ACTIVITY_COUNT_MORE_THAN_THREE
    {
        //idle adv interval, music adv interval, sco adv interval
        {BLE_ADV_INVALID_INTERVAL},
        {USER_BLE_CUSTOMER_3},
        BLE_ADV_TX_POWER_LEVEL_2
    },
#endif
};
```

3. Register a data filling function. This function will be called when adv activity traverses the adv user.

```
app_ble_register_data_fill_handle(USER_BLE_DEMO,
(BLE_DATA_FILL_FUNC_T)ble_demo_user_data_fill_handler, false);
```

4. Specify advertising data and set advertising parameters.

   Specify the advertising data and parameters in the registered callback function.The data and parameters of advertising can be set by using the structure BLE_ADV_PARAM_T as follows. Then enable or disable the advertising by using the interface app_ble_data_fill_enable().

```
typedef struct {
   BLE_ADV_ACTIVITY_USER_E adv_actv_user;
```

```
    bool isBleFlagsAdvDataConfiguredByAppLayer;
    /// Advertising filtering policy (@see enum adv_filter_policy)
    uint8_t filter_pol;
    BLE_ADV_TYPE_E advType;
    ADV_MODE_E advMode;
    ADV_DISC_MODE_E discMode;
    uint32_t advInterval;
    uint32_t advUserInterval[BLE_ADV_USER_NUM];
    /// only ADV_MODE_PERIODIC adv mode use
    uint32_t PeriodicIntervalMin;
    uint32_t PeriodicIntervalMax;
    // Maximum power level
    BLE_ADV_TX_POWER_LEVEL_E advTxPwr;
    uint8_t advDataLen;
    uint8_t advData[EXT_ADV_DATA_LEN];
    uint8_t scanRspDataLen;
    uint8_t scanRspData[EXT_ADV_DATA_LEN];
    uint8_t localAddrType;
    uint8_t localAddr[BTIF_BD_ADDR_SIZE];
    ble_bdaddr_t peerAddr;
} __attribute__((__packed__)) BLE_ADV_PARAM_T;

static void ble_demo_user_data_fill_handler(void *param)
{
    BLE_ADV_PARAM_T *cmd = (BLE_ADV_PARAM_T*)param;

    memcpy(&cmd->advData[cmd->advDataLen], APP_DEMO_UUID,
APP_DEMO_UUID_LEN);
    cmd->advDataLen += APP_DEMO_UUID_LEN;

    memcpy(&cmd->scanRspData[cmd->scanRspDataLen], APP_DEMO_DATA,
APP_DEMO_DATA_LEN);
    cmd->scanRspDataLen += APP_DEMO_DATA_LEN;

    cmd->filter_pol = 0;
    cmd->advType = ADV_TYPE_CONN_EXT_ADV;
    cmd->advMode = ADV_MODE_EXTENDED;
    cmd->advInterval = BLE_ADVERTISING_INTERVAL;
    cmd->advUserInterval[USER_BLE_DEMO] = BLE_ADVERTISING_INTERVAL;
```

```
    cmd->advTxPwr = BLE_ADV_TX_POWER_LEVEL_1;

    cmd->localAddr[0] = 0x31;
    cmd->localAddr[1] = 0x32;
    cmd->localAddr[2] = 0x33;
    cmd->localAddr[3] = 0x24;
    cmd->localAddr[4] = 0x11;
    cmd->localAddr[5] = 0x11;
    cmd->localAddrType = 1;

    //Add flag
    cmd->isBleFlagsAdvDataConfiguredByAppLayer = true;
    memcpy(&cmd->advData[cmd->advDataLen], FLAG_DATA, FLAG_DATA_LEN);
    cmd->advDataLen += FLAG_DATA_LEN;

    app_ble_data_fill_enable(USER_BLE_DEMO, true);
}
```

Table 52 BLE_ADV_PARAM_T Description

Parameter	Description
BLE_ADV_ACTIVITY_USER_E adv_actv_user	Adv activity, used in SDK, shouldn't be set here.
bool isBleFlagsAdvDataConfiguredByAppLayer	False : Adv Flags is not Configured By App Layer. True : Adv Flags is Configured By App Layer.
uint8_t filter_pol	///Advertising filter policy enum adv_filter_policy { ///Allow both scan and connection requests ///from anyone ADV_ALLOW_SCAN_ANY_CON_ANY = 0x00, ///Allow both scan req from White List devices ///only and connection req from anyone ADV_ALLOW_SCAN_WLST_CON_ANY, ///Allow both scan req from anyone and ///connection req from White List devices only ADV_ALLOW_SCAN_ANY_CON_WLST, ///Allow scan and connection requests from

	``
	    ///White List devices only
	    ADV_ALLOW_SCAN_WLST_CON_WLST,
	};
	``

BLE_ADV_TYPE_E
advType
	``
	/// Advertise Types
	typedef enum app_ble_adv_type
	{
	    /// Connectable and scannable undirected
	    /// advertising
	    ADV_TYPE_UNDIRECT = 0,
	    /// Directed connectable with Low Duty Cycle
	    ADV_TYPE_DIRECT_LDC,
	    /// Directed connectable with High Duty
	Cycle
	    ADV_TYPE_DIRECT_HDC,
	    /// Non-connectable but Scannable
	    /// undirected advertising
	    ADV_TYPE_NON_CONN_SCAN,
	    /// Non-connectable and nonscannable
	    /// undirected
	    ADV_TYPE_NON_CONN_NON_SCAN,
	    /// Undirected connectable but scannable
	with
	    /// extended ADV
	    ADV_TYPE_CONN_EXT_ADV,
	    /// Directed connectable no scannable with
	    /// extended ADV
	    ADV_TYPE_EXT_CON_DIRECT,
	    /// Type MAx Number
	    ADV_TYPE_MAX,
	} BLE_ADV_TYPE_E;
	``

ADV_MODE_E
advMode
	``
	/// Advertising mode types
	typedef enum adv_mode
	{
	    /// Legacy advertising
	    ADV_MODE_LEGACY = 0,
	    /// Extended advertising
	    ADV_MODE_EXTENDED,
	``

CONFI

	/// Periodic advertising ADV_MODE_PERIODIC, }ADV_MODE_E;
ADV_DISC_MODE_E discMode	/// Advertising discoverable mode types typedef enum adv_disc_mode { /// Mode in non-discoverable ADV_DISC_MODE_NON_DISC = 0, /// Mode in general discoverable ADV_DISC_MODE_GEN_DISC, /// Mode in limited discoverable ADV_DISC_MODE_LIM_DISC, /// Broadcast mode without presence of /// AD_TYPE_FLAG in advertising data ADV_DISC_MODE_BEACON, /// Mode Max number ADV_DISC_MODE_MAX, }ADV_DISC_MODE_E;
uint32_t advInterval	Advertising interval, takes precedence over advUserInterval.
uint32_t advUserInterval[BLE_ADV_USER_NUM]	Interval takes effect when advInterval is invalid.
uint32_t PeriodicIntervalMin	Minimum interval, only work in advertising mode: ADV_MODE_PERIODIC.
uint32_t PeriodicIntervalMax	Maximum interval, only work in advertising mode: ADV_MODE_PERIODIC.
BLE_ADV_TX_POWER_LEVEL_E advTxPwr	/// Advertise TX power level typedef enum { BLE_ADV_TX_POWER_LEVEL_0 = 3, BLE_ADV_TX_POWER_LEVEL_1 = 4, BLE_ADV_TX_POWER_LEVEL_2 = 5, } BLE_ADV_TX_POWER_LEVEL_E;
uint8_t advDataLen	Advertising data length.
uint8_t advData[EXT_ADV_DATA_LEN]	Advertising data.
uint8_t	Scan response data length.

CONFIDENTIAL

scanRspDataLen	
uint8_t scanRspData[EXT_ADV_DATA_LEN]	Scan response data.
uint8_t localAddrType	Type of the local address. 0: public address. 1: private random address.
uint8_t localAddr[BTIF_BD_ADDR_SIZE]	Address of the device.

ble_bdaddr_t
peerAddr
	peer address.
	`typedef struct ble_bdaddr`
	`{`
	`/// BD Address of device`
	`uint8_t addr[BTIF_BD_ADDR_SIZE];`
	`/// Address type of the device`
	`/// 0: public address`
	`/// 1: private random address`
	`uint8_t addr_type;`
	`} ble_bdaddr_t;`

Sample Code 45: ble_adv_user_register

{"ble_adv_user_register", ble_adv_user_register_test},

5.8.3.1.2.1.3 Fill Data

Advertising data and response data need to be specified by using the following method, or the data may be cleared by other users. The BLE name is added by default at the end of the data.

```
static void ble_demo_user_data_fill_handler(void *param)
{
    BLE_ADV_PARAM_T *cmd = (BLE_ADV_PARAM_T*)param;

    memcpy(&cmd->advData[cmd->advDataLen], APP_DEMO_UUID,
APP_DEMO_UUID_LEN);
    cmd->advDataLen += APP_DEMO_UUID_LEN;

    memcpy(&cmd->scanRspData[cmd->scanRspDataLen], APP_DEMO_DATA,
APP_DEMO_DATA_LEN);
    cmd->scanRspDataLen += APP_DEMO_DATA_LEN;

    cmd->filter_pol = 0;
```

CONFIDENTIAL

```
    cmd->advType = ADV_TYPE_CONN_EXT_ADV;
    cmd->advMode = ADV_MODE_EXTENDED;
    cmd->advInterval = BLE_ADVERTISING_INTERVAL;
    cmd->advUserInterval[USER_BLE_DEMO] = BLE_ADVERTISING_INTERVAL;
    cmd->advTxPwr = BLE_ADV_TX_POWER_LEVEL_1;

    //Add flag
    cmd->isBleFlagsAdvDataConfiguredByAppLayer = true;
    memcpy(&cmd->advData[cmd->advDataLen], FLAG_DATA, FLAG_DATA_LEN);
    cmd->advDataLen += FLAG_DATA_LEN;

    app_ble_data_fill_enable(USER_BLE_DEMO, true);
}
```

5.8.3.1.2.1.4 Set a Flag

The BLE stack sets the adv flag according to the advertising type automatically. If you want to set a custom flag for the user adv, set the parameter isBleFlagsAdvDataConfiguredByAppLayer to be true, and add the adv flag to the data as follows.

```
#define FLAG_DATA         "\x02\x01\x01"
#define FLAG_DATA_LEN         (3)

static void ble_demo_user_data_fill_handler(void *param)
{
    BLE_ADV_PARAM_T *cmd = (BLE_ADV_PARAM_T*)param;

    memcpy(&cmd->advData[cmd->advDataLen], APP_DEMO_UUID,
APP_DEMO_UUID_LEN);
    cmd->advDataLen += APP_DEMO_UUID_LEN;

    memcpy(&cmd->scanRspData[cmd->scanRspDataLen], APP_DEMO_DATA,
APP_DEMO_DATA_LEN);
    cmd->scanRspDataLen += APP_DEMO_DATA_LEN;

    cmd->filter_pol = 0;
    cmd->advType = ADV_TYPE_CONN_EXT_ADV;
    cmd->advMode = ADV_MODE_EXTENDED;
    cmd->advInterval = BLE_ADVERTISING_INTERVAL;
    cmd->advUserInterval[USER_BLE_DEMO] = BLE_ADVERTISING_INTERVAL;
```

```
    cmd->advTxPwr = BLE_ADV_TX_POWER_LEVEL_1;

    cmd->localAddr[0] = 0x31;
    cmd->localAddr[1] = 0x32;
    cmd->localAddr[2] = 0x33;
    cmd->localAddr[3] = 0x24;
    cmd->localAddr[4] = 0x11;
    cmd->localAddr[5] = 0x11;
    cmd->localAddrType = 1;

    //Add flag
    cmd->isBleFlagsAdvDataConfiguredByAppLayer = true;
    memcpy(&cmd->advData[cmd->advDataLen], FLAG_DATA, FLAG_DATA_LEN);
    cmd->advDataLen += FLAG_DATA_LEN;

    app_ble_data_fill_enable(USER_BLE_DEMO, true);
}
```

5.8.3.1.2.1.5 Advertising Control

There are two ways to control advertising, one is to enable or disable advertising in the data filling function, which can be understood as start and stop, and the other is to force switching, which can be understanded as open and close. Advertising can be opened or closed no matter whether the device is advertising or not.

1. Start and stop advertising

In the data filling function, advertising can be enabled or disabled by using the interface app_ble_data_fill_enable. Any adv user can enable adv, and if any adv user enables adv, the corresponding adv activity is enabled.

```
static void ble_demo_user_data_fill_handler(void *param)
{
    BLE_ADV_PARAM_T *cmd = (BLE_ADV_PARAM_T*)param;

    memcpy(&cmd->advData[cmd->advDataLen], APP_DEMO_UUID,
APP_DEMO_UUID_LEN);
    cmd->advDataLen += APP_DEMO_UUID_LEN;

    memcpy(&cmd->scanRspData[cmd->scanRspDataLen], APP_DEMO_DATA,
APP_DEMO_DATA_LEN);
    cmd->scanRspDataLen += APP_DEMO_DATA_LEN;

    cmd->filter_pol = 0;
```

```
    cmd->advType = ADV_TYPE_CONN_EXT_ADV;
    cmd->advMode = ADV_MODE_EXTENDED;
    cmd->advInterval = BLE_ADVERTISING_INTERVAL;
    cmd->advUserInterval[USER_BLE_DEMO] = BLE_ADVERTISING_INTERVAL;
    cmd->advTxPwr = BLE_ADV_TX_POWER_LEVEL_1;

    cmd->localAddr[0] = 0x31;
    cmd->localAddr[1] = 0x32;
    cmd->localAddr[2] = 0x33;
    cmd->localAddr[3] = 0x24;
    cmd->localAddr[4] = 0x11;
    cmd->localAddr[5] = 0x11;
    cmd->localAddrType = 1;

    //Add flag
    cmd->isBleFlagsAdvDataConfiguredByAppLayer = true;
    memcpy(&cmd->advData[cmd->advDataLen], FLAG_DATA, FLAG_DATA_LEN);
    cmd->advDataLen += FLAG_DATA_LEN;

    app_ble_data_fill_enable(USER_BLE_DEMO, true);
}
```

Table 53 Function app_ble_data_fill_enable Description

Parameter	Description
enum user	The user of the ble advertising.
bool enable	False : disable this advertising True : enable this advertising

2. Open and close advertising

An adv activity can be forcibly disabled by using the interface app_ble_force_switch_adv. Every adv user can disable adv activity. If any adv user disables adv, the corresponding adv activity is disabled.

```
void app_ble_force_switch_adv(enum BLE_ADV_SWITCH_USER_E user, bool
isToEnableAdv)
```

Table 54 Function app_ble_force_switch_adv Description

Parameter	Description

enum
BLE_ADV_SWITCH_USER_E user
	It is used to set the disable bit map. One user corresponds to one bit. When the disable bit map is not 0, that is, there is at least one user disable adv, the advertising is disabled.
	enum BLE_ADV_SWITCH_USER_E {
	BLE_SWITCH_USER_RS = 0, // used for role switch
	BLE_SWITCH_USER_BOX = 1, // used for box open/close
	BLE_SWITCH_USER_AI = 2, // used for ai
	BLE_SWITCH_USER_BT_CONNECT = 3, // used for bt connect
	BLE_SWITCH_USER_SCO = 4, // used for sco
	BLE_SWITCH_USER_IBRT = 5, // used for ibrt
	BLE_SWITCH_USER_FPGA = 6, // used for fpga
	BLE_SWITCH_USER_BLE_AUDIO = 7, // used for ble audio
	BLE_SWITCH_USER_NUM,
	};

bool isToEnableAdv	False: Disable adv True: Cancel disable adv

Sample Code 46: Enable and Disable advertising

{"close_adv", ble_audio_close_adv_test},

{"open_adv", ble_audio_open_adv_test},

5.8.3.1.2.1.6 Set an Advertising Type

There are two ways to set an advertising type. The first one is to set an advertising type by using the interface app_ble_set_adv_type, the other is to set the advType directly in the data filling function. The first one has a higher priority.

1. Set an advertising type by using the interface app_ble_set_adv_type.

```
void app_ble_set_adv_type(BLE_ADV_TYPE_E advType, ble_bdaddr_t *peer_addr)
```

Table 55 Function app_ble_set_adv_type Description

Parameter	Description
enum BLE_ADV_TYPE_E advType	typedef enum app_ble_adv_type { /// Connectable and scannable undirected advertising ADV_TYPE_UNDIRECT = 0, /// Directed connectable with Low Duty Cycle

	ADV_TYPE_DIRECT_LDC, /// Directed connectable with High Duty Cycle ADV_TYPE_DIRECT_HDC, /// Non-connectable but Scannable undirected advertising ADV_TYPE_NON_CONN_SCAN, /// Non-connectable and nonscannable undirected ADV_TYPE_NON_CONN_NON_SCAN, /// Undirected connectable but scannable with extended ADV ADV_TYPE_CONN_EXT_ADV, /// Directed connectable no scannable with extended ADV ADV_TYPE_EXT_CON_DIRECT, /// Type MAx Number ADV_TYPE_MAX, } BLE_ADV_TYPE_E;
struct ble_bdaddr_t *peer_addr	typedef struct ble_bdaddr { /// BD Address of device uint8_t addr[BTIF_BD_ADDR_SIZE]; /// Address type of the device 0=public/1=private random uint8_t addr_type; } ble_bdaddr_t;

Sample Code 47: ble_set_adv_type

{"ble_set_adv_type", ble_set_adv_type_test},

2. Set an advertising type in the data filling function

The advertising type is set to ADV_TYPE_CONN_EXT_ADV because the current advertsing mode is ADV_MODE_EXTENDED.

```
static void ble_demo_user_data_fill_handler(void *param)
{
    BLE_ADV_PARAM_T *cmd = (BLE_ADV_PARAM_T*)param;

    memcpy(&cmd->advData[cmd->advDataLen], APP_DEMO_UUID,
APP_DEMO_UUID_LEN);
    cmd->advDataLen += APP_DEMO_UUID_LEN;
```

CONFIDENTIAL

```
    memcpy(&cmd->scanRspData[cmd->scanRspDataLen], APP_DEMO_DATA,
APP_DEMO_DATA_LEN);
    cmd->scanRspDataLen += APP_DEMO_DATA_LEN;

    cmd->filter_pol = 0;
    cmd->advType = ADV_TYPE_CONN_EXT_ADV;
    cmd->advMode = ADV_MODE_EXTENDED;
    cmd->advInterval = BLE_ADVERTISING_INTERVAL;
    cmd->advUserInterval[USER_BLE_DEMO] = BLE_ADVERTISING_INTERVAL;
    cmd->advTxPwr = BLE_ADV_TX_POWER_LEVEL_1;

    cmd->localAddr[0] = 0x31;
    cmd->localAddr[1] = 0x32;
    cmd->localAddr[2] = 0x33;
    cmd->localAddr[3] = 0x24;
    cmd->localAddr[4] = 0x11;
    cmd->localAddr[5] = 0x11;
    cmd->localAddrType = 1;

    //Add flag
    cmd->isBleFlagsAdvDataConfiguredByAppLayer = true;
    memcpy(&cmd->advData[cmd->advDataLen], FLAG_DATA, FLAG_DATA_LEN);
    cmd->advDataLen += FLAG_DATA_LEN;

    app_ble_data_fill_enable(USER_BLE_DEMO, true);
}
```

5.8.3.1.2.1.7 Set an Advertsing Interval

There are two ways to set an adv interval. One is to set an adv activity interval, and the other is to set an advertising interval in the data filling function. The first one has a higher priority.

1. Set adv activity interval

When the system is in some states, the advertising interval can be set in the scene. The activity interval is set by the corresponding adv user. If there are multiple user requests to configure the advertising interval, the maximum value takes effect.

```
void app_ble_param_set_adv_interval(BLE_ADV_INTERVALREQ_USER_E
adv_intv_user,
                                    BLE_ADV_USER_E adv_user,
```

```
                                    uint32_t interval)
```

Table 56 Function app_ble_param_set_adv_interval Description

Parameter	Description
enum BLE_ADV_INTERVALREQ_USER_E adv_intv_user	A user corresponds to a scene typedef enum { BLE_ADV_INTERVALREQ_USER_A2DP, BLE_ADV_INTERVALREQ_USER_SCO, BLE_ADV_INTERVALREQ_USER_TWS_STM, BLE_ADV_INTERVALREQ_USER_NUM, } BLE_ADV_INTERVALREQ_USER_E;
enum BLE_ADV_USER_E adv_user	USER_ALL: Configure the interval for all adv activities 0xXX: Configure the interval of the adv activity corresponding to the adv user
uint32_t interval	Configure the interval of adv activity

Sample Code 48: set_adv_interval

{"ble_param_set_adv_interval", ble_param_set_adv_interval_test},

2. Set an adv user interval

The advertising interval can be configured by using the parameters advUserInterval[adv user] and advInterval in the data filling function. The parameter advInterval has a higher priority. If there are multiple adv user requests to configure the interval, the minimum value takes effect.

```
static void ble_demo_user_data_fill_handler(void *param)
{
    BLE_ADV_PARAM_T *cmd = (BLE_ADV_PARAM_T*)param;

    memcpy(&cmd->advData[cmd->advDataLen], APP_DEMO_UUID,
APP_DEMO_UUID_LEN);
    cmd->advDataLen += APP_DEMO_UUID_LEN;

    memcpy(&cmd->scanRspData[cmd->scanRspDataLen], APP_DEMO_DATA,
APP_DEMO_DATA_LEN);
    cmd->scanRspDataLen += APP_DEMO_DATA_LEN;

    cmd->filter_pol = 0;
```

CONFIDENTIAL

```
    cmd->advType = ADV_TYPE_CONN_EXT_ADV;
    cmd->advMode = ADV_MODE_EXTENDED;
    cmd->advInterval = BLE_ADVERTISING_INTERVAL;
    cmd->advUserInterval[USER_BLE_DEMO] = BLE_ADVERTISING_INTERVAL;
    cmd->advTxPwr = BLE_ADV_TX_POWER_LEVEL_1;

    cmd->localAddr[0] = 0x31;
    cmd->localAddr[1] = 0x32;
    cmd->localAddr[2] = 0x33;
    cmd->localAddr[3] = 0x24;
    cmd->localAddr[4] = 0x11;
    cmd->localAddr[5] = 0x11;
    cmd->localAddrType = 1;

    //Add flag
    cmd->isBleFlagsAdvDataConfiguredByAppLayer = true;
    memcpy(&cmd->advData[cmd->advDataLen], FLAG_DATA, FLAG_DATA_LEN);
    cmd->advDataLen += FLAG_DATA_LEN;

    app_ble_data_fill_enable(USER_BLE_DEMO, true);
}
```

5.8.3.1.2.1.8 Set the TX power

Each adv activity can have its own TX power configured. There are four ways to configure the TX power:

Set the TX power in the data fillling function;

Set the TX power by using the interface app_ble_set_adv_TXpwr_by_actv_user;

Set the TX power by using the interface app_ble_set_adv_TXpwr_by_adv_user;

Set the TX power by using the interface app_ble_set_all_adv_TXpwr.

The last three have higher priorities. TX power can be modified in real time after adv is up.

1. Set the TX power in the data fillling function.

TX power is divided into three levels. You can set the TX power level as needed.

```
typedef enum
{
    BLE_ADV_TX_POWER_LEVEL_0 = 3,
    BLE_ADV_TX_POWER_LEVEL_1 = 4,
```

```
    BLE_ADV_TX_POWER_LEVEL_2 = 5,
} BLE_ADV_TX_POWER_LEVEL_E;

static void ble_demo_user_data_fill_handler(void *param)
{
    BLE_ADV_PARAM_T *cmd = (BLE_ADV_PARAM_T*)param;

    memcpy(&cmd->advData[cmd->advDataLen], APP_DEMO_UUID,
APP_DEMO_UUID_LEN);
    cmd->advDataLen += APP_DEMO_UUID_LEN;

    memcpy(&cmd->scanRspData[cmd->scanRspDataLen], APP_DEMO_DATA,
APP_DEMO_DATA_LEN);
    cmd->scanRspDataLen += APP_DEMO_DATA_LEN;

    cmd->filter_pol = 0;
    cmd->advType = ADV_TYPE_CONN_EXT_ADV;
    cmd->advMode = ADV_MODE_EXTENDED;
    cmd->advInterval = BLE_ADVERTISING_INTERVAL;
    cmd->advUserInterval[USER_BLE_DEMO] = BLE_ADVERTISING_INTERVAL;
    cmd->advTxPwr = BLE_ADV_TX_POWER_LEVEL_1;

    cmd->localAddr[0] = 0x31;
    cmd->localAddr[1] = 0x32;
    cmd->localAddr[2] = 0x33;
    cmd->localAddr[3] = 0x24;
    cmd->localAddr[4] = 0x11;
    cmd->localAddr[5] = 0x11;
    cmd->localAddrType = 1;

    //Add flag
    cmd->isBleFlagsAdvDataConfiguredByAppLayer = true;
    memcpy(&cmd->advData[cmd->advDataLen], FLAG_DATA, FLAG_DATA_LEN);
    cmd->advDataLen += FLAG_DATA_LEN;

    app_ble_data_fill_enable(USER_BLE_DEMO, true);
}
```

2. Set the TX power by using the interface app_ble_set_adv_txpwr_by_actv_user.

The TX power of the activity can be set directly.

```
void app_ble_set_adv_txpwr_by_actv_user(BLE_ADV_ACTIVITY_USER_E actv_user,
int8_t txpwr_dbm)
```

Table 57 Function app_ble_set_adv_txpwr_by_actv_user Description

Parameter	Description
enum BLE_ADV_ACTIVITY_USER_E actv_user	Activities that need to be configured
int8_t txpwr_dbm	Tx power value in the range -21 -> 16

Sample Code 49: ble_set_adv_txpwr_by_actv_user

{"ble_set_adv_txpwr_by_actv_user", ble_set_adv_txpwr_by_actv_user_test},

3. Set the TX power by using the interface app_ble_set_adv_txpwr_by_adv_user.

The tx power of the activity can be set by the adv user.

```
void app_ble_set_adv_txpwr_by_adv_user(BLE_ADV_USER_E user, int8_t
txpwr_dbm)
```

Table 58 Function app_ble_set_adv_txpwr_by_adv_user Description

Parameter	Description
enum BLE_ADV_USER_E user	Used to get the activity that needs to be configured
int8_t txpwr_dbm	Tx power value in the range -21 -> 16

Sample Code 50: ble_set_adv_txpwr_by_adv_user

{"ble_set_adv_txpwr_by_adv_user", ble_set_adv_txpwr_by_adv_user_test},

4. Set the TX power by using the interface app_ble_set_all_adv_txpwr.

Set the TX power of all activities.

```
void app_ble_set_all_adv_txpwr(int8_t txpwr_dbm)
```

Table 59 Function app_ble_set_all_adv_txpwr Description

Parameter	Description
int8_t txpwr_dbm	Tx power value in the range -21 -> 16

Sample Code 51: ble_set_all_adv_txpwr

CONFIDENTIAL

{"ble_set_all_adv_txpwr", ble_set_all_adv_txpwr_test},

5.8.3.1.2.1.9 Set a Local Address

The advertising address of the user adv is a private random address by default. It can be set to a static address by enabling the macro switch BLE_ADV_RPA_ENABLED as follows:

BLE_ADV_RPA_ENABLED ?= 0

After enabling the macro, the advertising address is a BLE address by default. It can be set by the following methods:

Set a local address in the data filling function;

Set a local address by using the interface app_ble_set_adv_local_addr_by_actv_user;

Set local address by using the interface app_ble_set_adv_local_addr_by_adv_user.

The first one have a higher priority. When you use the last two ways to set a local address, the localAddrType needs to be set to 1 and the localAddr can not be set in the data filling function.

1. Set a local address in the data filling function.

```
static void ble_demo_user_data_fill_handler(void *param)
{
    BLE_ADV_PARAM_T *cmd = (BLE_ADV_PARAM_T*)param;

    memcpy(&cmd->advData[cmd->advDataLen], APP_DEMO_UUID,
APP_DEMO_UUID_LEN);
    cmd->advDataLen += APP_DEMO_UUID_LEN;

    memcpy(&cmd->scanRspData[cmd->scanRspDataLen], APP_DEMO_DATA,
APP_DEMO_DATA_LEN);
    cmd->scanRspDataLen += APP_DEMO_DATA_LEN;

    cmd->filter_pol = 0;
    cmd->advType = ADV_TYPE_CONN_EXT_ADV;
    cmd->advMode = ADV_MODE_EXTENDED;
    cmd->advInterval = BLE_ADVERTISING_INTERVAL;
    cmd->advUserInterval[USER_BLE_DEMO] = BLE_ADVERTISING_INTERVAL;
    cmd->advTxPwr = BLE_ADV_TX_POWER_LEVEL_1;

    cmd->localAddr[0] = 0x31;
    cmd->localAddr[1] = 0x32;
    cmd->localAddr[2] = 0x33;
```

```
    cmd->localAddr[3] = 0x24;
    cmd->localAddr[4] = 0x11;
    cmd->localAddr[5] = 0x11;
    cmd->localAddrType = 1;

    //Add flag
    cmd->isBleFlagsAdvDataConfiguredByAppLayer = true;
    memcpy(&cmd->advData[cmd->advDataLen], FLAG_DATA, FLAG_DATA_LEN);
    cmd->advDataLen += FLAG_DATA_LEN;

    app_ble_data_fill_enable(USER_BLE_DEMO, true);
}
```

2. Set a local address by using the interface app_ble_set_adv_local_addr_by_actv_user.

```
void app_ble_set_adv_local_addr_by_actv_user(BLE_ADV_ACTIVITY_USER_E
actv_user, uint8_t *addr)
```

Table 60 Function app_ble_set_adv_local_addr_by_user Description

Parameter	Description
enum BLE_ADV_ACTIVITY_USER_E actv_user	Activities that need to be configured
uint8_t * addr	Adv address that needs to configure

Sample Code 52: ble_set_adv_local_addr_by_actv_user

{"ble_set_adv_local_addr_by_actv_user", ble_set_adv_local_addr_by_actv_user_test},

3. Set a local address by using the interface app_ble_set_adv_local_addr_by_adv_user.

Set the local address of the activity through the adv user.

```
void app_ble_set_adv_local_addr_by_adv_user(BLE_ADV_USER_E user, uint8_t
*addr)
```

Table 61 Function app_ble_set_adv_local_adddr_adv_user Description

Parameter	Description
enum BLE_ADV_ACTIVITY_USER_E actv_user	Used to get the activity that needs to be configured
uint8_t * addr	Adv address that needs to configure

Sample Code 53: ble_set_adv_local_addr_by_adv_user

{"ble_set_adv_local_addr_by_adv_user", ble_set_adv_local_addr_by_adv_user_test},

5.8.3.1.2.1.10 Start Advertising

User adv can be started and stopped by using the interface app_ble_start_connectable_adv as follows.

The data filling function ble_demo_user_data_fill_handler is called after advertising starts.

If advertising starts successfully, BLE_ADV_STARTED_EVENT is triggered, and the callback functionn app_ble_customif_adv_started_event_handler() is called.

If advertising fails to start, BLE_ADV_STARTING_FAILED_EVENT is triggered, and the callback function app_ble_customif_adv_starting_failed_event_handler() is called. These two callback functions can be modified as needed.

```
void app_ble_start_connectable_adv(uint16_t advInterval)
```

Table 62 Function app_ble_refresh_adv_state Description

Parameter	Description
uint16_t advInterval	Invalid

Sample Code 54: ble_user_adv_start

{"ble_user_adv_start", ble_user_adv_start_test},

5.8.3.1.2.2 BLE Custom Adv

Inoder to choose a custom adv, the macro switch CUSTOMER_DEFINE_ADV_DATA needs to be enabled in target.mk as follows:

CUSTOMER_DEFINE_ADV_DATA ?= 1

5.8.3.1.2.2.1 Initiate a Custom Adv

Different from the user adv, the initialization of a custom adv has been writed in the SDK as follows. All custom users can be registered in the function app_ble_custom_init. Therefore, the way of initiating the custom adv is to call the interface app_ble_custom_init.

```
void app_ble_custom_init(void)
{
    static bool ble_custom_inited = false;
```

CONFIDENTIAL

```
    if (!ble_custom_inited)
    {
        LOG_I("%s", __func__);
        ble_custom_inited = true;
        for (uint8_t actv_user = BLE_ADV_ACTIVITY_USER_0; actv_user <
BLE_ADV_ACTIVITY_USER_NUM; actv_user++)
        {
            memset(&customer_adv_param[actv_user], 0,
sizeof(CUSTOMER_ADV_PARAM_T));
        }
        app_ble_custom_0_user_init();
        app_ble_custom_1_user_init();
        app_ble_custom_2_user_init();
#ifdef IS_BLE_ACTIVITY_COUNT_MORE_THAN_THREE
        app_ble_custom_3_user_init();
#endif
    }
}

static void app_ble_custom_0_user_init(void)
{
    app_ble_register_data_fill_handle(USER_BLE_CUSTOMER_0,
(BLE_DATA_FILL_FUNC_T)app_ble_custom_0_user_data_fill_handler, false);
}
```

5.8.3.1.2.2.2 Data Filling

The advertising parameters and data can be filled in by using the following interface:

```
void app_ble_custom_adv_write_data(BLE_ADV_ACTIVITY_USER_E actv_user,
                bool is_custom_adv_flags,
                ADV_ADDR_TYPE_E type,
                uint8_t *local_addr,
                ble_bdaddr_t *peer_addr,
                uint32_t adv_interval,
                BLE_ADV_TYPE_E adv_type,
                ADV_MODE_E adv_mode,
                int8_t tx_power_dbm,
                uint8_t *adv_data, uint8_t adv_data_size,
                uint8_t *scan_rsp_data, uint8_t scan_rsp_data_size)
```

Table 63 Function app_ble_custom_adv_write_data Description

Parameter	Description
enum BLE_ADV_ACTIVITY_USER_E actv_user	/// The user of the adv activity typedef enum { BLE_ADV_ACTIVITY_USER_0 = 0, BLE_ADV_ACTIVITY_USER_1, BLE_ADV_ACTIVITY_USER_2, BLE_ADV_ACTIVITY_USER_3, BLE_ADV_ACTIVITY_USER_NUM, } BLE_ADV_ACTIVITY_USER_E;
bool is_custom_adv_flags	False : Adv Flags is not Configured By App Layer. True : Adv Flags is Configured By App Layer.
enum ADV_ADDR_TYPE_E type	Configure the interval of adv activity
uint8_t * local_addr	Type of the local address. 0: public address. 1: private random address.

struct
ble_bdaddr_t *peer_addr
	// peer address.
	typedef struct ble_bdaddr
	{
	/// BD Address of device
	uint8_t addr[BTIF_BD_ADDR_SIZE];
	/// Address type of the device
	/// 0: public address
	/// 1: private random address
	uint8_t addr_type;
	} ble_bdaddr_t;

uint32_t adv_interval	The interval of the adv activity
enum BLE_ADV_TYPE_E adv_type	/// Advertise Types typedef enum app_ble_adv_type { /// Connectable and scannable undirected /// advertising ADV_TYPE_UNDIRECT = 0,

CONFIDENTIAL

	/// Directed connectable with Low Duty Cycle ADV_TYPE_DIRECT_LDC, /// Directed connectable with High Duty Cycle ADV_TYPE_DIRECT_HDC, /// Non-connectable but Scannable /// undirected advertising ADV_TYPE_NON_CONN_SCAN, /// Non-connectable and nonscannable /// undirected ADV_TYPE_NON_CONN_NON_SCAN, /// Undirected connectable but scannable with /// extended ADV ADV_TYPE_CONN_EXT_ADV, /// Directed connectable no scannable with /// extended ADV ADV_TYPE_EXT_CON_DIRECT, /// Type MAx Number ADV_TYPE_MAX, } BLE_ADV_TYPE_E;
enum ADV_MODE_E adv_mode	/// Advertising mode types typedef enum adv_mode { /// Legacy advertising ADV_MODE_LEGACY = 0, /// Extended advertising ADV_MODE_EXTENDED, /// Periodic advertising ADV_MODE_PERIODIC, }ADV_MODE_E;
int8_t tx_power_dbm	Tx power value, the range is -21 -> 16
uint8_t * adv_data	Advertising data.
uint8_t adv_data_size	Size of advertising data
uint8_t * scan_rsp_data	Scan response data.
uint8_t scan_rsp_data_size	Size of scan response data.

CONFIDENTIAL

5.8.3.1.2.2.3 Start Advertising

A custom adv can be started by using the following interface.

If advertising starts successfully, BLE_ADV_STARTED_EVENT is triggered, and the callback function app_ble_customif_adv_started_event_handler() is executed.

If advertising fails to start, BLE_ADV_STARTING_FAILED_EVENT is triggered, and the callback functionn app_ble_customif_adv_starting_failed_event_handler() is executed.

These two callback functions can be modified as needed.

```
void app_ble_custom_adv_start(BLE_ADV_ACTIVITY_USER_E actv_user)
```

Table 64 Function app_ble_custom_adv_start Description

Parameter	Description
enum BLE_ADV_ACTIVITY_USER_E actv_user	Activities that need to be configured

Sample Code 55: ble_custom_adv_start

{"ble_custom_adv_start", ble_custom_adv_start_test},

5.8.3.1.2.3 Advertising Event

In order to feedback the specific advertising operations, a series of callback interfaces is provided for users. When the advertising event is triggered, the corresponding callback handler is called.

```
static const ble_event_handler_t app_ble_customif_event_handler_tab[] =
{
    … …
   {BLE_ADV_STARTED_EVENT, app_ble_customif_adv_started_event_handler},
   {BLE_ADV_STARTING_FAILED_EVENT,
app_ble_customif_adv_starting_failed_event_handler},
   {BLE_ADV_STOPPED_EVENT, app_ble_customif_adv_stopped_event_handler},
    … …
};
```

Table 65 Adv Event List

Event	Description	Event Code	Event Parameters
BLE_ADV_STARTED_EVENT	Advertising started event	0x0B	// Event type enum ble_event_type_e evt_type

			// Activity index
			uint8_t act_user
			// Something to output
			void* output

BLE_ADV_STARTING_FAILED_EVENT	Advertising starting failed event	0x0C	// event type
			enum ble_event_type_e evt_type
			// Activity index
			uint8_t act_user
			// Something to output
			void* output

BLE_ADV_STOPPED_EVENT	Advertising stopped event	0x0D	// Event type
			enum ble_event_type_e evt_type
			// Activity index
			uint8_t act_user
			// Error reason
			uint8_t err_code
			// Something to output
			void* output

5.8.3.1.3 BLE Scan

5.8.3.1.3.1 Start a Scan

You can start a scan by using the following interface.

If scanning starts successfully, BLE_SCAN_STARTED_EVENT is triggered, and the callback function app_ble_customif_scan_started_event_handler() is executed.

If scaning failed to start, BLE_SCAN_STARTING_FAILED_EVENT is triggered, and the callback function app_ble_customif_scan_starting_failed_event_handler() is executed.

These two callback functions can be modified as needed.

```
void app_ble_start_scan(enum BLE_SCAN_FILTER_POLICY scanFilterPolicy,
uint16_t scanWindow, uint16_t scanInterval)
```

Table 66 Function app_ble_start_scan Description

Parameter	Description
enum BLE_SCAN_FILTER_POLICY scanFilterPolicy	enum BLE_SCAN_FILTER_POLICY { ///Allow advertising packets from anyone BLE_SCAN_ALLOW_ADV_ALL = 0x00,

CONFIDENTIAL

	///Allow advertising packets from White List ///devices only BLE_SCAN_ALLOW_ADV_WLST, ///Allow advertising packets from anyone and ///Direct adv using RPA in InitA BLE_SCAN_ALLOW_ADV_ALL_AND_INIT_RPA, ///Allow advertising packets from White List ///devices only and Direct adv using RPA in ///InitA BLE_SCAN_ALLOW_ADV_WLST_AND_INIT_RPA, };
uint16_t scanWindow	Scan duration
uint16_t scanInterval	Scan interval

Sample Code 56: ble_start_scan

{"ble_start_scan", ble_start_scan_test},

5.8.3.1.3.2 Stop a Scan

You can stop a scan by using the following interface.

If scanning stops, BLE_SCAN_STOPED_EVENT is triggered, and the callback function app_ble_customif_scan_stopped_event_handler() is executed. The callback functions can be modified as needed.

```
void app_ble_stop_scan(void)
```

Sample Code 57: ble_stop_scan

{"ble_stop_scan", ble_stop_scan_test},

5.8.3.1.3.3 Scan Event

In order to feedback the specific scan operations, a series of callback interfaces is provided for users. When the scan event is triggered, the corresponding callback handler is called.

```
static const ble_event_handler_t app_ble_customif_event_handler_tab[] =
{
    … …
```

```
    {BLE_SCAN_STARTED_EVENT, app_ble_customif_scan_started_event_handler},
    {BLE_SCAN_DATA_REPORT_EVENT,
app_ble_customif_scan_data_report_event_handler},
    {BLE_SCAN_STARTING_FAILED_EVENT,
app_ble_customif_scan_starting_failed_event_handler},
    {BLE_SCAN_STOPPED_EVENT, app_ble_customif_scan_stopped_event_handler},
     … …
};
```

Table 67 Scan Event List

Event	Description	Event Code	Event Parameters
BLE_SCAN_STARTED_EVENT	Scan start event,this event will be triggered when scan started.	0x0E	/// Event type enum ble_event_type_e evt_type /// Something to output void* output
BLE_SCAN_DATA_REPORT_EVENT	Scan data report event, this event will be triggered when scan report data.	0x0F	/// event type enum ble_event_type_e evt_type /// BD Address of device uint8_t addr[6]; /// Address type of the peer ///device 0=public/1=private /// random uint8_t addr_type; /// Adv signal strength uint8_t rssi /// report data length uint16_t length /// scan report data uint8_t data[251] /// Something to output void* output
BLE_SCAN_STARTING_FAILED_EVENT	Scan starting failed event, this event will be triggered when scan start failed.	0x10	/// event type enum ble_event_type_e evt_type /// Activity index uint8_t actv_idx; /// Error reason uint8_t err_code; /// Something to output

CONFIDENTIAL

			void* output
BLE_SCAN_STOPPED_EVENT	Scan stop event, this event will be triggered when scan stopped.	0x11	/// event type enum ble_event_type_e evt_type /// Something to output void* output

5.8.3.1.4 BLE Initiating

5.8.3.1.4.1 Connection

There are two connection methods: direct connection and connection with the initaited type. Direct connection is realized by connecting with the initiated type APP_BLE_INIT_TYPE_DIRECT_CONN_EST.

If the connection is successful, BLE_LINK_CONNECTED_EVENT is triggered, and the callback function app_ble_customif_connect_event_handler() is executed.

If the connection fails, BLE_CONNECTING_FAILED_EVENT is triggered, and the callback function app_ble_customif_scan_starting_failed_event_handler() is executed.

These two callback functions can be modified as needed.

1. Direct connection

```
void app_ble_start_connect(ble_bdaddr_t *addr)
```

Table 68 Function app_ble_start_connect Description

Parameter	Description
struct ble_bdaddr_t *addr	The address of device to be connected typedef struct ble_bdaddr { /// BD Address of device struct addr[BTIF_BD_ADDR_SIZE]; /// Address type of the device 0: public / 1: private random uint8_t addr_type; } ble_bdaddr_t;

Sample Code 58: ble_start_connect

{"ble_start_connect", ble_start_connect_test},

2. Connection with the initiated type

CONFIDENTIAL

```
void app_ble_start_connect_with_init_type(ble_bdaddr_t *addr,
BLE_INIT_TYPE_E init_type, uint16_t conn_to)
```

Table 69 Function app_ble_start_connect_with_init_type Description

Parameter	Description
struct ble_bdaddr_t *addr	The address of device to be connected typedef struct ble_bdaddr { /// BD Address of device uint8_t addr[BTIF_BD_ADDR_SIZE]; /// Address type of the device 0=public/1=private random uint8_t addr_type; } ble_bdaddr_t;
enum BLE_INIT_TYPE_E init_type	typedef enum app_ble_init_type { /// Direct connection establishment, establish a /// connection with an indicated device APP_BLE_INIT_TYPE_DIRECT_CONN_EST = 0, /// Automatic connection establishment, establish a /// connection with all devices whose address is present /// in the white list APP_BLE_INIT_TYPE_AUTO_CONN_EST, /// Name discovery, Establish a connection with an /// indicated device in order to read content of its device /// Name characteristic. Connection is closed once this /// operation is stopped. APP_BLE_INIT_TYPE_NAME_DISC, } BLE_INIT_TYPE_E;
uint16_t conn_to	Connection timeout

Sample Code 59: ble_start_connect_with_init_type

{"ble_start_connect_with_init_type", ble_start_connect_with_init_type_test},

CONFIDENTIAL

5.8.3.1.4.2 Connection Status

In order to determine whether the two devices are connected, you can use the following methods to get the connection status:

1. Get the connection status by address

```
bool app_ble_is_connection_on_by_addr(uint8_t *addr)
```

Table 70 Function app_ble_is_connection_on_by_addr Description

Parameter	Description
uint8_t * addr	Pointer to the peer address
bool	Return Parameter True: It's connected False: It's disconnected

Sample Code 60: ble_is_connection_on_by_addr

{"ble_is_connection_on_by_addr", ble_is_connection_on_by_addr_test},

2. Get the connection status by connetion index

Each connection is assigned an index from 0 increasing by 1. In the case of knowing the connection index, the connection state can be obtained by using the following interface:

```
bool app_ble_is_connection_on(uint8_t index)
```

Table 71 Function app_ble_is_connection_on Description

Parameter	Description
uint8_t index	Connection index
bool	Return Parameter True: It's connected False: It's disconnected

Sample Code 61: ble_is_connection_on_by_conIdx

{"ble_is_connection_on_by_conIdx", ble_is_connection_on_by_conIdx_test},

5.8.3.1.4.3 Cancel on-going Connections

All on-going connections can be canceled by using the following intereface.

CONFIDENTIAL

If an on-going connection is canceled, BLE_CONNECTING_STOPPED_EVENT is triggered, and the callback function app_ble_customif_stopped_connecting_event_handler() is executed.

```
void app_ble_cancel_connecting(void)
```

Sample Code 62: ble_cancel_connecting

{"ble_cancel_connecting", ble_cancel_connecting_test},

5.8.3.1.4.4 Disconnection

The BLE connection can be diconnected by using the following interface.

If the disconnection starts, BLE_DISCONNECT_EVENT is triggered, and the callback function app_ble_customif_disconnect_event_handler() is executed.

```
void app_ble_start_disconnect(uint8_t conIdx)
```

Table 72 Function app_ble_start_disconnect Description

Parameter	Description
uint8_t conIdx	Connection index

Sample Code 63: ble_start_disconnect

{"ble_start_disconnect", ble_start_disconnect_test},

5.8.3.1.4.5 Connection Event

In order to feedback the specific connection operations, a series of callback interfaces is provided for users. When the connection event is triggered, the corresponding callback handler is called.

```
static const ble_event_handler_t app_ble_customif_event_handler_tab[] =
{
    {BLE_LINK_CONNECTED_EVENT, app_ble_customif_connect_event_handler},
    {BLE_CONNECT_BOND_EVENT, app_ble_customif_connect_bond_event_handler},
    {BLE_CONNECT_NC_EXCH_EVENT,
app_ble_customif_connect_nc_exch_event_handler},
    {BLE_CONNECT_ENCRYPT_EVENT,
app_ble_customif_connect_encrypt_event_handler},
    {BLE_CONNECTING_STOPPED_EVENT,
app_ble_customif_stopped_connecting_event_handler},
    {BLE_CONNECTING_FAILED_EVENT,
app_ble_customif_connecting_failed_event_handler},
```

CONFIDENTIAL

```
    {BLE_DISCONNECT_EVENT, app_ble_customif_disconnect_event_handler},
    … …
};
```

Table 73 Connection Event List

Event	Description	Event Code	Event Parameters

BLE_LINK_CONNECTED_EVENT	BLE connection event, this event will be triggered when the TWS is actively or passively connected to a ble link.	0x00	// Event type
			enum ble_event_type_e evt_type
			// Connection index
			uint8_t conidx;
			// Peer address
			uint8_t addr[6];
			/// Address type of the peer
			/// device 0: public
			/// 1: private random
			uint8_t addr_type;
			/// Something to output
			void* output

BLE_CONNECT_BOND_EVENT	BLE connection bound event, this event will be triggered when two BLE device are pairing.	0x01	/// event type enum ble_event_type_e evt_type /// Connection index int8_t conidx; /// True: pair successfully /// False: pair failed bool success; /// failed reason uint16_t reason /// Something to output void* output
BLE_CONNECT_NC_EXCH_EVENT	BLE Numeric Comparison exchange event, when BLE is encrypted, this event will be triggered after the value is exchanged, and the customer can check the value here. Choose to agree or reject	0x02	/// event type enum ble_event_type_e evt_type /// Connection index uint8_t conidx; /// Something to output void* output
BLE_CONNECT_ENCRYPT_EVENT	BLE encryption event, ,this event will be triggered when the encryption is completed	0x03	/// event type enum ble_event_type_e evt_type /// Connection index

CONFIDENTIAL

			uint8_t conidx; /// BLE pairing level uint8_t pairing_lvl; /// Something to output void* output
BLE_CONNECTING_STOPPED_EVENT	BLE cancel connection complete event	0x04	/// event type enum ble_event_type_e evt_type /// Peer address uint8_t peer_bdaddr[6]; /// Something to output void* output
BLE_CONNECTING_FAILED_EVENT	BLE connection failure event	0x05	/// event type enum ble_event_type_e evt_type /// Activity index uint8_t actv_idx; /// Error reason uint8_t err_code; /// Peer address uint8_t addr[6]; /// Address type of the peer /// device 0: public /// 1: private random uint8_t addr_type; /// Something to output void* output
BLE_DISCONNECT_EVENT	BLE disconnection event	0x06	/// event type enum ble_event_type_e evt_type /// Disconnected BLE link connection index uint8_t conidx; /// Reason for disconnection uint8_t errCode; /// Disconnected peer address uint8_t addr[6]; /// Address type of the peer /// device 0: public /// 1: private random uint8_t addr_type; /// Something to output

CONFIDENTIAL

			void* output

5.8.3.1.5 Connection Parameter Management

5.8.3.1.5.1 Update Connection Parameters

There are different connection modes in BLE. Each connection mode has corresponding parameters. If you want to customize connection parameters, add the specific connection mode to the table ble_conn_param_config.

```
typedef enum
{
    BLE_CONN_PARAM_MODE_DEFAULT       = 0,
    BLE_CONN_PARAM_MODE_AI_STREAM_ON,
    BLE_CONN_PARAM_MODE_A2DP_ON,
    BLE_CONN_PARAM_MODE_HFP_ON,
    BLE_CONN_PARAM_MODE_OTA,
    BLE_CONN_PARAM_MODE_OTA_SLOWER,
    BLE_CONN_PARAM_MODE_SNOOP_EXCHANGE,
    BLE_CONN_PARAM_MODE_BLE_AUDIO,
    BLE_CONN_PARAM_MODE_SVC_DISC_START,
    BLE_CONN_PARAM_MODE_SVC_DISC_CMP,
    BLE_CONN_PARAM_MODE_ISO_DATA_ACT,
    BLE_CONN_PARAM_MODE_ISO_DATA_STOP,
    BLE_CONN_PARAM_MODE_NUM,
} BLE_CONN_PARAM_MODE_E;

typedef enum
{
    BLE_CONN_PARAM_PRIORITY_NORMAL = 0,
    BLE_CONN_PARAM_PRIORITY_ABOVE_NORMAL0,
    BLE_CONN_PARAM_PRIORITY_ABOVE_NORMAL1,
    BLE_CONN_PARAM_PRIORITY_ABOVE_NORMAL2,
    BLE_CONN_PARAM_PRIORITY_HIGH,
} BLE_CONN_PARAM_PRIORITY_E;

// interval in the unit of 1.25ms
static const BLE_CONN_PARAM_CONFIG_T ble_conn_param_config[] =
{
    // default value: for the case of BLE just connected and the BT idle
state
    {BLE_CONN_PARAM_MODE_DEFAULT, BLE_CONN_PARAM_PRIORITY_NORMAL, 36, 0},
```

```
    {BLE_CONN_PARAM_MODE_AI_STREAM_ON,
BLE_CONN_PARAM_PRIORITY_ABOVE_NORMAL1, 36, 0},
     … …
     … …
    // TODO: add mode cases if needed
};
```

Multiple connection modes can be enabled at the same time, but only one connection mode takes effect. The connection mode with higher priority takes effect first, and the connection mode with the same priority has a smaller connection interval.

If a connection mode is disabled, other enabled connection mode takes effect.

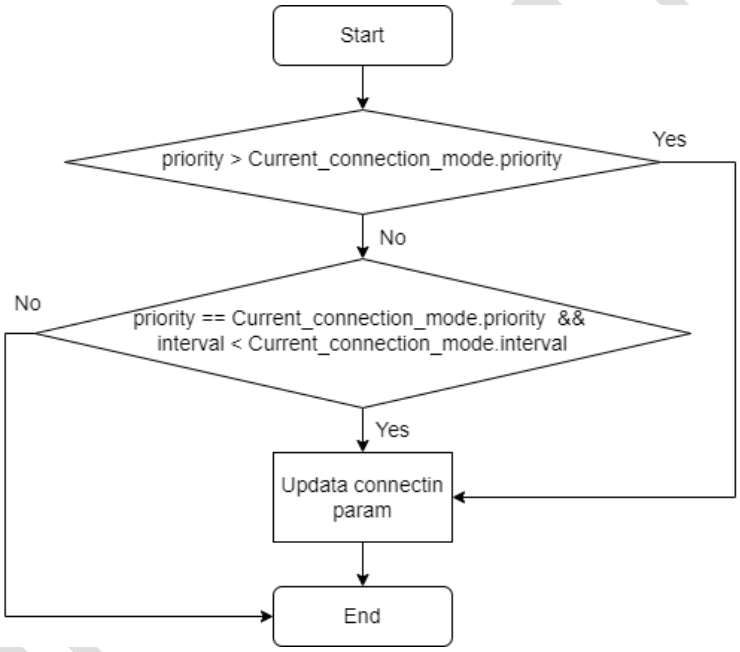


Figure 48 Connection Parameter Update Policy

A connection mode can be enabled or disabled by using the following interface.

If the connection mode is enbaled, the event BLE_CONN_PARAM_UPDATE_REQ_EVENT is triggered, and the callback function app_ble_customif_conn_param_update_req_event_handler is executed.

If the connection parameters change, the event BLE_CONN_PARAM_UPDATE_SUCCESSFUL_EVENT is triggered, and the callback function app_ble_customif_conn_param_update_successful_event_handler is executed.

If the connection parameters fail to change, the event BLE_CONN_PARAM_UPDATE_FAILED_EVENT is triggered, and the callback function app_ble_customif_conn_param_update_failed_event_handler is executed.

```
void app_ble_update_conn_param_mode(BLE_CONN_PARAM_MODE_E mode, bool
isEnable)
```

Table 74 Function app_ble_update_conn_param_mode Description

Parameter	Description
enum BLE_CONN_PARAM_MODE_E mode	Connection mode that needs to be configured
bool isEnable	True: enable update connection mode False: disable update connection mode

Sample Code 64: ble_update_conn_param_mode

{"ble_update_conn_param_mode", ble_update_conn_param_mode_test},

5.8.3.1.5.2 Get Connection Parameters

The connection parameters can be obtained by using the following interface.

```
bool app_ble_get_conn_param(uint8_t conidx,  APP_BLE_CONN_PARAM_T*
pConnParam)
```

Table 75 Function app_ble_conn_param Description

Parameter	Description
uint8_t conidx	Connection index
struct APP_BLE_CONN_PARAM_T* pConnParam	typedef struct { /// Connection interval value uint16_t con_interval; /// Connection latency value uint16_t con_latency; /// Supervision timeout uint16_t sup_to; } APP_BLE_CONN_PARAM_T;

Sample Code 65: ble_get_conn_param

{"ble_get_conn_param", ble_get_conn_param_test},

CONFIDENTIAL

5.8.3.1.5.3 Connection Parameter Management Event

In order to feedback the specific connection parameter management operations, a series of callback interfaces is provided for users. When the connection parameter management event is triggered, the corresponding callback handler is called.

```
static const ble_event_handler_t app_ble_customif_event_handler_tab[] =
{
    … …
   {BLE_CONN_PARAM_UPDATE_REQ_EVENT,
app_ble_customif_conn_param_update_req_event_handler},
   {BLE_CONN_PARAM_UPDATE_FAILED_EVENT,
app_ble_customif_conn_param_update_failed_event_handler},
   { BLE_CONN_PARAM_UPDATE_SUCCESSFUL_EVENT,
app_ble_customif_conn_param_update_successful_event_handler},
    … …
};
```

Table 76 Connection Parameter Management Event List

Event	Description	Event Code	Event Parameters

BLE_CONN_PARAM_UPDATE_REQ_EVENT	BLE connection parameter update request event	0x0E	/// Event type
			enum ble_event_type_e evt_type
			// Connection index
			uint16_t conidx;
			// Connection interval minimum
			uint16_t intv_min;
			// Connection interval maximum
			uint16_t intv_max;
			// Latency
			uint16_t latency;
			// Supervision timeout
			uint16_t time_out;
			// Something to output
			void* output

BLE_CONN_PARAM_UPDATE_FAILED_EVENT	BLE connection parameter update failure event	0x0F	// event type
			enum ble_event_type_e evt_type
			// Connection index
			uint8_t conidx;
			// Error reason

CONFIDEN

			uint8_t err_code; // Something to output void* output

BLE_CONN_PARAM_UPDATE_SUCCESSFUL_EVENT	BLE connection parameter update success event	0x10	// event type
			enum ble_event_type_e evt_type
			// Connection index
			uint16_t conidx;
			// Connection interval value
			uint16_t con_interval;
			// Latency
			uint16_t con_latency;
			// Supervision timeout
			uint16_t sup_out;
			// Something to output
			void* output

5.8.3.1.6 BLE Connection Process Case

The BLE connection process is shows as follows:

CONFIDENTIAL

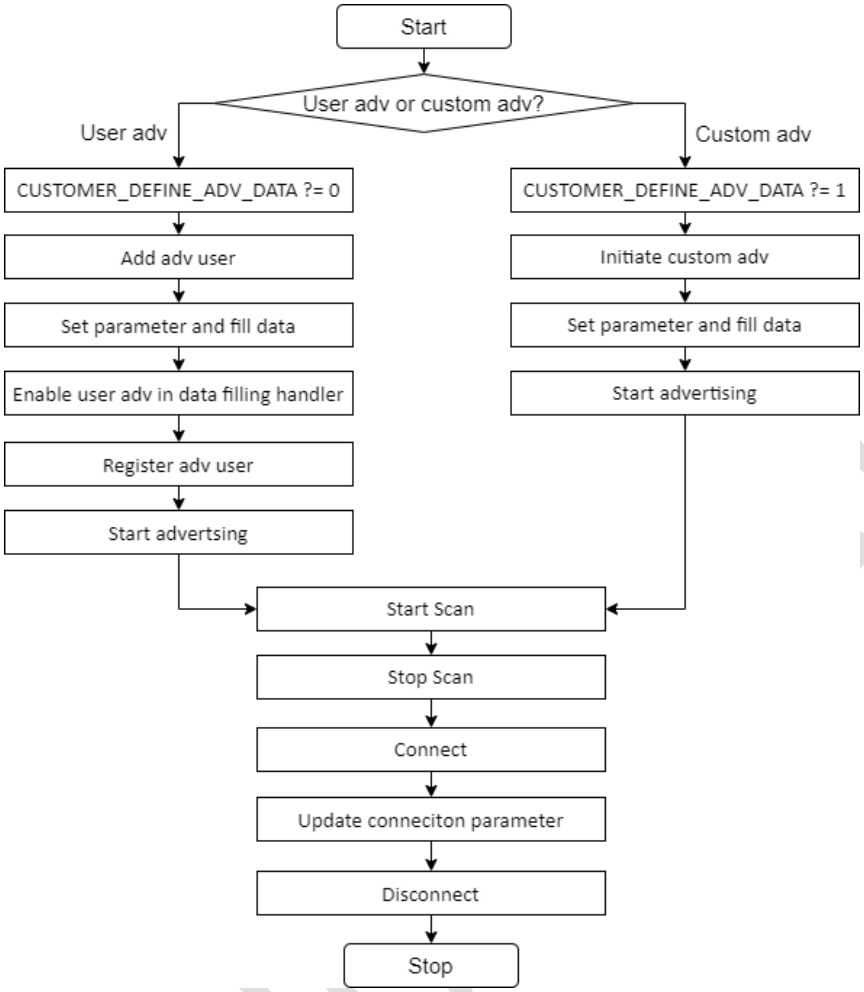


Figure 49 BLE Connection Process

Steps:

1. Advertising

Choose advertising : custom adv or user adv

1.1 User adv

- Disable the macro switch CUSTOMER_DEFINE_ADV_DATA in target.mk.
- Add the adv user to BLE_ADV_USER_E, ble_adv_user2str and add the adv user to ble_adv_fill_param.
- Fill in advertising data and scan response data and set advertising parameters in the data filling function.
- Enable the user adv by using the interface app_ble_data_fill_enable() in the data filling function.
- Set advertising parameters by using interfaces. (This part is not neccessary)

  Set an advertising type by using the interface app_ble_set_adv_type(). See Sample code 47 for details.

  Set an advertising interval by using the interface app_ble_param_set_adv_interval(). See Sample code 48 for details.

  Set the TX power by using the interface app_ble_set_adv_txpwr_by_actv_user(), app_ble_set_adv_txpwr_ by_act_user() or app_ble_set_all_adv_txpwr(). See Sample code 49, 50 and 51 for details.

Set a local address by using the interface app_ble_set_adv_local_addr_by_actv_user() or app_ble_set_adv_ local_addr_by_adv_user(). See Sample code 52 and 53 for details.

- Register the user adv by using the interface app_ble_register_data_fill_handle(). See Sample code 45 for details.
- Start the user adv by using the interface app_ble_start_connectable_adv(). See Sample code 54 for details.

1.2 Custom adv

- Enable the macro switch CUSTOMER_DEFINE_ADV_DATA in target.mk.
- Initialize the custom user by using the interface app_ble_custom_init().
- Set advertising parameters in the data filling function app_ble_custom_adv_write_data().
- Start the custom adv by using the interface app_ble_custom_adv_start(). See Sample code 55 for details.

2. Scan

- Start a scan by using the interface app_ble_start_scan(). See Sample code 56 for details.
- Stop a scan by using the interface app_ble_stop_scan(). See Sample code 57 for details.

3. Connection

- Start connection by using the interface ble_start_connect() or app_ble_start_connect_with_init_type(). See Sample code 58 and 59 for details.
- Get the connection status by using the interface app_ble_is_connection_on_by_addr() or app_ble_is_connection_on(). See Sample code 60 and 61 for details.

4. Update connection parameters (This part is not necessary)

- Add connection parameters in ble_conn_param_config.
- Enable connection mode by using the interface app_ble_update_conn_param_mode(). See Sample code 64 for details.
- Get connection parameters by the using interface app_ble_get_conn_param(). See Sample code 65 for details.

5. Disconnection

- Start disconnection by using the interface app_ble_start_disconnect() or app_ble_disconnect_all(). See Sample code 63 for details.

5.8.3.2 Add a Custom BLE Profile

5.8.3.2.1 Custom BLE Profile Overview

1. BLE Profile functional division

CONFIDENTIAL

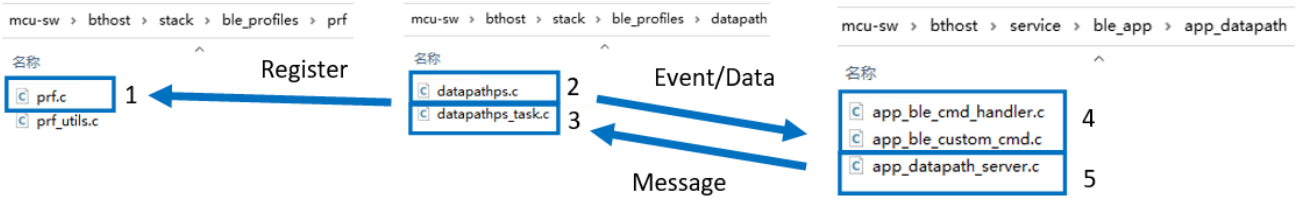


Figure 50 BLE Profile Functional Division

1) Profile manager. To add the BLE profile into the BLE database.

2) BLE profile definition. Define the profile's UUID/attribute database and the profile initializer, creator, clean-up and destroy functions here, for the registration to the profile manager.

3) The bridge between application layer and the profile layer. (Optional)
   Define the message APIs for the application layer to call:
   - Enable the notification/indication
   - Send out the data via Write req/command or notification/indication
     Distribute the following event and data to the application layer:
   - Receive the enabling notification/indication request
   - Receive the data from write req/command or notification/indication
   - Receive the sent done event of write req/command or notification/indication

4) Application layer. Define the APIs for the top level to call, and distribute the event and data to the top level.

5) Application. Make use of the APIs and event/data callback functions of application layer.

2. Steps to Add a BLE Custom Profile

Based on the BLE profile functional division, the following steps are needed to create a BLE custom profile.

1) Define a BLE profile.
2) Register the BLE profile to profile manager.
3) Define the bridge APIs and event/data distributors.
4) Define the application layer APIs and event/data distributors.

5.8.3.2.2 Define the BLE Profile

1. Turn to rwprf_config.h, add the macro switch of the profile. Then all the profile related functions and variables can be wrapped by the macro BLE_DATAPATH_SERVER.

```
#define CFG_PRF_DATAPATH_SERVER

/// Data Path Service Server Role
#if defined(CFG_PRF_DATAPATH_SERVER)
#define BLE_DATAPATH_SERVER    1
```

CONFIDENTIAL

```
#else
#define BLE_DATAPATH_SERVER     0
#endif // defined(CFG_PRF_DATAPATH_SERVER)
```

2. Turn to datapathps.c, specify the profile attribute database datapathps_att_db.

The default database is as follows:

```
/// Full DATAPATH SERVER Database Description - Used to add attributes into
the /// database
const struct gatt_att_desc datapathps_att_db[DATAPATHPS_IDX_NB] =
{
    // Service Declaration
    [DATAPATHPS_IDX_SVC]        =   {GATT_DECL_PRIMARY_SERVICE, PROP(RD), 0},

    // TX Characteristic Declaration
    [DATAPATHPS_IDX_TX_CHAR]    =   {GATT_DECL_CHARACTERISTIC_UUID, PROP(RD),
0},
    // TX Characteristic Value
    [DATAPATHPS_IDX_TX_VAL]     =   {datapath_tx_char_val_uuid_128_content,
PROP(N) | PROP(RD) | ATT_UUID(128), DATAPATHPS_MAX_LEN},

    // TX Characteristic - Client Characteristic Configuration Descriptor
#if defined(CTKD_ENABLE) && !defined(BLE_AUDIO_STARLORD_COMPATIBLE_SUPPORT)
    // Let app write ccc to trigger secure connection pairing
    [DATAPATHPS_IDX_TX_NTF_CFG] =   {GATT_DESC_CLIENT_CHAR_CFG_UUID,
PROP(RD) | PROP(WR) | SEC_LVL(WP, AUTH), PRF_SVC_DESC_CLI_CFG_LEN},
#else
    [DATAPATHPS_IDX_TX_NTF_CFG] =   {GATT_DESC_CLIENT_CHAR_CFG_UUID,
PROP(RD) | PROP(WR), PRF_SVC_DESC_CLI_CFG_LEN},
#endif
    // TX Characteristic - Characteristic User Description Descriptor
    [DATAPATHPS_IDX_TX_DESC]    =   {GATT_DESC_CHAR_USER_DESCRIPTION_UUID,
 PROP(RD), 32},

    // RX Characteristic Declaration
    [DATAPATHPS_IDX_RX_CHAR]    =   {GATT_DECL_CHARACTERISTIC_UUID, PROP(RD),
0},
    // RX Characteristic Value
    [DATAPATHPS_IDX_RX_VAL]     =   {datapath_rx_char_val_uuid_128_content,
PROP(RD) | PROP(WR) | PROP(WC) | ATT_UUID(128), DATAPATHPS_MAX_LEN},
```

```
    // RX Characteristic - Characteristic User Description Descriptor
    [DATAPATHPS_IDX_RX_DESC]    =   {GATT_DESC_CHAR_USER_DESCRIPTION_UUID,
PROP(RD), 32},
};
```

The UUID of the service and characteristics should be defined here, there are two choices, 128 bit or 16 bit.

For 128 bit, define USE_128BIT_UUID as 1 and change following UUID as the customized:

```
#define datapath_service_uuid_128_content      {0x12, 0x34, 0x56, 0x78,
0x90, 0x00, 0x00, 0x80, 0x00, 0x10, 0x00, 0x00, 0x00, 0x01, 0x00, 0x01 }
#define datapath_tx_char_val_uuid_128_content   {0x12, 0x34, 0x56, 0x78,
0x91, 0x00, 0x00, 0x80, 0x00, 0x10, 0x00, 0x00, 0x00, 0x02, 0x00, 0x02 }
#define datapath_rx_char_val_uuid_128_content   {0x12, 0x34, 0x56, 0x78,
0x92, 0x00, 0x00, 0x80, 0x00, 0x10, 0x00, 0x00, 0x00, 0x03, 0x00, 0x03 }
```

For 16 bit, define USE_128BIT_UUID as 0 and change following UUID as the customized:

```
#define DATAPATHPS_SERVICE_UUID_16BIT      0xFEF8
#define DATAPATHPS_TX_CHAR_VAL_UUID_16BIT   0xFEF9
#define DATAPATHPS_RX_CHAR_VAL_UUID_16BIT   0xFEFA
```

3. Define the profile init, destroy, con_create, con_clean_up and con_updated APIs and fill them into the profile callback list:

```
/// DATAPATHPS Task interface required by profile manager
const struct prf_task_cbs datapathps_itf =
{
    _init_cb,
    _destroy_cb,
    _con_create_cb,
    _con_cleanup_cb,
};

/// exported functions definitions
const struct prf_task_cbs* datapathps_prf_itf_get(void)
{
    return &datapathps_itf;
}
```

Add database to gatt user client:

```
#if USE_128BIT_UUID
       status = gatt_db_svc_add(datapathps_env->user_lid,
```

```
                                SVC_SEC_LVL(NOT_ENC) | SVC_UUID(128),
                                DATAPATH_SERVICE_UUID_128,
                                DATAPATHPS_IDX_NB, NULL, datapathps_att_db,
                                DATAPATHPS_IDX_NB,
                                &datapathps_env->shdl);
#else
       status = gatt_db_svc16_add(datapathps_env->user_lid,
                                  SVC_SEC_LVL(NOT_ENC),
                                  ATAPATHPS_SERVICE_UUID_16BIT,
                                  DATAPATHPS_IDX_NB, NULL,
                                  datapathps_att_db, DATAPATHPS_IDX_NB,
                                  &datapathps_env->shdl);
#endif
```

4. Define the set of callbacks functions for communication with GATT as a GATT User Client:

```
// cb_event_sent:
// This function is called when GATT server user has initiated event  send
to peer // device or if an error occurs.

// cb_att_read_get:
// This function is called when peer want to read local attribute database
value.

// cb_att_event_get:
// This function is called when GATT server user has initiated event send
procedure.

// cb_att_info_get:
// This function is called during a write procedure to get information
about a // specific attribute handle.

// cb_att_val_set:
// This function is called during a write procedure to modify attribute
handle.

/// Set of callbacks functions for communication with GATT as a GATT User
Client
__STATIC const gatt_srv_cb_t datapath_gatt_cb = {
    .cb_event_sent = datapath_gatt_cb_event_sent,
```

```
    .cb_att_read_get = datapath_gatt_cb_att_read_get,
    .cb_att_event_get = datapath_gatt_cb_att_event_get,
    .cb_att_info_get = datapath_gatt_cb_att_info_get,
    .cb_att_val_set = datapath_gatt_cb_att_set,
};
```

Register as GATT User Client in function _init_cb

```
status = gatt_user_srv_register(PREFERRED_BLE_MTU, 0, &datapath_gatt_cb,
                                &(datapathps_env->user_lid));
```

5.8.3.2.3 Register the BLE Profile

Turn to the profile manager source file prf.c. Register the profile by adding the following lines into prf_itf[PRF_ID_MAX] (array of the profile interface structure):

```
#if (BLE_DATAPATH_SERVER)
PRF_ITF(DATAPATHPS, datapathps_prf_itf_get),
#endif //(BLE_DATAPATH_SERVER)
```

5.8.3.2.4 Define Bridge APIs and Event/Data Distributors

Turn to file datapathps_task.c, and define the following message APIs for the application layer to call:

```
KE_MSG_HANDLER_TAB(datapathps)
{
    // send data via notification, can be called when the device is the BLE
peripheral
    {DATAPATHPS_SEND_DATA_VIA_NOTIFICATION,     (ke_msg_func_t)
                                    send_data_via_notification_handler},
    // send data via write command, can be called when the device is the BLE
central
    {DATAPATHPS_SEND_DATA_VIA_WRITE_COMMAND,    (ke_msg_func_t)
                                    send_data_via_write_command_handler},
};
```

5.8.3.2.5 Define Application Layer APIs and Event/Data Distributors

1. Turn to rwapp_config.h, and add the macro switch of the profile as follows:

```
#define CFG_APP_DATAPATH_SERVER

/// Data Path Server Application
#if defined(CFG_APP_DATAPATH_SERVER)
```

```
#define BLE_APP_DATAPATH_SERVER          1
#else
#define BLE_APP_DATAPATH_SERVER          0
#endif
```

Then all the profile related application layer functions and variables can be wrapped by the macro BLE_APP_DATAPATH_SERVER.

2. Turn to file app_datapath_server.c, and define the following APIs for the application top layer to call.

```
// initialize the profile application layer
void app_datapath_server_init(void);

// register the profile service into the service manager
void app_datapath_add_datapathps(void);

// send notification when the device is BLE peripheral
void app_datapath_server_send_data_via_notification(uint8_t* ptrData,
uint32_t length);

// send write command when the device is BLE central
void app_datapath_server_send_data_via_write_command(uint8_t* ptrData,
uint32_t length);

// register a callback function when the write command or notification has
been // sent out. when the data needs to be sent continously, this callback
function // can be used to trigger the next transmission
void app_datapath_server_register_tx_done(app_datapath_server_tx_done_t
callback);

// enable or disable the notification when the device is BLE central
void app_datapath_server_control_notification(bool isEnable);
```

3. Turn to file app_datapath_server.c, and define the following callback functions for internal handling:

```
// the BLE connection is connected
void app_datapath_server_connected_evt_handler(uint8_t conidx);
// the BLE connection is disconnected
void app_datapath_server_disconnected_evt_handler(uint8_t conidx);
```

4. Register the profile service into the APP service manger.

1) Add the adding profile API into the BLE database creation function list:

```
static const appm_add_svc_func_t appm_add_svc_func_list[APPM_SVC_LIST_STOP]
=
{
    … …
    #if (BLE_APP_DATAPATH_SERVER)
        (appm_add_svc_func_t)app_datapath_add_datapathps,
        #endif //(BLE_APP_DATAPATH_SERVER)
    … …
}
```

2) In the function appm_init, add the profile application layer initialization function:

```
#if (BLE_APP_DATAPATH_SERVER)
    // Data Path Server Module
    app_datapath_server_init();
#endif //(BLE_APP_DATAPATH_SERVER)
```

5. Register the profile service into the APP message handler.

1) Turn to rwip_task.h, add the profile task ID into enum TASK_API_ID:

```
TASK_ID_DATAPATHPS   = 91,   // Datapath Server Task
```

2) Turn to app_task.c and include the header file:

```
#if (BLE_APP_DATAPATH_SERVER)
#include "app_datapath_server.h"  // Data Path Server Application
Definitions
#endif //(BLE_APP_DATAPATH_SERVER)
```

3) In function appm_msg_handler, add the following content:

```
#if (BLE_APP_DATAPATH_SERVER)
       case (TASK_ID_DATAPATHPS):
       {
       // Call the Battery Module
       msg_pol = app_get_handler(&app_datapath_server_table_handler, msgid,
param, src_id);
       } break;
#endif //(BLE_APP_DATAPATH_SERVER)
```

5.8.3.3 Add a Custom l2cap Connection-oriented Channel

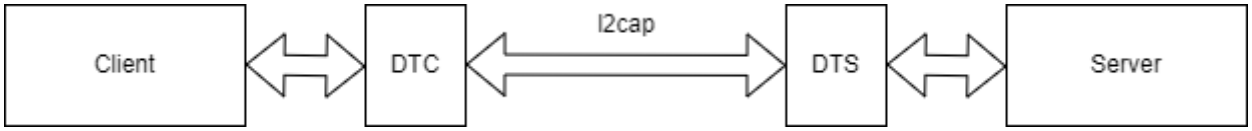


Figure 51 l2cap Connection

As the figure shows, the client and server transmit data on l2cap through DTC and DTS. Before data transmission, the client has to estabilsh a connection with the server.

Steps to add a custom l2cap connection_oriented channel:

1. The server registers the connection channel on l2cap;
2. The client initiates a connection and connects to the server;
3. The server and client transfer data on l2cap through DTS and DTC respectively.
4. The server and client are disconnected after data transfer is completed.

5.8.3.3.1 DTS

5.8.3.3.1.1 DTS Register

The l2cap connection channel can be registered by the server using the following interface.

If a l2cap connection channel is registered, the event APP_DTS_COC_REGISTERED_IND is triggered, and the callback function aob_dts_coc_registered_cb() is executed.

```
void aob_dts_register_spsm(uint16_t spsm, uint16_t initial_credits)
```

Table 77 Function aob_dts_register_spsm Description

Parameter	Description
uint16_t spsm	Simplified Protocol/Service Multiplexer
uint16_t initial_credits	Initial credits

Sample Code 66: DTS Registration

```
{"dts_config", aob_dts_configure},
```

5.8.3.3.1.2 DTS Data Sending

The server can send data to the client on the l2cap connection channel by using the following interface.

CONFIDENTIAL

When the data was sent by the server, the event APP_DTS_COC_SEND_IND is triggered, and the callback function aob_dts_coc_send_cb() is executed.

```
void aob_dts_send_data(uint8_t con_lid, uint16_t spsm, uint16_t length,
const uint8_t *sdu)
```

Table 78 Function aob_dts_send_data Description

Parameter	Description
uint16_t spsm	Simplified Protocol/Service Multiplexer
uint16_t initial_credits	Initial credits

Sample Code 67: dts_send_data

{"dts_send_data", aob_dts_send_data},

5.8.3.3.1.3 DTS Disconnection

The server can disconnect the client on the l2cap connection channel by using the following interface.

When the sever is disconnected from the client, the event APP_DTS_COC_DISCONNECTED_IND is triggered, and the callback function aob_dts_coc_disconnected_cb() is executed.

```
void aob_dts_disconnect(uint8_t con_lid, uint16_t spsm)
```

Table 79 Function aob_dts_disconnect Description

Parameter	Description
uint16_t con_lid	Connection local index . Used to identify the connected device
uint16_t spsm	Simplified Protocol/Service Multiplexer

Sample Code 68: dts_disconnect

{"dts_disconnect", aob_dts_disconnect},

5.8.3.3.1.4 DTS Events

In order to feedback the specific operations, a series of callback interfaces is provided for users. When the DTS event is triggered, the corresponding callback handler is called.

```
static dts_coc_event_handler_t dts_coc_event_cb = {
    .dts_coc_registered_cb = aob_dts_coc_registed_cb,
    .dts_coc_connected_cb = aob_dts_coc_connected_cb,
```

CONFIDENTIAL

```
    .dts_coc_disconnected_cb = aob_dts_coc_disconnected_cb,
    .dts_coc_data_cb = aob_dts_coc_data_cb,
    .dts_coc_send_cb = aob_dts_coc_send_cb,
};
```

Table 80 DTS Callback handler Description

Callback handler	Description	Event Parameters
dts_coc_registered_cb	The server registers the connection channel on l2cap	/// register status uint8_t status, /// Simplified Protocol/ServiceMultiplexer uint16_t spsm
dts_coc_connected_cb	The server connects the client on l2cap channel	/// Connection local index uint8_t con_lid, /// Maximum SDU size that the peer on /// the link can receive uint16_t tx_mtu, /// Maximum packet size that the peer on the /// link can receive uint16_t tx_mps, /// Simplified Protocol/Service Multiplexer uint16_t spsm, /// initial credits uint16_t initial_credits
dts_coc_disconnected_cb	The server disconnects the client on the l2cap channel	/// Connection local index uint8_t con_lid, /// Disconnection reason uint16_t reason, /// Simplified Protocol/Service Multiplexer uint16_t spsm
dts_coc_data_cb	The server receives data from the client on the l2cap channel	/// Connection local index uint8_t con_lid, /// Simplified Protocol/Service Multiplexer uint16_t spsm, /// SDU data length uint16_t length, /// SDU data uint8_t *sdu

CONFIDENTIAL

dts_coc_send_cb	The server sends data to the client on the l2cap channel	/// Connection local index uint8_t con_lid, /// Simplified Protocol/Service Multiplexer uint16_t spsm

5.8.3.3.2 DTC

5.8.3.3.2.1 DTC Connection

The client can connect with the server on the l2cap channel by using the following interface.

When the client is connected, the event APP_DTC_COC_CONNECTED_IND is triggered, and the callback function aob_dtc_coc_connected_cb() is executed.

```
void aob_dtc_connect(uint8_t con_lid, uint16_t local_max_sdu, uint16_t
spsm)
```

Table 81 Function aob_dtc_connect Decription

Parameter	Description
uint8_t con_lid	Connection local index . Used to identify the connected device
uint16_t local_max_sdu	Maximum length of the SDU
uint16_t spsm	Simplified Protocol/Service Multiplexer

Sample Code 69: dts_connect

{"dtc_connect", aob_dtc_connect},

5.8.3.3.2.2 DTC Data Sending

The client can send data to the server on the l2cap channel by using the following interface.

```
void aob_dtc_send_data(uint8_t con_lid, uint16_t spsm, uint16_t length,
                       uint8_t *sdu)
```

Table 82 Function aob_dtc_send_data Description

Parameter	Description
uint8_t con_lid	Connection local index. Used to identify the connected device
uint16_t spsm	Connection local index
uint16_t	SDU data length

CONFIDENTIAL

length	
uint8_t * sdu	SDU data to be tranferred to the peer device

Sample Code 70: dts_send_data

{"dtc_send_data", aob_dtc_send_data},

5.8.3.3.2.3 DTC Disconnection

The client can disconnect from the server on the l2cap channel by using the following interface.

When the client is disconnected, the event APP_DTC_COC_DISCONNECTED_IND is triggered, and the callback function aob_dtc_coc_disconnected_cb() is executed.

```
void aob_dtc_disconnect(uint8_t con_lid, uint16_t spsm)
```

Table 83 Function aob_dtc_disconnect Description

Parameter	Description
uint8_t con_lid	Connection local index . Used to identify the connected device
uint16_t spsm	Simplified Protocol/Service Multiplexer

Sample Code 71: dts_disconnect

{"dtc_disconnect", aob_dtc_disconnect},

5.8.3.3.2.4 DTC Events

In order to feedback the specific operations, a series of callback interfaces is provided for users. When the DTC event is triggered, the corresponding callback handler is called.

```
static dtc_coc_event_handler_t dtc_coc_event_cb = {
    .dtc_coc_connected_cb = aob_dtc_coc_connected_cb,
    .dtc_coc_disconnected_cb = aob_dtc_coc_disconnected_cb,
    .dtc_coc_data_cb = aob_dtc_coc_data_cb,
};
```

Table 84 DTC Callback Handler Description

Callback handler	Description	Event Parameters
dtc_coc_connected_cb	The Client connect to the Server on the l2cap channel	/// Connection local index uint8_t con_lid,

		/// Maximum SDU size that the peer on the link /// can receive uint16_t tx_mtu, /// Maximum packet size that the peer on the /// link can receive uint16_t tx_mps, /// Simplified Protocol/Service Multiplexer uint16_t spsm
dtc_coc_disconnected_cb	The Client disconnects to the Server on l2cap channel	/// Connection local index uint8_t con_lid, /// Disconnection reason uint16_t reason, /// Simplified Protocol/Service Multiplexer uint16_t spsm
dtc_coc_data_cb	Client receives data from Server on l2cap channel	/// Connection local index uint8_t con_lid, /// SDU data length uint16_t length, /// SDU data uint8_t *sdu, /// Simplified Protocol/Service Multiplexer uint16_t spsm

5.8.4 TWS Communication Data Path

Custom L2CAP is implemented as the communication data path between the master and slave buds.

The L2CAP connection flow is as follows:

CONFIDENTIAL

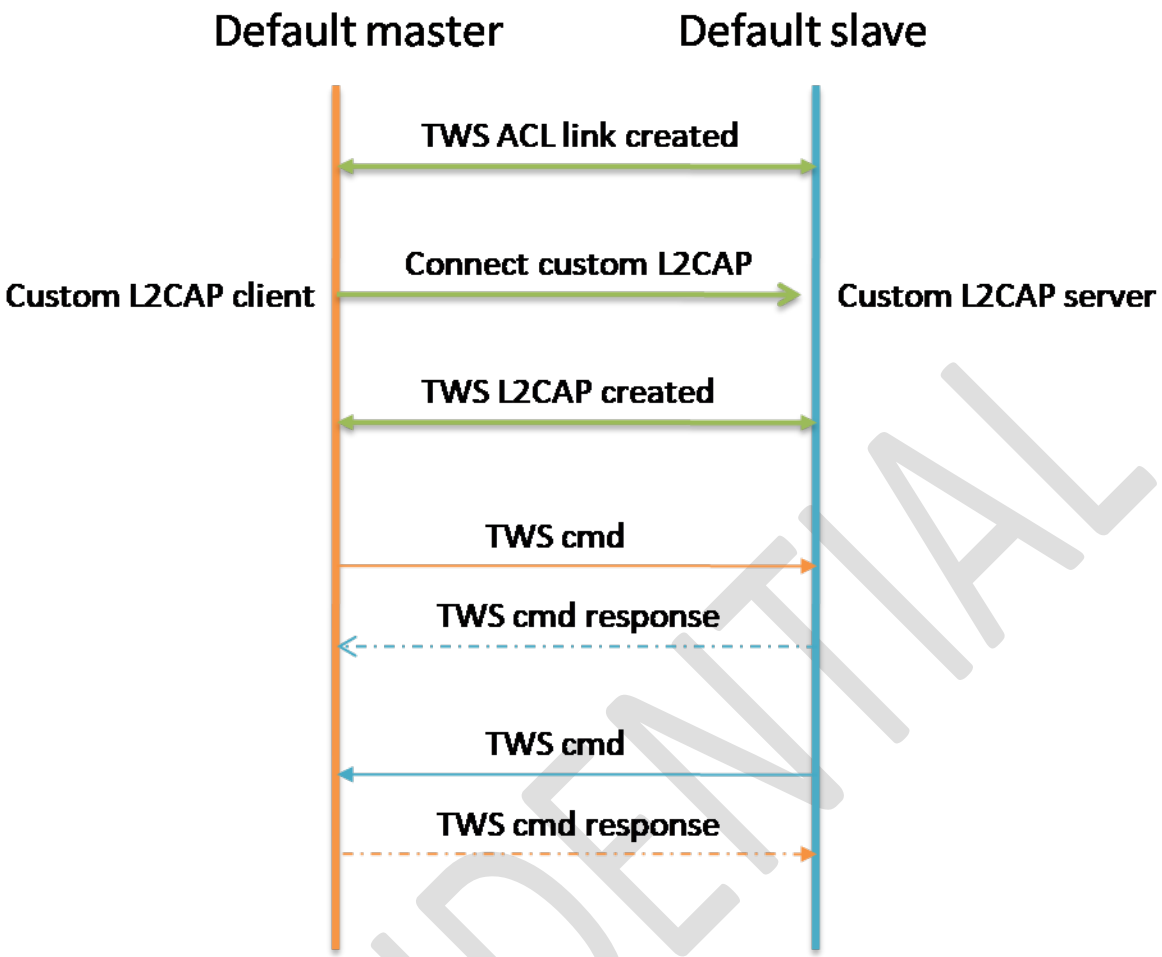


Figure 52 L2CAP Connection Datapath

Both earbuds register custom L2CAP in their classic BT profile database. When a TWS ACL link is created, the default master bud (the one that is chosen as the master during TWS pairing) triggers the L2CAP connection as the client.

After the TWS custom L2CAP connection is created. Either side can send TWS commands to the other side. The TWS command handler is designed to support “wait response” and “don’t wait response” modes, to meet different requirements.

TWS custom L2CAP isn’t influenced by TWS role switch, and across the lifetime of TWS link, TWS custom L2CAP is alive for use by the TWS command handler layer.

There is an existing custom TWS command table in app_ibrt_customif_cmd.cpp for developers to add their own commands.

The command entry definition:

Table 85 Command Entry Definition:

Item	Description	Notes

CONFIDENTIAL

Struct app_tws_cmd_instance_t	The command entry definition	typedef struct { uint32_t cmdcode; const char *log_cmd_code_str; app_tws_cmd_send_handler_t tws_cmd_send; app_tws_cmd_receivd_handler_t cmdhandler; /**< command handler function */ uint32_t timeout_ms; app_tws_rsp_timeout_handle_t app_tws_rsp_timeout_handle; app_tws_rsp_handle_t app_tws_rsp_handle; app_tws_cmd_tx_done_handler_t app_tws_cmd_tx_done_handler; } __attribute__((packed)) app_tws_cmd_instance_t;

Use the following sample code to configure GPIO_3_2 as a general GPIO in the 1.8v voltage domain, no pull state.

For example:

No-wait-response TWS custom command:

```
{
    APP_IBRT_CUSTOM_CMD_TEST1,                              "TWS_CMD_TEST1",
    app_ibrt_customif_test1_cmd_send,
    app_ibrt_customif_test1_cmd_send_handler,                  0,
    app_ibrt_custom_cmd_rsp_timeout_handler_null,
app_ibrt_custom_cmd_rsp_handler_null
},
tws_ctrl_send_cmd(APP_IBRT_CUSTOM_CMD_TEST1, (uint8_t*)cmd_test,
sizeof(ibrt_custom_cmd_test_t));
```

Wait-response TWS custom command:

```
{
    APP_IBRT_CUSTOM_CMD_TEST2,                              "TWS_CMD_TEST2",
    app_ibrt_customif_test2_cmd_send,
```

```
    app_ibrt_customif_test2_cmd_send_handler,
RSP_TIMEOUT_DEFAULT,
    app_ibrt_customif_test2_cmd_send_rsp_timeout_handler,
app_ibrt_customif_test2_cmd_send_rsp_handler
},
tws_ctrl_send_cmd(APP_IBRT_CUSTOM_CMD_TEST2, (uint8_t*)cmd_test,
sizeof(ibrt_custom_cmd_test_t));
```

5.8.5 GATT over BR/EDR

The Generic Attribute profile (GATT) defines a service framework using the Attribute protocol. This framework defines the procedures and formats of services and their characteristics. The procedures defined include discovering, reading, writing, notifying and indicating characteristics, as well as configuring the broadcast of characteristics. GATT is just a description of a simple protocol and is not limited to BLE or BR/EDR. The Bluetooth core specification 4.0 updated to support GATT over BR/EDR in 2010. Define the macro GATT_OVER_BR_EDR to enable this feature. After defining this macro, GATT and GATT over BR/EDR are used in the same way.

```
GATT_OVER_BR_EDR ?= 1
```

Take TOTA as an example. More details about TOTA can be found in 5.5

The user callback set of the GATT server is as follows.

Table 86 GATT Server User Callback Set

Item	Description	Notes
typedef struct gatt_srv_cb	GATT server user callback set	typedef struct gatt_srv_cb { void (*cb_event_sent) (uint8_t conidx, uint8_t user_lid, uint16_t dummy, uint16_t status); void (*cb_att_read_get) (uint8_t conidx, uint8_t user_lid, uint16_t token, uint16_t hdl, uint16_t offset, uint16_t max_length); void (*cb_att_event_get) (uint8_t conidx, uint8_t user_lid, uint16_t token, uint16_t dummy, uint16_t hdl, uint16_t max_length);

CONFIDENTIAL

		void (*cb_att_info_get) (uint8_t conidx, uint8_t user_lid, uint16_t token, uint16_t hdl); void (*cb_att_val_set) (uint8_t conidx, uint8_t user_lid, uint16_t token, uint16_t hdl, uint16_t offset, co_buf_t* p_data); } gatt_srv_cb_t;

This function is called when the GATT server user initiates an event sending request to the peer. The parameter conidx is the connection index, user_lid is the GATT user local identifier, dummy is the dummy parameter provided by upper layer for command execution and status is the status of the procedure.

```
__STATIC void tota_gatt_cb_event_sent(uint8_t conidx, uint8_t user_lid,
uint16_t dummy, uint16_t status)
```

This function is called when the peer wants to read local attribute database values. The parameter conidx is the connection index, user_lid is the GATT user local identifier, token is the procedure token that must be returned in confirmation function, hdl is the attribute handle, offset is the value offset, and max_length is the maximum value length to return.

```
__STATIC void tota_gatt_cb_att_read_get(uint8_t conidx, uint8_t user_lid,
uint16_t token, uint16_t hdl, uint16_t offset, uint16_t max_length)
```

This function is called when the GATT server user initiates an event sending procedure. The parameter conidx is the connection index, user_lid is the GATT user local identifier, token is the procedure token that must be returned in confirmation function, dummy is the dummy parameter provided by the upper layer for command execution, hdl is the attribute handle, offset is the value offset, and max_length is the maximum value length to return.

```
__STATIC void tota_gatt_cb_att_event_get(uint8_t conidx, uint8_t user_lid,
uint16_t token, uint16_t dummy, uint16_t hdl, uint16_t max_length)
```

This function is called during a write procedure to get information about a specific attribute handle. The parameter conidx is the connection index, user_lid is the GATT user local identifier, token is the procedure token that must be returned in confirmation function, hdl is attribute handle, offset is the value offset, and max_length is the maximum value length to return.

```
__STATIC void tota_gatt_cb_att_info_get(uint8_t conidx, uint8_t user_lid,
uint16_t token, uint16_t hdl)
```

This function is called during a write procedure to modify the attribute handle. The parameter conidx is the connection index, user_lid is the GATT user local identifier, token is the procedure token that must be returned in confirmation

function, hdl is the attribute handle, offset is the value offset, and p_data is the pointer to buffer that contains data to write starting from offset.

```
__STATIC void tota_gatt_cb_att_set(uint8_t conidx, uint8_t user_lid,
uint16_t token, uint16_t hdl, uint16_t offset, co_buf_t* p_buf)
```

Set of callback functions for communication with GATT as a GATT user server

```
__STATIC const gatt_srv_cb_t tota_gatt_srv_cb = {
    .cb_event_sent      = tota_gatt_cb_event_sent,
    .cb_att_read_get    = tota_gatt_cb_att_read_get,
    .cb_att_event_get   = tota_gatt_cb_att_event_get,
    .cb_att_info_get    = tota_gatt_cb_att_info_get,
    .cb_att_val_set     = tota_gatt_cb_att_set,
};
```

Register as a GATT User Server

```
status = gatt_user_srv_register(PREFERRED_BLE_MTU, 0, &tota_gatt_srv_cb,
&(tota_env->srv_user_lid));
```

5.8.6 DUT Test Mode

5.8.6.3 Signaling Test Mode

5.8.6.3.1 Test Setup

The flow of setup a test is shown in the following figure.

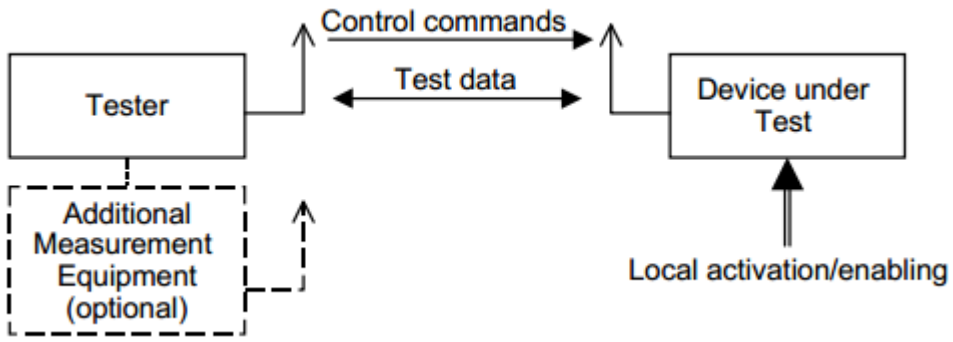


Figure 53 Test Setup

5.8.6.3.2 Software API

Software API for signal tests is as follows:

```
extern osTimerId app_env_checker_timer;
```

Interface to stop the battery measurement timer:

```
int app_battery_stop(void);
```

Interface for signal tests in factory mode:

```
void app_factorymode_bt_signalingtest(APP_KEY_STATUS *status, void *param)
{
    factory_section_data_t *factory_section_data =
factory_section_data_ptr_get();
    APP_FACTORY_TRACE("%s",__func__);
#ifdef __WATCHER_DOG_RESET__
    app_wdt_close();
#endif
    hal_cpu_wake_lock(APP_FACT_CPU_WAKE_LOCK);
    app_stop_10_second_timer(APP_PAIR_TIMER_ID);
    app_stop_10_second_timer(APP_POWEROFF_TIMER_ID);
    app_status_indication_set(APP_STATUS_INDICATION_TESTMODE);
    osTimerStop(app_env_checker_timer);

    app_battery_stop();
    pmu_sleep_en(0);
    BESHCI_Close();
    btdrv_hciopen();
    btdrv_sleep_config(0);
    //btdrv_hcioff();
    osDelay(2000);
    //btdrv_testmode_start();
    //btdrv_sleep_config(0);
    btdrv_ins_patch_test_init();
    btdrv_feature_default();
    //btdrv_test_mode_addr_set();

    if (factory_section_data){
        btdrv_write_localinfo((char *)factory_section_data->device_name,
strlen((char *)(factory_section_data->device_name)) + 1,
factory_section_data->bt_address);
    }

    btdrv_enable_dut();
}
```

5.8.6.4 Non-signaling Test Mode

5.8.6.4.1 Test Setup

The flow of setting up a test is shown in the following figure.

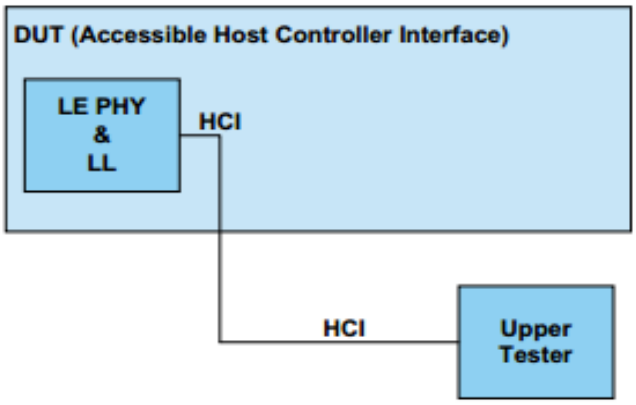


Figure 54 Test Setup

5.8.6.4.2 Software API

Interface for non-signal tests in factory mode:

```
void app_factorymode_bt_nosignalingtest(APP_KEY_STATUS *status, void
*param)
{
    factory_section_data_t *factory_section_data =
factory_section_data_ptr_get();
    APP_FACTORY_TRACE("%s",__func__);
#ifdef __WATCHER_DOG_RESET__
    app_wdt_close();
#endif
    hal_cpu_wake_lock(APP_FACT_CPU_WAKE_LOCK);
    app_stop_10_second_timer(APP_PAIR_TIMER_ID);
    app_stop_10_second_timer(APP_POWEROFF_TIMER_ID);
    app_status_indication_set(APP_STATUS_INDICATION_TESTMODE1);
    osTimerStop(app_env_checker_timer);

    app_battery_stop();
    pmu_sleep_en(0);
    BESHCI_Close();
    btdrv_hciopen();
    btdrv_sleep_config(0);
    //btdrv_hcioff();
```

```
    osDelay(2000);
    //btdrv_testmode_start();
    //btdrv_sleep_config(0);
    btdrv_ins_patch_test_init();

#ifdef CHIP_BEST2300
    *(volatile uint32_t *)0xd03503a0 |= 1;
#endif

    btdrv_feature_default();

    if (factory_section_data){
        btdrv_write_localinfo((char *)factory_section_data->device_name,
strlen((char *)(factory_section_data->device_name)) + 1,
factory_section_data->bt_address);
    }
    //bt_drv_check_calib();
    btdrv_hcioff();
    btdrv_uart_bridge_loop();
}
```

In some version of SDK, the preceding function is replaced by app_factorymode_bt_bridgemode(), but the content should be the same.

5.8.6.5 Test Proceduce

The test procedure is as follows:

1. Code entry

   Method 1: enter test mode in the key process handler (for quick test)

```
void app_bt_key_simulation(APP_KEY_STATUS *status, void *param)
{
    TRACE("Get key event %d", status->event);
    switch(status->event)
    {
…
        case APP_KEY_EVENT_TRIPLECLICK:
            //clear_dump_log();
            app_factorymode_bt_signalingtest(status, param);
            break;
```

```
        case APP_KEY_EVENT_ULTRACLICK:
            app_factorymode_bt_nosignalingtest(status, param);
            break;

        case APP_KEY_EVENT_LONGPRESS:
            app_tws_simulate_pairing();
            break;
        default:
            break;
    }
}
```

Method 2: enter test mode in reboot mode (Recommended because the initial condition is the same every time when entering test mode). Define the macro __ENGINEER_MODE_SUPPORT__ in target.mk

```
int app_init(void)
{
…
        TRACE("!!!!!ENGINEER_MODE!!!!!\n");
        nRet = 0;
        btdrv_start_bt();
        //BesbtInit();

        app_nvrecord_rebuild();
        if (hal_sw_bootmode_get() & HAL_SW_BOOTMODE_TEST_SIGNALINGMODE){
            hal_sw_bootmode_clear(HAL_SW_BOOTMODE_TEST_MASK);
            app_factorymode_bt_signalingtest(NULL, NULL);
        }
        if (hal_sw_bootmode_get() & HAL_SW_BOOTMODE_TEST_BRIDGEMODE){
            hal_sw_bootmode_clear(HAL_SW_BOOTMODE_TEST_MASK);
            app_factorymode_bt_bridgemode(NULL, NULL);
        }
…
}

void app_bt_key_simulation(APP_KEY_STATUS *status, void *param)
{
    TRACE("Get key event %d", status->event);
    switch(status->event)
    {
```

```
…
        case APP_KEY_EVENT_TRIPLECLICK:
            //clear_dump_log();
            //app_factorymode_bt_signalingtest(status, param);
            hal_sw_bootmode_clear(HAL_SW_BOOTMODE_REBOOT);

hal_sw_bootmode_set(HAL_SW_BOOTMODE_TEST_MODE|HAL_SW_BOOTMODE_TEST_SIGN
ALINGMODE);
            hal_cmu_sys_reboot();
            break;
        case APP_KEY_EVENT_ULTRACLICK:
            //app_factorymode_bt_nosignalingtest(status, param);
            hal_sw_bootmode_clear(HAL_SW_BOOTMODE_REBOOT);

hal_sw_bootmode_set(HAL_SW_BOOTMODE_TEST_MODE|HAL_SW_BOOTMODE_TEST_BRID
GEMODE);
            hal_cmu_sys_reboot();
            break;

        case APP_KEY_EVENT_LONGPRESS:
            app_tws_simulate_pairing();
            break;
        default:
            break;
    }
}
```

2. Set up the connection with the tester

For BR/EDR:

- Reset the target board.
- Press the PWR_ON button to start the application.
- Click the PWR button three times quickly to let the application enter DUT test mode.

Then the device can be found by the tester over the air.

For BLE:

- Reset the target board.
- Press the PWR button to start the application.

- Click the PWR button for four times quickly to let the application enter Direct test mode.
- Connect the UART (debug port) to the tester, configure the baud rate to 115200.

Then, the tester can communicate with the target board.

5.8.6.6 Test Sample Data

DTM (standard HCI command)

Reset

<UART>TX:[01:03:0C:00]

<UART>RX:[04:0E:04:05:03:0C:00]

BLE transfer test is as follows.

Table 87 BLE TX Test

Test item	Test condition	Command raw data
Case 1	TX Channel: 0 (2402MHz) Payload Pattern: 4:11111111 Payload Size:37	<UART>TX:[01:1E:20:03:00:25:04] <UART>RX:[04:0E:04:05:1E:20:00]-
Case 2	TX Channel: 1 (2404MHz) Payload Pattern: 2 (10101010) Payload Size:30	<UART>TX:[01:1E:20:03:01:1E:02] <UART>RX:[04:0E:04:05:1E:20:00]

BLE receive test is as follows.

Table 88 BLE RX Test

Test item	Test condition	Command raw data
Case 1	RX channel: 0 (2402MHz)	RX start <UART>TX:[01:1D:20:01:00] <UART>RX:[04:0E:04:05:1D:20:00] Receiving packet from tester… Test end (report received packet number) <UART>TX:[01:1F:20:00] <UART>RX:[04:0E:06:05:1F:20:00:00:00]
Case 2	RX channel: 39 (2480MHz)	RX start <UART>TX:[01:1D:20:01:27] <UART>RX:[04:0E:04:05:1D:20:00]

CONFIDENTIAL

		Receiving packet from tester… Test End (report received packet number) <UART>TX:[01:1F:20:00] <UART>RX:[04:0E:06:05:1F:20:00:00:00]

5.8.7 Throughput Usage Guide

There are two applications for testing throughput, Bestechnic and BES ThroughPut. Before using these two applications, be sure that the phone is connected to the board. These two applications can test the throughput or BLE or SPP, and SPP throughput test is taken as an example in this article.

5.8.7.3 Throughput Command Specification

5.8.7.3.1 BLE Data Path

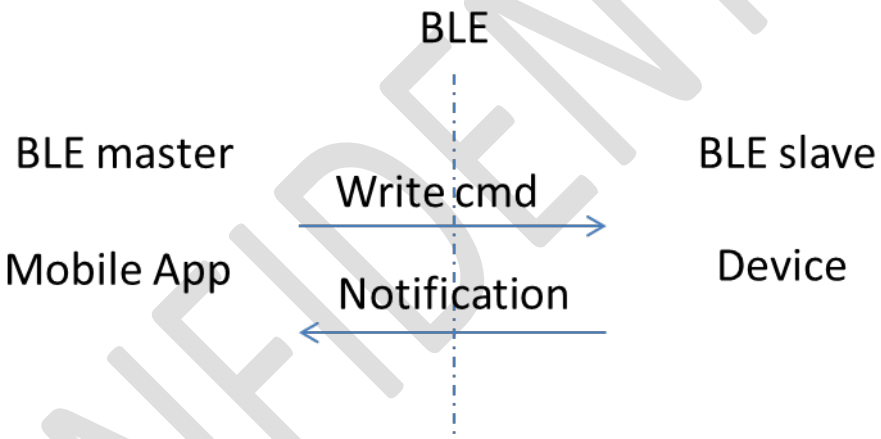


Figure 55 BLE Data Path

For the BLE data path, the smartphone application connects to the audio device via BLE. The smartphone acts as the master while the audio device is the slave. The application makes use of BLE write without response type requests to send the command and audio data. On the contrary direction, the device sends the data and command via BLE notifications.

Table 89 BLE service profile

Item	Description
Service UUID	0x00,0x00,0x82,0x6f,0x63,0x2e,0x74,0x6e,0x69,0x6f,0x70,0x6c,0x65,0x63,0x78,0x65
Cmd TX Value	0x01,0x00,0x82,0x6f,0x63,0x2e,0x74,0x6e,0x69,0x6f,0x70,0x6c,0x65,0x63,0x78,0x65
Cmd CCC	0x02, 0x29
Cmd RX Value	0x02,0x00,0x82,0x6f,0x63,0x2e,0x74,0x6e,0x69,0x6f,0x70,0x6c,0x65,0x63,0x78,0x65
Voice Data TX Value	0x03,0x00,0x82,0x6f,0x63,0x2e,0x74,0x6e,0x69,0x6f,0x70,0x6c,0x65,0x63,0x78,0x65

CONFIDENTIAL

Voice Data CCC	0x02, 0x29
Data RX Value	0x04,0x00,0x82,0x6f,0x63,0x2e,0x74,0x6e,0x69,0x6f,0x70,0x6c,0x65,0x63,0x78,0x65

The command and the data are split into two characteristic paths.

5.8.7.3.2 BT SPP Data Path

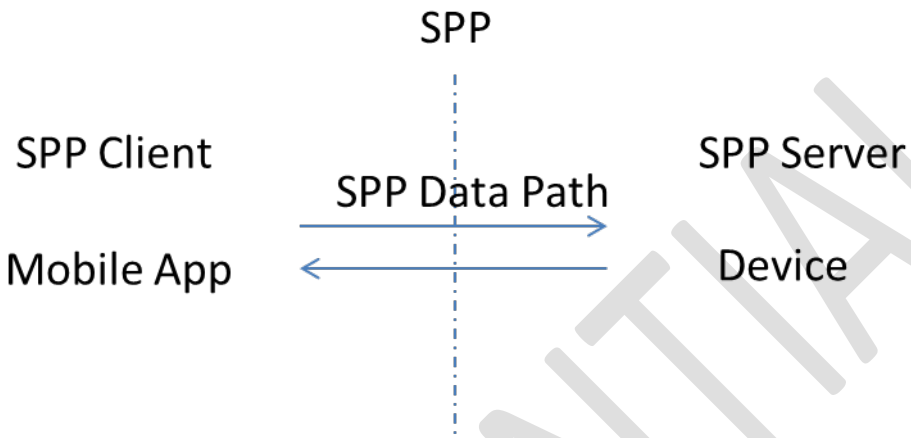
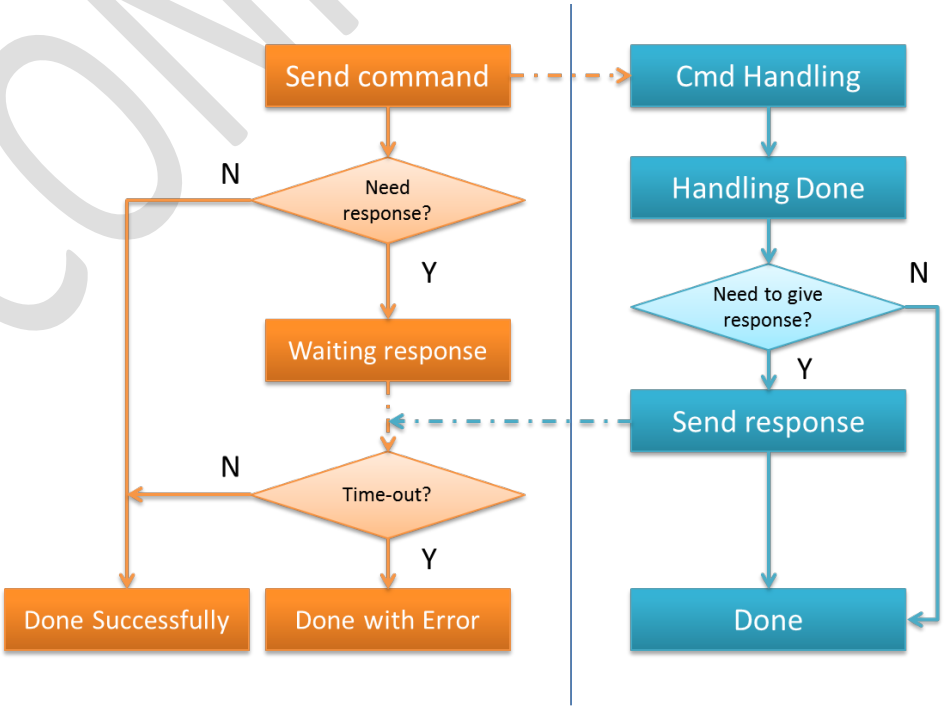


Figure 56 BT Data Path

For BT SPP data path, the mobile application connects to the audio device via classic BT SPP service. The audio device registers itself as the SPP server, waiting for the connection request from the smartphone. Both the command and the data are exchanged via the SPP data path.

5.8.7.3.3 Communication Procedure

The communicated content between the smartphone and the audio device includes commands, and data:



CONFIDENTIAL

Figure 57 Command

The command can be defined as “need to wait for the response” or “no need to wait for the response”, while the waiting time-out period can be customized.

The data format of the command request:

Table 90 Data Format

Offset (bytes)	Length(bytes)	Type	Description
0	2	uint16_t	Command code
2	2	uint16_t	The length of the following parameter
4 to (N-1)	N-4	Array of uint8_t	Parameter, N = 20~508, depending the MTU size

The command code list:

Table 91 Command Code List

Command Code	Value	Description
OP_RESPONSE_TO_CMD	0x8000	Response to cmd (for internal use)
OP_INFORM_THROUGHPUT_TEST_CONFIG	0x8009	Used by the smart-phone to inform the expected throughput test configuration.Used by the audio device to inform the actual negotiated configuration.
OP_THROUGHPUT_TEST_DATA	0x800A	Throughput test data packet
OP_THROUGHPUT_TEST_DATA_ACK	0x800B	Ack of the throughput test data packet
OP_THROUGHPUT_TEST_DONE	0x800C	Signal of the throughput test done

The data format of the command response:

Table 92 Data Format of Command Response

Offset (bytes)	Length(bytes)	Type	Description
0	2	uint16_t	Rsp cmd code, fixed value: 0x8000
2	2	uint16_t	The length of the followingdata
4	2	uint16_t	The responded cmd code
6	2	uint16_t	handling result of the command, from the error code table
8	2	uint16_t	The length of the response parameter
10-N	N-10	Array of uint8_t	Parameter, N = 20~508, depending the MTU size

CONFIDENTIAL

The error code list:

Table 93 Error Code List

Command Code	Value	Description
NO_ERROR	0	No error
INVALID_CMD	1	Invalid command code
INVALID_DATA_PACKET	2	Invalid data packet format
PARAM_LEN_OUT_OF_RANGE	3	Parameter length out of range
PARAM_LEN_TOO_SHORT	4	Parameter length too short
CMD_HANDLING_FAILED	5	Command handling failed
WAITING_RSP_TIMEOUT	6	Waiting response time-out
DATA_XFER_ALREADY_STARTED	7	The data transfer has already been started
DATA_XFER_NOT_STARTED_YET	8	The data transfer hasn't been started
DATA_SEGMENT_CRC_CHECK_FAILED	9	Data segment CRC check failed
WHOLE_DATA_CRC_CHECK_FAILED	10	Whole data CRC check failed
DATA_XFER_LEN_NOT_MATCHED	11	The data transfer length doesn't match the actually received data length
MUSIC_ALREADY_PLAYING	12	The music playing on the cloud side is already on
MUSIC_NOT_PLAYING_YET	13	The music playing on the cloud side is not started yet

CONFIDENTIAL

5.8.7.3.4 Throughput Test Procedure

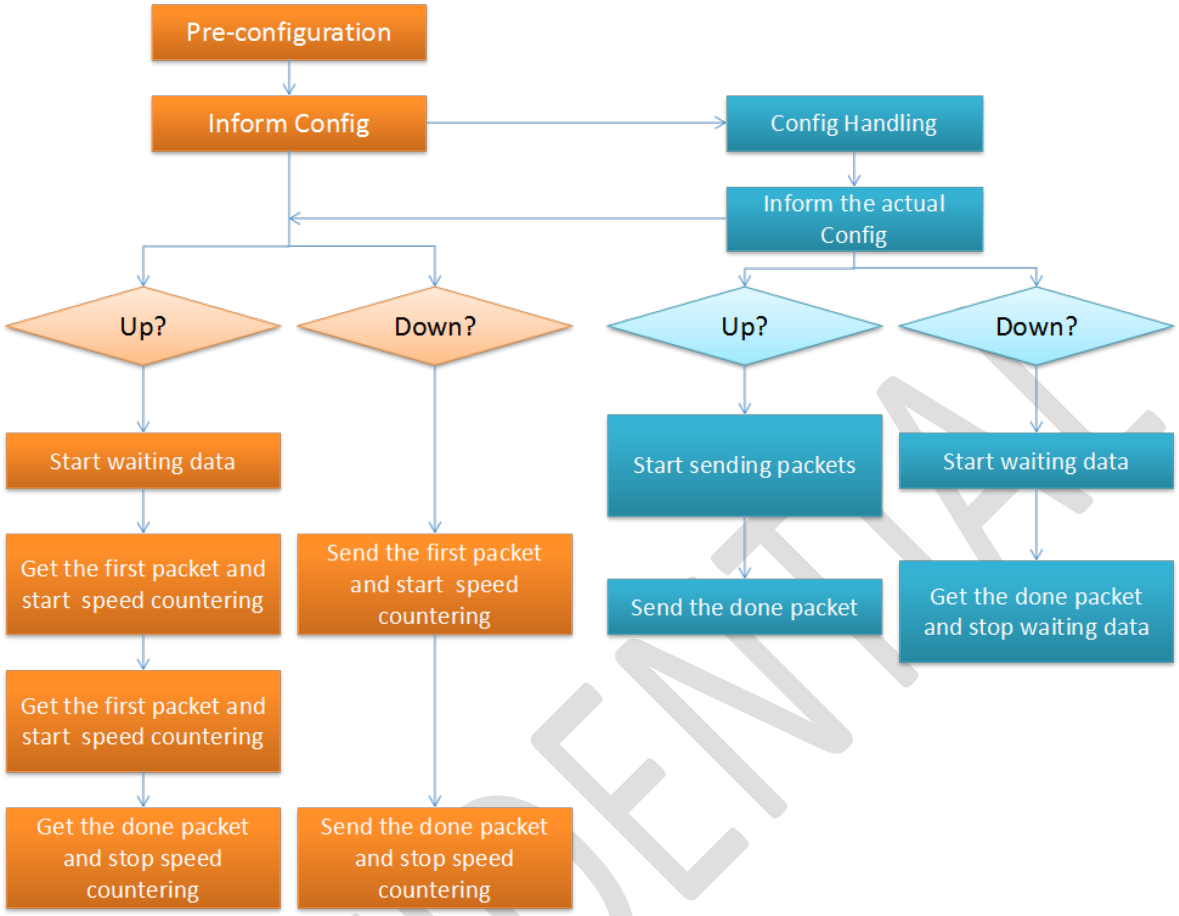


Figure 58 Throughput Test Procedure

The format of the throughput test commands:

The parameters are small-endian. For example, the test lasting time is 10 seconds, so the bytes are 0x0a 0x00.

Table 94 Data Format

Offset (bytes)	Length(bytes)	Type	Description
Inform the configuration of the throughput test	0x8009	16	Byte 0: Test data pattern: 0x00 Random data (default) 0x01 Pattern of alternating bits '11110000' 0x02 Pattern of alternating bits '10101010' 0x03 Pattern of All '1' bits 0x04 Pattern of All '0' bits 0x05 Pattern of alternating bits '00001111' 0x06 Pattern of alternating bits '0101'

			Byte 1-2: Test lasting time in second Default value is 10 seconds Byte 3-4: Size of the data packets. For BLE, the maximum value is (512-3=509).For SPP, the maximum value is 672. Default value is the maximum value. Byte 5: Direction 1 - Down 0 - Up Down means the data is from smart-phone to device. Up means the data is from device to smart-phone. Default value is 0. Bytes 6: With or without response. 1 - with response 0 - without response For BLE, 1 means the up test uses indication and the down stream uses write request. 0 means the up test uses notification. For SPP, 1 means the data packet transmission has to wait until getting the ACK of the last data packet. Default value is 0. Byte 7: Is to use the specific BLE connection parameter. Only valid for BLE. 0 - No 1 - Yes Default value: 1

CONFIDENTIAL

			Byte 8-9: min connection interval in ms. Only valid for BLE. Default value 10 Byte 10-11: max connection interval in ms. Only valid for BLE. Default value 20 Byte 12-15: reserve
Throughput test data packet	0x800A	Data packet size - 4	Depends on the test data pattern of the configuration: 0x00 Random data (default) 0x01 Pattern of alternating bits '11110000' 0x02 Pattern of alternating bits '10101010' 0x03 Pattern of All '1' bits 0x04 Pattern of All '0' bits 0x05 Pattern of alternating bits '00001111' 0x06 Pattern of alternating bits '0101'
The ACK of Throughput test data packet	0x800B	0	Only valid for SPP. If the configuration of the SPP is "with response". The data packet transmission has to wait until getting the ACK of the last data packet.
Throughput test done	0x800C	0	For up test, the device will send this cmd to inform smart-phone that the test lasting time has arrived and the test is done. For down test, the smart-phone will send this cmd to inform device that the test lasting time has arrived and the test is done.

5.8.7.4 Throughput Test with Bestechnic

Click these buttons in sequence after opening the APP, as shown in the following figure.

CONFIDENTIAL

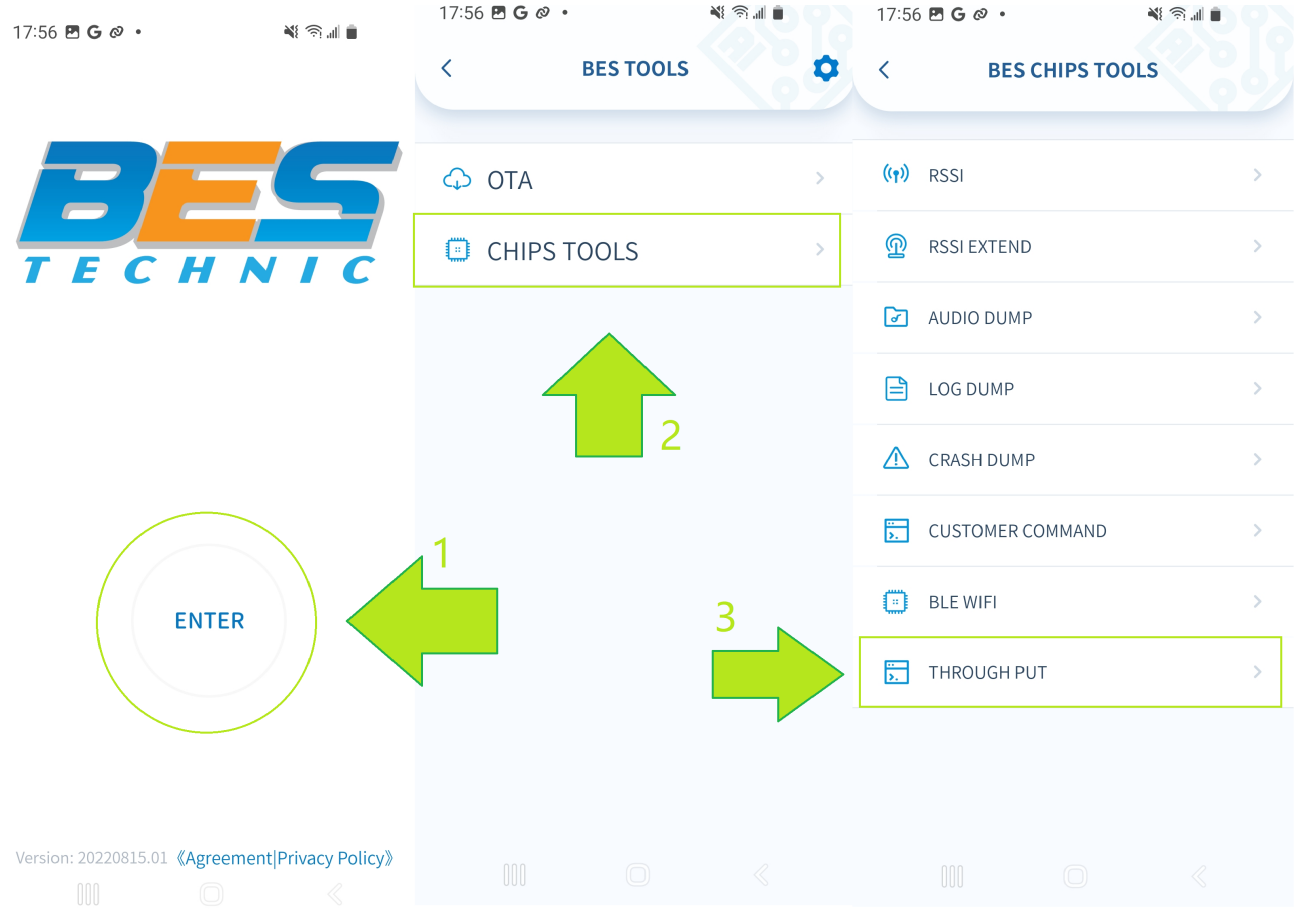


Figure 59 Bestechnic for Testing Throughput

The THROUGH PUT interface is as follows.

Steps to test SPP throughput:

1. Click Choose Spp Device(To test BLE throughput, click Choose Ble Device)
2. Choose the device which to be tested.
3. Click Connect Device

CONFIDE

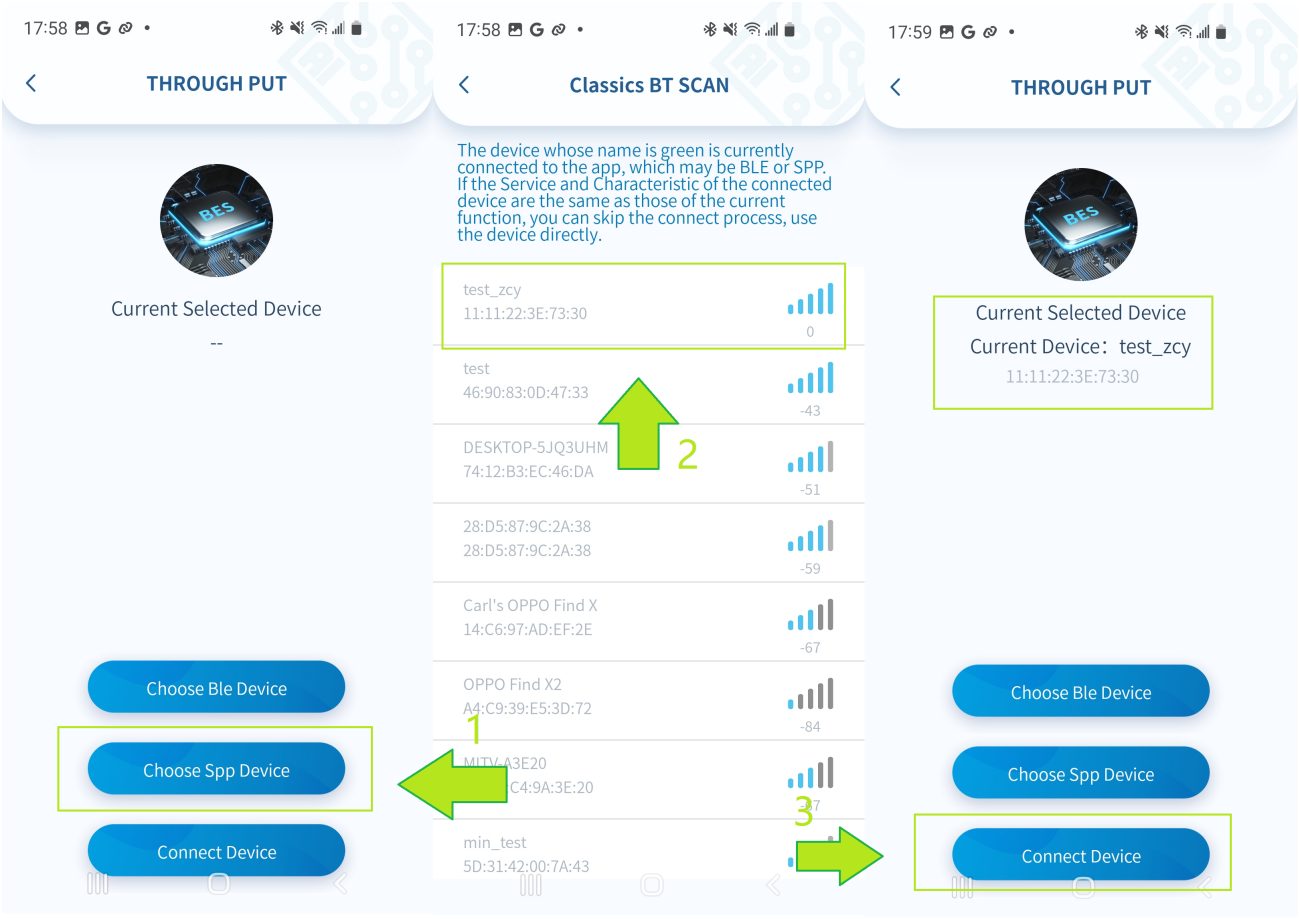


Figure 60 THROUGH PUT Interface

The THROUGH PUT interface is as follows. The left is for upstream tests while the right is for downstream tests.

- Data size
- Response type
- Up-Throughput rate
- Down-Throughput rate
- ThroughPut Data
- Lasting time in second
- Data Size
- Is With response
- Through Up Start
- Through Down Start

CONFIDENTIAL

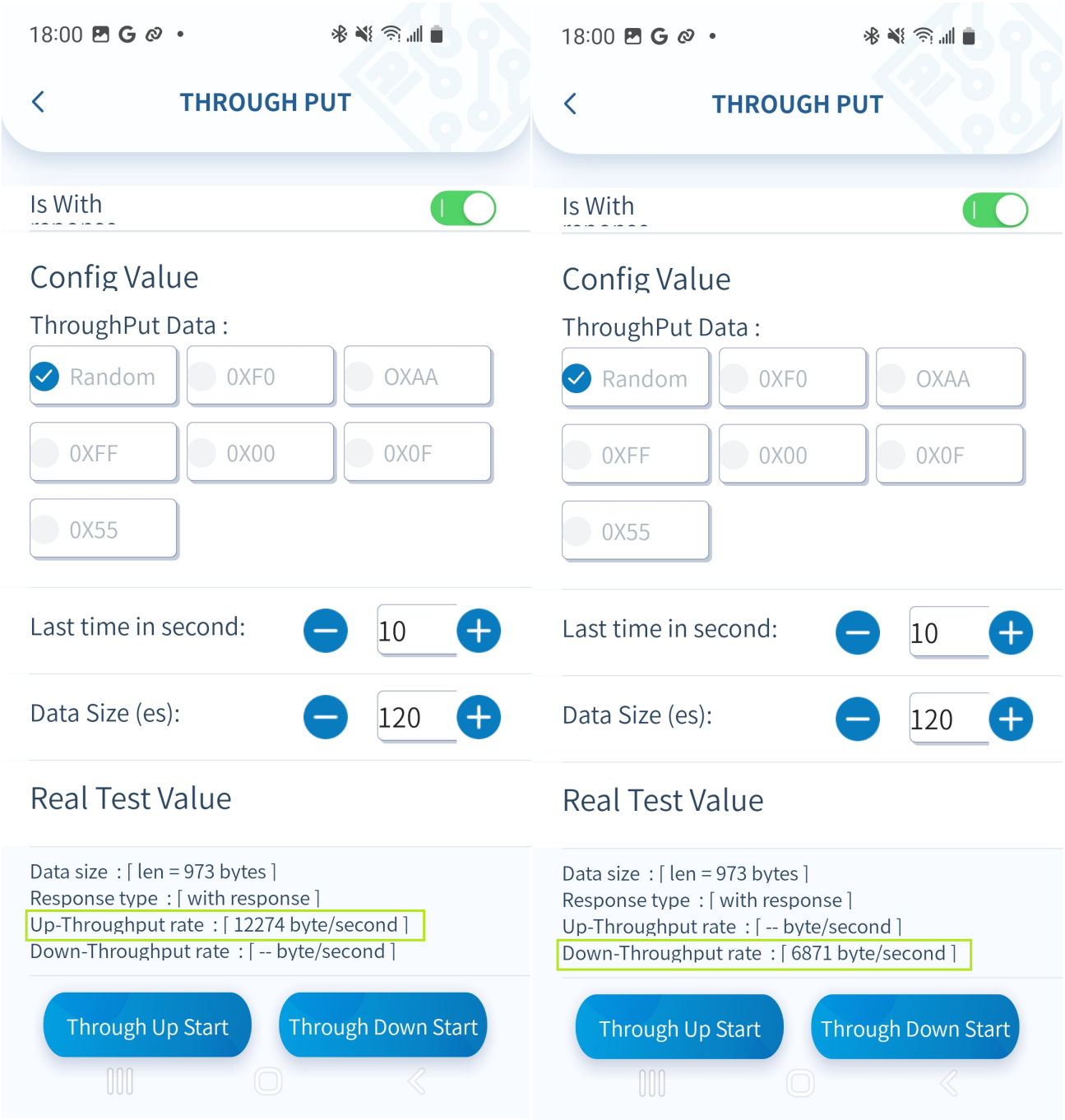


Figure 61 SW Image Power Off Flow

5.8.7.5 Throughput test with BES ThroughtPut

BES ThroughtPut is dedicated to testing throughput.

After the phone is connected to Bluetooth, open the through put APP. Choose BLE or SPP

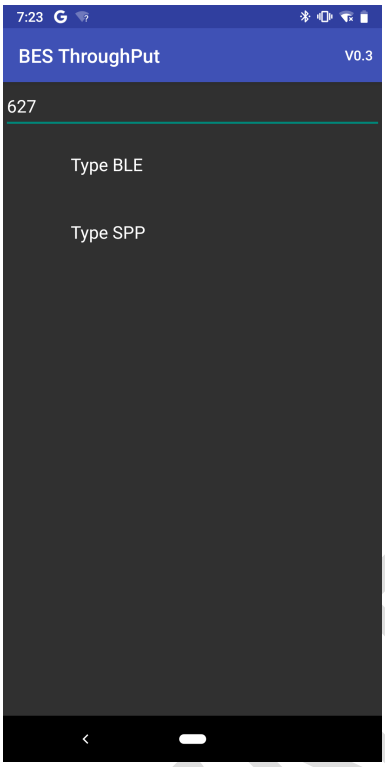


Figure 62 BES ThroughPut Main Interface

Connect the Bluetooth to be tested.

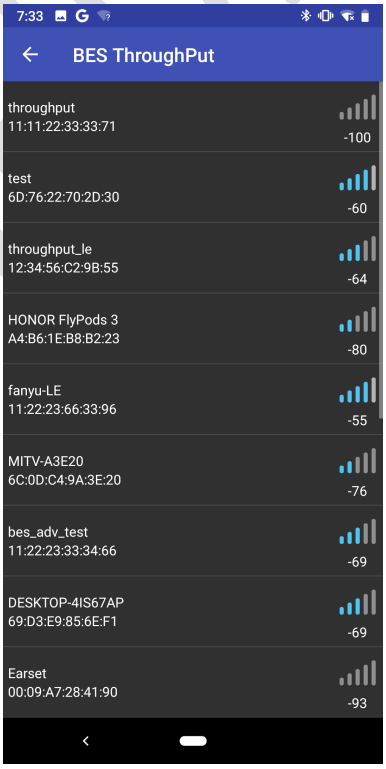


Figure 63 Choose Bluetooth

CONFIDENTIAL

After the connection, the interface is as follows

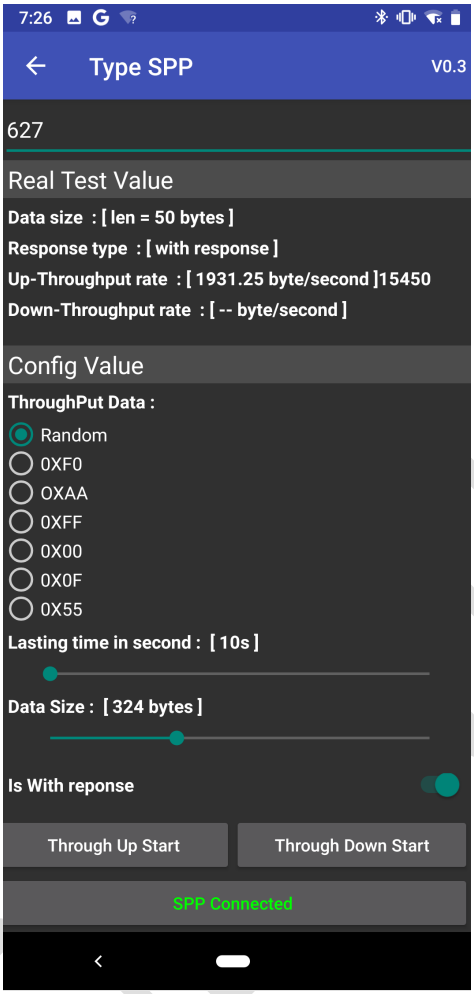


Figure 64 SPP Test

Real Test Value:

- Data size
- Response type
- Up-Throughput rate
- Down-Throughput rate

Config Value:

- ThroughPut Data
- Lasting time in second
- Data Size
- Is With response
- Through Up Start
- Through Down Start

CONFIDENTIAL

5.9 OTA

5.9.1 OTA Flash Memory Map

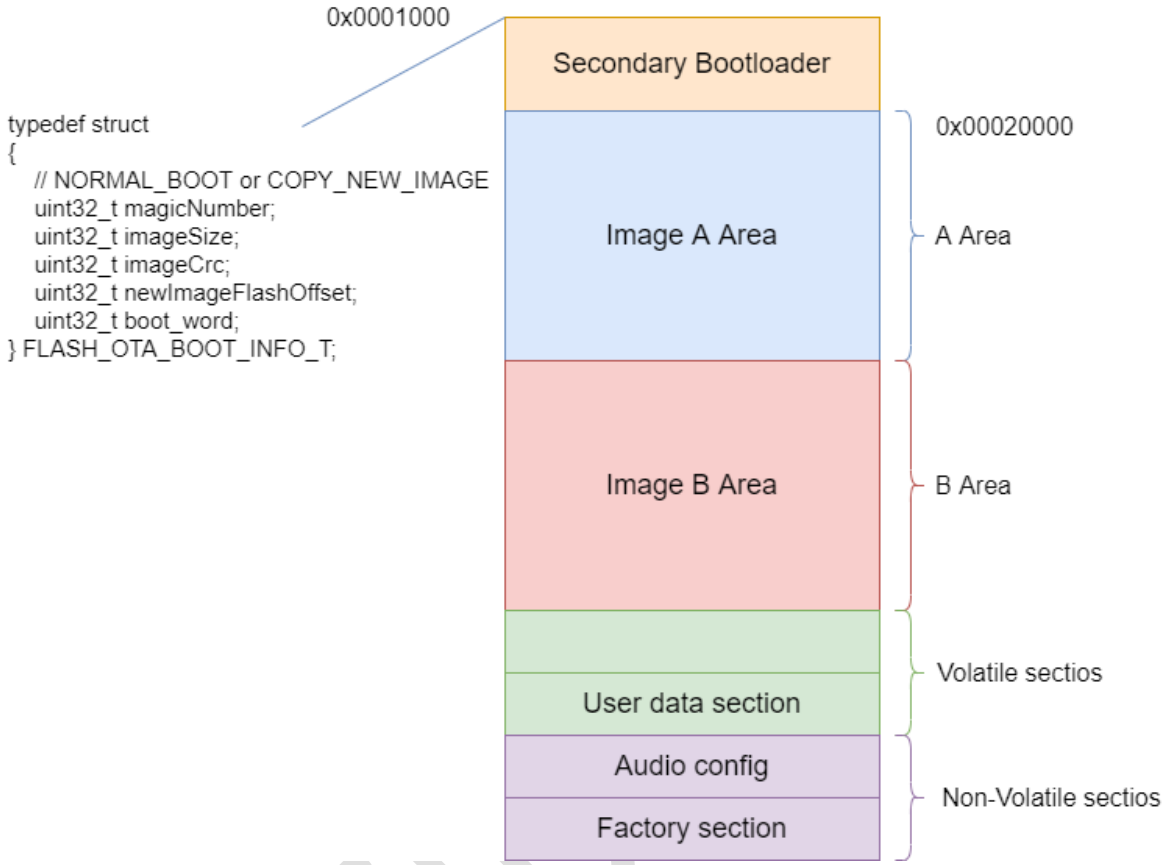


Figure 65 Main Flash Section

There are four main flash sections:

1. Secondary bootloader

   Once programmed, it never changes. It is in charge of doing image mapping and jumping to the dedicated image.

   FLASH_OTA_BOOT_INFO_T is located at 0x1000, to tell the secondary bootloader which image area shall be mapped and jumped to.

2. A area and B area

   Dual copy sections including the application image, voice prompt and other user binaries that needs to be updated all-together over the OTA.

   A and B areas have the same size and same start offset of each sub-sections.

   As the BES QSPI controller support flash physical->logic address mapping, the OTA procedure could program the new image to the area flip-flop according the current applied area.

3. Volatile sections

   To store the informations that are changing all over the lifetime. E.g. TWS & mobile pairing information,

volume, etc.

4. Non-volatile sections

   Never changed once programmed on the production line.

   The factory section is always located at the last 4KB flash page, for the BES download tool to program the non-volatile information: classic BT mac address, BT name. BLE mac address, BLE name. XTAL trim value.

   The ANC audio configuration section is to store the ANC configuration and the ANC related calibration values gathered from the production line.

5.9.2 OTA Flow

The OTA flow chart is as follows:

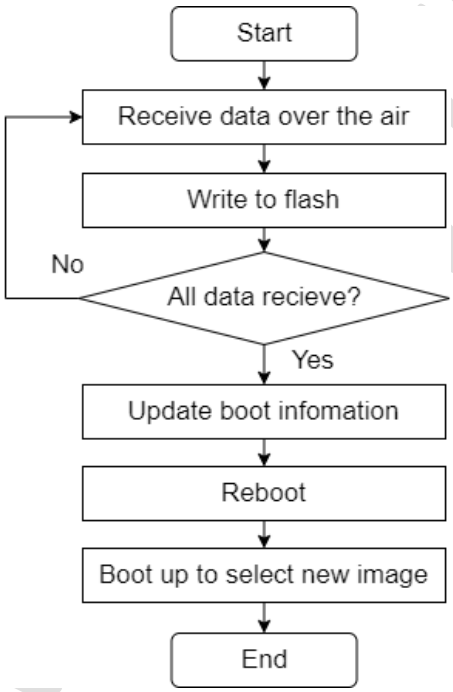


Figure 66 OTA Flow

As the figure shows, the OTA flow is divided into 6 parts:

1. Receive data over the air.

   After the connection is established between the APP and OTA device, the OTA device receives the image sent by APP.

2. Write the data to flash.

   OTA device writes the received data to flash.

3. Receive all data.

   If all data is received, end the transmission and disconnect the device. Otherwise, it continues to receive data.

4. Update boot information.

   BES code will update FLASH_OTA_BOOT_INFO_T as follows when the whole upgrade packet is received, distributed and verified:

   magicNumber: the boot type. This value is changed from NORMAL_BOOT to COPY_NEW_IMAGE after upgrade.

   imageSize: the size of image.

   imageCrc: the CRC of image.

   newImageFlashoffset: the offset where the new image is stored in flash.

   boot_word: Identify A area and B area, indicating the area of new image stored.

5. Reboot.

   Reboot the OTA device to select a new image.

6. Boot up to set the new image.

   No matter whether the image is in the A area or B area, the logical address for booting up is the address of the A area. Therefore, the system checks the Boot word of FLASH_OTA_BOOT_INFO_T when the secondary bootloader boots up. If Boot word is B, the system calls the QSPI controller API to map B physical area to A logic area. Then all the access to the logic address belonging to A will actually access the physical address of B area. If Boot word is A, it directly jumps to the application image via A logic address.

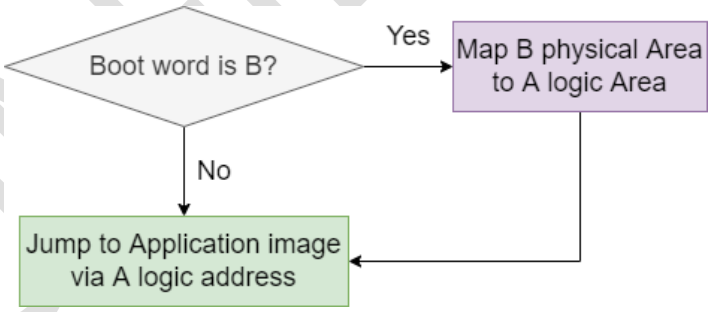


Figure 67 Boot Area Selection

5.9.3 OTA Protocol

5.9.3.1 Framework

5.9.3.1.1 Data Structure Type

Little endian, for example, the image size is 0x1234, and the data order is 0x34 0x12.

CONFIDENTIAL

5.9.3.1.2 Data Transfer Path

- IOS APP

  BLE: Using write requests to send data, and using notificaion to receive the feedback.

  GATT OVER BR/EDR: Using the GATT command to send data and receive the responses

- Android APP

  BLE: Using write requests to send data, and using notificaions to receive the responses.

  SPP: Using the SPP command to send data and receive the responses

5.9.3.1.3 OTA Command Type and Description

Table 95 OTA Command Type

APP command packet		Device response packet	
Transmission resuming at break-point check	0x8C	Transmission resuming at break-point verify response	0x8D
Start OTA	0x80	Start OTA response	0x81
OTA configure	0x86	OTA configure response	0x87
Data packet	0x85		
Segment CRC verify	0x82	Segment CRC verify response	0x83
Whole image CRC verify	0x88	OTA result response	0x84
Read flash content	0x89	Flash content response	0x8A
Data ack for SPP	0x8B		
Transmission side selection	0x90	Side selection response	0x91
Image overwriting confirmation	0x92	Image overwriting response	0x93
Set OTA user	0x97	Set OTA user response	0x98
Get OTA protocol version	0x99	Response OTA protocol version	0x9A

Command details:

Table 96 OTA Command Descriprion

Packet type	Direction	Description
0x8C	APP->device	To check transmission resuming at break-point
0x8D	Device->APP	Transmission resuming at break-point verify response
0x80	APP->device	Fetch OTA
0x81	Device->APP	Inform APP that device is ready for OTA and provide the basic information
0x86	APP->device	Configure the following OTA flow
0x87	Device->APP	OTA configure response
0x85	APP->device	OTA Data packet

CONFIDENTIAL

0x82	Device->APP	Do the segment CRC check
0x83	APP->device	Segment CRC verify response
0x88	Device->APP	Ask the device to do whole image CRC check
0x84	APP->device	OTA result response
0x89	Device->APP	Read flash content
0x8A	APP->device	Flash content response
0x8B	Device->APP	Data ack for using SPP transfer path
0x8E	APP->device	Get current version
0x8F	Device->APP	Read version response
0x90	APP->device	Select side
0x91	Device->APP	Select side response
0x92	APP->device	Confirm boot and overwrite image
0x93	Device->APP	Response to overwrite command
0x97	APP->device	Set OTA user
0x98	Device->APP	Response to set OTA user command
0x99	APP->device	Get OTA protocol version
0x9A	Device->APP	Response the OTA protocol version

5.9.3.2 OTA Command Packet Structure Definition

5.9.3.2.1 Transmission Resuming at Break Point (Check Packet)

Table 97 Transmission Resuming at Break Point (Request Packet)

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x8c
2	10bit	bit	Length of the following data
2	22bit	bit	RFU(Reserved for Future Use), must be all 0 for now
3	4	uint32_t	MagicCode, should be 0x54534542
4	32	uint8_t[32]	Random code
5	4	uint32_t	Segmentsize
6	4	uint32_t	Crc32,crc of randomcode and segment size

5.9.3.2.2 Transmission Resuming at Break Point (Verification Response Packet)

Table 98 Transmission Resuming at Break Point (Verification Response Packet)

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x8d
2	10bit	bit	Length of the following data
2	22bit	bit	RFU, must be all 0 for now

CONFIDENTIAL

3	4	uint32_t	Breakpoint, 0 if NOT a resuming transmission
4	32	uint8_t[32]	New random code if NOT a resuming transmission, otherwise equal to 0
5	4	uint32_t	Crc32, crc of breakpoint and randomcode

5.9.3.2.3 Get a Current Version (Request Packet)

Table 99 Get a Current Version (Request Packet)

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x8e
2	10bit	bit	Length of the following data
2	22bit	bit	RFU(Reserved for Future Use), must be all 0 for now
3	4	uint32_t	MagicCode, should be 0x54534542

5.9.3.2.4 Get a Current Version (Response Packet)

Table 100 Get a Current Version (Response Packet)

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x8e
2	10bit	bit	Length of the following data
2	22bit	bit	RFU(Reserved for Future Use), must be all 0 for now
3	4	uint32_t	MagicCode, should be 0x54534542
4	1	uint8_t	0x00: Stereo device 0x01: TWS device, current connected device is left earbud 0x02: TWS device, current connected device is right earbud
5	4	uint8_t	Version of the stereo device, or the left earbuds for the TWS devices.
6	4	uint8_t	Version of the right earbud for the TWS devices.

5.9.3.2.5 Start an OTA Task (Request Packet)

Table 101 Start an OTA Task (Request Packet)

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x80
2	10bit	bit	Length of the following data
2	22bit	bit	RFU(Reserved for Future Use), must be all 0 for now
3	4	uint32_t	MagicCode, should be 0x54534542
4	4	uint32_t	Imagesize, size of image file - 4

CONFIDENTIAL

5	4	uint32_t	Crc32ofimage, crc32 of the image, the image size is (size of image file – 4)

5.9.3.2.6 Start an OTA Task (Response Packet)

Table 102 Start an OTA Task (Response Packet)

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x81
2	10bit	bit	Length of the following data
2	22bit	bit	RFU(Reserved for Future Use), must be all 0 for now
3	4	uint32_t	MagicCode, should be 0x54534542
4	2	uint16_t	SwVersion, software version
5	2	uint16_t	HwVersion, hardware version
6	2	uint16_t	MTU exchange result, APP will use it as the maximum data size per data packet

5.9.3.2.7 OTA Configuration Packet

Table 103 OTA Configuration Packet

Order	Length	Type	Description
1	1 byte	uint8_t	PacketType, should be 0x81
2	10 bit	bit	Length of the following data
2	22 bit	bit	RFU(Reserved for Future Use), must be all 0 for now
3	4 byte	uint32_t	LengthOfFollowingData, The length of the OTA configuration content, including the last CRC32
4	4 byte	uint32_t	Startlocationtowriteimage, the offset of the flash to start writing the image
5	1 bit	bit	Istoclearuserdata
5	1 bit	bit	Istorenamebt
5	1 bit	bit	Istorenameble
5	1 bit	bit	Istoupdatebtaddr
5	1 bit	bit	Istoupdatebleaddr
5	27 bit	bit	Reserved
6	32 byte	uint8_t	Newbtname
7	32 byte	uint8_t	Newblename
8	6 byte	uint8_t	Newbtaddr
9	6 byte	uint8_t	Newbleaddr
10	4 byte	uint32_t	Crcofconfiguration

CONFIDENTIAL

5.9.3.2.8 OTA Configuration Respone Packet

Table 104 OTA Configuration Respone Packet

Order	Length	Type	Description
1	1	uint8_t	Packettype, should be 0x87
2	10 bit	bit	Length of the following data
2	22 bit	bit	RFU, must be all 0 for now
3	1	uint8_t	Isconfigurationdone, 1 if the configuration has been done successfully, otherwise, 0

5.9.3.2.9 Data Packet

Table 105 Data Packet Description

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x85
2	10 bit	bit	Length of the following data
2	22 bit	bit	RFU, must be all 0 for now
3	n	uint8_t	OTA data

5.9.3.2.10 Segment CRC Verification Packet

Table 106 Segment CRC Verification Packet

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x82
2	10 bit	bit	Length of the following data
2	22 bit	bit	RFU, must be all 0 for now
3	4	uint32_t	MagicCode, should be 0x54534542
4	4	uint32_t	Crc32ofsegment, crc32 of one segment

5.9.3.2.11 Segement CRC Verification Response Packet

Table 107 Segement CRC Verification Response Packet

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x83
2	10 bit	bit	Length of the following data
2	22 bit	bit	RFU, must be all 0 for now
3	1	uint8_t	IsVerificationPassed, 1 is pass, 0 is fail

CONFIDENTIAL

5.9.3.2.12 Whole-image CRC Check Packet

Table 108 Whole-image CRC Check Packet

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x88
2	10bit	bit	Length of the following data, should be 0
2	22bit	bit	RFU, must be all 0 for now

5.9.3.2.13 OTA Result Response Packet

Table 109 OTA Result Response Packet

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x84
2	10 bit	bit	Length of the following data
2	22 bit	bit	RFU, must be all 0 for now
3	1	uint8_t	Bit1 isVerificationPassed, 1 is pass, 0 is fail, Bit2 is received size error, Bit3 is write flash off set error, Bit4 is segment verify error, Bit5 is break point error, Bit6 is image size error

5.9.3.2.14 New-image Side Selection Packet

Table 110 New-image Side Selection Packet

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x90
2	10 bit	bit	Length of the following data
2	22 bit	bit	RFU, must be all 0 for now
3	1	uint8_t	0x00: Apply stereo device 0x01: Apply left earbud only 0x10: Apply right earbud only 0x11: Apply both earbuds

5.9.3.2.15 New-image Side Selection Response Packet

Table 111 New-image Side Selection Response Packet

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x91

CONFIDENTIAL

2	10 bit	bit	Length of the following data
2	22 bit	bit	RFU, must be all 0 for now
3	1	uint8_t	IsAppliedSuccessfully, 1 is pass, 0 is fail

5.9.3.2.16 Image Overwriting Confirmation Packet

Table 112 Image Overwriting Confirmation Packet

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x92
2	10 bit	bit	Length of the following data
2	22 bit	bit	RFU, must be all 0 for now
3	4	uint32_t	MagicCode, should be 0x54534542

5.9.3.2.17 Image Overwriting Confirmation Response Packet

Table 113 Image Overwriting Confirmation Response Packet

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x93
2	10 bit	bit	Length of the following data
2	22 bit	bit	RFU, must be all 0 for now
3	1	uint8_t	IsVerificationPassed, 1 is pass, 0 is fail

5.9.3.2.18 Set an OTA User

Table 114 Set an OTA User

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x97
2	10 bit	bit	Length of the following data
2	22 bit	bit	RFU, must be all 0 for now
3	1	uint8_t	1 for FIRMWARE, 2 for LANGUAGE_PACKAGE

5.9.3.2.19 Set an OTA Version (Response Packet)

Table 115 Set an OTA Version (Response Packet)

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x98
2	10 bit	bit	Length of the following data
2	22 bit	bit	RFU, must be all 0 for now

CONFIDENTIAL

3	1	uint8_t	IsVerificationPassed, 1 is pass, 0 is fail

5.9.3.2.20 Get an OTA Version

Table 116 Get an OTA Version

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x99
2	10 bit	bit	Length of the following data
2	22 bit	bit	RFU, must be all 0 for now
3	4	uint8_t	OTA protocol version on APP

5.9.3.2.21 Get an OTA Version (Response Packet)

Table 117Get an OTA Version (Response Packet)

Order	Length	Type	Description
1	1	uint8_t	PacketType, should be 0x9A
2	10 bit	bit	Length of the following data
2	22 bit	bit	RFU, must be all 0 for now
3	4	uint8_t	OTA protocol version on APP

CONFIDENTIAL

5.9.3.3 Upgrade Process

5.9.3.3.1 OTA Process Overview

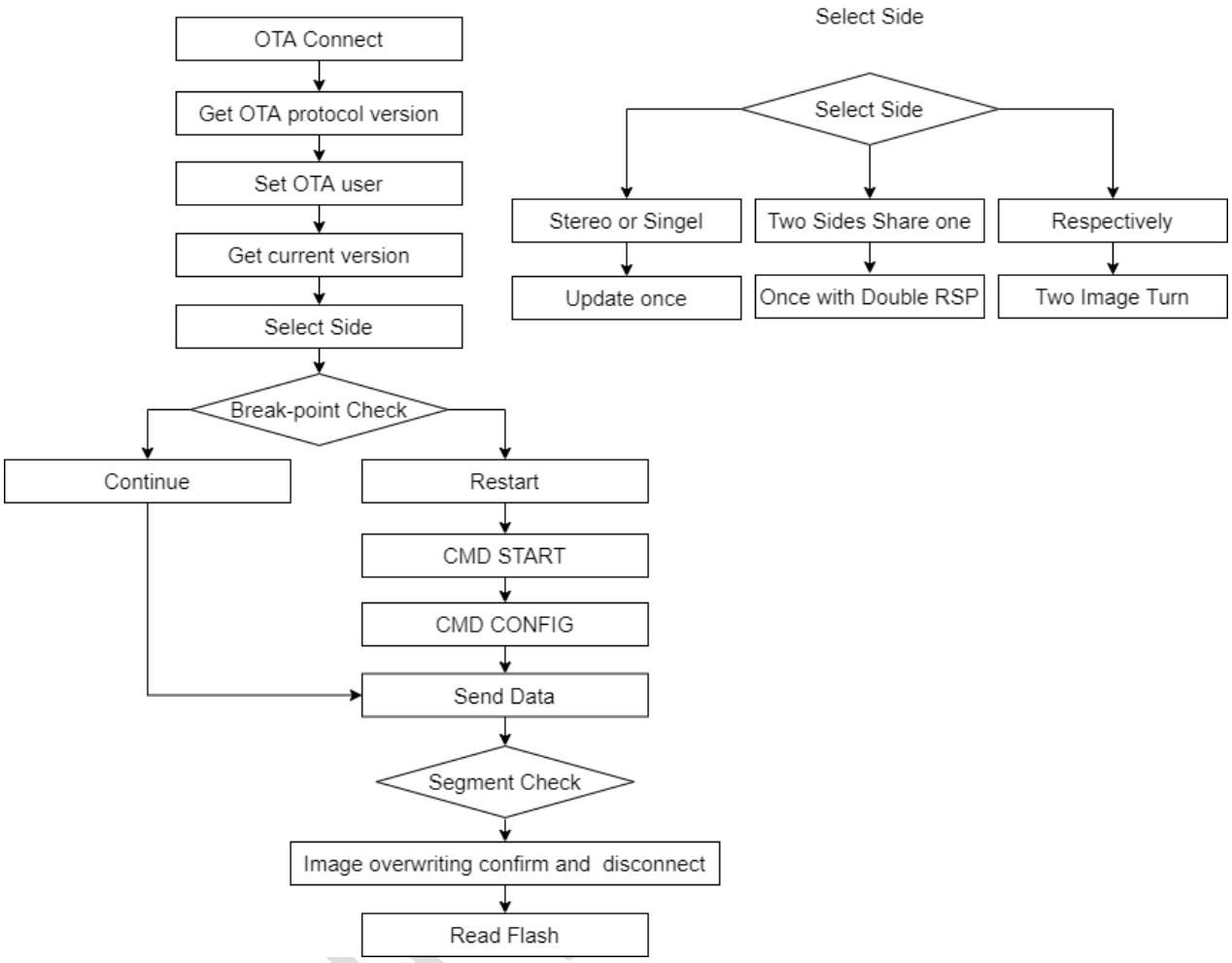


Figure 68 Upgrade Process

As shown in the preceding figure, the upgrade procedure is as follows:

1. OTA Connection: The APP and OTA device establish a connection.
2. Get an OTA protocol version: Get an OTA protocol version. The current latest version is 2.0.
3. Set the OTA user: Select the upgrade section: image, voice prompts or both.
4. Get the current version: Get the OTA device firmware version.
5. Select a side:

   If the OTA device is a stereo earphone or single earphone, select update once.

   If the OTA device is an earphone and two earphones share a flash, select update once but double response.

   If the OTA device is an earphone and two earphones have independent flashes, select Two Image Turn
6. Breakpoint check: Check the breakpoint, choose to send the whole image, or resume transmission from the breakpoint. If transmission resumes, skip step 6 and 7.
7. CMD START: Send the OTA start command.

8. CMD CONFIG: Send the OTA configure command.
9. Send data: The APP send segement data to the OTA device.
10. Segment check: The APP sends the segment CRC check code. If APP receives a reply that the segment data is correct, turn to step 9 to send the next segment data. If APP receives a reply that the segment data is uncorrect, return to step 9 to retransmit the previous segment. When the whole image transmission is finished, turn to step 11.
11. Image overwriting confirmation and disconnection: The APP sends the overwriting confirmation command. If upgrade is successful, the APP recieves a disconnection event.
12. Read the flash: Reboot and read the flash.

During the OTA process, the APP waits for a timeout period of 5 seconds, and if a reply is not received, the upgrade fails.

The APP data flow and Device data flow are as follows:

App data flow ->
Device data flow <<<

5.9.3.3.2 OTA Connection

➔ App establishes a connection with the OTA device.

If the phone supports MTU exchanges, send an exchange MTU command. The MTU value is 512 bytes. If not, go to the next step.

5.9.3.3.3 Get an OTA Protocol Version

➔ Get the OTA protocol version:

99 // packet type
N // length of following data, should be 4 here
xx[4] // should be 00 00 00 01 for current version, 00 00 00 00 for legacy version

<<< Get OTA protocol version response:

9A // packet type
N // length of following data, should be 4 here
xx[4] // should be 00 00 00 01 for current version, 00 00 00 00 for legacy version

5.9.3.3.4 Set an OTA User

➔ Set the OTA user:

97 // packet type
N // length of following data, should be 1 here
USER // 1 for FIRMWARE, 2 for LANGUAGE_PACKAGE

CONFIDENTIAL

<<< Set the OTA user response:

98 // packet type

N // length of following data, should be 1 here

Result // 0 for ok, other for fail

5.9.3.3.5 Get a Current Version

➔ Get the current version:

8E // packet type

N // length of following data, should be 4 here

42 45 53 54 // magic code

<<< Current version response

8F // packet type

N // length of following data, should be 13 here

42 45 53 54 // magic code

0x // device type

xx[4] // stereo of left side version

xx[4] // right side version

5.9.3.3.6 Select a Side

➔ Select a side:

90 // packet type

N // length of following data, should be 1 here

0x // side number

<<< Side selection response

91 // packet type

N // length of following data, should be 1 here

0x // status

5.9.3.3.7 Break-point Check

➔ Check transmission resuming at break-point:

Check packet:

8C // packet type

N // length of following data, should be 36 here

42 45 53 54 // magic code

xx[32] // The random code generated by the device at last upgrade

Waiting for device response

CONFIDENTIAL

<<< Receive transmission-resuming verification response

Received data:

8D // packet type

N // length of following data, should be 36 here

aa bb cc dd // breakpoint , 0 if NOT a resuming transmisson

xx[32] // new checkpoint If breakpoint equal to 0, otherwise equal to 0.

If the breakpoint in response is NOT 0, the checkpoint in response doesn't need to be recorded, then transmit the remaining image from the breakpoint

If the breakpoint in response is 0, the App needs to record the new checkpoint in response firstly,and transmit the whole image

5.9.3.3.8 CMD START

➔ Start the OTA process

Request packet:

80 // packet type

N // length of following data, should be 8 here

42 45 53 54 // magic code

xx yy mm nn // image data size, it's image file size - 4

aa bb cc dd // CRC32 value

<<< Receive the response within 2 seconds.

Received data:

81 // PacketType

N // length of following data, should be 10 here

42 45 53 54 // magic code

aa bb // SwVersion

cc dd // HwVersion

xx yy // MTU, the maximum size per packet during the OTA

Send the OTA process configuration information according to the contents of the configuration UI. The configuration information is as follows:

xx yy mm nn // The length of the following OTA configuration content, including the last CRC32

aa bb cc dd // the offset of the flash to start writing the image,

ee ff gg hh // enable switch, bit is 1 means enable, bit is 0 means disable:

// bit 0-isToClearUserData

// bit 1-isToRenameBT

// bit 2-isToRenameBLE

CONFIDENTIAL

// bit 3-isToUpdateBTAddr

// bit 4-isToUpdateBLEAddr

BT name, size is 32 bytes // If isToRenameBT bit is 0, BT name is zero

BLE name, size is 32 bytes // If isToRenameBLE bit is not selected, BLE name is zero

BT mac addr, size is 6 bytes // If isToUpdateBTAddr bit is not selected, BT mac is zero

BLE mac addr, size is 6 bytes // If isToUpdateBLEAddr bit is not selected, BLE mac is zero

CRC32 // CRC32 values for all of the above configuration contents

5.9.3.3.9 CMD CONFIG

➔ Send the configuration command:

86 // packet type

N // length of following data

Configure content // The number of the packets are decided by the size of the received MTU, until all the configuration content are sent

Waiting for device response

<<< Receive the response from the device

87 // packet type

N // length of following data, should be 1 here

Aa // 1 if the configuration has been done successfully, otherwise, 0

If the response is 1, the APP starts sending OTA data.

If the response is 0, the OTA process fails.

5.9.3.3.10 Send Data

➔ Send OTA data

85 // packet type

N // length of following data

data // maximum value is MTU-3, the actual value depends on the segment size.

One segment is 1% of the image size, used as the unit of the segment CRC check.

In order to be compatible with the property of the flash programming that data needs to be programmed in the unit of 256 bytes page, the segment size needs to be coordinated to be 256 bytes aligned.

So there are (Segment Size + (MTU – 1) – 1) /(MTU – 1) data packets per segment, and the size of the last data packet in the segment is Segment Size – (n-1)*(MTU-1).

5.9.3.3.11 Segment Check

➔ Send the segment CRC check command

CRC check packet:

CONFIDENTIAL

82 // packet type

N // length of following data, should be 8 here

42 45 53 54 // magic code

aa bb cc dd // The CRC32 value of the segment data sent before

Waiting for device response

<<< Receive the CRC check response

Received data:

83 // packet type

N // length of following data, should be 1 here

xx // CRC result: 01 is successful, 00 is failed

If the response is 01, continue with the next segment of the data packets.

If the response is 00, retransmit the current segment.

5.9.3.3.12 Image Overwriting Confirmation and Disconnection

➔ Image overwriting confirmation packet:

92 //packet type

N // length of following data, should be 4 here

42 45 53 54 // magic code

Waiting for a reply.

<<< Image overwriting confirmation response:

Received data:

93 // packet type

N // length of following data, should be 1 here

Xx // isVerificationPassed, 1 is pass, 0 is fail

If the upgrade is successful, the APP display is successful and initiates the disconnection.

<<< The device receives a disconnected event and reboots to use the new image.

5.9.3.3.13 Read Flash Data

➔ Read flash content:

89 // packet type

N // length of following data, should be 9 here

Xx // 1 for start reading, 0 for stop reading

aa bb cc dd // small endian, start offset of the flash to read

ee ff gg hh // small endian, size of flash content to read

CONFIDENTIAL

Waiting for a reply.

<<< response:

8A // packet type

N // length of following data, max value is 128

uint8_t[N] // flash content

8A // packet type

N // length of following data, max value is 128

uint8_t[N] // flash content

…

Repeat response to APP until required data length reaches

5.9.4 OTA User Guide

5.9.4.1 Image Config and Burning Option

There are two kinds of OTAs:

OTA and TOTA OTA

Different from OTA, TOTA OTA is based on TOTA. The following section describes the usage of these two OTAs respectively.

1. OTA

   Before the executable image file is generated, some basic configuration of target.mk is required to support OTA upgrade.

   BES_OTA ?= 1

   If it is an iphone, and the OTA device is upgraded by GATT OVER BR/EDR, and two macro switches GATT_OVER_BR_ERD and IS_BLE_TOTA_ENABLED also need to be configured in target.mk.

   GATT_OVER_BR_EDR ?= 1

   TOTA ?= 0

   TOTA_v2 ?= 0

2. TOTA OTA

   Before the executable image file is generated, some basic configuration of target.mk is required to support TOTA OTA upgrade.

   BES_OTA=1 GATT_OVER_BR_EDR ?= 1

   TOTA_v2 ?= 1

   OTA_OVER_TOTA_ENABLED ?= 1

CONFIDENTIAL

If it is an iphone, and the OTA device is upgraded by GATT OVER BR/EDR, and the macro switch GATT_OVER_BR_ERD also need to be enabled in target.mk.

```
GATT_OVER_BR_EDR ?= 1
```

After generating the executable image file, the ota_boot.bin needs to be generated by compiling the command “make T=ota_boot DEBUG=0”. DEBUG must be set 0 to make the generated file less than 96KB, depending on its storage space.

After generating the image file and ota_boot.bin, these two files need to be burned to the flash together.

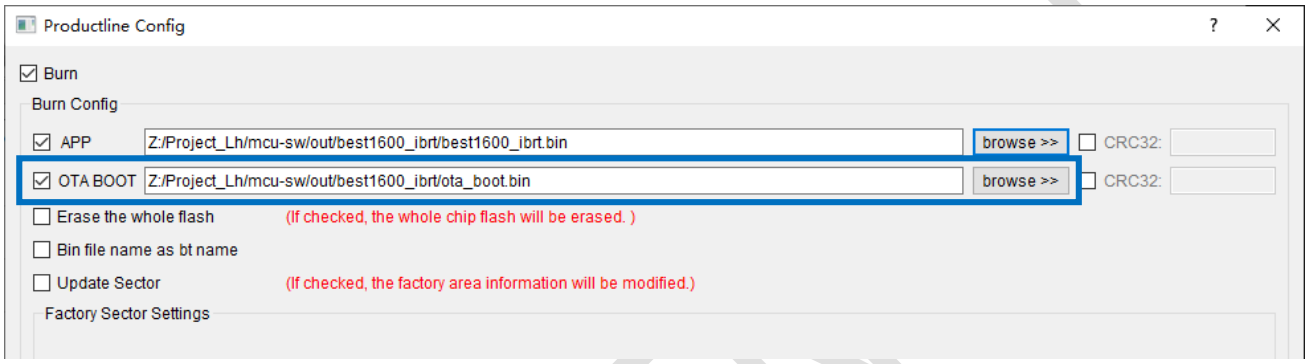


Figure 69 Burn with OTA BOOT

5.9.4.2 Operating Instructions

5.9.4.2.1 OTA Base on Android

5.9.4.2.1.1 Install and Open the APP

Install the APP named Bestechnic. Open it, enter the main view and click “**OTA**”.

Choose version2.0 because BES devices no longer support Version1.0.

The OTA device can be upgraded in two ways: SPP OTA and BLE OTA. Most of the steps are the same. Some differences are explained in the following section.

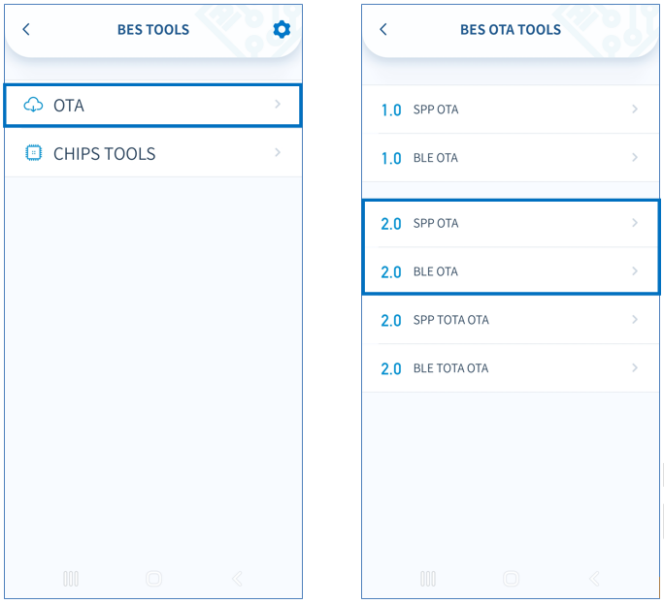


Figure 70 Main View

5.9.4.2.1.2 Bluetooth Pairing

If you choose BLE OTA, skip this step.

Press and hold the "**PWR_ON**" button. When hearing the "**pairing**" tone after the "**power on**" tone, pair the OTA device with phone.

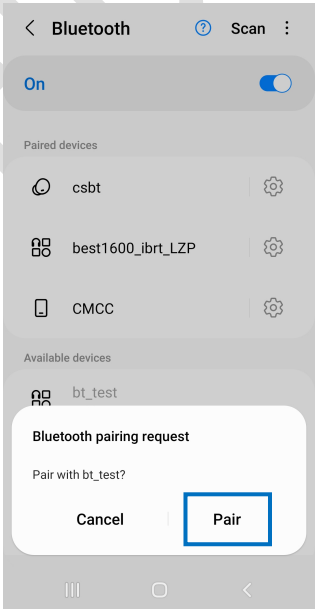


Figure 71 Pairing

CONFIDENTIAL

5.9.4.2.1.3 Select a Device

Click **“Please Select Device”** and choose OTA device, then the selected device’s name and address are viewed as follows:

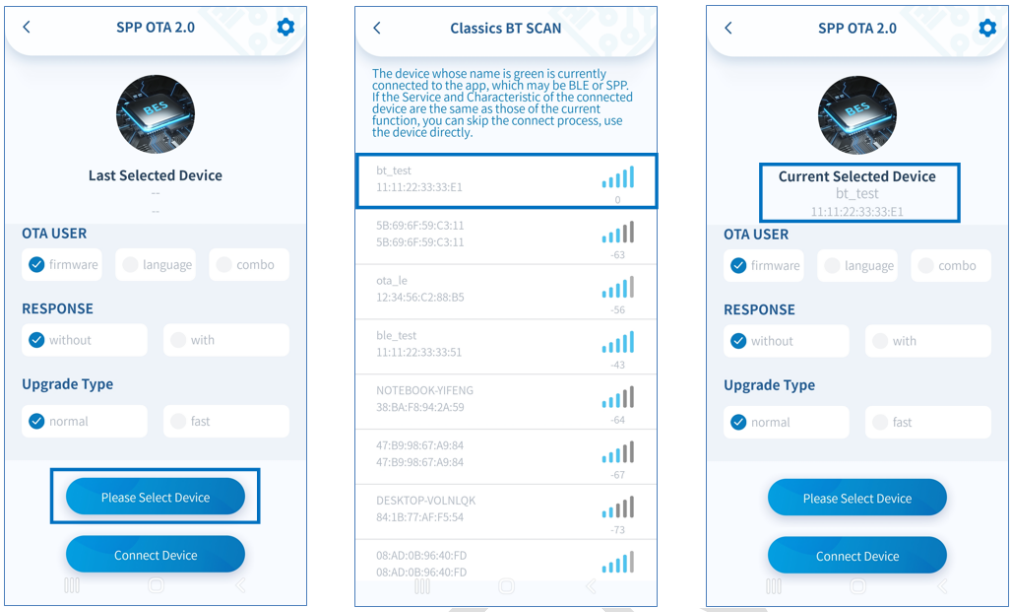


Figure 72 Device Selection

5.9.4.2.1.4 Configure Upgrade

- Select OTA USER:
  firmware: Only image.
  language: Only voice prompt.
  combo: Combo image and voice prompt.
- Select RESPONSE:
  without: No response.
  with: Have response.
- Select Upgrade Type:
  normal: Normal upgrading rate, user can use OTA device normally in this mode.
  fast: Faster upgrading rate but disable some function, such as voice calls.

CONFIDENTIAL

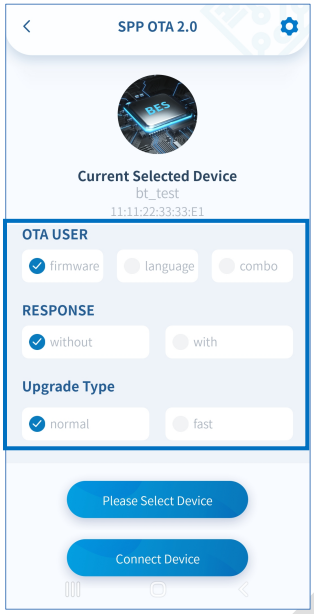


Figure 73 Configure Upgrade

5.9.4.2.1.5 Connect a Device

Click “**Connect Device**”.

After a few seconds, the firmware version of OTA device are viewed as follows if the connection is successful.

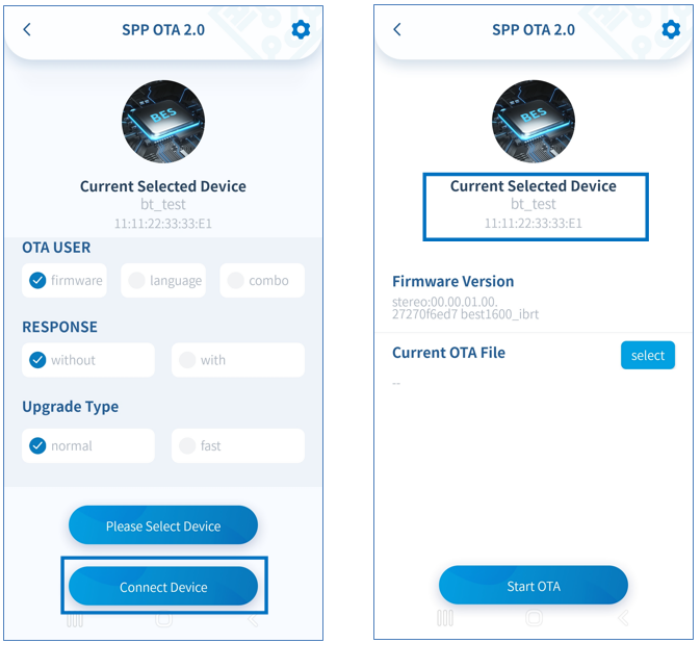


Figure 74 Device Connection

5.9.4.2.1.6 Select an Image

Click “**select**” to select the new image.

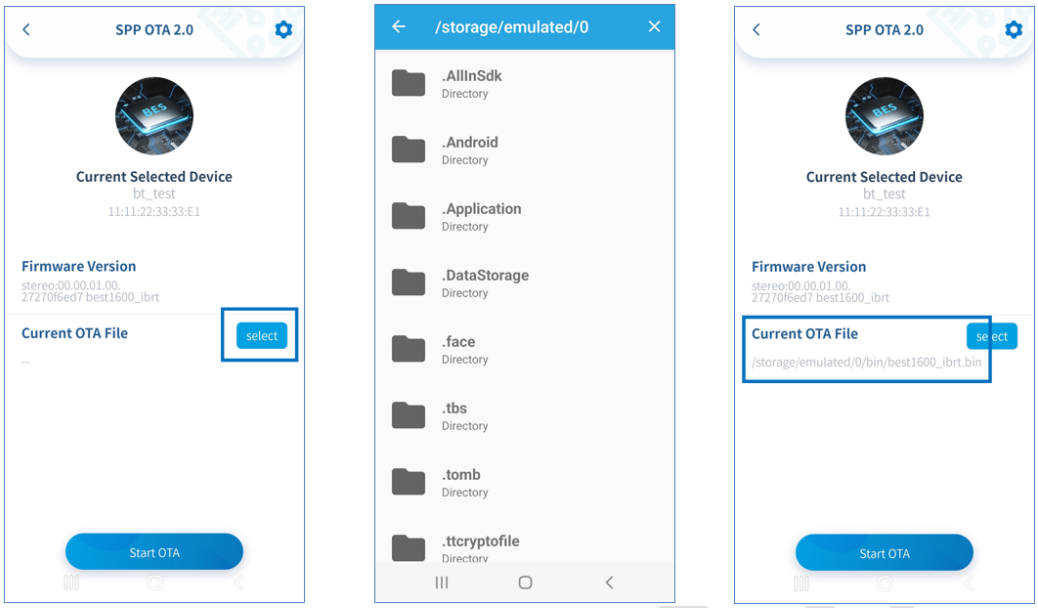


Figure 75 Image Selection

5.9.4.2.1.7 Start the OTA Process

Click "**Start OTA"**, and an option box appears. The content of the option box can edited only when the corresponding checkbox is selected with "**Y**". Click "**OK**" to start upgrading.

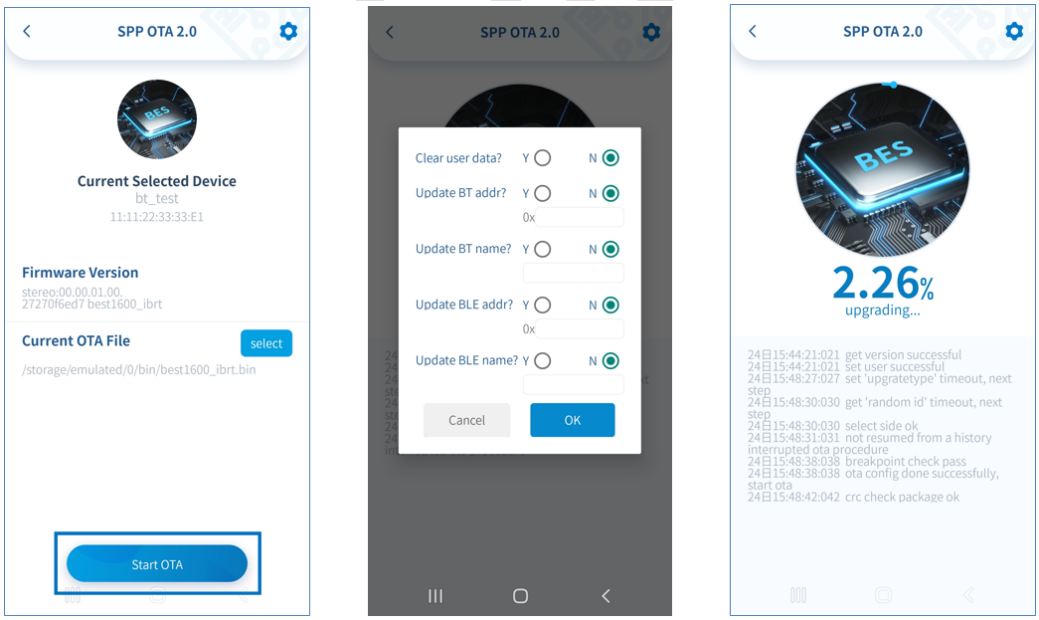


Figure 76 Start OTA Process

5.9.4.2.1.8 Finish the OTA Process

When the upgrade is completed, the new application image is applied and the rebooting to execute the new image is automatically done.

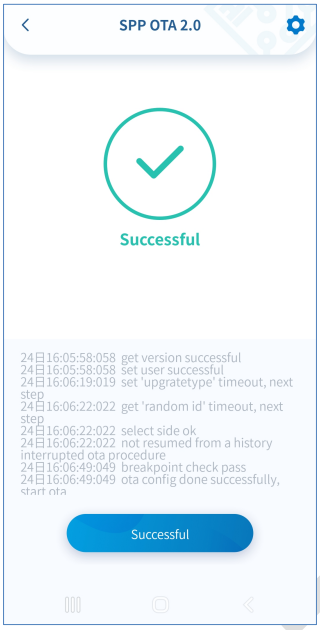


Figure 77 OTA Process Completed

5.9.4.2.2 OTA Base on iOS

5.9.4.2.2.1 Install and open the APP

Install the APP named Bestechnic. Open it, enter the main view and click "**OTA**".

Choose version2.0 because BES devices no longer support Version1.0.

The OTA device can be upgraded in two ways: GATT OVER BR/EDR OTA and BLE OTA. Most of the steps are the same, and some differences are explained in the following section.

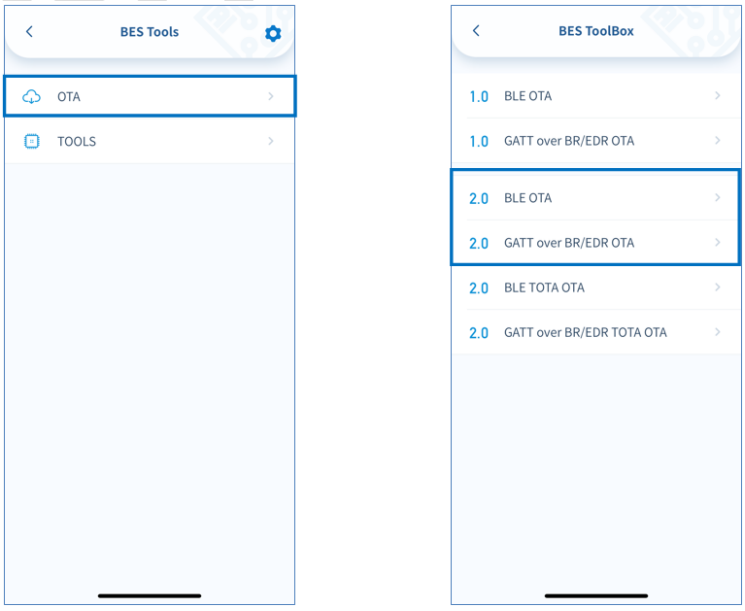


Figure 78 Main View

CONFIDENTIAL

5.9.4.2.2.2 Bluetooth Pairing and Connection

If you choose BLE OTA, skip this step.

Press and hold the "**PWR_ON**" button. When hearing the "**pairing**" tone after the "**power on**" tone, pair the OTA device with iphone.

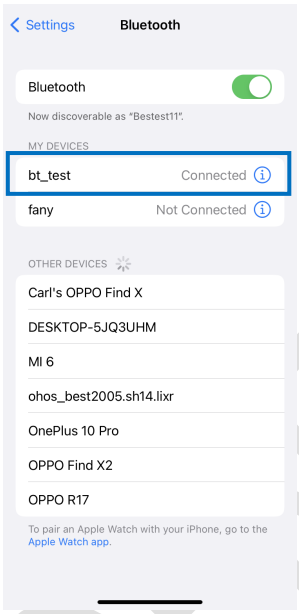


Figure 79 Pairing and Connection

5.9.4.2.2.3 Select a Device

Click **"Choose Device"** and select the OTA device, then the selected device's name can be viewed as follows:

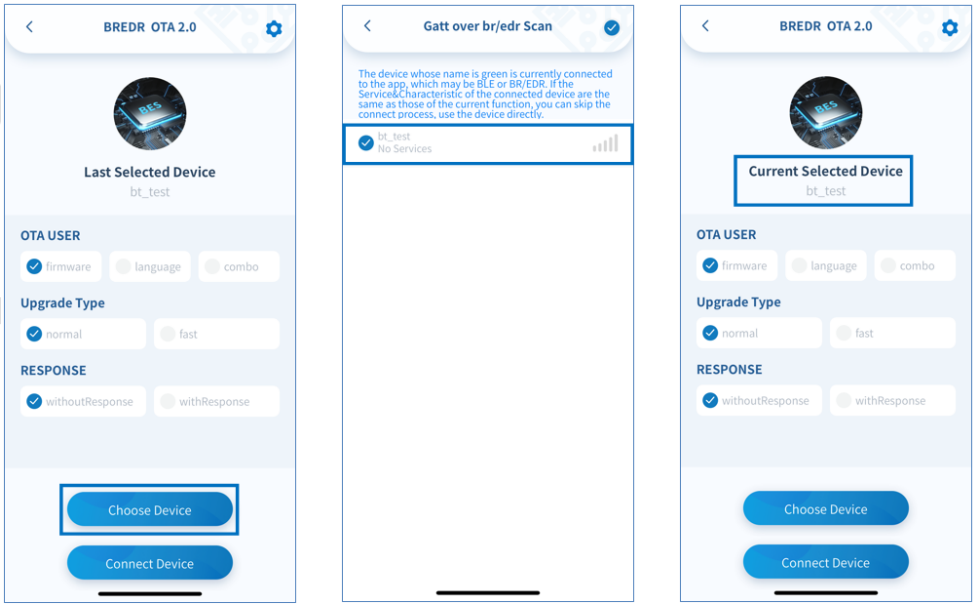


Figure 80 Device Selection

5.9.4.2.2.4 Configure Upgrade

- Select OTA USER:

  firmware: Only image.

  language: Only voice prompt.

  combo: Combo image and voice prompt.

- Select RESPONSE:

  without: No response.

  with: Have response.

- Select Upgrade type:

  normal: Normal upgrade rate. Users can use OTA device normally in this mode.

  fast: Faster upgrade rate but disable some function, such as voice calls.

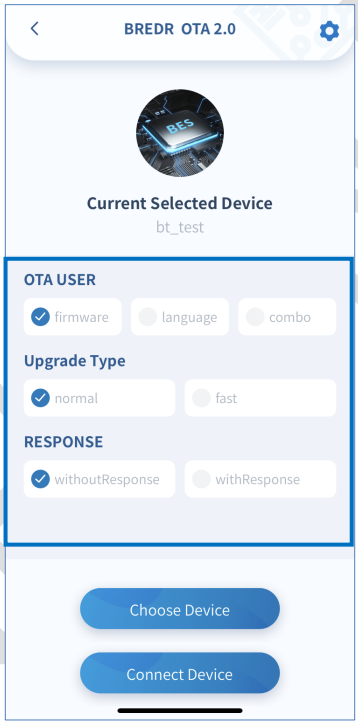


Figure 81 Configure Upgrade

5.9.4.2.2.5 Connect a Device

Click “**Connect Device”.**

After a few seconds, the firmware version of OTA device can be viewed as follows if the connection is successful.

CONFIDENTIAL

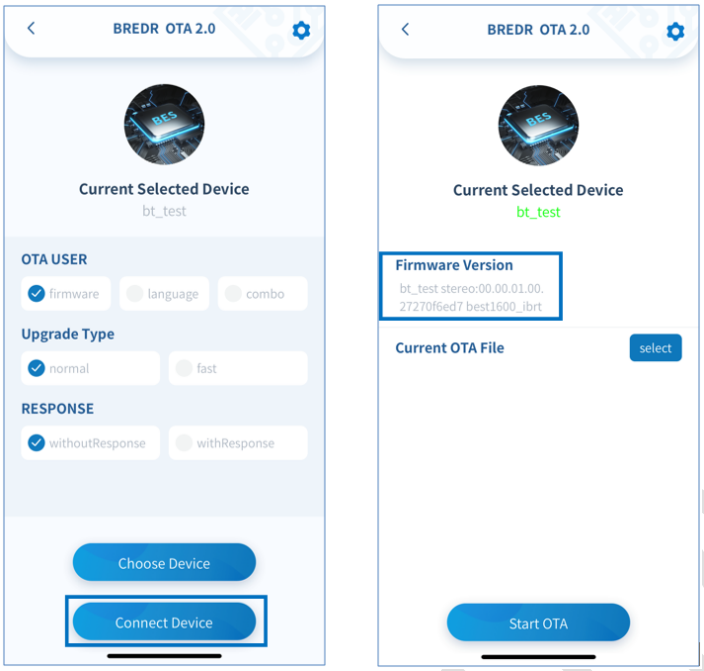


Figure 82 Device Connection

5.9.4.2.2.6 Select an Image

Click "**select**" to select the new image.

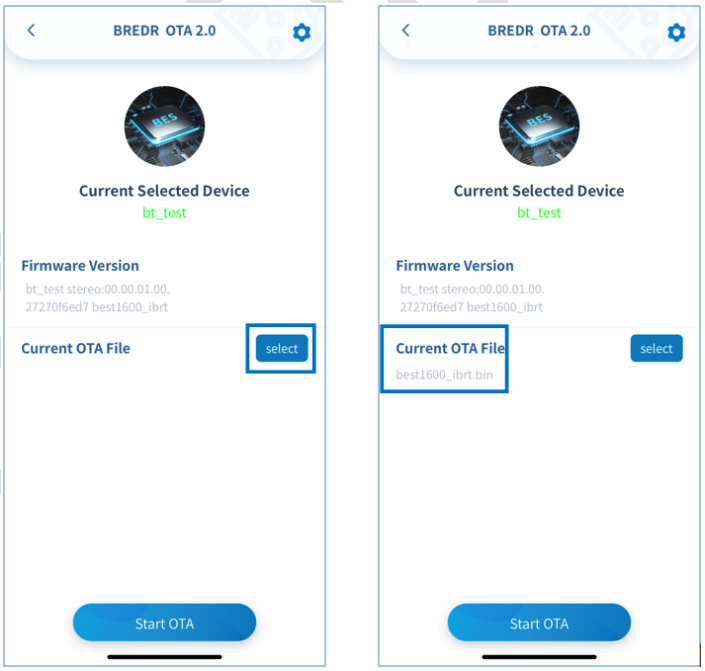


Figure 83 Image Selection

5.9.4.2.2.7 Start the OTA Process

Click "**Start OTA"**, and an option box appears. The content of the option box can edited only when the corresponding checkbox is selected with "**Y**". Click "**OK**" to start upgrading.

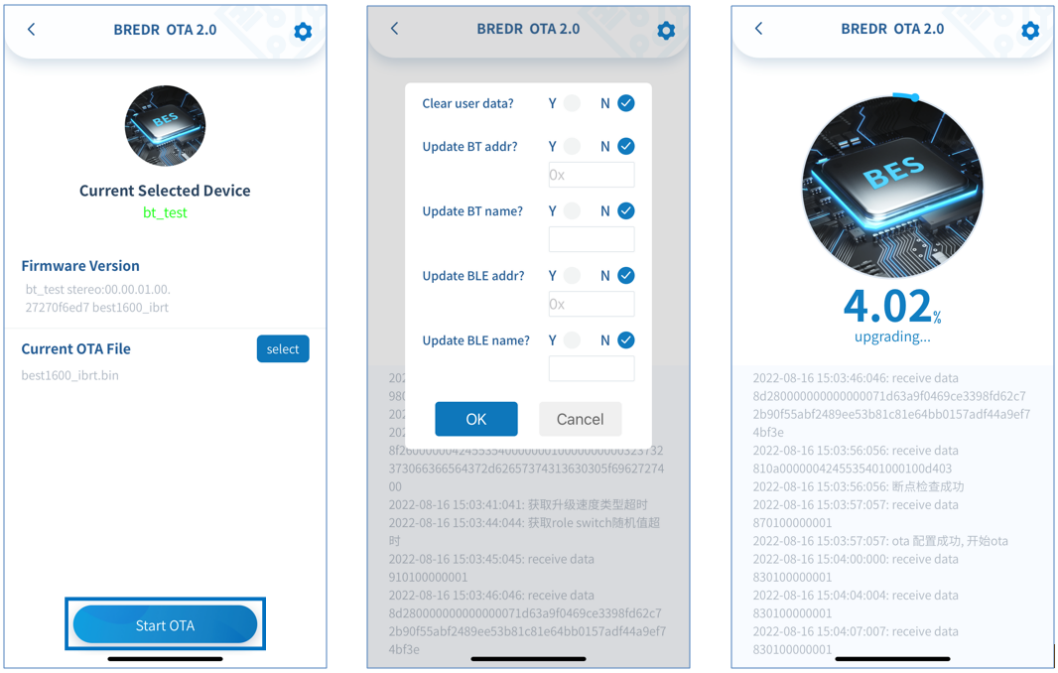


Figure 84 Start OTA Process

5.9.4.2.2.8 Finish the OTA Process

When the upgrade is completed, the new application image is applied and the rebooting to execute the new image is automatically done.

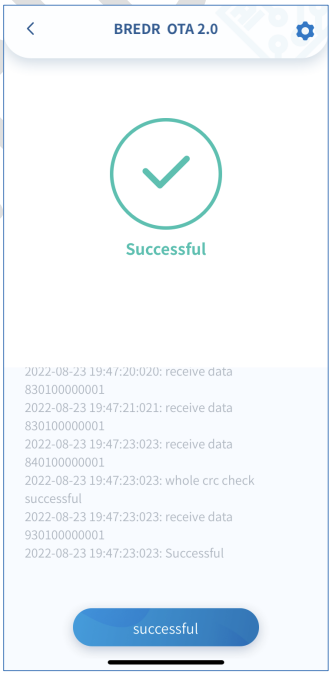


Figure 85 Upgrade Completed

5.9.4.2.3 TOTA OTA Base on Android

5.9.4.2.3.1 Install and open the APP

Install the APP named Bestechnic. Open it, enter the main view and click "**OTA**".

Choose version2.0 because BES devices no longer support Version1.0.

There are two kinds of TOTA OTA: SPP OTA and BLE TOTA OTA. Most of the steps are the same, and some differences are explained in the following section.

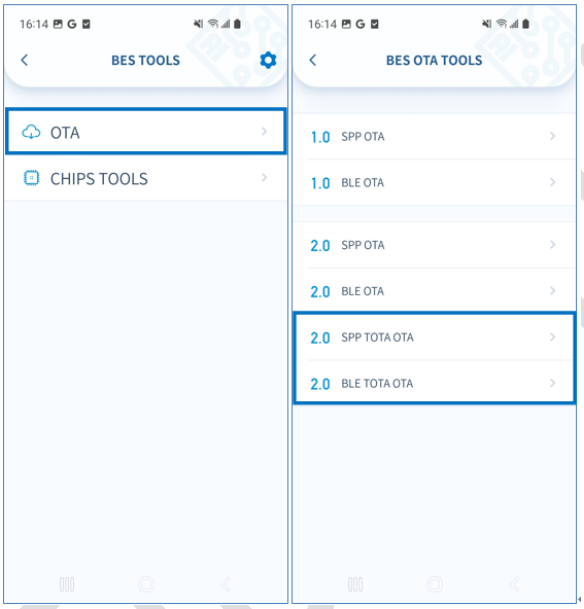


Figure 86 Main View

5.9.4.2.3.2 Bluetooth Pairing and Connection

If you choose BLE TOTA OTA, skip this step.

Press and hold the "**PWR_ON**" button. When hearing the "**pairing**" tone after the "**power on**" tone, pair the OTA device with phone.

CONFIDENTIAL

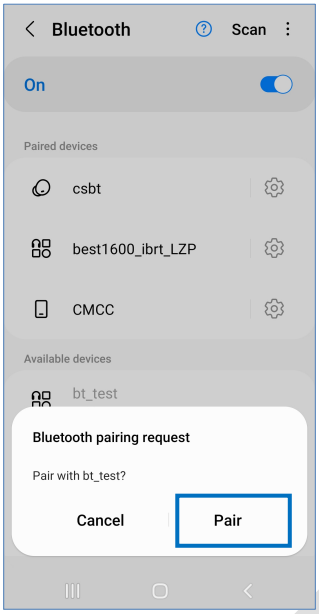


Figure 87 Pairing

5.9.4.2.3.3 Select a Device

Click **"Please Select Device"** and choose OTA device, then the selected device's name and address can be viewed as follows:

Figure 88 Device Selection

5.9.4.2.3.4 Configure Upgrade

- Select OTA USER:

  firmware: Only image.

  language: Only voice prompt.

combo: Combo image and voice prompt.

- Select RESPONSE:

  without: No response.

  with: Have response.

- Select Upgrade type:

  normal: Normal upgrading rate. Users can use OTA device normally in this mode.

  fast: Faster upgrading rate but disable some function, such as voice calls.

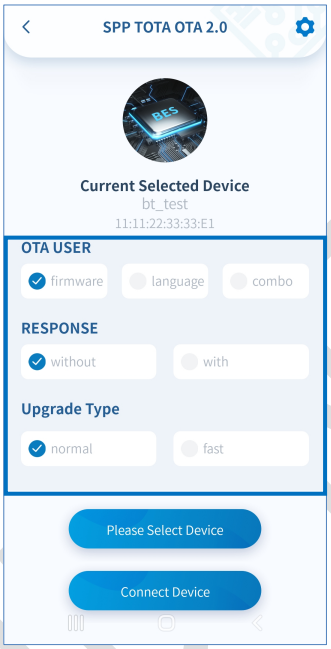


Figure 89 Configure Upgrade

5.9.4.2.3.5 Connect a Device

Click “**Connect Device**”.

After a few seconds, the firmware version of OTA device can be viewed as follows if the connection is successful.

CONFIDENTIAL

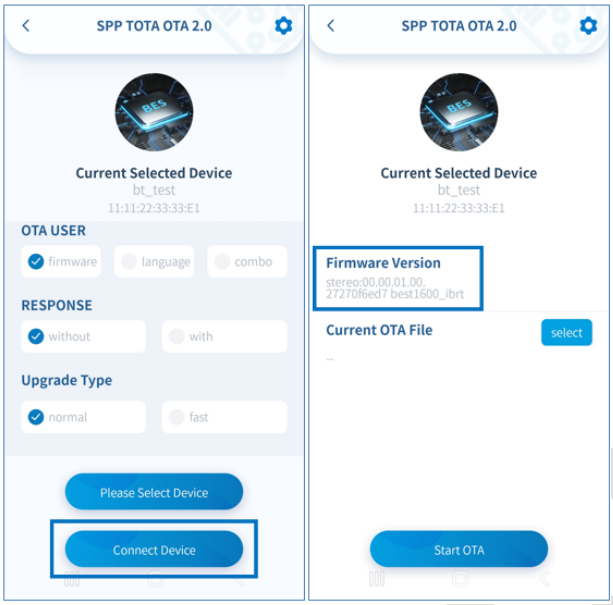


Figure 90 Device Connection

5.9.4.2.3.6 Select an Image

Click "**select**" to select the new image.

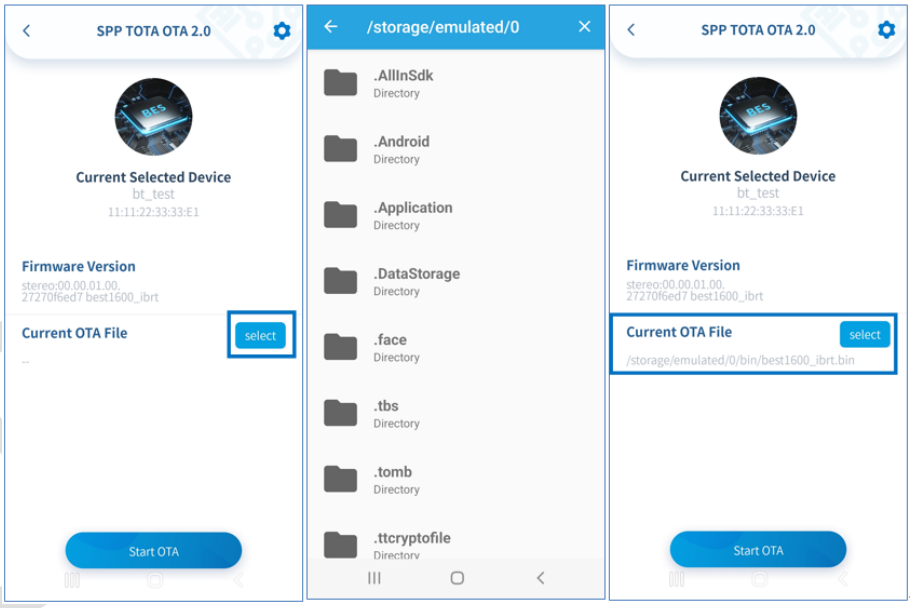


Figure 91 Image Selection

5.9.4.2.3.7 Start the OTA Process

Click "**Start OTA"**, and an option box appears. The content of the option box can edited only when the corresponding checkbox is selected with "**Y**". Click "**OK**" button to start upgrading.

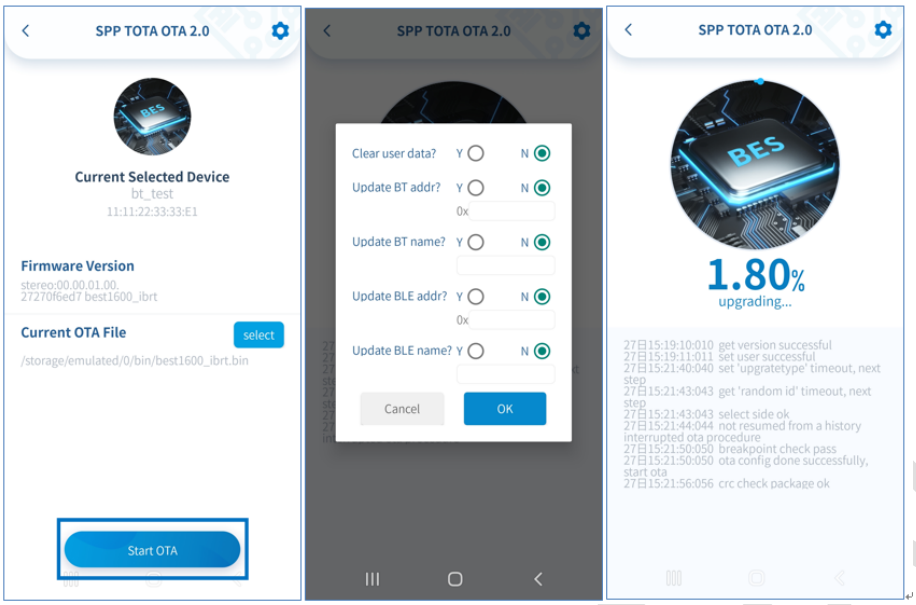


Figure 92 Start OTA Process

5.9.4.2.3.8 Finish the OTA Process

When the upgrade is completed, the new application image is applied and the rebooting to execute the new image is automatically done.

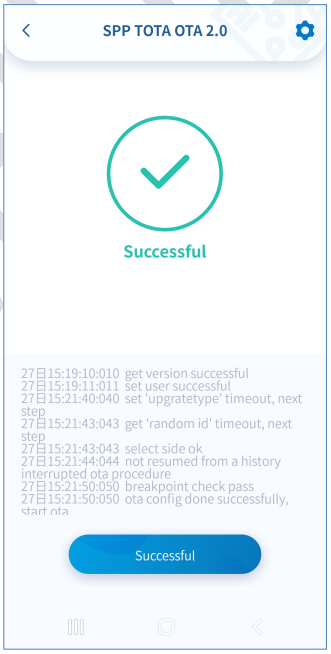


Figure 93 OTA Completed

5.9.4.2.4 TOTA OTA Base on IOS

5.9.4.2.4.1 Install and open the APP

Install the APP named Bestechnic. Open it, enter the main view and click "**OTA**".

Choose version2.0 because BES devices no longer support Version1.0.

There are two kinds of TOTA OTA: GATT OVER BR/EDR TOTA OTA and BLE TOTA OTA. Most of the steps are the same, and some differences are explained in the following section.

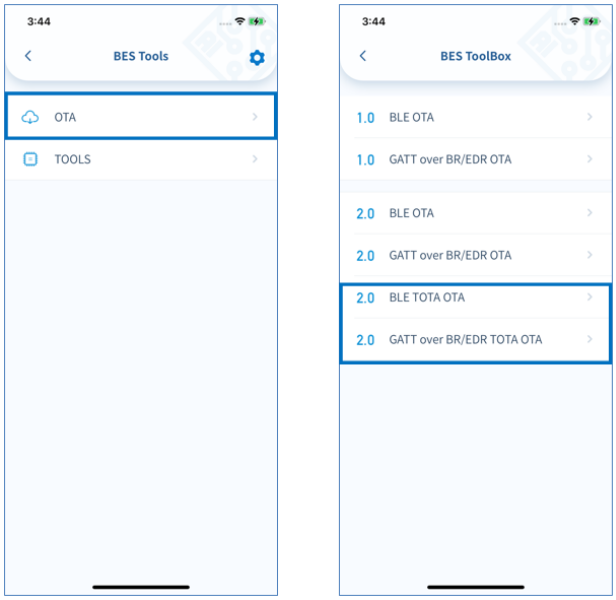


Figure 94 Main View

5.9.4.2.4.2 Bluetooth Pairing and Connection

If you choose BLE TOTA OTA, skip this step.

Press and hold the "**PWR_ON**" button. When hearing the "**pairing**" tone after the "**power on**" tone, pair the OTA device with iphone.

CONFIDENTIAL

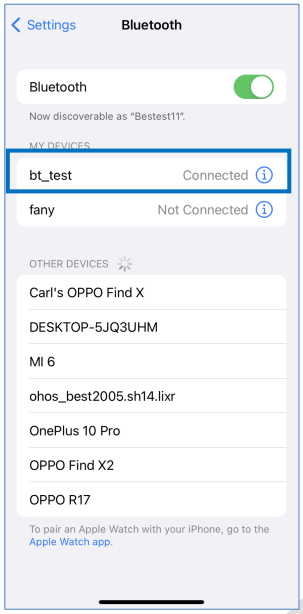


Figure 95 Pairing and Connection

5.9.4.2.4.3 Select a Device

Click **"Choose Device"** and select the OTA device, then the selected device's name can be viewed as follows:

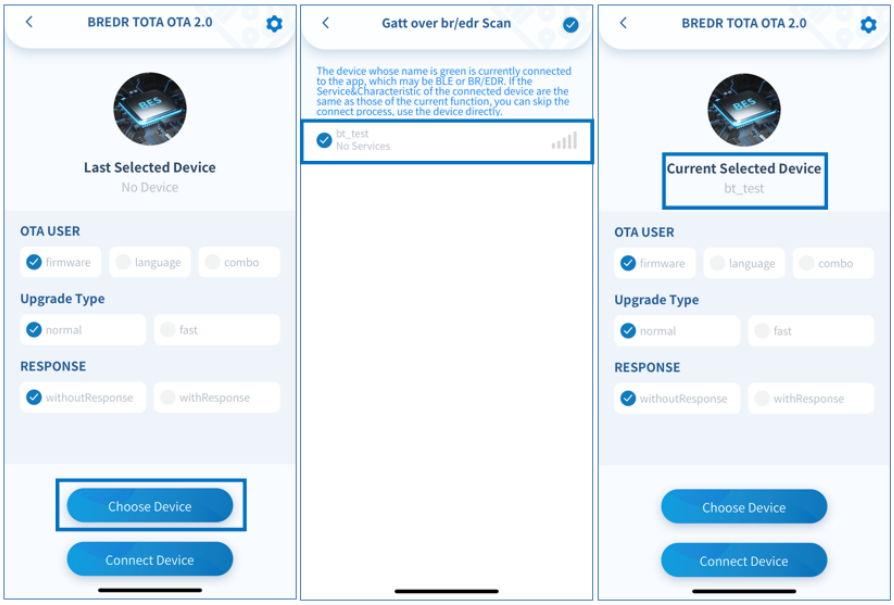


Figure 96 Device Selection

5.9.4.2.4.4 Configure upgrade

- Select OTA USER:

  firmware: Only image.

  language: Only voice prompt.

  combo: Combo image and voice prompt.

- Select RESPONSE:

  withoutResponse: No response.

  withRespons: Have response.

- Select Upgrade type:

  normal: Normal upgrade rate. Users can use OTA device normally in this mode.

  fast: Faster upgrade rate but disable some functions, such as voice calls.

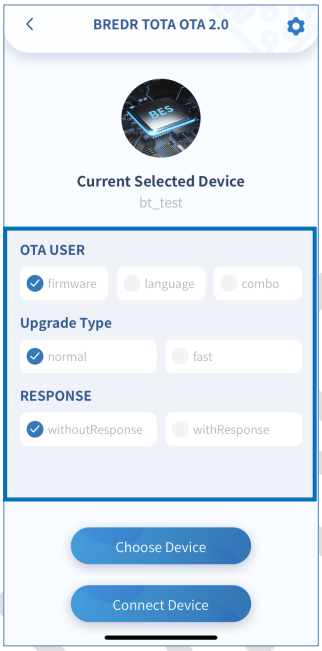


Figure 97 Configure Upgrade

5.9.4.2.4.5 Connect a Device

Click “**Connect Device”.**

After a few seconds, the firmware version of OTA device can be viewed as follows if the connection is successful.

CONFIDENTIAL

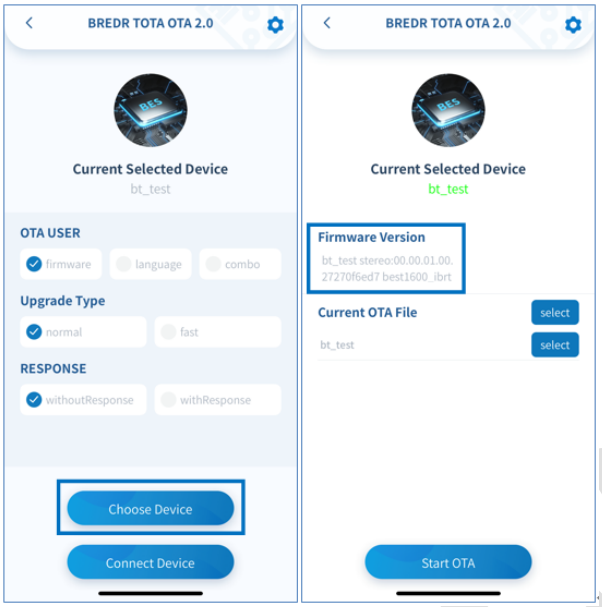


Figure 98 Device Connection

5.9.4.2.4.6 Select an Image

Click "**select**" to select the new image.

If you select multiple devices, multiple buttons can be viewed to select multiple OTA files. There are only two buttons here because only one OTA device was selected. The top button is used to select OTA files for all OTA devices. If you want to select different OTA files for different devices, click the button corresponding to the device to select OTA file respectively.

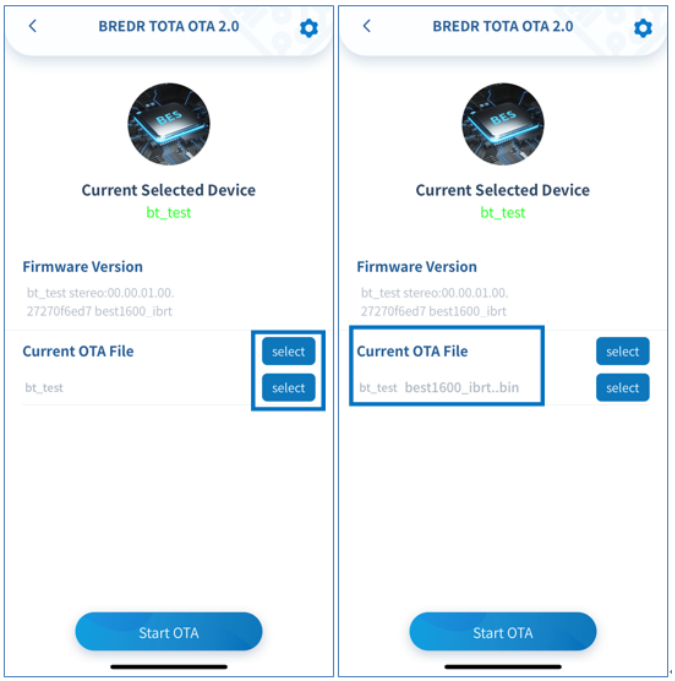


Figure 99 Image Selection

CONFIDENTIAL

5.9.4.2.4.7 Start the OTA Process

Click “**Start OTA”**, and an option box appears. The content of the option box can edited only when the corresponding checkbox is selected with “**Y**”. Click “**OK**” button to start upgrading.

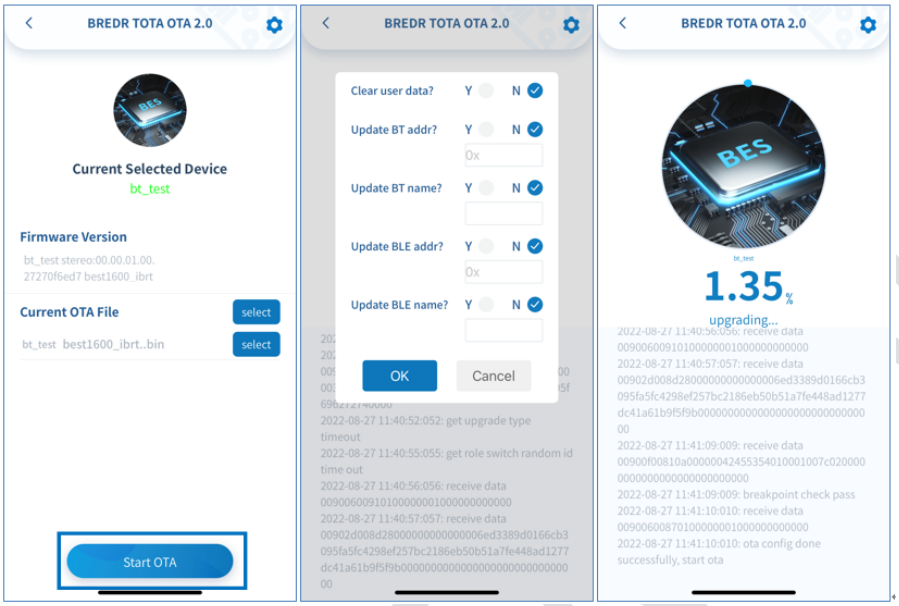


Figure 100 Start OTA Process

5.9.4.2.4.8 Finish the OTA Process

When the upgrade is completed, the new application image is applied and the rebooting to execute the new image is automatically done.

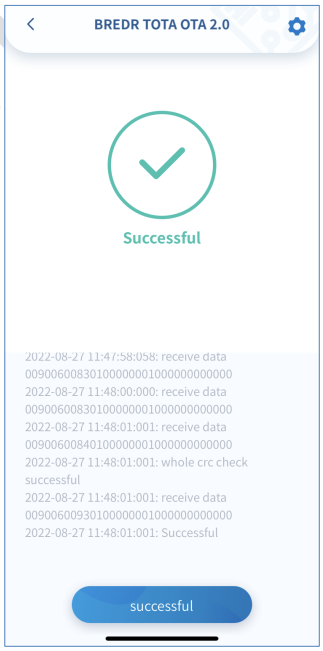


Figure 101 Upgrade Completed

5.10 TOTA

5.10.1 TOTA Protocol

- TOTA packet

The dongle packet has a maximum length of 208 bytes, and the data content has a maximum length of 192 bytes, as defined in the following macro in “app_tota.cpp”:

```
#define DONGLE_PACKET_FRAME 208
#define DONGLE_DATA_FRAME 192
```

Packet format:

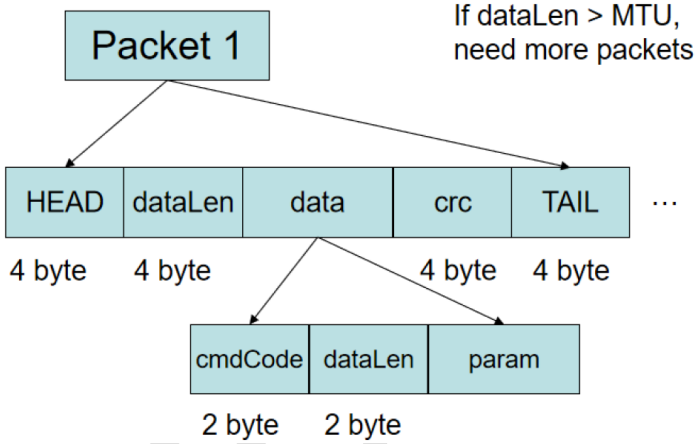


Figure 102 TOTA Packet Format

A basic rate packet consists of 5 entities: the HEAD, the dataLen, the data, the crc and the TAIL.
Head and Tail are used to distinguish a packet of data. Data is encrypted and can be decrypted only after receiving the entire packet.

The data fields include cmdCode, dataLen and param.

cmdCode: tota command
dataLen: param length
param: cmdCode parameter

Note that the dataLen in the data field does not necessarily correspond to the dataLen in the packet due to encryption and subcontracting:

Encryption requires 16-byte alignment. If the value is less than 16 bytes, fill in 0. So the packet length contains padding bytes in addition to content. If the data content is larger than 192 bytes, the packet must be subcontracted and sent. The length of the first packet is 208 bytes (the data part is 192 bytes, and no padding is required). The last packet is filled with bytes before transmission.

- Information interaction between the Dongle and earbuds

The TOTA test between the Dongle and the earbuds uses SPP connection. After the SPP connection, the secret key is negotiated through BASIC commands, and then other commands are encrypted and transmitted.

The communication process is as follows:

1. SPP is connected, but TOTA is not connected. Only the BASIC CMD is allowed to interact with each other, and other commands are filtered out.
2. Complete TOTA connection and encryption through the BASIC CMD. The interaction process is shown in the following figure.
3. Encrypt and transmit data.

The Dongle communicates with the earbuds only through CMD and RSP, and there is no other message such as ACK.

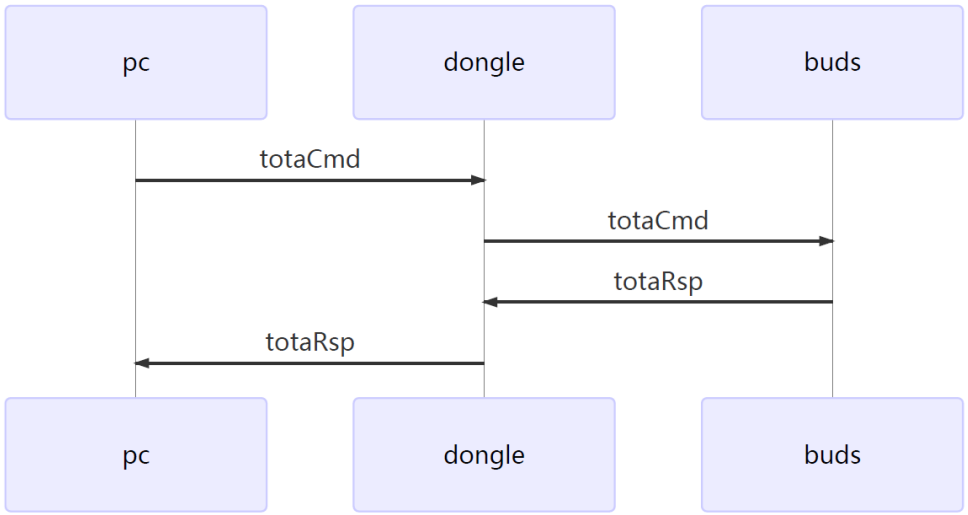


Figure 103 Interaction Process of TOTA Test

For the read command, the earbuds directly respond if the command is successfully executed. If the earbuds fail to execute, the response cmd is returned with the error number. For write or set commands, the earbuds return the response cmd with the execution result.

- Test handler function format

Define the code segment “tota_handler_table” to which all handlers are added. When you receive the total command, look for the corresponding handler in this section.

```
#define TOTA_COMMAND_TO_ADD(cmdCode, cmdHandler, isNeedResponse,
timeoutWaitingRspInMs,
cmdRspHandler) \
static const APP_TOTA_CMD_INSTANCE_T cmdCode##_entry __attribute__((used,
section(".tota_handler_table"))) = \
{(cmdCode), (cmdHandler), (isNeedResponse), (timeoutWaitingRspInMs),
(cmdRspHandler)};
```

```
extern uint32_t __tota_handler_table_start[];
extern uint32_t __tota_handler_table_end[];
```

Test handler function format:

```
typedef struct
{
uint32_t cmdCode;
app_tota_cmd_handler_t cmdHandler; /**< command handler function */
uint16_t isNeedResponse; /**< true if needs the response from the peer
device */
uint16_t timeoutWaitingRspInMs; /**< time-out of waiting for response in
milli-seconds */
app_tota_cmd_response_handler_t cmdRspHandler; /**< command response
handler function */
} APP_TOTA_CMD_INSTANCE_T;
```

- Processes of receiving and sending messages

The receiving process is shown in the following figure:

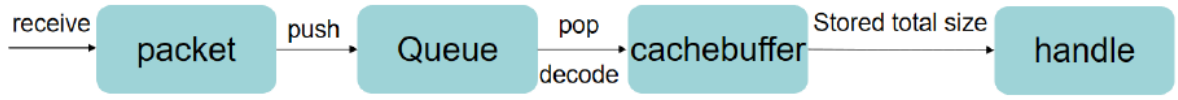


Figure 104 Message Receiving Process

The data received by SPP is stored in “totaEnv.rxDataQueue” to cache data packets.

Call the unpack function, process the data according to the connection status, and save the packet to the “totaEnv.rxDataQueue”.

If the connection status is TOTA_CONNECTED, the data is cached directly.

If the connection status is TOTA_SHAKE_HANDED, the packet needs to be decrypted and cached.

Call the parsing function to parse the packet.

Packet-receiving handler function:

```
//receiving spp packet
void s_app_tota_rx(uint8_t * cmd_buf, uint16_t len)
//buffer queue
int tota_rx_queue_push(uint8_t *buff, uint16_t len)
//unpacking
int app_tota_rx_unpack(uint8_t * pdata, uint16_t dataLen)
//cache packet
```

```
TOTA_STORE_STATUS_E app_tota_store(uint8_t * pdata, uint16_t dataLen)
//command handler function
APP_TOTA_CMD_RET_STATUS_E app_tota_cmd_received(uint8_t* ptrData, uint32_t
dataLength)
```

The sending process is shown in the following figure:

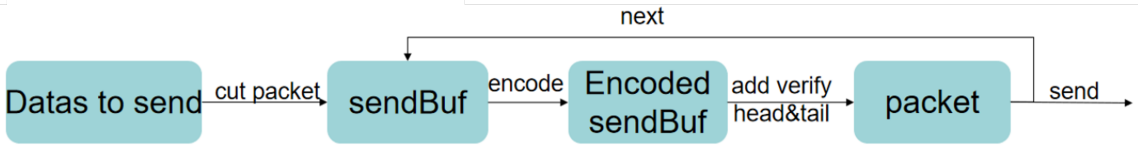


Figure 105 Message Sending Process

For large data packets, they need to be unpacked first.

Packet-sending handler function:

```
//sending function
bool app_tota_send(uint8_t * pdata, uint16_t dataLen, APP_TOTA_CMD_CODE_E
opCode)
bool app_tota_send_rsp(APP_TOTA_CMD_CODE_E rsp_opCode,
APP_TOTA_CMD_RET_STATUS_E rsp_status,
uint8_t * pdata, uint16_t dataLen)
bool app_tota_data_send(APP_TOTA_CMD_CODE_E opCode, uint8_t * data,
uint16_t dataLen)
//encrypt function
uint16_t tota_encrypt(uint8_t * out, uint8_t * in, uint16_t inLen)
//pack function
uint16_t app_tota_tx_pack(uint8_t * pdata, uint16_t dataLen)
//spp sending function
bool app_spp_tota_send_data(uint8_t* ptrData, uint16_t length)
```

5.10.2 TOTA User Guide

1. To make firmware best1600_ibrt.bin, use the compilation instruction:

"make T=best1600_ibrt TOTA_v2=1 POWER_ON_ENTER_TWS_PAIRING_ENABLED=1 BLE=1 -j"

bin path: /out/best1600_ibrt/best1600_ibrt.bin

2. Burn the firmware best1600_ibrt.bin into the earbuds.

3. In the OTA function of the App, the option with TOTA is TOTA_v2 by default, such as BLE TOTA OTA;

In the App code, the TOTA_v2 function is determined by the totaConnect and useTotaV2 attributes in

"BesServiceConfig". TOTA_v2 is enabled if both attributes are set to YES.

4. Click Choose Device to choose a device, and click Connect Device to connect the device. The process of establishing a TOTA connection can be seen through the App console or the log in the headset.

APP name: Bestechnic.

IOS App:

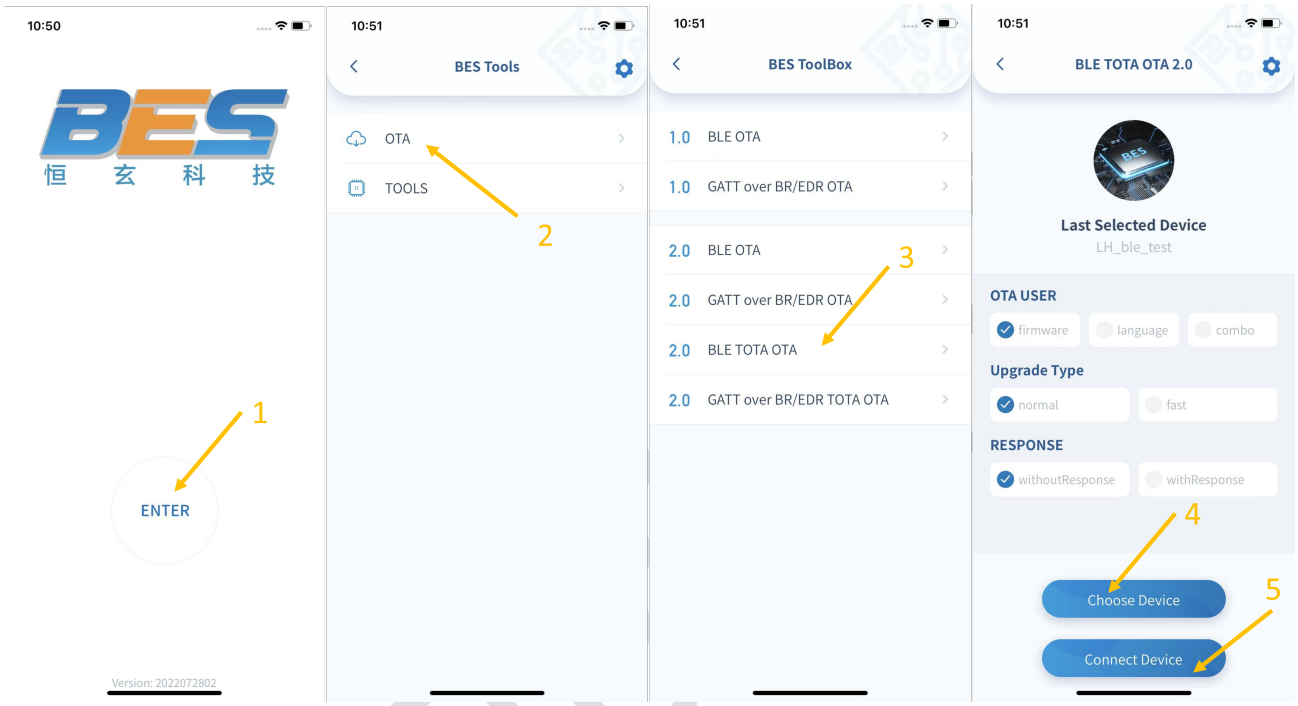


Figure 106 IOS App Operation Process

Android App:

CONFI

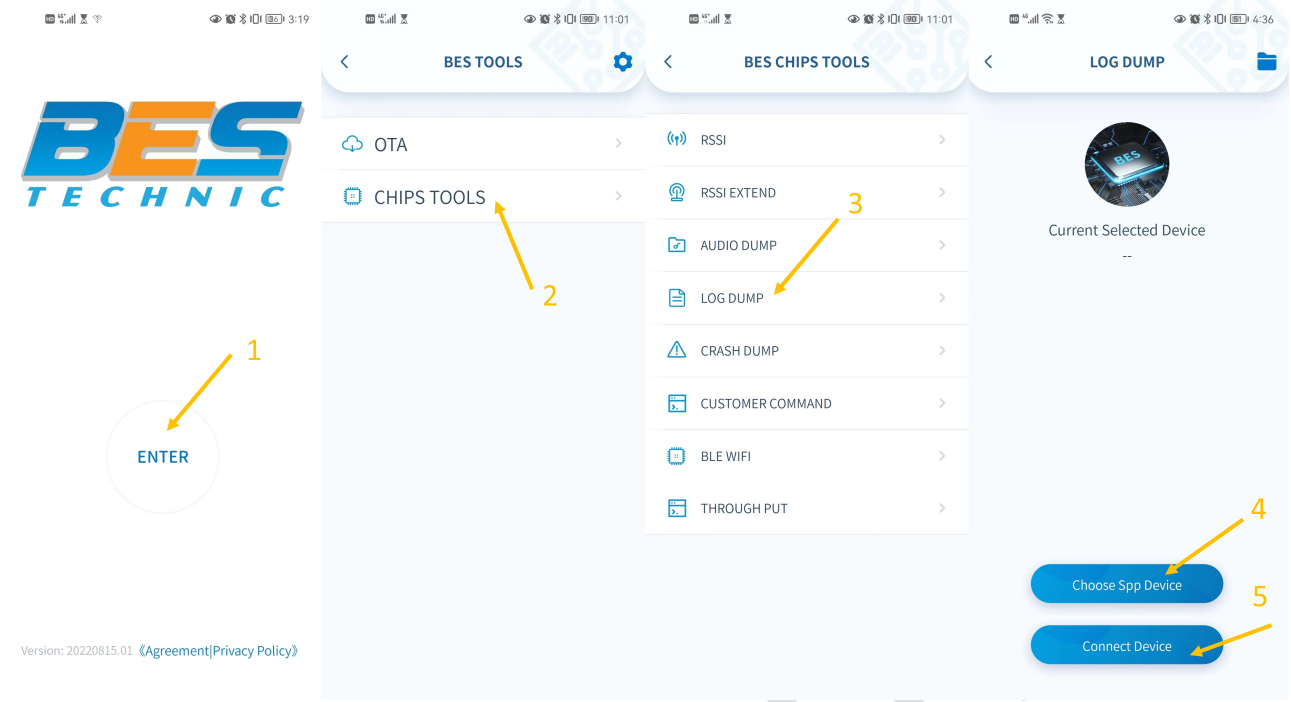


Figure 107 Android App Operation Process

5. The App console log is shown as follows. The connection process of TOTA_v2 can be understood through the short line, and receiving the long line command indicates that the TOTA connection is successful.

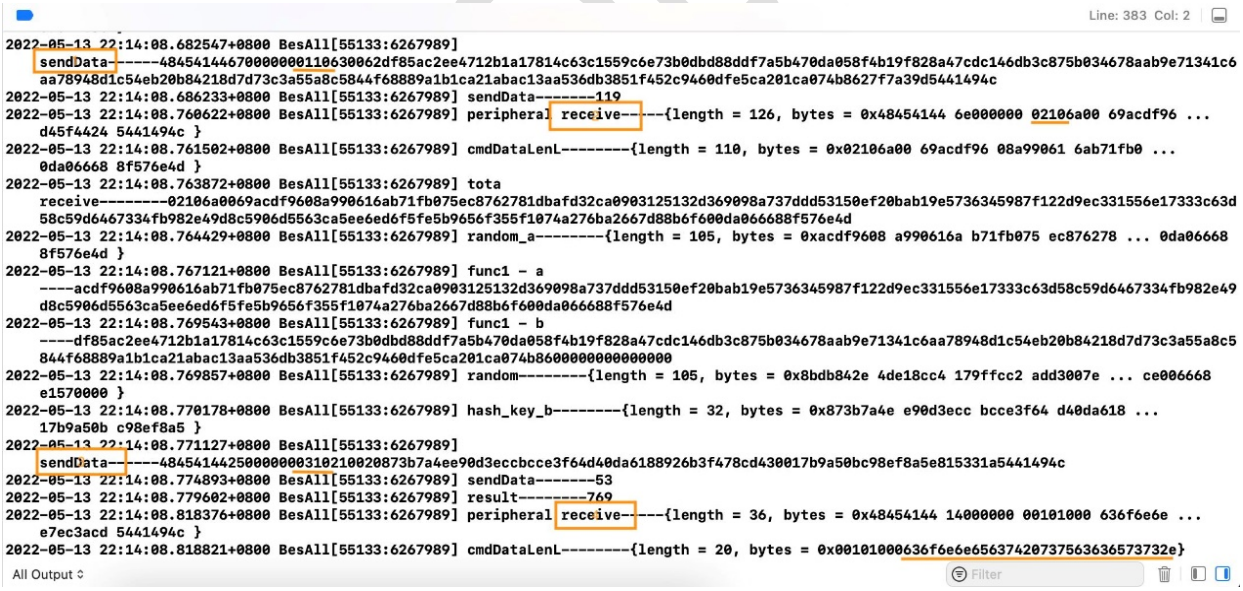


Figure 108 App Console Log

6. The following firmware serial port log shows corresponding to the App log. The short line part is the same as THE App. If "tota connect success" is displayed, the TOTA connection is successful.

```
[22:14:09:149]02 02 20 05 00 01 00 04 00 13
[22:14:09:149]   263807/R-M/I/NONE  /  8 | [TX2]:
[22:14:09:149]02 02 20 85 00 81 00 04 00 1b 20 00 48 45 41 44 6e 00 00 00 02 10 6a 00 69 ac df 96 08 a9 90 61 6a b7 1f b0 75 ec 87 62 78 1d
ba fd 32 ca 09 03 12 51 32 d3 69 09 8a 73 7d dd 53 15
[22:14:09:149]   263836/R-M/I/NONE  /  9 | [RX]:
[22:14:09:149]04 13 05 01 02 00 01 00
[22:14:09:149]   263837/R-M/I/NONE  /  9 | [RX]:
[22:14:09:149]04 13 05 01 02 00 01 00
[22:14:09:149]   263872/R-M/I/NONE  /  9 | [RX]:
[22:14:09:149]02 02 20 3c 00 38 00 04 00 12 20 00 48 45 41 44 25 00 00 00 03 10 21 00 20 87 3b 7a 4e e9 0d 3e cc bc ce 3f 64 d4 0d a6 18 89
26 b3 f4 78 cd 43 00 17 b9 a5 0b c9 8e f8 a5 e8 15 33
[22:14:09:149]   263879/R-M/I/NONE  /  8 | tota_gatt_cb_att_set buds_env 0x2006ef4c write handle 32 shdl 30
[22:14:09:149]   263879/R-M/I/NONE  /  8 | [TOTA_CONTROL] [app_tota_handle_received_data]data receive data length = 53
[22:14:09:149][0x48][0x45][0x41][0x44][0x25][0x0][0x0][0x0][0x3][0x10][0x21][0x0][0x20][0x87][0x3b][0x7a][0x4e][0xe9][0xd][0x3e]
[22:14:09:149]   263880/R-M/I/NONE  /  8 | [TOTA_CONTROL] Tota rx.
[22:14:09:149]   263881/R-M/I/NONE  /  8 | [TOTA_CONTROL] queueLen = 0x35
[22:14:09:149]   263881/R-M/I/NONE  /  8 | [TOTA_CONTROL] app_tota_rx_unpack
[22:14:09:149]   263881/R-M/I/NONE  /  8 | [TOTA_CONTROL] bufLen:0x25
[22:14:09:149]   263882/R-M/I/NONE  /  8 | [TOTA_CONTROL] TOTA_CONNECTED
[22:14:09:149]   263882/R-M/I/NONE  /  8 | [TOTA_CONTROL] app_tota_store status=0,DataCacheBufferOffset=0x25
[22:14:09:149]   263882/R-M/I/NONE  /  8 | [TOTA_CONTROL] TOTA Receive data:
[22:14:09:149]0x03 0x10 0x21 0x00 0x20 0x87 0x3b 0x7a 0x4e 0xe9 0x0d 0x3e 0xcc 0xbc 0xce 0x3f 0x64 0xd4 0x0d 0xa6
[22:14:09:149]   263883/R-M/I/NONE  /  8 | [TOTA_CONTROL] [_tota_conn_and_update_key_handle]: opCode:0x1003, paramLen:33
[22:14:09:149]   263884/R-M/I/NONE  /  8 | [TOTA_CONTROL] aes key:
[22:14:09:149]87 7a e9 3e bc 3f d4 a6 89 b3 78 43 17 a5 c9 f8
[22:14:09:149]   263885/R-M/I/NONE  / 10 | ibrt_core:-------IRS:004a cmdcode=SYNC_TOTA_ENCODE_STATUS------->
[22:14:09:149]   263885/R-M/I/NONE  /  8 | [TOTA_CONTROL] tota connect success
[22:14:09:149]   263886/R-M/I/NONE  /  8 | [TOTA_CONTROL] [app_tota_send_notification]TOTA
[22:14:09:149]   263886/R-M/I/NONE  /  8 | [TOTA_CONTROL] queueLen = 0x0
[22:14:09:149]   263887/R-M/I/NONE  /  8 | tota_gatt_cb_event_sent conidx 0x0
[22:14:09:149]   263889/R-M/I/NONE  /  8 | ibrt_core:TXDONE:004a cmdcode=SYNC_TOTA_ENCODE_STATUS
```

Figure 109 Firmware Serial Port Log

5.11 Add a Custom UART Command

Uart commands are used to implement corresponding functions. You can add the command string and its corresponding function to the structure array.

```
static const ble_audio_uart_handle_with_param_t
aob_uart_test_handle_with_param[]=
{
    // Example: [AOB,mobile_connect|214 204 251 165 185 226]
    {"mobile_connect", ble_audio_mobile_connect_test},
    {"mobile_disconnect", ble_audio_mobile_disconnect_test},
}
```

Here is an example to show how to add a custom uart command list (support parameters) with a prefix string.

The function ble_audio_uart_cmd_init can call the function app_trace_rx_register with two arguments: the prefix name and its callback function ble_audio_uart_cmd_callback.

```
void ble_audio_uart_cmd_init(void)
{
#ifdef APP_TRACE_RX_ENABLE
    TRACE(0,"ble_audio_uart_cmd_init");
    app_trace_rx_register("AOB", ble_audio_uart_cmd_callback);
#endif
}
```

For the format of the UART instruction with arguments, the callback function calls the function ble_audio_uart_cmd_with_param_handler to match the input character instruction in UART with the prefix string

defined in the aob_uart_test_handle_with_param array. When the same string is found, the corresponding function instruction is executed.

```
ble_audio_uart_test_function_handle_with_param
ble_audio_test_find_uart_handle_with_param(char* buf)
{
    ble_audio_uart_test_function_handle_with_param p = NULL;
    for(uint32_t i = 0; i < ARRAY_SIZE(aob_uart_test_handle_with_param);
i++)
    {
        if (strncmp((char*)buf, aob_uart_test_handle_with_param[i].string,
            strlen(aob_uart_test_handle_with_param[i].string))==0 ||
            strstr(aob_uart_test_handle_with_param[i].string, (char*)buf))
        {
            TRACE(1, "AOB cmd:%s",
aob_uart_test_handle_with_param[i].string);
            p = aob_uart_test_handle_with_param[i].function;
            break;
        }
    }
    return p;
}
```

5.12 Debugging Methods

5.12.1 UART Log Print

5.12.2 Crash Analysis

5.12.2.1 Exception Analysis

Exception refers to the abnormal interrupt generated by the CPU, which includes the following four types:

1. Hard Fault
2. Mem Manage Fault
3. Bus Fault
4. Usage Fault

The CPU saves the status of the system interrupt in SHCSR (System Handler Control and Status Registers). The bit that is set to 1 indicates the fault that occurred.

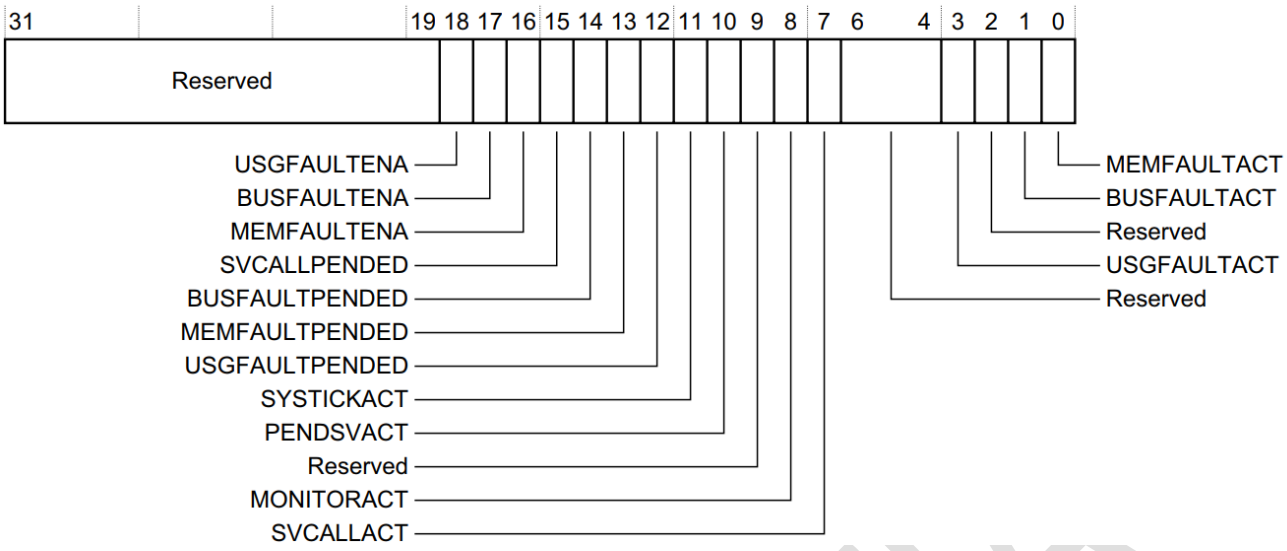


Figure 110 SHCSR Bit Assignmonts

5.12.2.1.1 Hard Fault

For the cause of Hard Fault, you can query the register HFSR, as shown in the following figure.

There are two reasons for the CPU to generate Hard Fault:

1. VECTTBL

   This reason means that when the interrupt is triggered, the CPU encounters Bus Fault when reading the interrupt vector from the vector table.

2. FORCED

   This reason means that when other faults are triggered, the faults cannot be responded to due to lower priority or interrupts being turned off. Therefore, Hard Fault is triggered. Because the priority of the Hard Fault interrupt is -1, which is second only to the Reset interrupt and NMI interrupt, the Hard Fault interrupt is not masked. When other fault interrupts cannot be responded to, the Hard Fault interrupt is triggered to stop the CPU.

   Therefore, when encountering an exception caused by this reason, you also need to check other Fault states and reasons.

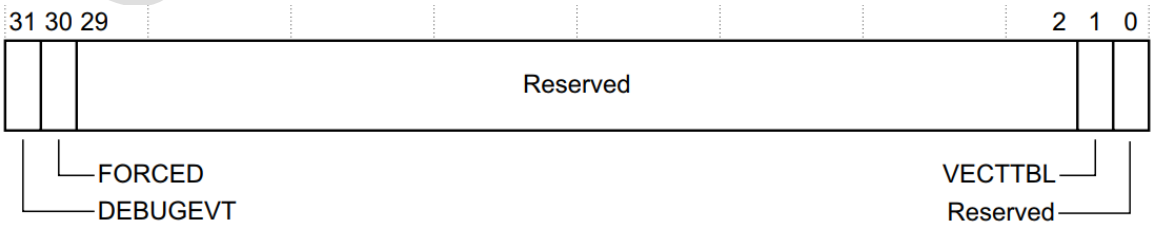


Figure 111 HardFault Status Register

CONFIDENTIAL

Table 118 HFSR Bit Assigments

Bits	Name	Function
[31]	DEBUGEVT	Reserved for Debug use. When writing to the register you must write 0 to this bit, otherwise the behavior is unpredictable.
[30]	FORCED	Indicates a forced hard fault, generated by escalation of a fault with configurable priority that cannot be handled, because of priority or because it is disabled: 0 = no forced HardFault 1 = forced HardFault. When this bit is set to 1, the HardFault handler must read the other fault status registers to find the cause of the fault.
[29:2]	-	Reserved.
[1]	VECTTBL	Indicates a BusFault on a vector table read during exception processing: 0 = no BusFault on vector table read 1 = BusFault on vector table read. This error is always handled by the hard fault handler. When this bit is set to 1, the PC value stacked for the exception points to the instruction that was preempted by the exception.
[0]	-	Reserved

5.12.2.1.2 Mem Manage Fault

For the cause of Mem Manage Fault, you can query the register MMFSR, as shown in the following figure.

There are many reasons for Mem Manage Fault.

Common reasons:

1. IACCVIOL

   This reason means that the access address is the MPU protection area when the CPU fetches instructions.

2. DACCVIOL

   This reason means that when the CPU reads or writes data, the access address is the MPU protected area.

Other reasons:

3. MUNSTKERR

   This reason means that when returning from an interrupt, the CPU accesses the MPU protection area when restoring the thread stack.

4. MSTKERR

   This reason means that when entering the interrupt, the CPU accesses the MPU protection area when pushing the stack.

CONFIDENTIAL

5. MLSPERR

This reason means that the MPU protection area is accessed when the reduced FPU is pushed and popped.

As the following figure shows, the bit 7 of MMFSR is the MMARVALID signal, which indicates whether the MMAR is valid. If it is valid, the illegally accessed address is stored in the MMAR. For example, if the 0 address is illegally accessed, 0 is stored in the MMAR.

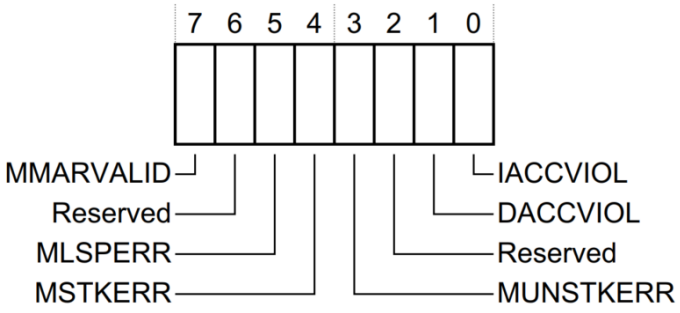


Figure 112 MemManage Fault Status Register

Table 119 MMFSR Bit Assignments

Bits	Name	Function
[7]	MMARVALID	MemManage Fault Address Register (MMFAR) valid flag: 0 = value in MMAR is not a valid fault address 1 = MMAR holds a valid fault address. If a MemManage fault occurs and is escalated to a HardFault because of priority, the HardFault handler must set this bit to 0. This prevents problems on return to a stacked active MemManage fault handler whose MMAR value has been overwritten.
[6]	-	Reserved
[5]	MLSPERR	0 = no MemManage fault occurred during floating-point lazy state preservation 1 = a MemManage fault occurred during floating-point lazy state preservation.
[4]	MSTKERR	MemManage fault on stacking for exception entry: 0 = no stacking fault 1 = stacking for an exception entry has caused one or more access violations. When this bit is 1, the SP is still adjusted but the values in the context area on the stack might be incorrect. The processor has not written a fault address to the MMAR.
[3]	MUNSTKERR	MemManage fault on unstacking for a return from exception: 0 = no unstacking fault 1 = unstack for an exception return has caused one or more access violations. This fault is chained to the handler. This means that when this bit is 1, the original return stack is still present. The processor has not adjusted the SP from

CONFIDENTIAL

		the failing return, and has not performed a new save. The processor has not written a fault address to the MMAR.
[2]	-	Reserved
[1]	DACCVIOL	Data access violation flag: 0 = no data access violation fault 1 = the processor attempted a load or store at a location that does not permit the operation. When this bit is 1, the PC value stacked for the exception return points to the faulting instruction. The processor has loaded the MMAR with the address of the attempted access.
[0]	IACCVIOL	Instruction access violation flag: 0 = no instruction access violation fault 1 = the processor attempted an instruction fetch from a location that does not permit execution. This fault occurs on any access to an XN region, even when the MPU is disabled or not present. When this bit is 1, the PC value stacked for the exception return points to the faulting instruction. The processor has not written a fault address to the MMAR.

Taking bes1600_ibrt which is ARM_ARCH_8M architecture as an example, the MPU protection area in the BES code is shown in the following code. The area from address 0 to ROM BASE area is executable only, and the RAMX area is readable and executable. So when you write a value to any address in the RAMX area, the Mem Manage Fault is triggered.

```
typedef struct
{
    uint32_t addr;
    uint32_t len;
    // MPU attribute
    enum MPU_ATTR_T ap_attr;
    // memory attribute
    enum MEM_ATTR_T mem_attr;
} mpu_regions_t;

#define ROM_DATA_MAP_BASE                       0x20000
#define RAM_TOTAL_SIZE                          0xE0000

static const mpu_regions_t mpu_boot_table[] = {
    {0,          ROM_DATA_MAP_BASE, MPU_ATTR_EXEC,       MEM_ATTR_INT_SRAM },
    {RAMX_BASE, RAM_TOTAL_SIZE,     MPU_ATTR_READ_EXEC,  MEM_ATTR_INT_SRAM },
```

CONFIDENTIAL

```
};
```

5.12.2.1.3 Bus Fault

For the cause of Bus Fault, you can query the register BFSR, as shown in the following figure.

There are many reasons for Bus Fault.

Common reasons:

1. IBUSERR

   Instruction bus error. The CPU accesses an illegal address when fetching an instruction.

2. PRECISERR

   Data bus error. The CPU accesses an illegal address when reading and writing data.

3. IMPRECISERR

   Data bus error. The difference from 2 is that the address accessed by the data bus has nothing to do with the currently executing instruction.

Other reasons:

4. UNSTKERR

   This reason means that the CPU resumes the thread stack with an illegal address access when exiting from an interrupt.

5. STKERR

   This reason means that the CPU pushes the stack and there is an illegal address access when entering the interrupt.

6. LSPERR

   This reason means that the CPU access illegal address when the compact FPU is pushed and popped.

Bus Fault is very similar to Mem Manage Fault. The only difference is whether the accessed address is protected by MPU or cannot be found on the address bus.

The bit 7 of BFSR is the BFARVALID signal, which indicates whether BFAR is valid. If it is valid, BFAR saves the address of illegal access. For example, if address 0 is illegally accessed, 0 is saved in BFAR.

CONFIDENTIAL

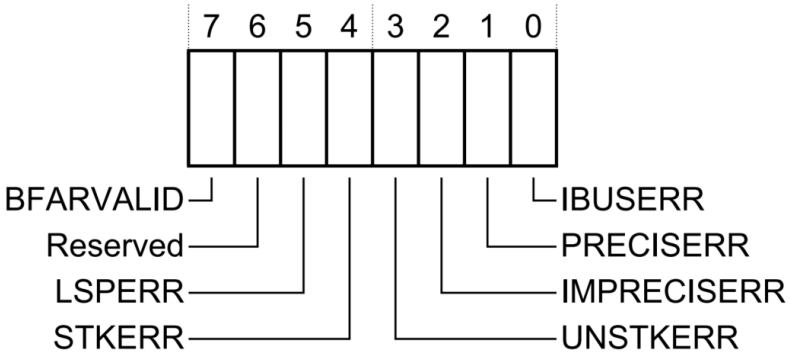


Figure 113 BusFault Status Register

Table 120 BFSR Bit Assignments

Bits	Name	Function
[7]	BFARVALID	BusFault Address Register (BFAR) valid flag: 0 = value in BFAR is not a valid fault address 1 = BFAR holds a valid fault address. The processor sets this bit to 1 after a BusFault where the address is known. Other faults can set this bit to 0, such as a MemManage fault occurring later. If a BusFault occurs and is escalated to a hard fault because of priority, the hard fault handler must set this bit to 0. This prevents problems if returning to a stacked active BusFault handler whose BFAR value has been overwritten.
[6]	-	Reserved
[5]	LSPERR	0 = no bus fault occurred during floating-point lazy state preservation 1 = a bus fault occurred during floating-point lazy state preservation.
[4]	STKERR	BusFault on stacking for exception entry: 0 = no stacking fault 1 = stacking for an exception entry has caused one or more BusFaults. When the processor sets this bit to 1, the SP is still adjusted but the values in the context area on the stack might be incorrect. The processor does not write a fault address to the BFAR.
[3]	UNSTKERR	BusFault on unstacking for a return from exception: 0 = no unstacking fault 1 = unstack for an exception return has caused one or more BusFaults. This fault is chained to the handler. This means that when the processor sets this bit to 1, the original return stack is still present. The processor does not adjust the SP from the failing return, does not performed a new save, and does not write a fault address to the BFAR.

CONFIDENTIAL

[2]	IMPRECISERR	Imprecise data bus error: 0 = no imprecise data bus error 1 = a data bus error has occurred, but the return address in the stack frame is not related to the instruction that caused the error. When the processor sets this bit to 1, it does not write a fault address to the BFAR. This is an asynchronous fault. Therefore, if it is detected when the priority of the current process is higher than the BusFault priority, the BusFault becomes pending and becomes active only when the processor returns from all higher priority processes. If a precise fault occurs before the processor enters the handler for the imprecise BusFault, the handler detects both IMPRECISERR set to 1 and one of the precise fault status bits set to 1.
[1]	PRECISERR	Precise data bus error: 0 = no precise data bus error 1 = a data bus error has occurred, and the PC value stacked for the exception return points to the instruction that caused the fault. When the processor sets this bit is 1, it writes the faulting address to the BFAR.
[0]	IBUSERR	Instruction bus error: 0 = no instruction bus error 1 = instruction bus error. The processor detects the instruction bus error on prefetching an instruction, but it sets the IBUSERR flag to 1 only if it attempts to issue the faulting instruction. When the processor sets this bit is 1, it does not write a fault address to the BFAR.

5.12.2.1.4 Usage Fault

For the reason for Usage Fault, you can query the special register UFSR, as shown in the following figure.

There are many reasons for Usage Fault.

Common reasons:

1. UNDEFINSTR

   This error indicates that the instruction found by the CPU is an unrecognized instruction.

2. INVSTATE

   This error indicates that the CPU incorrectly modifies EPSR to an illegal value after executing the instruction.

   The EPSR contains the Thumb State Bit. The lowest bit of all Thumb instruction addresses should be 1. If the lowest bit of the instruction address is 0, it is considered by the CPU as not a Thumb instruction, thus

CONFIDENTIAL

modifying the Thumb State in the EPSR. The Cortex-M series only supports the Thumb instruction set, so when the Thumb State in EPSR is not 1, the CPU triggers the Usage Fault.

3. INVPC

   This error indicates that EXC_RETURN is an illegal value when returning from interrupt.

4. NOCP

   This error indicates that the instruction found by the CPU reference requires a coprocessor, but there is no coprocessor or the coprocessor is turned off.

5. UNALIGNED

   The error indicates that the access is not aligned. For example, to do a 4-byte read and write, the address is odd. This error is configurable and will not be generated by the CPU when unaligned accesses are supported.

6. DIVBYZERO

   This error indicates that a division by 0 error occurred. The error is also configurable.

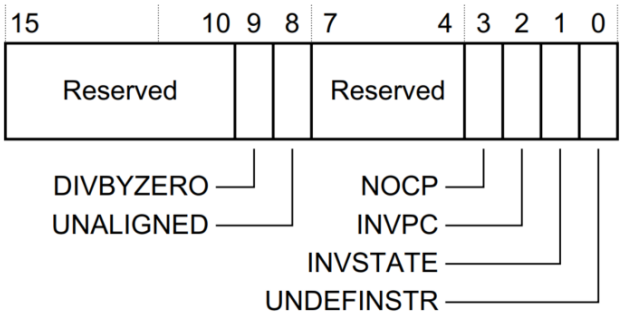


Figure 114 UsageFault Status Register

Table 121 UFSR Bit Assignments

Bits	Name	Function
[15:10]	-	Reserved.
[9]	DIVBYZERO	Divide by zero UsageFault: 0 = no divide by zero fault, or divide by zero trapping not enabled 1 = the processor has executed an SDIV or UDIV instruction with a divisor of 0. When the processor sets this bit to 1, the PC value stacked for the exception return points to the instruction that performed the divide by zero. Enable trapping of divide by zero by setting the DIV_0_TRP bit in the CCR to 1.
[8]	UNALIGNED	Unaligned access UsageFault: 0 = no unaligned access fault, or unaligned access trapping not enabled 1 = the processor has made an unaligned memory access. Enable trapping of unaligned accesses by setting the UNALIGN_TRP bit in the CCR to 1.

CONFIDENTIAL

		Unaligned LDM, STM, LDRD, and STRD instructions always fault irrespective of the setting of UNALIGN_TRP.
[7:4]	-	Reserved.
[3]	NOCP	No coprocessor UsageFault. The processor does not support coprocessor instructions: 0 = no UsageFault caused by attempting to access a coprocessor 1 = the processor has attempted to access a coprocessor.
[2]	INVPC	Invalid PC load UsageFault, caused by an invalid PC load by EXC_RETURN: 0 = no invalid PC load UsageFault 1 = the processor has attempted an illegal load of EXC_RETURN to the PC, as a result of an invalid context, or an invalid EXC_RETURN value. When this bit is set to 1, the PC value stacked for the exception return points to the instruction that tried to perform the illegal load of the PC.
[1]	INVSTATE	Invalid state UsageFault: 0 = no invalid state UsageFault 1 = the processor has attempted to execute an instruction that makes illegal use of the EPSR. When this bit is set to 1, the PC value stacked for the exception return points to the instruction that attempted the illegal use of the EPSR. This bit is not set to 1 if an undefined instruction uses the EPSR.
[0]	UNDEFINSTR	Undefined instruction UsageFault: 0 = no undefined instruction UsageFault 1 = the processor has attempted to execute an undefined instruction. When this bit is set to 1, the PC value stacked for the exception return points to the undefined instruction. An undefined instruction is an instruction that the processor cannot decode.

5.12.2.1.5 Some Examples

The following examples are given to explain how to analyze exceptions.

5.12.2.1.5.1 Mem Fault-Data Access Violation

The fault type and the fault reason are parsed and printed out in log. The log directly shows that it is Mem Fault, and the reason is Data access violation. MMFAR records the last accessed address, which can be seen to be 0x0000. According to the MPU protection range, the protected address is from 0 to ROM BASE, and this area is executable only. The triggering crash means that a value was written to the protected area.

The way to determine which instruction accesses this address is to read the lst file, and find the corresponding code by PC. As the following figure shows, the PC stores the value of R2 in the address of R3. It can be seen that 1 is written to 0x0000. This indicates this type of crash.

CONFIDENTIAL

```
1408be5a:   601a        str r2, [r3, #0]
```

```
### EXCEPTION ###
PC =1408BE5A, ExceptionNumber=D'-12, EXC_RETURN=FFFFFFAC

R0 =34280D38, R1 =3427CB38, R2 =00000001, R3 =00000000
R4 =20563BB0, R5 =205417A0, R6 =00000019, R7 =1408BE51
R8 =0001388F, R9 =205450A8, R10=00000000, R11=00000000
R12=40130000, SP =20547238, LR =140A541F

MSP   =205BE7D8, PSP   =205471D0
MSPLIM=205BD700, PSPLIM=205450B0

PRIMASK   =00, FAULTMASK   =00, BASEPRI   =00, CONTROL   =00

XPSR=61000000, APSR=nZCvq, EPSR=01000000, IPSR=000 (NoException)

ICSR =00000804, AIRCR=FA052000, SCR  =00000000, CCR  =00000611
SHCSR=00070001, CFSR =00000082, HFSR =00000000, AFSR =00000000
DFSR =00000000
MMFAR=00000000, BFAR =00000000

FaultInfo    : (MemFault)
FaultCause   : (Data access violation) (MMFAR valid)
DebugEvent   : (None)
```

Figure 115 Mem Fault Dump

Sample Code 72: Data Access Error

{"data_access_error", data_access_error_test},

5.12.2.1.5.2 Bus Fault-Instruction Bus Error

As the following figure shows, it's an instruction bus error, and the value of the PC is 0x4000000, which is no longer a code segment. Becase the PC is abnormal, the way of finding out the position in code triggered crash is to query the LR register. The value saved in LR is 0x1408B6F1 which means the address that executes after the jump return. Because of the Thumb instruction, the lowest bit is set to 1. Remove the lowest bit and get 0x1408B6F0. Therefore, the jump instruction 0x1408B6F0 is being executed. Query register R3, and you can see that the value is 0x4000000 as PC.

```
1408be3a:   2300        ldr    r3, [sp, #0]
1408be3c:   4798        blx r3
```

CONFIDENTIAL

```
### EXCEPTION ###
PC =04000000, ExceptionNumber=D'-11, EXC_RETURN=FFFFFFBC

R0 =00000001, R1 =00000600, R2 =34261D76, R3 =04000000
R4 =20562612, R5 =00000007, R6 =0000001C, R7 =0000001C
R8 =205625FD, R9 =20523498, R10=205234B0, R11=00000000
R12=FFFFFF9F, SP =205240E8, LR =1408B6F1

MSP   =205BE7D8, PSP   =205240C8
MSPLIM=205BD700, PSPLIM=20523940

PRIMASK   =00, FAULTMASK   =00, BASEPRI   =00, CONTROL   =00

XPSR=60000000, APSR=nZCvq, EPSR=00000000, IPSR=000 (NoException)

ICSR =00000805, AIRCR=FA052000, SCR  =00000000, CCR  =00000611
SHCSR=00070002, CFSR =00000100, HFSR =00000000, AFSR =00000000
DFSR =00000000
MMFAR=E000ED34, BFAR =E000ED38

FaultInfo    : (BusFault)
FaultCause   : (Instruction bus error)
DebugEvent   : (None)
```

Figure 116 Instruction bus error Dump

Sample Code 73: Instruction Bus Error

{"instruction_bus_error", instruction_bus_error_test},

5.12.2.1.5.3 Bus Fault-Precise Data Bus Error

As the following figure shows, the crash is similar to the Mem Manage Fault, and the analysis method is also similar. The instruction to access the abnormal address can be found out according to the PC register. A number is read from the address corresponding to the value stored in R3. The value stored in R3 is 0x4000000, which is an illegal address, this address cannot be found on the bus. So Precise data bus fault is triggered.

```
1408c65e:   681b        ldr r3, [r3, #0]
```

CONFIDENTIAL

```
### EXCEPTION ###
PC =1408B6BC, ExceptionNumber=D'-11, EXC_RETURN=FFFFFFBC

R0 =34280D68, R1 =3427C87C, R2 =00000001, R3 =04000000
R4 =2056260C, R5 =00000007, R6 =00000016, R7 =00000016
R8 =205625FD, R9 =20523498, R10=205234B0, R11=00000000
R12=FFFFFF9F, SP =205240E8, LR =1408B6B5

MSP   =205BE7D8, PSP   =205240C8
MSPLIM=205BD700, PSPLIM=20523940

PRIMASK   =00, FAULTMASK   =00, BASEPRI   =00, CONTROL   =00

XPSR=21000000, APSR=nzCvq, EPSR=01000000, IPSR=000 (NoException)

ICSR =00000805, AIRCR=FA052000, SCR  =00000000, CCR  =00000611
SHCSR=00070002, CFSR =00008200, HFSR =00000000, AFSR =00000000
DFSR =00000000
MMFAR=04000000, BFAR =04000000

FaultInfo    : (BusFault)
FaultCause   : (Precise data bus error) (BFAR valid)
DebugEvent   : (None)
```

Figure 117 Precise Data Bus Error Dump

Sample Code 74: Precise Data Bus Error

```
{"data_bus_error", data_bus_error_test},
```

5.12.2.1.5.4 Usage Fault-Invalid State

The Invalid State that causes a CPU crash is explained as the Bit [24] Thumb State state in the EPSR register is cleared. It can be seen that the EPSR register becomes 0. Any operation that changes the PC value may change the Thumb State state, so the crash is not caused by the current instruction. To locate the crash address, you need to query the LR register. The instruction before jump can be found out from LR, which is BLX R3. Although R3 is a function address, the lowest bit is 0, which is considered by the CPU as not a Thumb instruction.

```
1408be22:   4798        blx r3
```

CONFIDENTIAL

```
### EXCEPTION ###
PC =1408B502, ExceptionNumber=D'-10, EXC_RETURN=FFFFFFAC

R0 =00000000, R1 =00000000, R2 =00000000, R3 =1408B502
R4 =20563BB0, R5 =205417A0, R6 =00000019, R7 =1408B505
R8 =0001388F, R9 =205450A8, R10=00000000, R11=00000000
R12=40130000, SP =20547240, LR =140A53BF

MSP   =205BE7D8, PSP   =205471D8
MSPLIM=205BD700, PSPLIM=205450B0

PRIMASK   =00, FAULTMASK   =00, BASEPRI   =00, CONTROL   =00

XPSR=00000000, APSR=nzcvq, EPSR=00000000, IPSR=000 (NoException)

ICSR =00000806, AIRCR=FA052000, SCR  =00000000, CCR  =00000611
SHCSR=00070008, CFSR =00020000, HFSR =00000000, AFSR =00000000
DFSR =00000000
MMFAR=E000ED34, BFAR =E000ED38

FaultInfo    : (UsageFault)
FaultCause   : (Invalid state UsageFault)
DebugEvent   : (None)
```

Figure 118 Usage Fault Dump

Sample Code 75: Usage Fault

{"usage_fault", usage_fault_test},

5.12.2.1.5.5 Hard Fault-Usage Fault

As the following figure shows, the FaultInfo is HardFault, but the fault cause is Invalid state UsageFault. And the PRIMASK is 1, which means that it is turned off. Therefore, the usage fault interrupt cannot be responded, the HardFault is triggered by CPU. It is actually a usage fault.

```
1408c650:   4798        blx r3
```

```
### EXCEPTION ###
PC =1408B502, ExceptionNumber=D'-13, EXC_RETURN=FFFFFFAC

R0 =00000000, R1 =00000000, R2 =00000000, R3 =1408B502
R4 =20502E8C, R5 =205417A0, R6 =00000019, R7 =1408C5F9
R8 =0001388F, R9 =205450A8, R10=00000000, R11=00000000
R12=40130000, SP =20547238, LR =1408C613

MSP   =205BE7D8, PSP   =205471D0
MSPLIM=205BD700, PSPLIM=205450B0

PRIMASK   =01, FAULTMASK   =00, BASEPRI   =00, CONTROL   =00

XPSR=00000000, APSR=nzcvq, EPSR=00000000, IPSR=000 (NoException)

ICSR =00000803, AIRCR=FA052000, SCR  =00000000, CCR  =00000611
SHCSR=00070004, CFSR =00020000, HFSR =40000000, AFSR =00000000
DFSR =00000000
MMFAR=E000ED34, BFAR =E000ED38

(Escalation HardFault)
FaultInfo    : (HardFault)
FaultCause   : (Invalid state UsageFault)
DebugEvent   : (None)
```

Figure 119 Hard Fault Dump

Sample Code 76: Hard Fault

{"hard_fault", hard_fault_test},

5.12.2.2 ASSERT Analysis

Assert is used to catch the scene in some impossible events or results in the software code. Each Assert has a corresponding Assert string. The location of the crash can be found quikly by searching the Assert string in code. This positioning is much easier than Exception, but you may not understand why the Assert is here or what causes the Assert. Several common Assert positioning methods and ideas are summarized as follows.

5.12.2.2.1 app_wt-irq__handle

The reason for the watchdog crash is that watchdog isn't fed before timeout. Now we are feeding the dog in the timer thread, so this crash occurs when the timer thread cannot be scheduled. There are usually two cases:

1. The system MIPS is not enough, and other threads occupy all the CPU, so the timer thread can't be scheduled.
2. The Timer thread and other threads are deadlocked, causing the subsequent timer callback to fail to execute.

The first type of crash can be easily identified by looking at the CPU usage. The solution is usually to optimize MIPS or increase the CPU frequency for certain scenario.

The second type of crash is showed in the following figure. The timer thread state is WAIT_MUT, which means the timer thread is stopped by waiting mutex. Looking for other threads, thread 14 named Thread_test is also waiting mutex. Then through the C code, it can be viewed that deadlock_mutex1 is taken first in the timer callback and then waits for deadlock_mutex2. Thread_test takes deadlock_mutex2 first and then waits for deadlock_mutex1. In this way, two threads cannot wait for each other's mutex and does not release their own mutex, so the two threads can never continue to execute.

So waiting for mutex in the timer thread is usually prohibitted. One reason is that it may cause a deadlock, and the other reason is that it may take a long time to get the mutex, which will cause other timers to fail to respond on time.

CONFIDENTIAL

```
--- Thread  14 name=Thread_test thread=0x20566c80, prio=40 state=WAIT_MUT  thread_addr=0x1408C80D
    thread_next=0x00000000 thread_prev=0x20566C30 delay_next=0x20565380 delay_prev=0x205669A0
    thread_join=0x00000000 flags_options=0 wait_flags=0 thread_flags=0
    stack_mem=0x20532DB0 stack_size=1024 sp:0x20533158 min_stack_free=936
    swap_in_time=161558(ticks)/10097(ms) swap_out_time=161559(ticks)/10097(ms)
    after last switch task runtime 1(ticks)/0 ms
    frame:0x20533178
    R0 =0xFFFFFFFE R1=0x20566C80 R2=0x00000000 R3  =0x00000000
    R12=0x0050669D LR=0x00506A37 PC=0x00506A62 XPSR=0x01000000

Possible Backtrace:
  506A32
1408C826

--- Thread  3 name=timer thread=0x2056ab38, prio=40 state=WAIT_MUT  thread_addr=0x005041DD
    thread_next=0x00000000 thread_prev=0x20566C58 delay_next=0x00000000 delay_prev=0x20563050
    thread_join=0x00000000 flags_options=0 wait_flags=0 thread_flags=0
    stack_mem=0x2056ABE8 stack_size=4096 sp:0x2056bb70 min_stack_free=3240
    swap_in_time=163157(ticks)/10197(ms) swap_out_time=163163(ticks)/10197(ms)
    after last switch task runtime 6(ticks)/0 ms
    frame:0x2056BB90
    R0 =0xFFFFFFFE R1=0x2056AB38 R2=0x00000000 R3  =0x00000000
    R12=0x0050669D LR=0x00506A37 PC=0x00506A62 XPSR=0x61000000

Possible Backtrace:
  506A32
1408C866
  50421E
```

Figure 120 Deadlock Causes Watchdog Crash

```
static void deadlock_test_timer_cb(void const *arg)
{
    osMutexWait(deadlock_mutex1_id, osWaitForever);
    osDelay(100);
    TRACE(0, "%s try to get deadlock mutex2", __func__);
    osMutexWait(deadlock_mutex2_id, osWaitForever);
    TRACE(0, "%s got deadlock mutex2", __func__);

    osMutexRelease(deadlock_mutex1_id);
    osMutexRelease(deadlock_mutex2_id);
}

static void Thread_test(void const *argument) {
    while (1)
    {
        osMutexWait(deadlock_mutex2_id, osWaitForever);
        osThreadYield();
        osMutexWait(deadlock_mutex1_id, osWaitForever);

        osMutexRelease(deadlock_mutex2_id);
```

```
        osMutexRelease(deadlock_mutex1_id);
    }
}
```

Sample Code 77: Deadlock Causes Watchdog Crash

{"dead_lock", dead_lock_test},

5.12.2.2.2 Stact Smashing Detected

The reason for this crash is that the thread stack is destroyed. As shown in the following assembly code in the lst file, GCC take a stack check, a value is loaded from the flash and stored at register SP offset 20 after entering the function stack_overflow_test. When the function is exited, the value of SP offset 20 is taken out and compared with the value in the flash. If the value is different, it jumps to __stack_chk_fail.

Therefore, it's necessary to find the previous function that jumps into __stack_chk_fail from the stack space when encountering such a crash, then check whether there is a possibility of buffer overflow in this function.

```
1408c6b8 <_ZL19stack_overflow_testv>:
_ZL19stack_overflow_testv():
/home/project/mcu-
sw/out/best1600_ibrt/../../services/../bthost/service/ble_audio/ble_audio_t
est/ble_audio_test.cpp:2515
{
1408c6b8:   b500        push    {lr}
1408c6ba:   4b11        ldr r3, [pc, #68]   ; (1408c700
<_ZL19stack_overflow_testv+0x48>)
1408c6bc:   b087        sub sp, #28
1408c6be:   681b        ldr r3, [r3, #0]
1408c6c0:   9305        str r3, [sp, #20]
1408c6c2:   f04f 0300   mov.w   r3, #0
/home/project/mcu-
sw/out/best1600_ibrt/../../services/../bthost/service/ble_audio/ble_audio_t
est/ble_audio_test.cpp:2517
    uint8_t test_cnt = GET_CURRENT_MS();
1408c6c6:   f0bf fafb   bl  1414bcc0 <__hal_sys_timer_get_veneer>
1408c6ca:   4602        mov r2, r0
/home/project/mcu-
sw/out/best1600_ibrt/../../services/../bthost/service/ble_audio/ble_audio_t
est/ble_audio_test.cpp:2518
    memset(test_buf, 0xbb, test_cnt);
1408c6cc:   21bb        movs    r1, #187    ; 0xbb
```

```
1408c6ce:   f3c2 1207   ubfx    r2, r2, #4, #8
1408c6d2:   a802        add r0, sp, #8
1408c6d4:   9201        str r2, [sp, #4]
1408c6d6:   f0bf fb33   bl  1414bd40 <__memset_veneer>
/home/project/mcu-
sw/out/best1600_ibrt/../../services/../bthost/service/ble_audio/ble_audio_t
est/ble_audio_test.cpp:2519
    DUMP8("%02x ", test_buf, test_cnt);
1408c6da:   9a01        ldr r2, [sp, #4]
1408c6dc:   ab02        add r3, sp, #8
1408c6de:   2101        movs    r1, #1
1408c6e0:   4808        ldr r0, [pc, #32]   ; (1408c704
<_ZL19stack_overflow_testv+0x4c>)
1408c6e2:   f79a fe4d   bl  14027380 <hal_trace_dump>
/home/project/mcu-
sw/out/best1600_ibrt/../../services/../bthost/service/ble_audio/ble_audio_t
est/ble_audio_test.cpp:2520
}
1408c6e6:   4b06        ldr r3, [pc, #24]   ; (1408c700
<_ZL19stack_overflow_testv+0x48>)
1408c6e8:   681a        ldr r2, [r3, #0]
1408c6ea:   9b05        ldr r3, [sp, #20]
1408c6ec:   405a        eors    r2, r3
1408c6ee:   f04f 0300   mov.w   r3, #0
1408c6f2:   d102        bne.n   1408c6fa <_ZL19stack_overflow_testv+0x42>
1408c6f4:   b007        add sp, #28
1408c6f6:   f85d fb04   ldr.w   pc, [sp], #4
1408c6fa:   f7b2 ffc5   bl  1403f688 <__stack_chk_fail>
1408c6fe:   bf00        nop
1408c700:   20502d90    .word   0x20502d90
1408c704:   3414eb20    .word   0x3414eb20
```

```
### ASSERT @ 0x1403F693 ###
*** stack smashing detected ***: terminated

R0 =342810A8, R1 =34254BCC, R2 =65160554, R3 =00000000
R4 =20563BB0, R5 =205417A8, R6 =00000019, R7 =1408C6B9
R8 =0001388F, R9 =205450B0, R10=00000000, R11=00000000
R12=40130000, SP =20547220, LR =1403F693

MSP   =205BE7D8, PSP   =20547220
MSPLIM=205BD700, PSPLIM=205450B8

Stack:
  205471E0: 62206262 62622062 20626220 0A206262
  205471F0: 00000000 00000000 00000001 2054E088
  20547200: 2054E3F8 DEADBEEF 00000000 20563BB0
  20547210: 205417A8 00000019 1408C6B9 0001388F
* 20547220: 00000000 1408C6FF 0001388F 0000004D
  20547230: BBBBBBBB BBBBBBBB BBBBBBBB BBBBBBBB
  20547240: BBBBBBBB BBBBBBBB BBBBBBBB BBBBBBBB
  20547250: BBBBBBBB BBBBBBBB BBBBBBBB BBBBBBBB
  20547260: BBBBBBBB BBBBBBBB BBBBBBBB BBBBBBBB
  20547270: BBBBBBBB BBBBBBBB BBBBBBBB 140AD2BB
  20547280: 00000010 000000FF 00000000 DEADBEEF
  20547290: 00000000 00000000 00000000 00000000

Possible Backtrace:
1408C6FA
140AD33E
140E0B0C
```

Figure 121 Stack Smashing

Sample Code 78: stack_overflow

{" stack_overflow", stack_overflow_test},

5.12.2.2.3 Generic Crash Analysis

There is a general crash analysis method, namely, using the software gcc-arm-none-eabi to find the crash location of code.

1. Download and install gcc-arm-none-eabi. Add the installation path to the environment variable if the installation OS is Windows.

CONFIDENTIAL

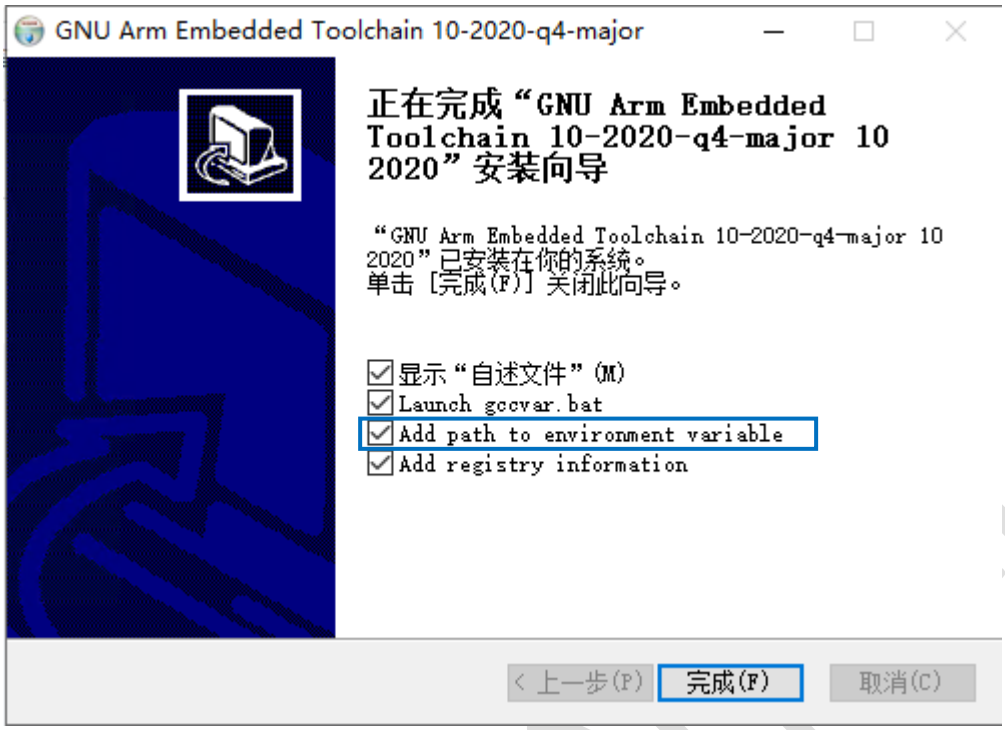


Figure 122 Install gcc-arm-none-eabi

2. Turn to the compiled elf file path, open the command shell, and enter the command arm-none-eabi-addr2line -e xxx.elf xxxxx as follows:

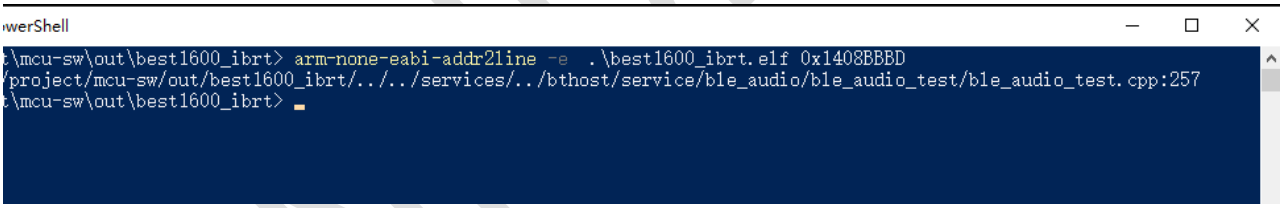


Figure 123 Get Crash Location

As the figure shows, the location before the crash in the code is /bthost/service/ble_audio_test.cpp line 257

3. Turn to the code location to find the bug.

5.12.3 Full CPU Workload Power Consumption Measurement

Before measuring the full CPU workload power consumption, the two macro switches need to be configured in the target.mk:

SPECIFIC_FREQ_POWER_CONSUMPTION_MEASUREMENT_ENABLE ?= 1

WATCHER_DOG ?= 0

The CPU frequency needs to be configured by APP_SYSFREQ_USER before measurement.

Multiple users can set CPU frequencies at the same time, but only one takes effect. Basecally, the highest requested frequency will be the configured value if several users ask for different frequencies. If a user doesn't need to frequency anymore, it will release the frequency and the existing frequency request list will be went through again to choose the highest one.

```
enum APP_SYSFREQ_USER_T {
    APP_SYSFREQ_USER_APP_INIT = HAL_SYSFREQ_USER_INIT,
    APP_SYSFREQ_USER_APP_0    = HAL_SYSFREQ_USER_APP_0,
    APP_SYSFREQ_USER_APP_1    = HAL_SYSFREQ_USER_APP_1,
    APP_SYSFREQ_USER_APP_2    = HAL_SYSFREQ_USER_APP_2,
    APP_SYSFREQ_USER_APP_3    = HAL_SYSFREQ_USER_APP_3,
    APP_SYSFREQ_USER_APP_4    = HAL_SYSFREQ_USER_APP_4,
    APP_SYSFREQ_USER_APP_5    = HAL_SYSFREQ_USER_APP_5,
    APP_SYSFREQ_USER_APP_6    = HAL_SYSFREQ_USER_APP_6,
    APP_SYSFREQ_USER_APP_7    = HAL_SYSFREQ_USER_APP_7,
    APP_SYSFREQ_USER_APP_8    = HAL_SYSFREQ_USER_APP_8,
    APP_SYSFREQ_USER_APP_9    = HAL_SYSFREQ_USER_APP_9,
    APP_SYSFREQ_USER_APP_10   = HAL_SYSFREQ_USER_APP_10,
    APP_SYSFREQ_USER_APP_11   = HAL_SYSFREQ_USER_APP_11,
    APP_SYSFREQ_USER_APP_12   = HAL_SYSFREQ_USER_APP_12,
    APP_SYSFREQ_USER_APP_13   = HAL_SYSFREQ_USER_APP_13,
    APP_SYSFREQ_USER_APP_14   = HAL_SYSFREQ_USER_APP_14,
    APP_SYSFREQ_USER_APP_15   = HAL_SYSFREQ_USER_APP_15,
    APP_SYSFREQ_USER_APP_16   = HAL_SYSFREQ_USER_APP_16,
    APP_SYSFREQ_USER_APP_17   = HAL_SYSFREQ_USER_APP_17,
    APP_SYSFREQ_USER_APP_18   = HAL_SYSFREQ_USER_APP_18,
    APP_SYSFREQ_USER_APP_19   = HAL_SYSFREQ_USER_APP_19,
    APP_SYSFREQ_USER_QTY
};
```

By default, each user is assigned with APP_SYSFREQ_32K and the system frequency is 32KHz, which means that system can enter deep sleep mode. If any user requests a frequency higher than 32KHz, the system won't be allowed to enter deep sleep mode.

The frequency is configured as 26MHz here.

```
enum APP_SYSFREQ_FREQ_T {
    APP_SYSFREQ_32K =  HAL_CMU_FREQ_32K,
    APP_SYSFREQ_26M =  HAL_CMU_FREQ_26M,
    APP_SYSFREQ_52M =  HAL_CMU_FREQ_52M,
    APP_SYSFREQ_78M =  HAL_CMU_FREQ_78M,
```

```
    APP_SYSFREQ_104M = HAL_CMU_FREQ_104M,
#ifdef HAL_CMU_FREQ_15M
    APP_SYSFREQ_12M =  HAL_CMU_FREQ_12M,
    APP_SYSFREQ_15M =  HAL_CMU_FREQ_15M,
    APP_SYSFREQ_30M =  HAL_CMU_FREQ_30M,
    APP_SYSFREQ_48M =  HAL_CMU_FREQ_48M,
    APP_SYSFREQ_120M = HAL_CMU_FREQ_120M,
    APP_SYSFREQ_144M = HAL_CMU_FREQ_144M,
    APP_SYSFREQ_168M = HAL_CMU_FREQ_168M,
#endif
    APP_SYSFREQ_208M = HAL_CMU_FREQ_208M,

    APP_SYSFREQ_FREQ_QTY =  HAL_CMU_FREQ_QTY
};

#define POWER_CONSUMPTION_MEASUREMENT_FREQ  APP_SYSFREQ_26M
```

A thread named power_consumption_thread is created when the system is initiated. This thread firstly releases all the allocated frequencies, makes the coprocessor work at full load if there is a coprocessor in the CPU, sets the frequency, and finally executes the while loop to make the CPU work at full load.

```
static void power_consumption_thread(const void *arg)
{
    // release all the allocated frequencies
    for (uint32_t user = APP_SYSFREQ_USER_APP_INIT;
        user < APP_SYSFREQ_USER_QTY; user++)
    {
        app_sysfreq_req((enum APP_SYSFREQ_USER_T)user, APP_SYSFREQ_32K);
    }

    // Let coprocessor work at full load
#if defined(CP_IN_SAME_EE)&&defined(IS_TEST_MCU_AND_CP)
    power_consumption_measurement_start_cp();
#endif

    app_sysfreq_req(APP_SYSFREQ_USER_APP_15,
        (enum APP_SYSFREQ_FREQ_T)POWER_CONSUMPTION_MEASUREMENT_FREQ);
    while(1);
}
```

If all thread information is printed out (see details in 7.4 Thread information periodic print), the log shows that the thread power_consumption_thread has made the CPU work at full load.

At this time, the full CPU workload power consumption can be measured by the electric current.

```
Thread List:
--- Thread name=timer cpu=%0 min-free-stack=3384
--- Thread name=power_consumption_thread cpu=%100 min-free-stack=352
--- Thread name=main cpu=%0 min-free-stack=688
--- Thread name=idle cpu=%0 min-free-stack=864
--- Thread name=app_trace_rx cpu=%0 min-free-stack=1912
--- Thread name=audio_flinger cpu=%0 min-free-stack=10072
--- Thread name=app_bt_cmd_thread cpu=%0 min-free-stack=1824
--- Thread name=app_thread cpu=%0 min-free-stack=2920
--- Thread name=app_prompt_handler cpu=%0 min-free-stack=840
```

Figure 124 power_consumption_thread Infomation

5.12.4 Thread Information Periodic Print

In order to periodically print thread information, enable the macro switch OS_THREAD_TIMING_STATISTICS_ENABLE in target.mk:

```
OS_THREAD_TIMING_STATISTICS_ENABLE ?= 1
```

The system creates a timer thread to print all the thread information periodically as follows:

```
static void cpu_usage_timer_handler(void const *param)
{
    rtx_show_all_threads_usage();
}

void rtx_show_all_threads_usage(void)
{
    const os_thread_t *thread;
    static bool first_time  = 1;
    uint32_t sample_time;
    static uint32_t start_sample_time  = 0;

    if (first_time) {
        start_sample_time  = rtx_get_hwticks();
        first_time = 0;
        return;
    }

    sample_time = HWTICKS_TO_MS(rtx_get_hwticks() - start_sample_time);
```

```
    REL_TRACE_IMM_NOTS(0," ");
    REL_TRACE_NOTS(0,"Thread List:");

    // Current List
    _rtx_show_thread_usage(osRtxInfo.thread.run.curr, sample_time);

    // Next List
    if (osRtxInfo.thread.run.next != osRtxInfo.thread.run.curr)
        _rtx_show_thread_usage(osRtxInfo.thread.run.next, sample_time);

    // Ready List
    for (thread = osRtxInfo.thread.ready.thread_list;
        thread != NULL; thread = thread->thread_next) {
        _rtx_show_thread_usage(thread, sample_time);
    }

    // Delay List
    for (thread = osRtxInfo.thread.delay_list;
        thread != NULL; thread = thread->delay_next) {
        _rtx_show_thread_usage(thread, sample_time);
    }

    // Wait List
    for (thread = osRtxInfo.thread.wait_list;
        thread != NULL; thread = thread->delay_next) {
        _rtx_show_thread_usage(thread, sample_time);
    }

    // Terminate List
    for (thread = osRtxInfo.thread.terminate_list;
        thread != NULL; thread = thread->thread_next) {
        _rtx_show_thread_usage(thread, sample_time);
    }
    start_sample_time  = rtx_get_hwticks();
    REL_TRACE_IMM_NOTS(0," ");
}
```

The timer thread prints out all information of the threads in Current List, Next List, Ready List, Delay List, Wait List and Terminate List thread through the function rtx_show_thread_usage.

The thread information consists of the thread name, CPU load and minimum free stack size as the follows:

```
Thread List:
--- Thread name=timer cpu=%0 min-free-stack=3240
--- Thread name=idle cpu=%99 min-free-stack=288
--- Thread name=app_trace_rx cpu=%0 min-free-stack=1912
--- Thread name=app_bt_cmd_thread cpu=%0 min-free-stack=1824
--- Thread name=tws_ctrl cpu=%0 min-free-stack=2536
--- Thread name=TOTA_STREAM_DATA_THREAD cpu=%0 min-free-stack=696
--- Thread name=main cpu=%0 min-free-stack=176
--- Thread name=audio_flinger cpu=%0 min-free-stack=9328
--- Thread name=app_prompt_handler cpu=%0 min-free-stack=680
--- Thread name=intersys_rx_thread cpu=%0 min-free-stack=1192
--- Thread name=app_ui_thread cpu=%0 min-free-stack=976
--- Thread name=app_thread cpu=%0 min-free-stack=1920
--- Thread name=bes_bt_main cpu=%0 min-free-stack=6656
```

Figure 125 Thread Information

CONFIDENTIAL

5.12.5 Audio Dump Tool Usage

GUI Interface

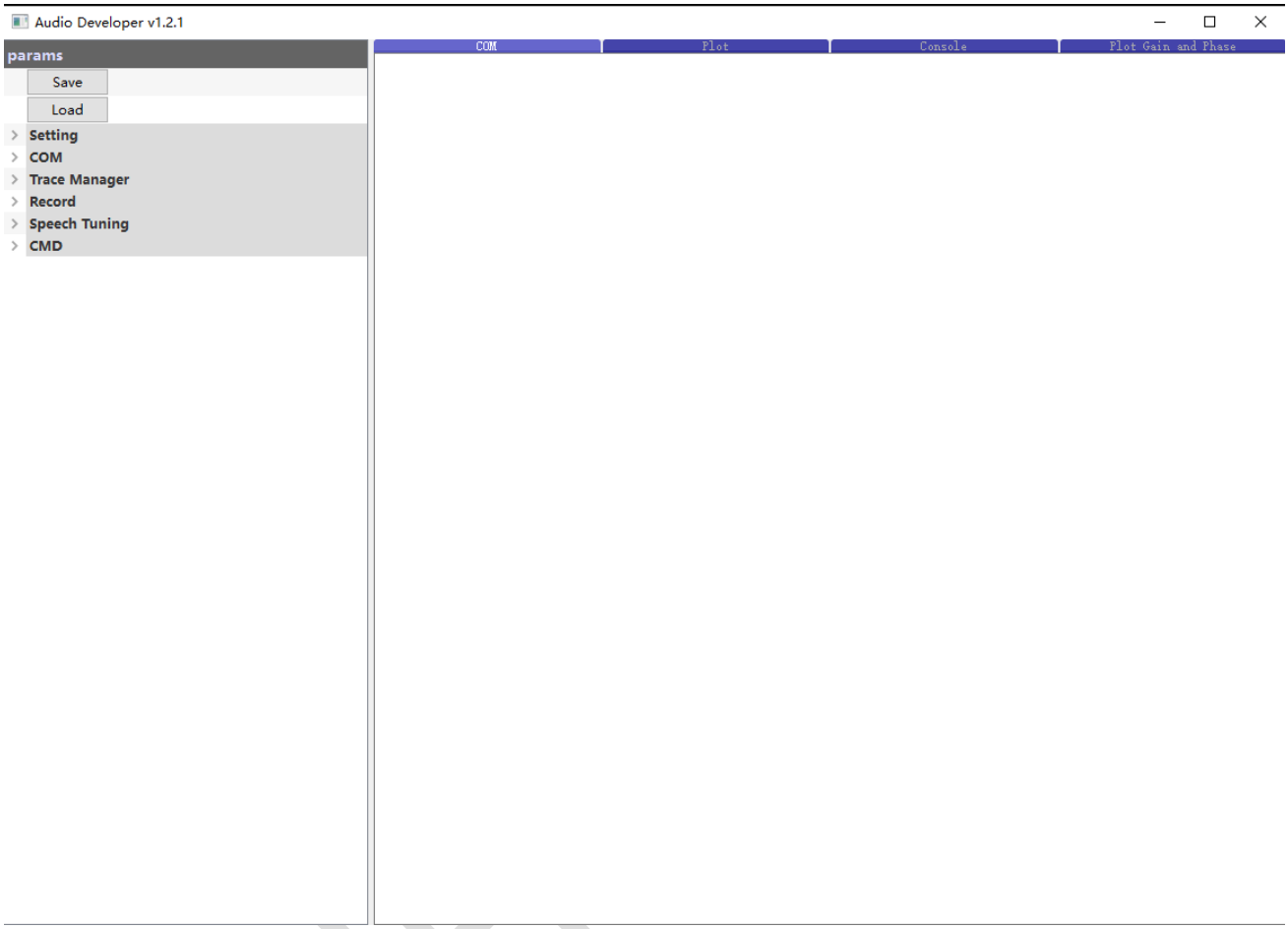


Figure 126 Audio Developer GUI

5.12.5.1 Preparation

Enable the realtive “DUMP” macro in the BES code

1. Download PCM data from the main kernel (monolithic kernel or multiple kernels):

   Build binary with AUDIO_DEBUG=1 in the M33 command line.

2. Download PCM data from other kernels (multiple kernels):

   Build binary with AUDIO_DEBUG=1 DSP_M55_TRC_TO_MCU=0 DEBUG_PORT=1 in M55 compile command, DSP_M55_TRC_TO_MCU=0 DEBUG_PORT=2 in the M33 command line.

   DSP_M55_TRC_TO_MCU is used for stop transmission between M33 and M55.

   DEBUG_PORT is used for seperate M55 TRACE print from M33. Please ask for help from the hardware engineer to verify the UART connection method according to the black board version:

NOTE: TRACE affects PCM data dump. If you print many TRACE in an instant, CHIP may not output PCM data, which will cause data lost in the PC. In this case, you can decrease the priority of log print(main.cpp): hal_trace_set_log_level().

5.12.5.2 COM

1. Run Audio Developer v0.5.3\dist\main.exe.
2. Unfold COM, and set the corresponding Port and Baud Rate. Normally, set the baudrate to 2000000.

   Check the platform\hal\hal_trace.c file for details if necessary. The BES chip can support up to 3M baudrate.

   Note: Make sure that your USB2UART tool can support bigger baudrate than your setting in the BES code.

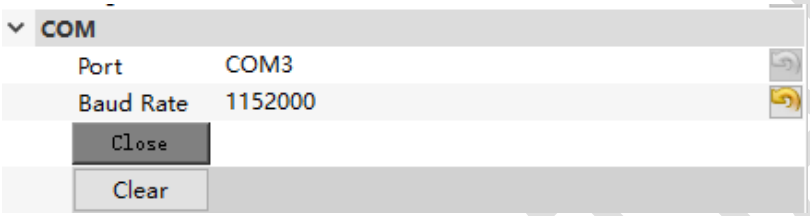


Figure 127 COM

To check the port, right click My Computer/This PC on the desktop, select Manage. The following window appears:

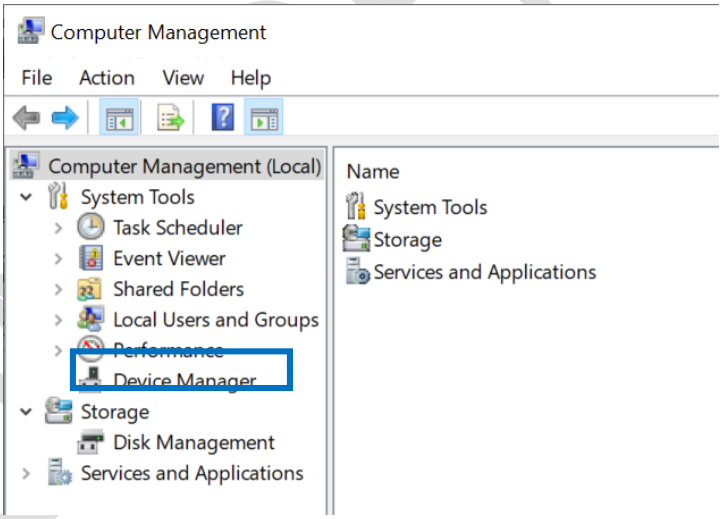


Figure 128 Computer Management

Click **Device Manager**, and unfold Ports (COM & LPT) in the right box. The current used port is displayed.

Note: For multiple kernels, there exists more than one COM list. Please choose the correct COM based on logs or GPIO list.

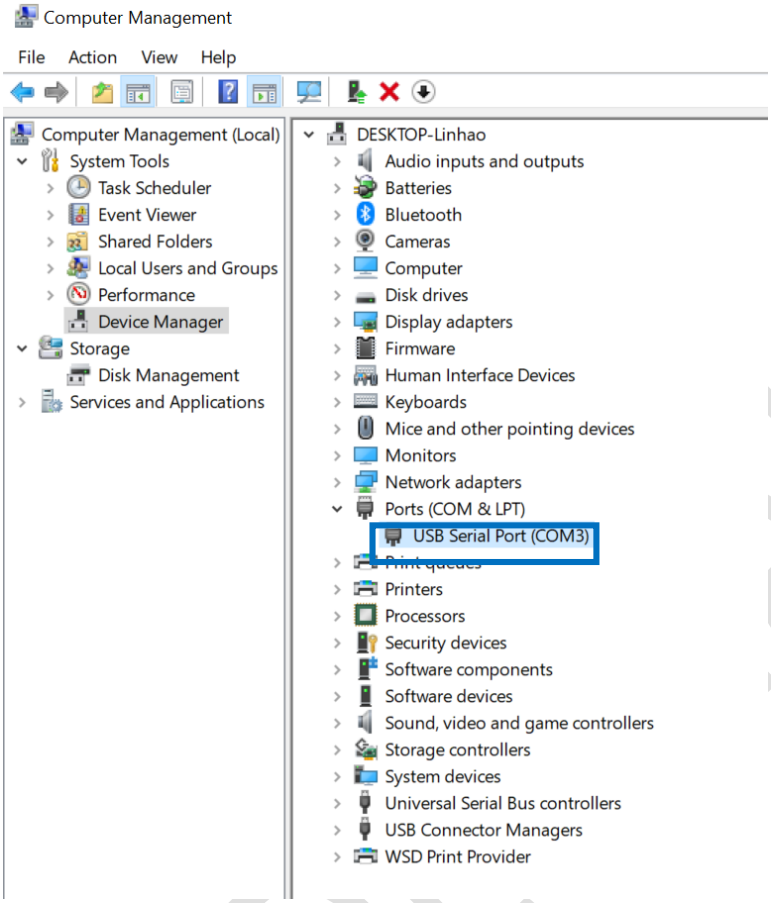


Figure 129 COM3 Port

After the settings is completed, click Close to change the status to Open (the port is opened now), and we can start tuning. Likewise, click Open will change the status back to Close (the port is closed now) and tuning is stopped.

5.12.5.3 Trace Manager

Save Data

1. Modify BES SDK to use the data_dump_run function to output data through UART.

   data_dump_run("[mic1_data]", mic1_buf, mic1_buf_len);

2. Unfold **TRACE Manager**, and choose **Save Data** > **Add New**.

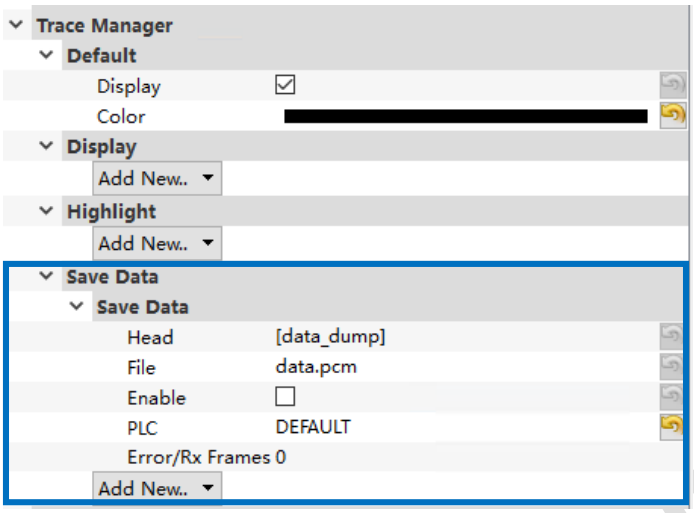


Figure 130 Save Data

3. Set the Head and File path, then click Enable to start saving data. If you want to finish saving data, click Enable again to save data into the file.

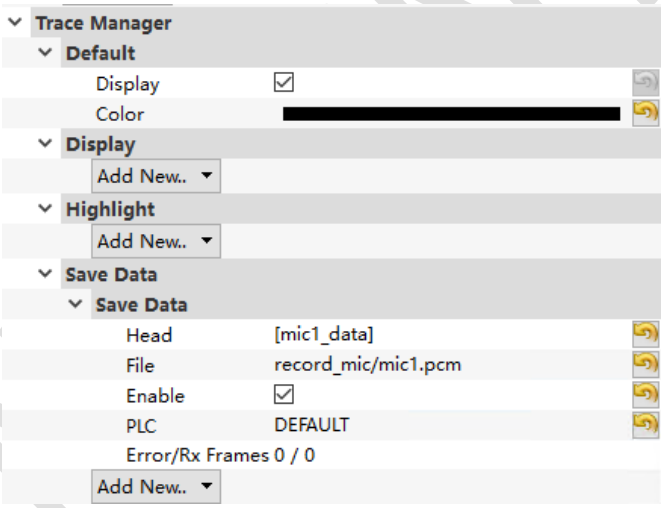


Figure 131 Trace Manager

- We recommend the Head string include [] and the string length > 6.
- File path can include the folder path and you can find saved file under out\Save_Data\ folder.
- If you disable DUMP_PLC_ENABLE in firmware, PLC should be DEFAULT. If you enable DUMP_PLC_ENABLE, PLC should be MAX, ZERO or LOST.

CONFIDENTIAL

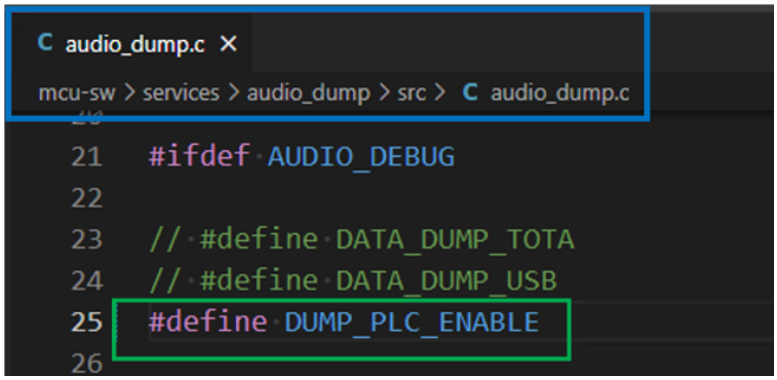


Figure 132 DUMP_PLC_ENABLE

- You can click Add New to add more channels.

4. Make sure that the Com baud Rate is enough to dump data.

5.12.5.4 Recording

1. Unfold Record, set the Wav File path, and click Enable to start recording. If you want to finish recording, click Enable again to save data into the file.

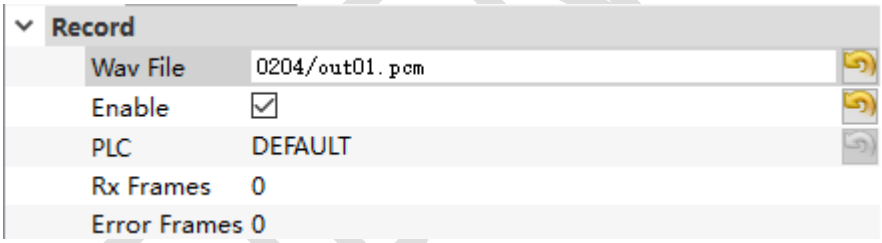


Figure 133 Record

- Wav File path can include the folder path and you can find save file under out\Record\ folder.
- If you disable DUMP_PLC_ENABLE in firmware, PLC should be DEFAULT. If you enable DUMP_PLC_ENABLE, PLC should be MAX, ZERO or LOST

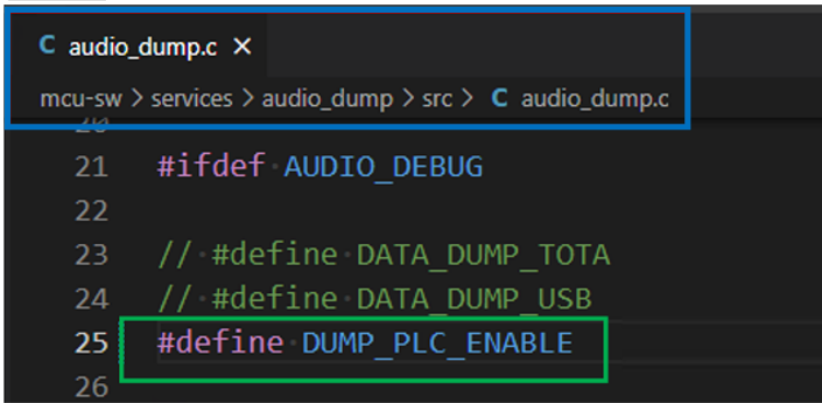


Figure 134 DUMP_PLC_ENABLE

CONFIDENTIAL

2. Make sure that the Com baud Rate is enough to Record data.

3. If you want to dump PCM data during phone call, you can enable BT_SCO_CHAIN_AUDIO_DUMP. PCM data is 16k sample rate, 16 bits and 2 channels by default.

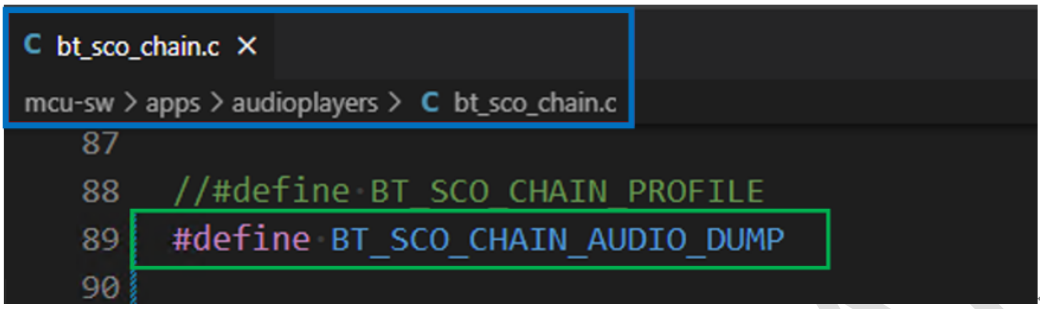


Figure 135 BT_SCO_CHAIN_AUDIO_DUMP

4. If you want to dump data in your stream, refer to BT_SCO_CHAIN_AUDIO_DUMP.

CONFIDENTIAL